

Actor-Critic Reinforcement Learning Algorithms and Frameworks

Table of Contents

OpenAI Gym	2
Actor-Critic Algorithm in Reinforcement Learning	5
High-Dimensional Continuous Control Using Generalized Advantage Estimation	12
DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills	25
A (Long) Peak into Reinforcement Learning	42
Real-time reinforcement learning by sequential Actor-Critics and experience replay	63
An Analysis of Actor/Critic Algorithms using Eligibility Traces: Reinforcement Learning with Imperfect Value Functions	77
Autonomous reinforcement learning with experience replay	85
Continuous control with deep reinforcement learning	96
Stable-Baselines3: Reliable Reinforcement Learning Implementations	109
Proximal Policy Optimization Algorithms	113
END	123

OpenAI Gym

Greg Brockman, Vicki Cheung, Ludwig Pettersson,
Jonas Schneider, John Schulman, Jie Tang, Wojciech Zaremba
OpenAI

Abstract

OpenAI Gym¹ is a toolkit for reinforcement learning research. It includes a growing collection of benchmark problems that expose a common interface, and a website where people can share their results and compare the performance of algorithms. This whitepaper discusses the components of OpenAI Gym and the design decisions that went into the software.

1 Introduction

Reinforcement learning (RL) is the branch of machine learning that is concerned with making sequences of decisions. RL has a rich mathematical theory and has found a variety of practical applications [1]. Recent advances that combine deep learning with reinforcement learning have led to a great deal of excitement in the field, as it has become evident that general algorithms such as policy gradients and Q-learning can achieve good performance on difficult problems, without problem-specific engineering [2, 3, 4].

To build on recent progress in reinforcement learning, the research community needs good benchmarks on which to compare algorithms. A variety of benchmarks have been released, such as the Arcade Learning Environment (ALE) [5], which exposed a collection of Atari 2600 games as reinforcement learning problems, and recently the RLLab benchmark for continuous control [6], to which we refer the reader for a survey on other RL benchmarks, including [7, 8, 9, 10, 11]. OpenAI Gym aims to combine the best elements of these previous benchmark collections, in a software package that is maximally convenient and accessible. It includes a diverse collection of tasks (called *environments*) with a common interface, and this collection will grow over time. The environments are versioned in a way that will ensure that results remain meaningful and reproducible as the software is updated.

Alongside the software library, OpenAI Gym has a website (gym.openai.com) where one can find scoreboards for all of the environments, showcasing results submitted by users. Users are encouraged to provide links to source code and detailed instructions on how to reproduce their results.

2 Background

Reinforcement learning assumes that there is an agent that is situated in an environment. Each step, the agent takes an *action*, and it receives an *observation* and *reward* from the environment. An RL algorithm seeks to maximize some measure of the agent’s total reward, as the agent interacts with the environment. In the RL literature, the environment is formalized as a partially observable Markov decision process (POMDP) [12].

OpenAI Gym focuses on the episodic setting of reinforcement learning, where the agent’s experience is broken down into a series of *episodes*. In each episode, the agent’s initial state is randomly sampled from a distribution, and the interaction proceeds until the environment reaches a terminal state. The goal in episodic reinforcement learning is to maximize the expectation of total reward per episode, and to achieve a high level of performance in as few episodes as possible.

The following code snippet shows a single episode with 100 timesteps. It assumes that there is an object called `agent`, which takes in the observation at each timestep, and an object called `env`, which is the

¹gym.openai.com

environment. OpenAI Gym does not include an agent class or specify what interface the agent should use; we just include an agent here for demonstration purposes.

```
ob0 = env.reset() # sample environment state, return first observation
a0 = agent.act(ob0) # agent chooses first action
ob1, rew0, done0, info0 = env.step(a0) # environment returns observation,
# reward, and boolean flag indicating if the episode is complete.
a1 = agent.act(ob1)
ob2, rew1, done1, info1 = env.step(a1)
...
a99 = agent.act(o99)
ob100, rew99, done99, info2 = env.step(a99)
# done99 == True => terminal
```

3 Design Decisions

The design of OpenAI Gym is based on the authors’ experience developing and comparing reinforcement learning algorithms, and our experience using previous benchmark collections. Below, we will summarize some of our design decisions.

Environments, not agents. Two core concepts are the agent and the environment. We have chosen to only provide an abstraction for the environment, not for the agent. This choice was to maximize convenience for users and allow them to implement different styles of agent interface. First, one could imagine an “online learning” style, where the agent takes (observation, reward, done) as an input at each timestep and performs learning updates incrementally. In an alternative “batch update” style, a agent is called with observation as input, and the reward information is collected separately by the RL algorithm, and later it is used to compute an update. By only specifying the agent interface, we allow users to write their agents with either of these styles.

Emphasize sample complexity, not just final performance. The performance of an RL algorithm on an environment can be measured along two axes: first, the final performance; second, the amount of time it takes to learn—the sample complexity. To be more specific, final performance refers to the average reward per episode, after learning is complete. Learning time can be measured in multiple ways, one simple scheme is to count the number of episodes before a threshold level of average performance is exceeded. This threshold is chosen per-environment in an ad-hoc way, for example, as 90% of the maximum performance achievable by a very heavily trained agent. Both final performance and sample complexity are very interesting, however, arbitrary amounts of computation can be used to boost final performance, making it a comparison of computational resources rather than algorithm quality.

Encourage peer review, not competition. The OpenAI Gym website allows users to compare the performance of their algorithms. One of its inspiration is [Kaggle](#), which hosts a set of machine learning contests with leaderboards. However, the aim of the OpenAI Gym scoreboards is not to create a competition, but rather to stimulate the sharing of code and ideas, and to be a meaningful benchmark for assessing different methods.

RL presents new challenges for benchmarking. In the supervised learning setting, performance is measured by prediction accuracy on a test set, where the correct outputs are hidden from contestants. In RL, it’s less straightforward to measure generalization performance, except by running the users’ code on a collection of unseen environments, which would be computationally expensive. Without a hidden test set, one must check that an algorithm did not “overfit” on the problems it was tested on (for example, through parameter tuning).

We would like to encourage a peer review process for interpreting results submitted by users. Thus, OpenAI Gym asks users to create a *Writeup* describing their algorithm, parameters used, and linking to code. Writeups should allow other users to reproduce the results. With the source code available, it is possible to make a nuanced judgement about whether the algorithm “overfit” to the task at hand.

Strict versioning for environments. If an environment changes, results before and after the change would be incomparable. To avoid this problem, we guarantee than any changes to an environment will be accompanied by an increase in version number. For example, the initial version of the CartPole task is named `Cartpole-v0`, and if its functionality changes, the name will be updated to `Cartpole-v1`.

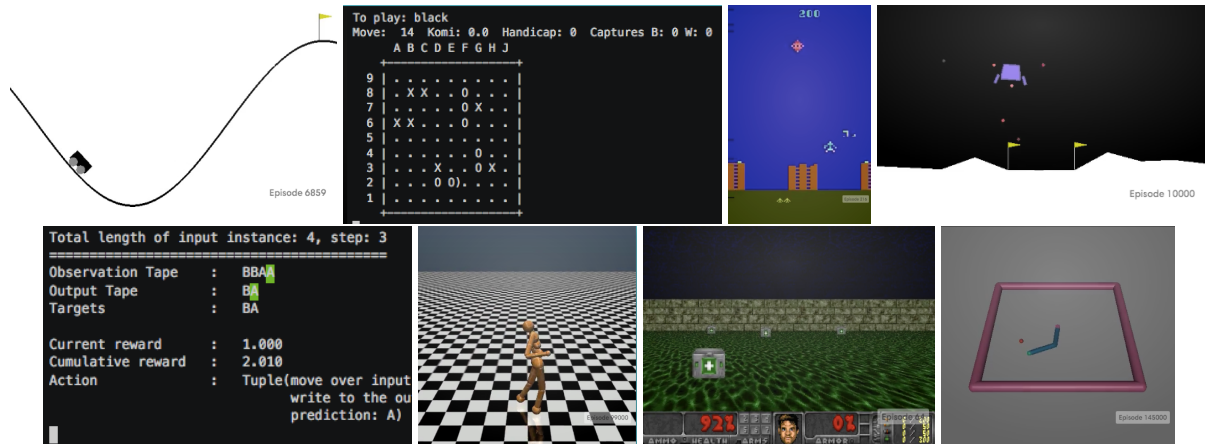


Figure 1: Images of some environments that are currently part of OpenAI Gym.

Monitoring by default. By default, environments are instrumented with a `Monitor`, which keeps track of every time step (one step of simulation) and `reset` (sampling a new initial state) are called. The `Monitor`’s behavior is configurable, and it can record a video periodically. It also is sufficient to produce learning curves. The videos and learning curve data can be easily posted to the OpenAI Gym website.

4 Environments

OpenAI Gym contains a collection of Environments (POMDPs), which will grow over time. See Figure 1 for examples. At the time of Gym’s initial beta release, the following environments were included:

- *Classic control and toy text*: small-scale tasks from the RL literature.
- *Algorithmic*: perform computations such as adding multi-digit numbers and reversing sequences. Most of these tasks require memory, and their difficulty can be chosen by varying the sequence length.
- *Atari*: classic Atari games, with screen images or RAM as input, using the Arcade Learning Environment [5].
- *Board games*: currently, we have included the game of Go on 9x9 and 19x19 boards, where the Pachi engine [13] serves as an opponent.
- *2D and 3D robots*: control a robot in simulation. These tasks use the MuJoCo physics engine, which was designed for fast and accurate robot simulation [14]. A few of the tasks are adapted from RLLab [6].

Since the initial release, more environments have been created, including ones based on the open source physics engine Box2D or the Doom game engine via VizDoom [15].

5 Future Directions

In the future, we hope to extend OpenAI Gym in several ways.

- *Multi-agent setting*. It will be interesting to eventually include tasks in which agents must collaborate or compete with other agents.
- *Curriculum and transfer learning*. Right now, the tasks are meant to be solved from scratch. Later, it will be more interesting to consider sequences of tasks, so that the algorithm is trained on one task after the other. Here, we will create sequences of increasingly difficult tasks, which are meant to be solved in order.
- *Real-world operation*. Eventually, we would like to integrate the Gym API with robotic hardware, validating reinforcement learning algorithms in the real world.

Actor-Critic Algorithm in Reinforcement Learning

Last Updated : 26 Feb, 2025

Actor-Critic Algorithm is a type of reinforcement learning algorithm that combines aspects of both **policy-based methods (Actor)** and **value-based methods (Critic)**. This hybrid approach is designed to address the limitations of each method when used individually.

In the actor-critic framework, an agent (the "actor") learns a policy to make decisions, and a value function (the "Critic") evaluates the actions taken by the Actor.

Simultaneously, the critic evaluates these actions by estimating their value or quality. This dual role allows the method to strike a balance between exploration and exploitation, leveraging the strengths of both policy and value functions.

Roles of Actor and Critic

- **Actor:** The actor makes decisions by selecting actions based on the current policy. Its responsibility lies in exploring the action space to maximize expected cumulative rewards. By continuously refining the policy, the actor adapts to the dynamic nature of the environment.
- **Critic:** The critic evaluates the actions taken by the actor. It estimates the value or quality of these actions by providing feedback on their performance. The critic's role is pivotal in guiding the actor towards actions that lead to higher expected returns, contributing to the overall improvement of the learning process.

Key Terms in Actor Critic Algorithm

There are two key terms:

- **Policy (Actor) :**

- The actor seeks to maximize the expected return by optimizing this policy.
- The policy is modeled by the actor network, and its parameters are denoted by θ
- **Value Function (Critic) :**
 - The value function, denoted as $V(s)$, estimates the expected cumulative reward starting from state s .
 - The value function is modeled by the critic network, and its parameters are denoted by w .

How Actor-Critic algorithm works?

Actor Critic Algorithm Objective Function

- The objective function for the Actor-Critic algorithm is a combination of the policy gradient (for the actor) and the value function (for the critic).
- The overall objective function is typically expressed as the sum of two components:

Policy Gradient (Actor)

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=0}^N \nabla_{\theta} \log \pi_{\theta}(a_i | s_i) \cdot A(s_i, a_i)$$

Here,

- $J(\theta)$ represents the expected return under the policy parameterized by θ
- $\pi_{\theta}(a | s)$ is the policy function
- N is the number of sampled experiences.
- $A(s, a)$ is the advantage function representing the advantage of taking action a in state s .
- i represents the index of the sample

Value Function Update (Critic)

$$\nabla_w J(w) \approx \frac{1}{N} \sum_{i=1}^N \nabla_w (V_w(s_i) - Q_w(s_i, a_i))^2$$

Here,

- $\nabla_w J(w)$ is the gradient of the loss function with respect to the critic's

- $V_w(s_i)$ is the critic's estimate of value of state s with parameter w
- $Q_w(s_i, a_i)$ is the critic's estimate of the action-value of taking action a
- i represents the index of the sample

Update Rules

The update rules for the actor and critic involve adjusting their respective parameters using gradient ascent (for the actor) and gradient descent (for the critic).

Actor Update

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t)$$

Here,

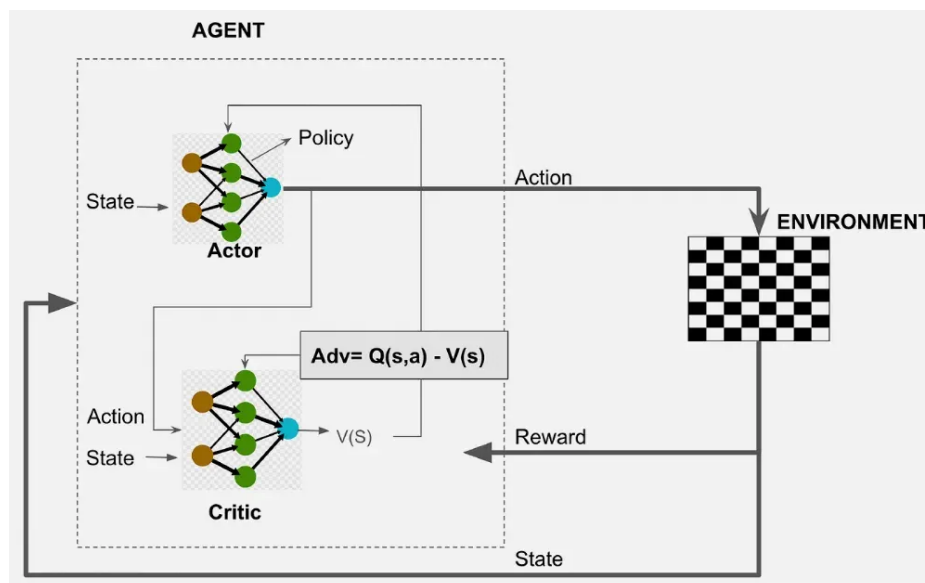
- α : learning rate for the actor
- t is the time step within an episode

Critic Update

$$w_t = w_t - \beta \nabla_w J(w_t)$$

Here,

- w represents the parameters of the critic network
- β is the learning rate for the critic



Advantage Function

state s over the expected value of the state under the current policy.

$$A(s, a) = Q(s, a) - V(s)$$

The advantage function, then, provides a measure of how much better or worse an action is compared to the average action.

These mathematical expressions highlight the essential computations involved in the Actor-Critic method. The actor is updated based on the policy gradient, encouraging actions with higher advantages, while the critic is updated to minimize the difference between the estimated value and the action-value.

Training Agent: Actor-Critic Algorithm

Let's understand how the Actor-Critic algorithm works in practice. Below is an implementation of a simple Actor-Critic algorithm using [TensorFlow](#) and OpenAI Gym to train an agent in the **CartPole environment**.

Step 1: Import Libraries

```
import numpy as np
import tensorflow as tf
import gym
```



Step 2: Creating CartPole Environment

Create the CartPole environment using the **`gym.make()`** function from the Gym library because it provides a standardized and convenient way to interact with various reinforcement learning tasks.

```
# Create the CartPole Environment
env = gym.make('CartPole-v1')
```



Step 3: Defining Actor and Critic Networks

- Actor and the Critic are implemented as neural networks using **TensorFlow's Keras API**.
- Actor network maps the state to a probability distribution over actions.

```
# Define the actor and critic networks
actor = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(env.action_space.n, activation='softmax')
])

critic = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)
])
```

Step 4: Defining Optimizers and Loss Functions

We use Adam optimizer for both networks.

```
# Define optimizer and loss functions
actor_optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
critic_optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```

Step 5: Training Loop

The training loop runs for 1000 episodes, with the agent interacting with the environment, calculating advantages, and updating both the actor and critic.

```
# Main training loop
num_episodes = 1000
gamma = 0.99

for episode in range(num_episodes):
    state = env.reset()
    episode_reward = 0

    with tf.GradientTape(persistent=True) as tape:
        for t in range(1, 10000): # Limit the number of time steps
            # Choose an action using the actor
            action_probs = actor(np.array([state]))
            action = np.random.choice(env.action_space.n,
p=action_probs.numpy()[0])

            # Take the chosen action and observe the next state and reward
            next_state, reward, done, _ = env.step(action)

            # Compute the advantage
            state_value = critic(np.array([state]))[0, 0]
            next_state_value = critic(np.array([next_state]))[0, 0]
            advantage = reward + gamma * next_state_value - state_value

            # Compute actor and critic losses
            actor_loss = -tf.math.log(action_probs[0, action]) * advantage
```

```

        # Update actor and critic
        actor_gradients = tape.gradient(actor_loss,
actor.trainable_variables)
        critic_gradients = tape.gradient(critic_loss,
critic.trainable_variables)
        actor_optimizer.apply_gradients(zip(actor_gradients,
actor.trainable_variables))
        critic_optimizer.apply_gradients(zip(critic_gradients,
critic.trainable_variables))

    if done:
        break

if episode % 10 == 0:
    print(f'Episode {episode}, Reward: {episode_reward}')

env.close()

```

Output:

```

Episode 0, Reward: 13.0
Episode 10, Reward: 19.0
Episode 20, Reward: 25.0
Episode 30, Reward: 42.0
Episode 40, Reward: 12.0
Episode 50, Reward: 20.0
Episode 60, Reward: 16.0
Episode 70, Reward: 20.0
Episode 80, Reward: 11.0
Episode 90, Reward: 20.0
Episode 100, Reward: 16.0
Episode 110, Reward: 23.0
Episode 120, Reward: 26.0
Episode 130, Reward: 25.0
Episode 140, Reward: 15.0
Episode 150, Reward: 20.0
Episode 160, Reward: 23.0
Episode 170, Reward: 16.0
Episode 180, Reward: 24.0

```

Advantages of Actor Critic Algorithm

The Actor-Critic method offer several advantages:

- **Improved Sample Efficiency:** The hybrid nature of Actor-Critic algorithms often leads to improved sample efficiency, requiring fewer interactions with the environment to achieve optimal performance.
- **Faster Convergence:** The method's ability to update both the policy and value function concurrently contributes to faster convergence during training, enabling quicker adaptation to the learning task.
- **Versatility Across Action Spaces:** Actor-Critic architectures can seamlessly handle both discrete and continuous action spaces, offering flexibility in addressing a wide range of RL problems.

Variants of Actor-Critic Algorithms

Several variants of the Actor-Critic algorithm have been developed to address specific challenges or improve performance in certain types of environments:

- **Advantage Actor-Critic (A2C):** A2C modifies the critic's value function to estimate the **advantage function**, which measures how much better or worse an action is compared to the average action. The advantage function is defined as:

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

A2C helps reduce the variance of the policy gradient, leading to better learning performance.

- **Asynchronous Advantage Actor-Critic (A3C):** A3C is an extension of A2C that uses multiple agents (threads) running in parallel to update the policy asynchronously. This allows for more stable and faster learning by reducing correlations between updates.

Comment

More info

Advertise with us

Next Article

How Interaction Design Enhances UI/
UX?

Similar Reads

Dyna Algorithm in Reinforcement Learning

The Dyna algorithm introduces a hybrid approach that leverages both real-world and simulated experiences, enhancing the agent's learning efficiency. This article delves into the key concepts, architecture, and benefits of...

5 min read

Epsilon-Greedy Algorithm in Reinforcement Learning

In Reinforcement Learning, the agent or decision-maker learns what to do—how to map situations to actions—so as to maximize a numerical reward signal. The agent is not explicitly told which actions to take,...

4 min read

HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan and Pieter Abbeel

Department of Electrical Engineering and Computer Science

University of California, Berkeley

{joschu, pcmoritz, levine, jordan, pabbeel}@eecs.berkeley.edu

ABSTRACT

Policy gradient methods are an appealing approach in reinforcement learning because they directly optimize the cumulative reward and can straightforwardly be used with nonlinear function approximators such as neural networks. The two main challenges are the large number of samples typically required, and the difficulty of obtaining stable and steady improvement despite the nonstationarity of the incoming data. We address the first challenge by using value functions to substantially reduce the variance of policy gradient estimates at the cost of some bias, with an exponentially-weighted estimator of the advantage function that is analogous to TD(λ). We address the second challenge by using trust region optimization procedure for both the policy and the value function, which are represented by neural networks.

Our approach yields strong empirical results on highly challenging 3D locomotion tasks, learning running gaits for bipedal and quadrupedal simulated robots, and learning a policy for getting the biped to stand up from starting out lying on the ground. In contrast to a body of prior work that uses hand-crafted policy representations, our neural network policies map directly from raw kinematics to joint torques. Our algorithm is fully model-free, and the amount of simulated experience required for the learning tasks on 3D bipeds corresponds to 1-2 weeks of real time.

1 INTRODUCTION

The typical problem formulation in reinforcement learning is to maximize the expected total reward of a policy. A key source of difficulty is the long time delay between actions and their positive or negative effect on rewards; this issue is called the *credit assignment problem* in the reinforcement learning literature (Minsky, 1961; Sutton & Barto, 1998), and the *distal reward problem* in the behavioral literature (Hull, 1943). Value functions offer an elegant solution to the credit assignment problem—they allow us to estimate the goodness of an action before the delayed reward arrives. Reinforcement learning algorithms make use of value functions in a variety of different ways; this paper considers algorithms that optimize a parameterized policy and use value functions to help estimate how the policy should be improved.

When using a parameterized *stochastic* policy, it is possible to obtain an unbiased estimate of the gradient of the expected total returns (Williams, 1992; Sutton et al., 1999; Baxter & Bartlett, 2000); these noisy gradient estimates can be used in a stochastic gradient ascent algorithm. Unfortunately, the variance of the gradient estimator scales unfavorably with the time horizon, since the effect of an action is confounded with the effects of past and future actions. Another class of policy gradient algorithms, called actor-critic methods, use a value function rather than the empirical returns, obtaining an estimator with lower variance at the cost of introducing bias (Konda & Tsitsiklis, 2003; Hafner & Riedmiller, 2011). But while high variance necessitates using more samples, bias is more pernicious—even with an unlimited number of samples, bias can cause the algorithm to fail to converge, or to converge to a poor solution that is not even a local optimum.

We propose a family of policy gradient estimators that significantly reduce variance while maintaining a tolerable level of bias. We call this estimation scheme, parameterized by $\gamma \in [0, 1]$ and

$\lambda \in [0, 1]$, the generalized advantage estimator (GAE). Related methods have been proposed in the context of online actor-critic methods (Kimura & Kobayashi, 1998; Wawrzyński, 2009). We provide a more general analysis, which is applicable in both the online and batch settings, and discuss an interpretation of our method as an instance of reward shaping (Ng et al., 1999), where the approximate value function is used to shape the reward.

We present experimental results on a number of highly challenging 3D locomotion tasks, where we show that our approach can learn complex gaits using high-dimensional, general purpose neural network function approximators for both the policy and the value function, each with over 10^4 parameters. The policies perform torque-level control of simulated 3D robots with up to 33 state dimensions and 10 actuators.

The contributions of this paper are summarized as follows:

1. We provide justification and intuition for an effective variance reduction scheme for policy gradients, which we call generalized advantage estimation (GAE). While the formula has been proposed in prior work (Kimura & Kobayashi, 1998; Wawrzyński, 2009), our analysis is novel and enables GAE to be applied with a more general set of algorithms, including the batch trust-region algorithm we use for our experiments.
2. We propose the use of a trust region optimization method for the value function, which we find is a robust and efficient way to train neural network value functions with thousands of parameters.
3. By combining (1) and (2) above, we obtain an algorithm that empirically is effective at learning neural network policies for challenging control tasks. The results extend the state of the art in using reinforcement learning for high-dimensional continuous control. Videos are available at <https://sites.google.com/site/gaepapersupp>.

2 PRELIMINARIES

We consider an undiscounted formulation of the policy optimization problem. The initial state s_0 is sampled from distribution ρ_0 . A trajectory $(s_0, a_0, s_1, a_1, \dots)$ is generated by sampling actions according to the policy $a_t \sim \pi(a_t | s_t)$ and sampling the states according to the dynamics $s_{t+1} \sim P(s_{t+1} | s_t, a_t)$, until a terminal (absorbing) state is reached. A reward $r_t = r(s_t, a_t, s_{t+1})$ is received at each timestep. The goal is to maximize the expected total reward $\sum_{t=0}^{\infty} r_t$, which is assumed to be finite for all policies. Note that we are not using a discount as part of the problem specification; it will appear below as an algorithm parameter that adjusts a bias-variance tradeoff. But the discounted problem (maximizing $\sum_{t=0}^{\infty} \gamma^t r_t$) can be handled as an instance of the undiscounted problem in which we absorb the discount factor into the reward function, making it time-dependent.

Policy gradient methods maximize the expected total reward by repeatedly estimating the gradient $g := \nabla_{\theta} \mathbb{E} [\sum_{t=0}^{\infty} r_t]$. There are several different related expressions for the policy gradient, which have the form

$$g = \mathbb{E} \left[\sum_{t=0}^{\infty} \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right], \quad (1)$$

where Ψ_t may be one of the following:

- | | |
|--|---|
| 1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory. | 4. $Q^{\pi}(s_t, a_t)$: state-action value function. |
| 2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t . | 5. $A^{\pi}(s_t, a_t)$: advantage function. |
| 3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula. | 6. $r_t + V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$: TD residual. |

The latter formulas use the definitions

$$V^{\pi}(s_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} r_{t+l} \right] \quad Q^{\pi}(s_t, a_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} r_{t+l} \right] \quad (2)$$

$$A^{\pi}(s_t, a_t) := Q^{\pi}(s_t, a_t) - V^{\pi}(s_t), \quad (\text{Advantage function}). \quad (3)$$

Here, the subscript of $\mathbb{E}$ enumerates the variables being integrated over, where states and actions are sampled sequentially from the dynamics model $P(s_{t+1} | s_t, a_t)$ and policy $\pi(a_t | s_t)$, respectively. The colon notation $a : b$ refers to the inclusive range $(a, a + 1, \dots, b)$. These formulas are well known and straightforward to obtain; they follow directly from Proposition 1, which will be stated shortly.

The choice $\Psi_t = A^\pi(s_t, a_t)$ yields almost the lowest possible variance, though in practice, the advantage function is not known and must be estimated. This statement can be intuitively justified by the following interpretation of the policy gradient: that a step in the policy gradient direction should increase the probability of better-than-average actions and decrease the probability of worse-than-average actions. The advantage function, by its definition $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$, measures whether or not the action is better or worse than the policy’s default behavior. Hence, we should choose Ψ_t to be the advantage function $A^\pi(s_t, a_t)$, so that the gradient term $\Psi_t \nabla_\theta \log \pi_\theta(a_t | s_t)$ points in the direction of increased $\pi_\theta(a_t | s_t)$ if and only if $A^\pi(s_t, a_t) > 0$. See Greensmith et al. (2004) for a more rigorous analysis of the variance of policy gradient estimators and the effect of using a baseline.

We will introduce a parameter γ that allows us to reduce variance by downweighting rewards corresponding to delayed effects, at the cost of introducing bias. This parameter corresponds to the discount factor used in discounted formulations of MDPs, but we treat it as a variance reduction parameter in an undiscounted problem; this technique was analyzed theoretically by Marbach & Tsitsiklis (2003); Kakade (2001b); Thomas (2014). The discounted value functions are given by:

$$V^{\pi, \gamma}(s_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} \gamma^l r_{t+l} \right] \quad Q^{\pi, \gamma}(s_t, a_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} \gamma^l r_{t+l} \right] \quad (4)$$

$$A^{\pi, \gamma}(s_t, a_t) := Q^{\pi, \gamma}(s_t, a_t) - V^{\pi, \gamma}(s_t). \quad (5)$$

The discounted approximation to the policy gradient is defined as follows:

$$g^\gamma := \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} \left[\sum_{t=0}^{\infty} A^{\pi, \gamma}(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \quad (6)$$

The following section discusses how to obtain biased (but not too biased) estimators for $A^{\pi, \gamma}$, giving us noisy estimates of the discounted policy gradient in Equation (6).

Before proceeding, we will introduce the notion of a γ -just estimator of the advantage function, which is an estimator that does not introduce bias when we use it in place of $A^{\pi, \gamma}$ (which is not known and must be estimated) in Equation (6) to estimate g^γ .¹ Consider an advantage estimator $\hat{A}_t(s_{0:\infty}, a_{0:\infty})$, which may in general be a function of the entire trajectory.

Definition 1. The estimator $\hat{A}_t$ is γ -just if

$$\mathbb{E}_{s_{0:\infty}, a_{0:\infty}} \left[\hat{A}_t(s_{0:\infty}, a_{0:\infty}) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] = \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} \left[A^{\pi, \gamma}(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \quad (7)$$

It follows immediately that if $\hat{A}_t$ is γ -just for all t , then

$$\mathbb{E}_{s_{0:\infty}, a_{0:\infty}} \left[\sum_{t=0}^{\infty} \hat{A}_t(s_{0:\infty}, a_{0:\infty}) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] = g^\gamma \quad (8)$$

One sufficient condition for $\hat{A}_t$ to be γ -just is that $\hat{A}_t$ decomposes as the difference between two functions Q_t and b_t , where Q_t can depend on any trajectory variables but gives an unbiased estimator of the γ -discounted Q -function, and b_t is an arbitrary function of the states and actions sampled before a_t .

Proposition 1. Suppose that $\hat{A}_t$ can be written in the form $\hat{A}_t(s_{0:\infty}, a_{0:\infty}) = Q_t(s_{t:\infty}, a_{t:\infty}) - b_t(s_{0:t}, a_{0:t-1})$ such that for all (s_t, a_t) , $\mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty} | s_t, a_t} [Q_t(s_{t:\infty}, a_{t:\infty})] = Q^{\pi, \gamma}(s_t, a_t)$. Then $\hat{A}$ is γ -just.

¹Note, that we have already introduced bias by using $A^{\pi, \gamma}$ in place of A^π ; here we are concerned with obtaining an unbiased estimate of g^γ , which is a biased estimate of the policy gradient of the undiscounted MDP.

The proof is provided in Appendix B. It is easy to verify that the following expressions are γ -just advantage estimators for $\hat{A}_t$:

- $\sum_{l=0}^{\infty} \gamma^l r_{t+l}$
- $Q^{\pi, \gamma}(s_t, a_t)$
- $A^{\pi, \gamma}(s_t, a_t)$
- $r_t + \gamma V^{\pi, \gamma}(s_{t+1}) - V^{\pi, \gamma}(s_t)$.

3 ADVANTAGE FUNCTION ESTIMATION

This section will be concerned with producing an accurate estimate $\hat{A}_t$ of the discounted advantage function $A^{\pi, \gamma}(s_t, a_t)$, which will then be used to construct a policy gradient estimator of the following form:

$$\hat{g} = \frac{1}{N} \sum_{n=1}^N \sum_{t=0}^{\infty} \hat{A}_t^n \nabla_{\theta} \log \pi_{\theta}(a_t^n | s_t^n) \quad (9)$$

where n indexes over a batch of episodes.

Let V be an approximate value function. Define $\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$, i.e., the TD residual of V with discount γ (Sutton & Barto, 1998). Note that δ_t^V can be considered as an estimate of the advantage of the action a_t . In fact, if we have the correct value function $V = V^{\pi, \gamma}$, then it is a γ -just advantage estimator, and in fact, an unbiased estimator of $A^{\pi, \gamma}$:

$$\begin{aligned} \mathbb{E}_{s_{t+1}} [\delta_t^{V^{\pi, \gamma}}] &= \mathbb{E}_{s_{t+1}} [r_t + \gamma V^{\pi, \gamma}(s_{t+1}) - V^{\pi, \gamma}(s_t)] \\ &= \mathbb{E}_{s_{t+1}} [Q^{\pi, \gamma}(s_t, a_t) - V^{\pi, \gamma}(s_t)] = A^{\pi, \gamma}(s_t, a_t). \end{aligned} \quad (10)$$

However, this estimator is only γ -just for $V = V^{\pi, \gamma}$, otherwise it will yield biased policy gradient estimates.

Next, let us consider taking the sum of k of these δ terms, which we will denote by $\hat{A}_t^{(k)}$

$$\hat{A}_t^{(1)} := \delta_t^V = -V(s_t) + r_t + \gamma V(s_{t+1}) \quad (11)$$

$$\hat{A}_t^{(2)} := \delta_t^V + \gamma \delta_{t+1}^V = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \quad (12)$$

$$\hat{A}_t^{(3)} := \delta_t^V + \gamma \delta_{t+1}^V + \gamma^2 \delta_{t+2}^V = -V(s_t) + r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V(s_{t+3}) \quad (13)$$

$$\hat{A}_t^{(k)} := \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{k-1} r_{t+k-1} + \gamma^k V(s_{t+k}) \quad (14)$$

These equations result from a telescoping sum, and we see that $\hat{A}_t^{(k)}$ involves a k -step estimate of the returns, minus a baseline term $-V(s_t)$. Analogously to the case of $\delta_t^V = \hat{A}_t^{(1)}$, we can consider $\hat{A}_t^{(k)}$ to be an estimator of the advantage function, which is only γ -just when $V = V^{\pi, \gamma}$. However, note that the bias generally becomes smaller as $k \rightarrow \infty$, since the term $\gamma^k V(s_{t+k})$ becomes more heavily discounted, and the term $-V(s_t)$ does not affect the bias. Taking $k \rightarrow \infty$, we get

$$\hat{A}_t^{(\infty)} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l}^V = -V(s_t) + \sum_{l=0}^{\infty} \gamma^l r_{t+l}, \quad (15)$$

which is simply the empirical returns minus the value function baseline.

The generalized advantage estimator $\text{GAE}(\gamma, \lambda)$ is defined as the exponentially-weighted average of these k -step estimators:

$$\begin{aligned}
\hat{A}_t^{\text{GAE}(\gamma, \lambda)} &:= (1 - \lambda) \left(\hat{A}_t^{(1)} + \lambda \hat{A}_t^{(2)} + \lambda^2 \hat{A}_t^{(3)} + \dots \right) \\
&= (1 - \lambda) \left(\delta_t^V + \lambda(\delta_t^V + \gamma \delta_{t+1}^V) + \lambda^2(\delta_t^V + \gamma \delta_{t+1}^V + \gamma^2 \delta_{t+2}^V) + \dots \right) \\
&= (1 - \lambda) \left(\delta_t^V (1 + \lambda + \lambda^2 + \dots) + \gamma \delta_{t+1}^V (\lambda + \lambda^2 + \lambda^3 + \dots) \right. \\
&\quad \left. + \gamma^2 \delta_{t+2}^V (\lambda^2 + \lambda^3 + \lambda^4 + \dots) + \dots \right) \\
&= (1 - \lambda) \left(\delta_t^V \left(\frac{1}{1 - \lambda} \right) + \gamma \delta_{t+1}^V \left(\frac{\lambda}{1 - \lambda} \right) + \gamma^2 \delta_{t+2}^V \left(\frac{\lambda^2}{1 - \lambda} \right) + \dots \right) \\
&= \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V
\end{aligned} \tag{16}$$

From Equation (16), we see that the advantage estimator has a remarkably simple formula involving a discounted sum of Bellman residual terms. Section 4 discusses an interpretation of this formula as the returns in an MDP with a modified reward function. The construction we used above is closely analogous to the one used to define $\text{TD}(\lambda)$ (Sutton & Barto, 1998), however $\text{TD}(\lambda)$ is an estimator of the value function, whereas here we are estimating the advantage function.

There are two notable special cases of this formula, obtained by setting $\lambda = 0$ and $\lambda = 1$.

$$\text{GAE}(\gamma, 0) : \hat{A}_t := \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{17}$$

$$\text{GAE}(\gamma, 1) : \hat{A}_t := \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t) \tag{18}$$

$\text{GAE}(\gamma, 1)$ is γ -just regardless of the accuracy of V , but it has high variance due to the sum of terms. $\text{GAE}(\gamma, 0)$ is γ -just for $V = V^{\pi, \gamma}$ and otherwise induces bias, but it typically has much lower variance. The generalized advantage estimator for $0 < \lambda < 1$ makes a compromise between bias and variance, controlled by parameter λ .

We’ve described an advantage estimator with two separate parameters γ and λ , both of which contribute to the bias-variance tradeoff when using an approximate value function. However, they serve different purposes and work best with different ranges of values. γ most importantly determines the scale of the value function $V^{\pi, \gamma}$, which does not depend on λ . Taking $\gamma < 1$ introduces bias into the policy gradient estimate, regardless of the value function’s accuracy. On the other hand, $\lambda < 1$ introduces bias only when the value function is inaccurate. Empirically, we find that the best value of λ is much lower than the best value of γ , likely because λ introduces far less bias than γ for a reasonably accurate value function.

Using the generalized advantage estimator, we can construct a biased estimator of g^γ , the discounted policy gradient from Equation (6):

$$g^\gamma \approx \mathbb{E} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t^{\text{GAE}(\gamma, \lambda)} \right] = \mathbb{E} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \right], \tag{19}$$

where equality holds when $\lambda = 1$.

4 INTERPRETATION AS REWARD SHAPING

In this section, we discuss how one can interpret λ as an extra discount factor applied after performing a reward shaping transformation on the MDP. We also introduce the notion of a response function to help understand the bias introduced by γ and λ .

Reward shaping (Ng et al., 1999) refers to the following transformation of the reward function of an MDP: let $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ be an arbitrary scalar-valued function on state space, and define the transformed reward function $\tilde{r}$ by

$$\tilde{r}(s, a, s') = r(s, a, s') + \gamma \Phi(s') - \Phi(s), \tag{20}$$

which in turn defines a transformed MDP. This transformation leaves the discounted advantage function $A^{\pi,\gamma}$ unchanged for any policy π . To see this, consider the discounted sum of rewards of a trajectory starting with state s_t :

$$\sum_{l=0}^{\infty} \gamma^l \tilde{r}(s_{t+l}, a_t, s_{t+l+1}) = \sum_{l=0}^{\infty} \gamma^l r(s_{t+l}, a_{t+l}, s_{t+l+1}) - \Phi(s_t). \quad (21)$$

Letting $\tilde{Q}^{\pi,\gamma}$, $\tilde{V}^{\pi,\gamma}$, $\tilde{A}^{\pi,\gamma}$ be the value and advantage functions of the transformed MDP, one obtains from the definitions of these quantities that

$$\tilde{Q}^{\pi,\gamma}(s, a) = Q^{\pi,\gamma}(s, a) - \Phi(s) \quad (22)$$

$$\tilde{V}^{\pi,\gamma}(s, a) = V^{\pi,\gamma}(s) - \Phi(s) \quad (23)$$

$$\tilde{A}^{\pi,\gamma}(s, a) = (Q^{\pi,\gamma}(s, a) - \Phi(s)) - (V^{\pi,\gamma}(s) - \Phi(s)) = A^{\pi,\gamma}(s, a). \quad (24)$$

Note that if Φ happens to be the state-value function $V^{\pi,\gamma}$ from the original MDP, then the transformed MDP has the interesting property that $\tilde{V}^{\pi,\gamma}(s)$ is zero at every state.

Note that (Ng et al., 1999) showed that the reward shaping transformation leaves the policy gradient and optimal policy unchanged when our objective is to maximize the discounted sum of rewards $\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, s_{t+1})$. In contrast, this paper is concerned with maximizing the undiscounted sum of rewards, where the discount γ is used as a variance-reduction parameter.

Having reviewed the idea of reward shaping, let us consider how we could use it to get a policy gradient estimate. The most natural approach is to construct policy gradient estimators that use discounted sums of shaped rewards $\tilde{r}$. However, Equation (21) shows that we obtain the discounted sum of the original MDP’s rewards r minus a baseline term. Next, let’s consider using a “steeper” discount $\gamma\lambda$, where $0 \leq \lambda \leq 1$. It’s easy to see that the shaped reward $\tilde{r}$ equals the Bellman residual term δ^V , introduced in Section 3, where we set $\Phi = V$. Letting $\Phi = V$, we see that

$$\sum_{l=0}^{\infty} (\gamma\lambda)^l \tilde{r}(s_{t+l}, a_t, s_{t+l+1}) = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V = \hat{A}_t^{\text{GAE}(\gamma,\lambda)}. \quad (25)$$

Hence, by considering the $\gamma\lambda$ -discounted sum of shaped rewards, we exactly obtain the generalized advantage estimators from Section 3. As shown previously, $\lambda = 1$ gives an unbiased estimate of g^γ , whereas $\lambda < 1$ gives a biased estimate.

To further analyze the effect of this shaping transformation and parameters γ and λ , it will be useful to introduce the notion of a response function χ , which we define as follows:

$$\chi(l; s_t, a_t) = \mathbb{E}[r_{t+l} | s_t, a_t] - \mathbb{E}[r_{t+l} | s_t]. \quad (26)$$

Note that $A^{\pi,\gamma}(s, a) = \sum_{l=0}^{\infty} \gamma^l \chi(l; s, a)$, hence the response function decomposes the advantage function across timesteps. The response function lets us quantify the temporal credit assignment problem: long range dependencies between actions and rewards correspond to nonzero values of the response function for $l \gg 0$.

Next, let us revisit the discount factor γ and the approximation we are making by using $A^{\pi,\gamma}$ rather than $A^{\pi,1}$. The discounted policy gradient estimator from Equation (6) has a sum of terms of the form

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi,\gamma}(s_t, a_t) = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{l=0}^{\infty} \gamma^l \chi(l; s_t, a_t). \quad (27)$$

Using a discount $\gamma < 1$ corresponds to dropping the terms with $l \gg 1/(1 - \gamma)$. Thus, the error introduced by this approximation will be small if χ rapidly decays as l increases, i.e., if the effect of an action on rewards is “forgotten” after $\approx 1/(1 - \gamma)$ timesteps.

If the reward function $\tilde{r}$ were obtained using $\Phi = V^{\pi,\gamma}$, we would have $\mathbb{E}[\tilde{r}_{t+l} | s_t, a_t] = \mathbb{E}[\tilde{r}_{t+l} | s_t] = 0$ for $l > 0$, i.e., the response function would only be nonzero at $l = 0$. Therefore, this shaping transformation would turn temporally extended response into an immediate response. Given that $V^{\pi,\gamma}$ completely reduces the temporal spread of the response function, we can hope that a good approximation $V \approx V^{\pi,\gamma}$ partially reduces it. This observation suggests an interpretation of Equation (16): reshape the rewards using V to shrink the temporal extent of the response function, and then introduce a “steeper” discount $\gamma\lambda$ to cut off the noise arising from long delays, i.e., ignore terms $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \delta_{t+l}^V$ where $l \gg 1/(1 - \gamma\lambda)$.

5 VALUE FUNCTION ESTIMATION

A variety of different methods can be used to estimate the value function (see, e.g., Bertsekas (2012)). When using a nonlinear function approximator to represent the value function, the simplest approach is to solve a nonlinear regression problem:

$$\underset{\phi}{\text{minimize}} \sum_{n=1}^N \|V_{\phi}(s_n) - \hat{V}_n\|^2, \quad (28)$$

where $\hat{V}_t = \sum_{l=0}^{\infty} \gamma^l r_{t+l}$ is the discounted sum of rewards, and n indexes over all timesteps in a batch of trajectories. This is sometimes called the Monte Carlo or TD(1) approach for estimating the value function (Sutton & Barto, 1998).²

For the experiments in this work, we used a trust region method to optimize the value function in each iteration of a batch optimization procedure. The trust region helps us to avoid overfitting to the most recent batch of data. To formulate the trust region problem, we first compute $\sigma^2 = \frac{1}{N} \sum_{n=1}^N \|V_{\phi_{\text{old}}}(s_n) - \hat{V}_n\|^2$, where ϕ_{old} is the parameter vector before optimization. Then we solve the following constrained optimization problem:

$$\begin{aligned} &\underset{\phi}{\text{minimize}} \quad \sum_{n=1}^N \|V_{\phi}(s_n) - \hat{V}_n\|^2 \\ &\text{subject to} \quad \frac{1}{N} \sum_{n=1}^N \frac{\|V_{\phi}(s_n) - V_{\phi_{\text{old}}}(s_n)\|^2}{2\sigma^2} \leq \epsilon. \end{aligned} \quad (29)$$

This constraint is equivalent to constraining the average KL divergence between the previous value function and the new value function to be smaller than ϵ , where the value function is taken to parameterize a conditional Gaussian distribution with mean $V_{\phi}(s)$ and variance σ^2 .

We compute an approximate solution to the trust region problem using the conjugate gradient algorithm (Wright & Nocedal, 1999). Specifically, we are solving the quadratic program

$$\begin{aligned} &\underset{\phi}{\text{minimize}} \quad g^T(\phi - \phi_{\text{old}}) \\ &\text{subject to} \quad \frac{1}{N} \sum_{n=1}^N (\phi - \phi_{\text{old}})^T H(\phi - \phi_{\text{old}}) \leq \epsilon. \end{aligned} \quad (30)$$

where g is the gradient of the objective, and $H = \frac{1}{N} \sum_n j_n j_n^T$, where $j_n = \nabla_{\phi} V_{\phi}(s_n)$. Note that H is the ‘‘Gauss-Newton’’ approximation of the Hessian of the objective, and it is (up to a σ^2 factor) the Fisher information matrix when interpreting the value function as a conditional probability distribution. Using matrix-vector products $v \rightarrow H v$ to implement the conjugate gradient algorithm, we compute a step direction $s \approx -H^{-1}g$. Then we rescale $s \rightarrow \alpha s$ such that $\frac{1}{2}(\alpha s)^T H(\alpha s) = \epsilon$ and take $\phi = \phi_{\text{old}} + \alpha s$. This procedure is analogous to the procedure we use for updating the policy, which is described further in Section 6 and based on Schulman et al. (2015).

6 EXPERIMENTS

We designed a set of experiments to investigate the following questions:

1. What is the empirical effect of varying $\lambda \in [0, 1]$ and $\gamma \in [0, 1]$ when optimizing episodic total reward using generalized advantage estimation?
2. Can generalized advantage estimation, along with trust region algorithms for policy and value function optimization, be used to optimize large neural network policies for challenging control problems?

²Another natural choice is to compute target values with an estimator based on the TD(λ) backup (Bertsekas, 2012; Sutton & Barto, 1998), mirroring the expression we use for policy gradient estimation: $\hat{V}_t^{\lambda} = V_{\phi_{\text{old}}}(s_t) + \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$. While we experimented with this choice, we did not notice a difference in performance from the $\lambda = 1$ estimator in Equation (28).

6.1 POLICY OPTIMIZATION ALGORITHM

While generalized advantage estimation can be used along with a variety of different policy gradient methods, for these experiments, we performed the policy updates using trust region policy optimization (TRPO) (Schulman et al., 2015). TRPO updates the policy by approximately solving the following constrained optimization problem each iteration:

$$\begin{aligned}
& \underset{\theta}{\text{minimize}} \quad L_{\theta_{old}}(\theta) \\
& \text{subject to} \quad \overline{D}_{KL}^{\theta_{old}}(\pi_{\theta_{old}}, \pi_{\theta}) \leq \epsilon \\
& \text{where} \quad L_{\theta_{old}}(\theta) = \frac{1}{N} \sum_{n=1}^N \frac{\pi_{\theta}(a_n | s_n)}{\pi_{\theta_{old}}(a_n | s_n)} \hat{A}_n \\
& \quad \overline{D}_{KL}^{\theta_{old}}(\pi_{\theta_{old}}, \pi_{\theta}) = \frac{1}{N} \sum_{n=1}^N D_{KL}(\pi_{\theta_{old}}(\cdot | s_n) \parallel \pi_{\theta}(\cdot | s_n))
\end{aligned} \tag{31}$$

As described in (Schulman et al., 2015), we approximately solve this problem by linearizing the objective and quadraticizing the constraint, which yields a step in the direction $\theta - \theta_{old} \propto -F^{-1}g$, where F is the average Fisher information matrix, and g is a policy gradient estimate. This policy update yields the same step direction as the natural policy gradient (Kakade, 2001a) and natural actor-critic (Peters & Schaal, 2008), however it uses a different stepsize determination scheme and numerical procedure for computing the step.

Since prior work (Schulman et al., 2015) compared TRPO to a variety of different policy optimization algorithms, we will not repeat these comparisons; rather, we will focus on varying the γ, λ parameters of policy gradient estimator while keeping the underlying algorithm fixed.

For completeness, the whole algorithm for iteratively updating policy and value function is given below:

```

Initialize policy parameter  $\theta_0$  and value function parameter  $\phi_0$ .
for  $i = 0, 1, 2, \dots$  do
    Simulate current policy  $\pi_{\theta_i}$  until  $N$  timesteps are obtained.
    Compute  $\delta_t^V$  at all timesteps  $t \in \{1, 2, \dots, N\}$ , using  $V = V_{\phi_i}$ .
    Compute  $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$  at all timesteps.
    Compute  $\theta_{i+1}$  with TRPO update, Equation (31).
    Compute  $\phi_{i+1}$  with Equation (30).
end for

```

Note that the policy update $\theta_i \rightarrow \theta_{i+1}$ is performed using the value function V_{ϕ_i} for advantage estimation, not $V_{\phi_{i+1}}$. Additional bias would have been introduced if we updated the value function first. To see this, consider the extreme case where we overfit the value function, and the Bellman residual $r_t + \gamma V(s_{t+1}) - V(s_t)$ becomes zero at all timesteps—the policy gradient estimate would be zero.

6.2 EXPERIMENTAL SETUP

We evaluated our approach on the classic cart-pole balancing problem, as well as several challenging 3D locomotion tasks: (1) bipedal locomotion; (2) quadrupedal locomotion; (3) dynamically standing up, for the biped, which starts off laying on its back. The models are shown in Figure 1.

6.2.1 ARCHITECTURE

We used the same neural network architecture for all of the 3D robot tasks, which was a feedforward network with three hidden layers, with 100, 50 and 25 tanh units respectively. The same architecture was used for the policy and value function. The final output layer had linear activation. The value function estimator used the same architecture, but with only one scalar output. For the simpler cart-pole task, we used a linear policy, and a neural network with one 20-unit hidden layer as the value function.

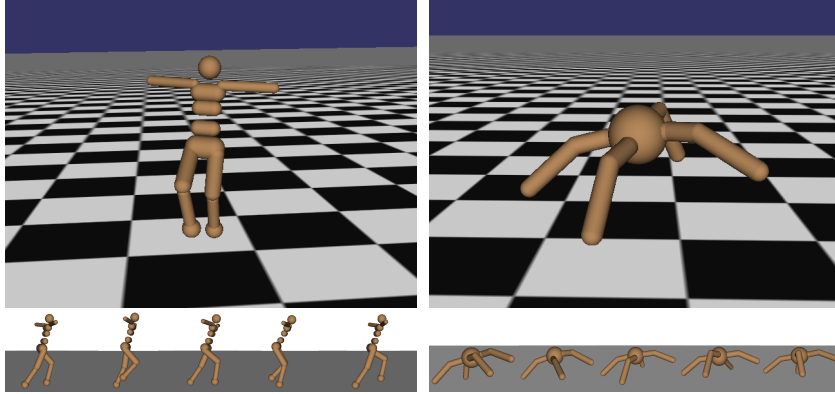


Figure 1: Top figures: robot models used for 3D locomotion. Bottom figures: a sequence of frames from the learned gaits. Videos are available at <https://sites.google.com/site/gaepapersupp>.

6.2.2 TASK DETAILS

For the cart-pole balancing task, we collected 20 trajectories per batch, with a maximum length of 1000 timesteps, using the physical parameters from Barto et al. (1983).

The simulated robot tasks were simulated using the MuJoCo physics engine (Todorov et al., 2012). The humanoid model has 33 state dimensions and 10 actuated degrees of freedom, while the quadruped model has 29 state dimensions and 8 actuated degrees of freedom. The initial state for these tasks consisted of a uniform distribution centered on a reference configuration. We used 50000 timesteps per batch for bipedal locomotion, and 200000 timesteps per batch for quadrupedal locomotion and bipedal standing. Each episode was terminated after 2000 timesteps if the robot had not reached a terminal state beforehand. The timestep was 0.01 seconds.

The reward functions are provided in the table below.

Task	Reward
3D biped locomotion	$v_{\text{fwd}} - 10^{-5}\ u\ ^2 - 10^{-5}\ f_{\text{impact}}\ ^2 + 0.2$
Quadruped locomotion	$v_{\text{fwd}} - 10^{-6}\ u\ ^2 - 10^{-3}\ f_{\text{impact}}\ ^2 + 0.05$
Biped getting up	$-(h_{\text{head}} - 1.5)^2 - 10^{-5}\ u\ ^2$

Here, $v_{\text{fwd}} :=$ forward velocity, $u :=$ vector of joint torques, $f_{\text{impact}} :=$ impact forces, $h_{\text{head}} :=$ height of the head.

In the locomotion tasks, the episode is terminated if the center of mass of the actor falls below a predefined height: .8 m for the biped, and .2 m for the quadruped. The constant offset in the reward function encourages longer episodes; otherwise the quadratic reward terms might lead to a policy that ends the episodes as quickly as possible.

6.3 EXPERIMENTAL RESULTS

All results are presented in terms of the cost, which is defined as negative reward and is minimized. Videos of the learned policies are available at <https://sites.google.com/site/gaepapersupp>. In plots, “No VF” means that we used a time-dependent baseline that did not depend on the state, rather than an estimate of the state value function. The time-dependent baseline was computed by averaging the return at each timestep over the trajectories in the batch.

6.3.1 CART-POLE

The results are averaged across 21 experiments with different random seeds. Results are shown in Figure 2, and indicate that the best results are obtained at intermediate values of the parameters: $\gamma \in [0.96, 0.99]$ and $\lambda \in [0.92, 0.99]$.

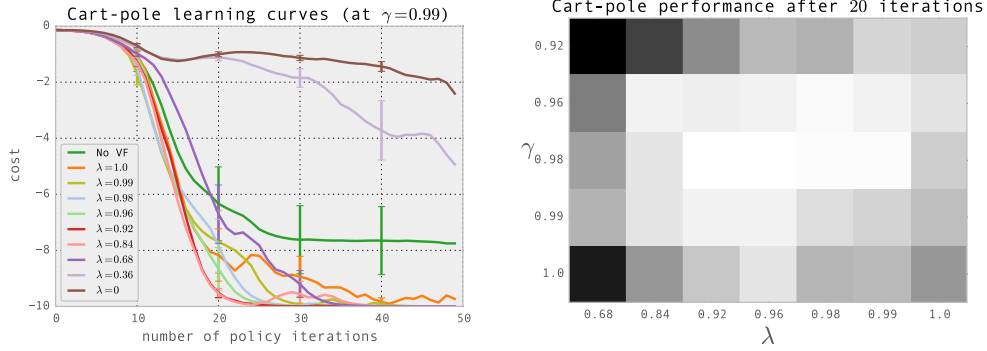


Figure 2: Left: learning curves for cart-pole task, using generalized advantage estimation with varying values of λ at $\gamma = 0.99$. The fastest policy improvement is obtained by intermediate values of λ in the range $[0.92, 0.98]$. Right: performance after 20 iterations of policy optimization, as γ and λ are varied. White means higher reward. The best results are obtained at intermediate values of both.

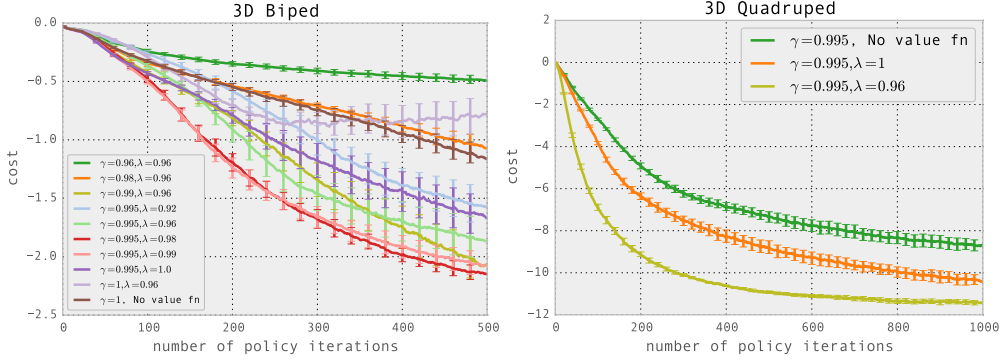


Figure 3: Left: Learning curves for 3D bipedal locomotion, averaged across nine runs of the algorithm. Right: learning curves for 3D quadrupedal locomotion, averaged across five runs.

6.3.2 3D BIPEDAL LOCOMOTION

Each trial took about 2 hours to run on a 16-core machine, where the simulation rollouts were parallelized, as were the function, gradient, and matrix-vector-product evaluations used when optimizing the policy and value function. Here, the results are averaged across 9 trials with different random seeds. The best performance is again obtained using intermediate values of $\gamma \in [0.99, 0.995]$, $\lambda \in [0.96, 0.99]$. The result after 1000 iterations is a fast, smooth, and stable gait that is effectively completely stable. We can compute how much “real time” was used for this learning process: $0.01 \text{ seconds/timestep} \times 50000 \text{ timesteps/batch} \times 1000 \text{ batches/3600} \cdot 24 \text{ seconds/day} = 5.8 \text{ days}$. Hence, it is plausible that this algorithm could be run on a real robot, or multiple real robots learning in parallel, if there were a way to reset the state of the robot and ensure that it doesn’t damage itself.

6.3.3 OTHER 3D ROBOT TASKS

The other two motor behaviors considered are quadrupedal locomotion and getting up off the ground for the 3D biped. Again, we performed 5 trials per experimental condition, with different random seeds (and initializations). The experiments took about 4 hours per trial on a 32-core machine. We performed a more limited comparison on these domains (due to the substantial computational resources required to run these experiments), fixing $\gamma = 0.995$ but varying $\lambda = \{0, 0.96\}$, as well as an experimental condition with no value function. For quadrupedal locomotion, the best results are obtained using a value function with $\lambda = 0.96$ Section 6.3.2. For 3D standing, the value function always helped, but the results are roughly the same for $\lambda = 0.96$ and $\lambda = 1$.

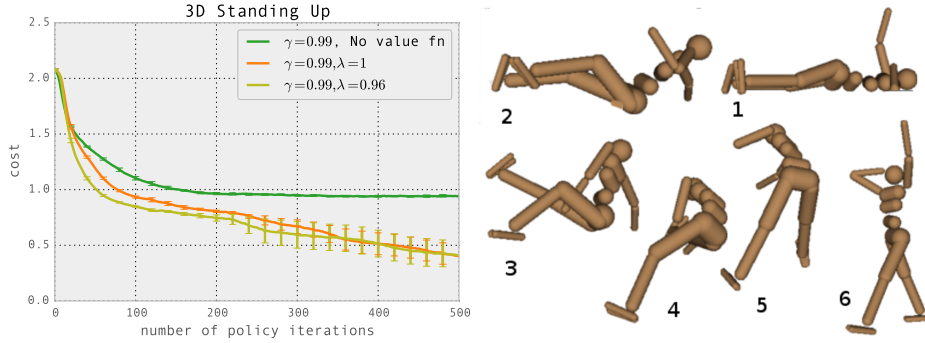


Figure 4: (a) Learning curve from quadrupedal walking, (b) learning curve for 3D standing up, (c) clips from 3D standing up.

7 DISCUSSION

Policy gradient methods provide a way to reduce reinforcement learning to stochastic gradient descent, by providing unbiased gradient estimates. However, so far their success at solving difficult control problems has been limited, largely due to their high sample complexity. We have argued that the key to variance reduction is to obtain good estimates of the advantage function.

We have provided an intuitive but informal analysis of the problem of advantage function estimation, and justified the generalized advantage estimator, which has two parameters γ, λ which adjust the bias-variance tradeoff. We described how to combine this idea with trust region policy optimization and a trust region algorithm that optimizes a value function, both represented by neural networks. Combining these techniques, we are able to learn to solve difficult control tasks that have previously been out of reach for generic reinforcement learning methods.

Our main experimental validation of generalized advantage estimation is in the domain of simulated robotic locomotion. As shown in our experiments, choosing an appropriate intermediate value of λ in the range $[0.9, 0.99]$ usually results in the best performance. A possible topic for future work is how to adjust the estimator parameters γ, λ in an adaptive or automatic way.

One question that merits future investigation is the relationship between value function estimation error and policy gradient estimation error. If this relationship were known, we could choose an error metric for value function fitting that is well-matched to the quantity of interest, which is typically the accuracy of the policy gradient estimation. Some candidates for such an error metric might include the Bellman error or projected Bellman error, as described in Bhatnagar et al. (2009).

Another enticing possibility is to use a shared function approximation architecture for the policy and the value function, while optimizing the policy using generalized advantage estimation. While formulating this problem in a way that is suitable for numerical optimization and provides convergence guarantees remains an open question, such an approach could allow the value function and policy representations to share useful features of the input, resulting in even faster learning.

In concurrent work, researchers have been developing policy gradient methods that involve differentiation with respect to the continuous-valued action (Lillicrap et al., 2015; Heess et al., 2015). While we found empirically that the one-step return ($\lambda = 0$) leads to excessive bias and poor performance, these papers show that such methods can work when tuned appropriately. However, note that those papers consider control problems with substantially lower-dimensional state and action spaces than the ones considered here. A comparison between both classes of approach would be useful for future work.

ACKNOWLEDGEMENTS

We thank Emo Todorov for providing the simulator as well as insightful discussions, and we thank Greg Wayne, Yuval Tassa, Dave Silver, Carlos Florensa Campo, and Greg Brockman for insightful discussions. This research was funded in part by the Office of Naval Research through a Young

Investigator Award and under grant number N00014-11-1-0688, DARPA through a Young Faculty Award, by the Army Research Office through the MAST program.

A FREQUENTLY ASKED QUESTIONS

A.1 WHAT’S THE RELATIONSHIP WITH COMPATIBLE FEATURES?

Compatible features are often mentioned in relation to policy gradient algorithms that make use of a value function, and the idea was proposed in the paper *On Actor-Critic Methods* by Konda & Tsitsiklis (2003). These authors pointed out that due to the limited representation power of the policy, the policy gradient only depends on a certain subspace of the space of advantage functions. This subspace is spanned by the compatible features $\nabla_{\theta_i} \log \pi_{\theta}(a_t | s_t)$, where $i \in \{1, 2, \dots, \dim \theta\}$. This theory of compatible features provides no guidance on how to exploit the temporal structure of the problem to obtain better estimates of the advantage function, making it mostly orthogonal to the ideas in this paper.

The idea of compatible features motivates an elegant method for computing the natural policy gradient (Kakade, 2001a; Peters & Schaal, 2008). Given an empirical estimate of the advantage function $\hat{A}_t$ at each timestep, we can project it onto the subspace of compatible features by solving the following least squares problem:

$$\underset{\mathbf{r}}{\text{minimize}} \sum_t \|\mathbf{r} \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) - \hat{A}_t\|^2. \quad (32)$$

If $\hat{A}$ is γ -just, the least squares solution is the natural policy gradient (Kakade, 2001a). Note that any estimator of the advantage function can be substituted into this formula, including the ones we derive in this paper. For our experiments, we also compute natural policy gradient steps, but we use the more computationally efficient numerical procedure from Schulman et al. (2015), as discussed in Section 6.

A.2 WHY DON’T YOU JUST USE A Q -FUNCTION?

Previous actor critic methods, e.g. in Konda & Tsitsiklis (2003), use a Q -function to obtain potentially low-variance policy gradient estimates. Recent papers, including Heess et al. (2015); Lillicrap et al. (2015), have shown that a neural network Q -function approximator can be used effectively in a policy gradient method. However, there are several advantages to using a state-value function in the manner of this paper. First, the state-value function has a lower-dimensional input and is thus easier to learn than a state-action value function. Second, the method of this paper allows us to smoothly interpolate between the high-bias estimator ($\lambda = 0$) and the low-bias estimator ($\lambda = 1$). On the other hand, using a parameterized Q -function only allows us to use a high-bias estimator. We have found that the bias is prohibitively large when using a one-step estimate of the returns, i.e., the $\lambda = 0$ estimator, $\hat{A}_t = \delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$. We expect that similar difficulty would be encountered when using an advantage estimator involving a parameterized Q -function, $\hat{A}_t = Q(s, a) - V(s)$. There is an interesting space of possible algorithms that would use a parameterized Q -function and attempt to reduce bias, however, an exploration of these possibilities is beyond the scope of this work.

B PROOFS

Proof of Proposition 1: First we can split the expectation into terms involving Q and b ,

$$\begin{aligned} & \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q_t(s_{0:\infty}, a_{0:\infty}) - b_t(s_{0:t}, a_{0:t-1}))] \\ &= \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (Q_t(s_{0:\infty}, a_{0:\infty}))] \\ &\quad - \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (b_t(s_{0:t}, a_{0:t-1}))] \end{aligned} \quad (33)$$

We'll consider the terms with Q and b in turn.

$$\begin{aligned}
& \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_t(s_{0:\infty}, a_{0:\infty})] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t}} [\mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q_t(s_{0:\infty}, a_{0:\infty})]] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} [Q_t(s_{0:\infty}, a_{0:\infty})]] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t-1}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi}(s_t, a_t)]
\end{aligned}$$

Next,

$$\begin{aligned}
& \mathbb{E}_{s_{0:\infty}, a_{0:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b_t(s_{0:t}, a_{0:t-1})] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t-1}} [\mathbb{E}_{s_{t+1:\infty}, a_{t:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b_t(s_{0:t}, a_{0:t-1})]] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t-1}} [\mathbb{E}_{s_{t+1:\infty}, a_{t:\infty}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] b_t(s_{0:t}, a_{0:t-1})] \\
&= \mathbb{E}_{s_{0:t}, a_{0:t-1}} [0 \cdot b_t(s_{0:t}, a_{0:t-1})] \\
&= 0.
\end{aligned}$$

REFERENCES

- Barto, Andrew G, Sutton, Richard S, and Anderson, Charles W. Neuronlike adaptive elements that can solve difficult learning control problems. *Systems, Man and Cybernetics, IEEE Transactions on*, (5):834–846, 1983.
- Baxter, Jonathan and Bartlett, Peter L. Reinforcement learning in POMDPs via direct gradient ascent. In *ICML*, pp. 41–48, 2000.
- Bertsekas, Dimitri P. *Dynamic programming and optimal control*, volume 2. Athena Scientific, 2012.
- Bhatnagar, Shalabh, Precup, Doina, Silver, David, Sutton, Richard S, Maei, Hamid R, and Szepesvári, Csaba. Convergent temporal-difference learning with arbitrary smooth function approximation. In *Advances in Neural Information Processing Systems*, pp. 1204–1212, 2009.
- Greensmith, Evan, Bartlett, Peter L, and Baxter, Jonathan. Variance reduction techniques for gradient estimates in reinforcement learning. *The Journal of Machine Learning Research*, 5:1471–1530, 2004.
- Hafner, Roland and Riedmiller, Martin. Reinforcement learning in feedback control. *Machine learning*, 84 (1-2):137–169, 2011.
- Heess, Nicolas, Wayne, Greg, Silver, David, Lillicrap, Timothy, Tassa, Yuval, and Erez, Tom. Learning continuous control policies by stochastic value gradients. *arXiv preprint arXiv:1510.09142*, 2015.
- Hull, Clark. Principles of behavior. 1943.
- Kakade, Sham. A natural policy gradient. In *NIPS*, volume 14, pp. 1531–1538, 2001a.
- Kakade, Sham. Optimizing average reward using discounted rewards. In *Computational Learning Theory*, pp. 605–615. Springer, 2001b.
- Kimura, Hajime and Kobayashi, Shigenobu. An analysis of actor/critic algorithms using eligibility traces: Reinforcement learning with imperfect value function. In *ICML*, pp. 278–286, 1998.
- Konda, Vijay R and Tsitsiklis, John N. On actor-critic algorithms. *SIAM journal on Control and Optimization*, 42(4):1143–1166, 2003.
- Lillicrap, Timothy P, Hunt, Jonathan J, Pritzel, Alexander, Heess, Nicolas, Erez, Tom, Tassa, Yuval, Silver, David, and Wierstra, Daan. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- Marbach, Peter and Tsitsiklis, John N. Approximate gradient methods in policy-space optimization of markov reward processes. *Discrete Event Dynamic Systems*, 13(1-2):111–148, 2003.
- Minsky, Marvin. Steps toward artificial intelligence. *Proceedings of the IRE*, 49(1):8–30, 1961.
- Ng, Andrew Y, Harada, Daishi, and Russell, Stuart. Policy invariance under reward transformations: Theory and application to reward shaping. In *ICML*, volume 99, pp. 278–287, 1999.
- Peters, Jan and Schaal, Stefan. Natural actor-critic. *Neurocomputing*, 71(7):1180–1190, 2008.

DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills

XUE BIN PENG, University of California, Berkeley
 PIETER ABBEEL, University of California, Berkeley
 SERGEY LEVINE, University of California, Berkeley
 MICHEL VAN DE PANNE, University of British Columbia

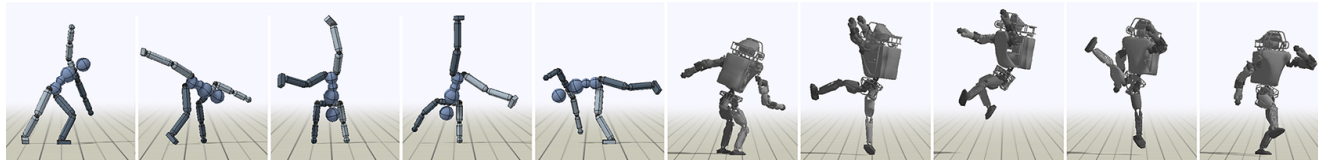


Fig. 1. Highly dynamic skills learned by imitating reference motion capture clips using our method, executed by physically simulated characters. **Left:** Humanoid character performing a cartwheel. **Right:** Simulated Atlas robot performing a spinkick.

A longstanding goal in character animation is to combine data-driven specification of behavior with a system that can execute a similar behavior in a physical simulation, thus enabling realistic responses to perturbations and environmental variation. We show that well-known reinforcement learning (RL) methods can be adapted to learn robust control policies capable of imitating a broad range of example motion clips, while also learning complex recoveries, adapting to changes in morphology, and accomplishing user-specified goals. Our method handles keyframed motions, highly-dynamic actions such as motion-captured flips and spins, and retargeted motions. By combining a motion-imitation objective with a task objective, we can train characters that react intelligently in interactive settings, e.g., by walking in a desired direction or throwing a ball at a user-specified target. This approach thus combines the convenience and motion quality of using motion clips to define the desired style and appearance, with the flexibility and generality afforded by RL methods and physics-based animation. We further explore a number of methods for integrating multiple clips into the learning process to develop multi-skilled agents capable of performing a rich repertoire of diverse skills. We demonstrate results using multiple characters (human, Atlas robot, bipedal dinosaur, dragon) and a large variety of skills, including locomotion, acrobatics, and martial arts.

CCS Concepts: • **Computing methodologies** → **Animation**; *Physical simulation*; *Control methods*; *Reinforcement learning*;

Additional Key Words and Phrases: physics-based character animation, motion control, reinforcement learning

ACM Reference Format:

Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. 2018. DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. *ACM Trans. Graph.* 37, 4, Article 143 (August 2018), 18 pages. <https://doi.org/10.1145/3197517.3201311>

Authors' addresses: Xue Bin Peng, University of California, Berkeley; Pieter Abbeel, University of California, Berkeley; Sergey Levine, University of California, Berkeley; Michiel van de Panne, University of British Columbia.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
 0730-0301/2018/8-ART143 \$15.00
<https://doi.org/10.1145/3197517.3201311>

1 INTRODUCTION

Physics-based simulation of passive phenomena, such as cloth and fluids, has become nearly ubiquitous in industry. However, the adoption of physically simulated characters has been more modest. Modeling the motion of humans and animals remains a challenging problem, and currently, few methods exist that can simulate the diversity of behaviors exhibited in the real world. Among the enduring challenges in this domain are generalization and directability. Methods that rely on manually designed controllers have produced compelling results, but their ability to generalize to new skills and new situations is limited by the availability of human insight. Though humans are adept at performing a wide range of skills themselves, it can be difficult to articulate the internal strategies that underly such proficiency, and more challenging still to encode them into a controller. Directability is another obstacle that has impeded the adoption of simulated characters. Authoring motions for simulated characters remains notoriously difficult, and current interfaces still cannot provide users with an effective means of eliciting the desired behaviours from simulated characters.

Reinforcement learning (RL) provides a promising approach for motion synthesis, whereby an agent learns to perform various skills through trial-and-error, thus reducing the need for human insight. While deep reinforcement learning has been demonstrated to produce a range of complex behaviors in prior work [Duan et al. 2016; Heess et al. 2016; Schulman et al. 2015b], the quality of the generated motions has thus far lagged well behind state-of-the-art kinematic methods or manually designed controllers. In particular, controllers trained with deep RL exhibit severe (and sometimes humorous) artifacts, such as extraneous upper body motion, peculiar gaits, and unrealistic posture [Heess et al. 2017].¹ A natural direction to improve the quality of learned controllers is to incorporate motion capture or hand-authored animation data. In prior work, such systems have typically been designed by layering a physics-based tracking controller on top of a kinematic animation system [Da Silva et al. 2008; Lee et al. 2010a]. This type of approach is challenging because the kinematic animation system must produce reference motions that

¹See, for example, https://youtu.be/hx_bgoTF7bs

are feasible to track, and the resulting physics-based controller is limited in its ability to modify the motion to achieve plausible recoveries or accomplish task goals in ways that deviate substantially from the kinematic motion. Furthermore, such methods tend to be quite complex to implement.

An ideal learning-based animation system should allow an artist or motion capture actor to supply a set of reference motions for style, and then generate goal-directed and physically realistic behavior from those reference motions. In this work, we take a simple approach to this problem by directly rewarding the learned controller for producing motions that resemble reference animation data, while also achieving additional task objectives. We also demonstrate three methods for constructing controllers from multiple clips: training with a multi-clip reward based on a max operator; training a policy to perform multiple diverse skills that can be triggered by the user; and sequencing multiple single-clip policies by using their value functions to estimate the feasibility of transitions.

The central contribution of our paper is a framework for physics-based character animation that combines goal-directed reinforcement learning with data, which may be provided in the form of motion capture clips or keyframed animations. Although our framework consists of individual components that have been known for some time, the particular combination of these components in the context of data-driven and physics-based character animation is novel and, as we demonstrate in our experiments, produces a wide range of skills with motion quality and robustness that substantially exceed prior work. By incorporating motion capture data into a phase-aware policy, our system can produce physics-based behaviors that are nearly indistinguishable in appearance from the reference motion in the absence of perturbations, avoiding many of the artifacts exhibited by previous deep reinforcement learning algorithms, e.g., [Duan et al. 2016]. In the presence of perturbations or modifications, the motions remain natural, and the recovery strategies exhibit a high degree of robustness without the need for human engineering. To the best of our knowledge, we demonstrate some of the most capable physically simulated characters produced by learning-based methods. In our ablation studies, we identify two specific components of our method, reference state initialization and early termination, that are critical for achieving highly dynamic skills. We also demonstrate several methods for integrating multiple clips into a single policy.

2 RELATED WORK

Modeling the skilled movement of articulated figures has a long history in fields ranging from biomechanics to robotics and animation. In recent years, as machine learning algorithms for control have matured, there has also been an increase in interest in these problems from the machine learning community. Here we focus on the most closely related work in animation and RL.

Kinematic Models: Kinematic methods have been an enduring avenue of work in character animation that can be effective when large amounts of data are available. Given a dataset of motion clips, controllers can be built to select the appropriate clip to play in a given situation, e.g., [Agrawal and van de Panne 2016; Lee et al. 2010b; Safonova and Hodgins 2007]. Gaussian processes have been used to

learn latent representations which can then synthesize motions at runtime [Levine et al. 2012; Ye and Liu 2010b]. Extending this line of work, deep learning models, such as autoencoders and phase-functioned networks, have also been applied to develop generative models of human motion in a kinematic setting [Holden et al. 2017, 2016]. Given high quality data, data-driven kinematic methods will often produce higher quality motions than most simulation-based approaches. However, their ability to synthesize behaviors for novel situations can be limited. As tasks and environments become complex, collecting enough motion data to provide sufficient coverage of the possible behaviors quickly becomes untenable. Incorporating physics as a source of prior knowledge about how motions should change in the presence of perturbations and environmental variation provides one solution to this problem, as discussed below.

Physics-based Models: Design of controllers for simulated characters remains a challenging problem, and has often relied on human insight to implement task-specific strategies. Locomotion in particular has been the subject of considerable work, with robust controllers being developed for both human and nonhuman characters, e.g., [Coros et al. 2010; Ye and Liu 2010a; Yin et al. 2007]. Many such controllers are the products of an underlying simplified model and an optimization process, where a compact set of parameters are tuned in order to achieve the desired behaviors [Agrawal et al. 2013; Ha and Liu 2014; Wang et al. 2012]. Dynamics-aware optimization methods based on quadratic programming have also been applied to develop locomotion controllers [Da Silva et al. 2008; Lee et al. 2010a, 2014]. While model-based methods have been shown to be effective for a variety of skills, they tend to struggle with more dynamics motions that require long-term planning, as well as contact-rich motions. Trajectory optimization has been explored for synthesizing physically-plausible motions for a variety of tasks and characters [Mordatch et al. 2012; Wampler et al. 2014]. These methods synthesize motions over an extended time horizon using an offline optimization process, where the equations of motion are enforced as constraints. Recent work has extended such techniques into online model-predictive control methods [Hämäläinen et al. 2015; Tassa et al. 2012], although they remain limited in both motion quality and capacity for long-term planning. The principal advantage of our method over the above approaches is that of generality. We demonstrate that a single model-free framework is capable of a wider range of motion skills (from walks to highly dynamic kicks and flips) and an ability to sequence these; the ability to combine motion imitation and task-related demands; compact and fast-to-compute control policies; and the ability to leverage rich high-dimensional state and environment descriptions.

Reinforcement Learning: Many of the optimization techniques used to develop controllers for simulated characters are based on reinforcement learning. Value iteration methods have been used to develop kinematic controllers to sequence motion clips in the context of a given task [Lee et al. 2010b; Levine et al. 2012]. Similar approaches have been explored for simulated characters [Coros et al. 2009; Peng et al. 2015]. More recently, the introduction of deep neural network models for RL has given rise to simulated agents that can perform a diverse array of challenging tasks [Brockman et al. 2016a; Duan et al. 2016; Liu and Hodgins 2017; Peng et al. 2016; Rajeswaran

et al. 2017; Teh et al. 2017]. Policy gradient methods have emerged as the algorithms of choice for many continuous control problems [Schulman et al. 2015a, 2017; Sutton and Barto 1998]. Although RL algorithms have been capable of synthesizing controllers using minimal task-specific control structures, the resulting behaviors generally appear less natural than their more manually engineered counterparts [Merel et al. 2017; Schulman et al. 2015b]. Part of the challenge stems from the difficulty in specifying reward functions for natural movement, particularly in the absence of biomechanical models and objectives that can be used to achieve natural simulated locomotion [Lee et al. 2014; Wang et al. 2012]. Naïve objectives for torque-actuated locomotion, such as forward progress or maintaining a desired velocity, often produce gaits that exhibit extraneous motion of the limbs, asymmetric gaits, and other objectionable artifacts. To mitigate these artifacts, additional objectives such as effort or impact penalties have been used to discourage these undesirable behaviors. Crafting such objective functions requires a substantial degree of human insight, and often yields only modest improvements. Alternatively, recent RL methods based on the imitation of motion capture, such as GAIL [Ho and Ermon 2016], address the challenge of designing a reward function by using data to induce an objective. While this has been shown to improve the quality of the generated motions, current results still do not compare favorably to standard methods in computer animation [Merel et al. 2017]. The DeepLoco system [Peng et al. 2017a] takes an approach similar to the one we use here, namely adding an imitation term to the reward function, although with significant limitations. It uses fixed initial states and is thus not capable of highly dynamic motions; it is demonstrated only on locomotion tasks defined by foot-placement goals computed by a high-level controller; and it is applied to a single armless biped model. Lastly, the multi-clip demonstration involves a hand-crafted procedure for selecting suitable target clips for turning motions.

Motion Imitation: Imitation of reference motions has a long history in computer animation. An early instantiation of this idea was in bipedal locomotion with planar characters [Sharon and van de Panne 2005; Sok et al. 2007], using controllers tuned through policy search. Model-based methods for tracking reference motions have also been demonstrated for locomotion with 3D humanoid characters [Lee et al. 2010a; Muico et al. 2009; Yin et al. 2007]. Reference motions have also been used to shape the reward function for deep RL to produce more natural locomotion gaits [Peng et al. 2017a,b] and for flapping flight [Won et al. 2017]. In our work, we demonstrate the capability to perform a significantly broader range of difficult motions: highly dynamic spins, kicks, and flips with intermittent ground contact, and we show that reference-state initialization and early termination are critical to their success. We also explore several options for multi-clip integration and skill sequencing.

The work most reminiscent of ours in terms of capabilities is the Sampling-Based Controller (SAMCON) [Liu et al. 2016, 2010]. An impressive array of skills has been reproduced by SAMCON, and to the best of our knowledge, SAMCON has been the only system to demonstrate such a diverse corpus of highly dynamic and acrobatic motions with simulated characters. However, the system is complex, having many components and iterative steps, and requires defining

a low dimensional state representation for the synthesized linear feedback structures. The resulting controllers excel at mimicking the original reference motions, but it is not clear how to extend the method for task objectives, particularly if they involve significant sensory input. A more recent variation introduces deep Q-learning to train a high-level policy that selects from a precomputed collection of SAMCON control fragments [Liu and Hodgins 2017]. This provides flexibility in the order of execution of the control fragments, and is demonstrated to be capable of challenging non-terminating tasks, such as balancing on a bongo-board and walking on a ball. In this work, we propose an alternative framework using deep RL, that is conceptually much simpler than SAMCON, but is nonetheless able to learn highly dynamic and acrobatic skills, including those having task objectives and multiple clips.

3 OVERVIEW

Our system receives as input a character model, a corresponding set of kinematic reference motions, and a task defined by a reward function. It then synthesizes a controller that enables the character to imitate the reference motions, while also satisfying task objectives, such as striking a target or running in a desired direction over irregular terrain. Each reference motion is represented as a sequence of target poses $\{\hat{q}_t\}$. A control policy $\pi(a_t|s_t, g_t)$ maps the state of the character s_t , a task-specific goal g_t to an action a_t , which is then used to compute torques to be applied to each of the character's joints. Each action specifies target angles for proportional-derivative (PD) controllers that then produce the final torques applied at the joints. The reference motions are used to define an imitation reward $r^I(s_t, a_t)$, and the goal defines a task-specific reward $r^G(s_t, a_t, g_t)$. The final result of our system is a policy that enables a simulated character to imitate the behaviours from the reference motions while also fulfilling the specified task objectives. The policies are modeled using neural networks and trained using the proximal policy optimization algorithm [Schulman et al. 2017].

4 BACKGROUND

Our tasks will be structured as standard reinforcement learning problems, where an agent interacts with an environment according to a policy in order to maximize a reward. In the interest of brevity, we will exclude the goal g , from the notation, but the following discussion readily generalizes to include this. A policy $\pi(a|s)$ models the conditional distribution over action $a \in A$ given a state $s \in S$. At each control timestep, the agent observes the current state s_t and samples an action a_t from π . The environment then responds with a new state $s' = s_{t+1}$, sampled from the dynamics $p(s'|s, a)$, and a scalar reward r_t that reflects the desirability of the transition. For a parametric policy $\pi_\theta(a|s)$, the goal of the agent is to learn the optimal parameters θ^* that maximizes its expected return

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^T \gamma^t r_t \right],$$

where $p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} p(s_{t+1}|s_t, a_t) \pi_\theta(a_t|s_t)$ is the distribution over all possible trajectories $\tau = (s_0, a_0, s_1, \dots, a_{T-1}, s_T)$ induced by the policy π_θ , with $p(s_0)$ being the initial state distribution. $\sum_{t=0}^T \gamma^t r_t$ represents the total return of a trajectory, with a horizon

of T steps. T may or may not be infinite, and $\gamma \in [0, 1]$ is a discount factor that can be used to ensure the return is finite. A popular class of algorithms for optimizing a parametric policy is policy gradient methods [Sutton et al. 2001], where the gradient of the expected return $\nabla_{\theta} J(\theta)$ is estimated with trajectories sampled by following the policy. The policy gradient can be estimated according to

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s_t \sim d_{\theta}(s_t), a_t \sim \pi_{\theta}(a_t|s_t)} [\nabla_{\theta} \log(\pi_{\theta}(a_t|s_t)) \mathcal{A}_t],$$

where $d_{\theta}(s_t)$ is the state distribution under the policy π_{θ} . $\mathcal{A}_t$ represents the advantage of taking an action a_t at a given state s_t

$$\mathcal{A}_t = R_t - V(s_t).$$

$R_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$ denotes the return received by a particular trajectory starting from state s_t at time t . $V(s_t)$ is a value function that estimates the average return of starting in s_t and following the policy for all subsequent steps

$$V(s_t) = \mathbb{E}[R_t | \pi_{\theta}, s_t].$$

The policy gradient can therefore be interpreted as increasing the likelihood of actions that lead to higher than expected returns, while decreasing the likelihood of actions that lead to lower than expected returns. A classic policy gradient algorithm for learning a policy using this empirical gradient estimator to perform gradient ascent on $J(\theta)$ is REINFORCE [Williams 1992].

Our policies will be trained using the proximal policy optimization algorithm [Schulman et al. 2017], which has demonstrated state-of-the-art results on a number of challenging control problems. The value function will be trained using multi-step returns with TD(λ). The advantages for the policy gradient will be computed using the generalized advantage estimator GAE(λ) [Schulman et al. 2015b]. A more in-depth review of these methods can be found in the supplementary material.

5 POLICY REPRESENTATION

Given a reference motion clip, represented by a sequence of target poses $\{\hat{q}_t\}$, the goal of the policy is to reproduce the desired motion in a physically simulated environment, while also satisfying additional task objectives. Since a reference motion only provides kinematic information in the form of target poses, the policy is responsible for determining which actions should be applied at each timestep in order to realize the desired trajectory.

5.1 States and Actions

The state s describes the configuration of the character's body, with features consisting of the relative positions of each link with respect to the root (designated to be the pelvis), their rotations expressed in quaternions, and their linear and angular velocities. All features are computed in the character's local coordinate frame, with the root at the origin and the x-axis along the root link's facing direction. Since the target poses from the reference motions vary with time, a phase variable $\phi \in [0, 1]$ is also included among the state features. $\phi = 0$ denotes the start of a motion, and $\phi = 1$ denotes the end. For cyclic motions, ϕ is reset to 0 after the end of each cycle. Policies trained to achieve additional task objectives, such as walking in a particular direction or hitting a target, are also provided with a goal g , which can be treated in a similarly fashion as the state. Specific

goals used in the experiments are discussed in section 9. The action a from the policy specifies target orientations for PD controllers at each joint. The policy is queried at 30Hz, and target orientations for spherical joints are represented in axis-angle form, while targets for revolute joints are represented by scalar rotation angles. Unlike the standard benchmarks, which often operate directly on torques, our use of PD controllers abstracts away low-level control details such as local damping and local feedback. Compared to torques, PD controllers have been shown to improve performance and learning speed for certain motion control tasks [Peng et al. 2017b].

5.2 Network

Each policy π is represented by a neural network that maps a given state s and goal g to a distribution over action $\pi(a|s, g)$. The action distribution is modeled as a Gaussian, with a state dependent mean $\mu(s)$ specified by the network, and a fixed diagonal covariance matrix Σ that is treated as a hyperparameter of the algorithm:

$$\pi(a|s) = \mathcal{N}(\mu(s), \Sigma).$$

The inputs are processed by two fully-connected layers with 1024, and 512 units each, followed by a linear output layer. ReLU activations are used for all hidden units. The value function is modeled by a similar network, with exception of the output layer, which consists of a single linear unit.

For vision-based tasks, discussed in section 9, the inputs are augmented with a heightmap H of the surrounding terrain, sampled on a uniform grid around the character. The policy and value networks are augmented accordingly with convolutional layers to process the heightmap. A schematic illustration of this visuomotor policy network is shown in Figure 2. The heightmap is first processed by a series of convolutional layers, followed by a fully-connected layer. The resulting features are then concatenated with the input state s and goal g , and processed by a similar fully-connected network as the one used for tasks that do not require vision.

5.3 Reward

The reward r_t at each step t consists of two terms that encourage the character to match the reference motion while also satisfying

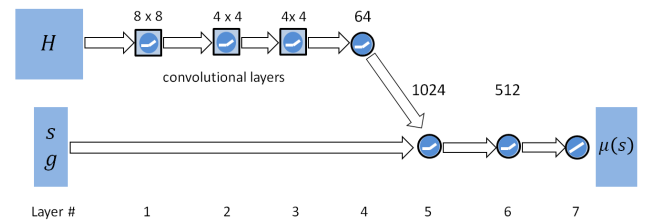


Fig. 2. Schematic illustration of the visuomotor policy network. The heightmap H is processed by 3 convolutional layers with 16 8x8 filters, 32 4x4 filters, and 32 4x4 filters. The feature maps are then processed by 64 fully-connected units. The resulting features are concatenated with the input state s and goal g and processed by two fully-connected layer with 1024 and 512 units. The output $\mu(s)$ is produced by a layer of linear units. ReLU activations are used for all hidden layers. For tasks that do not require a heightmap, the networks consist only of layers 5-7.

additional task objectives:

$$r_t = \omega^I r_t^I + \omega^G r_t^G.$$

Here, r_t^I and r_t^G represent the imitation and task objectives, with ω^I and ω^G being their respective weights. The task objective r_t^G incentivizes the character to fulfill task-specific objectives, the details of which will be discussed in the following section. The imitation objective r_t^I encourages the character to follow a given reference motion $\{\hat{q}_t\}$. It is further decomposed into terms that reward the character for matching certain characteristics of the reference motion, such as joint orientations and velocities, as follows:

$$r_t^I = w^p r_t^p + w^v r_t^v + w^e r_t^e + w^c r_t^c$$

$$w^p = 0.65, w^v = 0.1, w^e = 0.15, w^c = 0.1$$

The pose reward r_t^p encourages the character to match the joint orientations of the reference motion at each step, and is computed as the difference between the joint orientation quaternions of the simulated character and those of the reference motion. In the equation below, q_t^j and $\hat{q}_t^j$ represent the orientations of the j th joint from the simulated character and reference motion respectively, $q_1 \ominus q_2$ denotes the quaternion difference, and $\|q\|$ computes the scalar rotation of a quaternion about its axis in radians:

$$r_t^p = \exp \left[-2 \left(\sum_j \|\hat{q}_t^j \ominus q_t^j\|^2 \right) \right].$$

The velocity reward r_t^v is computed from the difference of local joint velocities, with $\dot{q}_t^j$ being the angular velocity of the j th joint. The target velocity $\hat{\dot{q}}_t^j$ is computed from the data via finite difference.

$$r_t^v = \exp \left[-0.1 \left(\sum_j \|\hat{\dot{q}}_t^j - \dot{q}_t^j\|^2 \right) \right].$$

The end-effector reward r_t^e encourages the character's hands and feet to match the positions from the reference motion. Here, p_t^e denotes the 3D world position in meters of end-effector $e \in [\text{left foot, right foot, left hand, right hand}]$:

$$r_t^e = \exp \left[-40 \left(\sum_e \|\hat{p}_t^e - p_t^e\|^2 \right) \right].$$

Finally, r_t^c penalizes deviations in the character's center-of-mass p_t^c from that of the reference motion $\hat{p}_t^c$:

$$r_t^c = \exp \left[-10 \left(\|\hat{p}_t^c - p_t^c\|^2 \right) \right].$$

6 TRAINING

Our policies are trained with PPO using the clipped surrogate objective [Schulman et al. 2017]. We maintain two networks, one for the policy $\pi_\theta(a|s, g)$ and another for the value function $V_\psi(s, g)$, with parameters θ and ψ respectively. Training proceeds episodically, where at the start of each episode, an initial state s_0 is sampled uniformly from the reference motion (section 6.1), and rollouts are generated by sampling actions from the policy at every step. Each episode is simulated to a fixed time horizon or until a termination condition has been triggered (section 6.2). Once a batch of data

has been collected, minibatches are sampled from the dataset and used to update the policy and value function. The value function is updated using target values computed with TD(λ) [Sutton and Barto 1998]. The policy is updated using gradients computed from the surrogate objective, with advantages $\mathcal{A}_t$ computed using GAE(λ) [Schulman et al. 2015b]. Please refer to the supplementary material for a more detailed summary of the learning algorithm.

One of the persistent challenges in RL is the problem of exploration. Since most formulations assume an unknown MDP, the agent is required to use its interactions with the environment to infer the structure of the MDP and discover high value states that it should endeavor to reach. A number of algorithmic improvements have been proposed to improve exploration, such as using metrics for novelty or information gain [Bellemare et al. 2016; Fu et al. 2017; Houthoofd et al. 2016]. However, less attention has been placed on the structure of the episodes during training and their potential as a mechanism to guide exploration. In the following sections, we consider two design decisions, the initial state distribution and the termination condition, which have often been treated as fixed properties of a given RL problem. We will show that appropriate choices are crucial for allowing our method to learn challenging skills such as highly-dynamic kicks, spins, and flips. With common default choices, such as a fixed initial state and fixed-length episodes, we find that imitation of these difficult motions is often unsuccessful.

6.1 Initial State Distribution

The initial state distribution $p(s_0)$ determines the states in which an agent begins each episode. A common choice for $p(s_0)$ is to always place the agent in a fixed state. However, consider the task of imitating a desired motion. A simple strategy is to initialize the character to the starting state of the motion, and allow it to proceed towards the end of the motion over the course of an episode. With this design, the policy must learn the motion in a sequential manner, by first learning the early phases of the motion, and then incrementally progressing towards the later phases. Before mastering the earlier phases, little progress can be made on the later phases. This can be problematic for motions such as backflips, where learning the landing is a prerequisite for the character to receive a high return from the jump itself. If the policy cannot land successfully, jumping will actually result in *worse* returns. Another disadvantage of a fixed initial state is the resulting exploration challenge. The policy only receives reward retrospectively, once it has visited a state. Therefore, until a high-reward state has been visited, the policy has no way of learning that this state is favorable. Both disadvantages can be mitigated by modifying the initial state distribution.

For many RL tasks, a fixed initial state can be more convenient, since it can be challenging to initialize the agent in other states (e.g., physical robots) or obtain a richer initial state distribution. For motion imitation tasks, however, the reference motion provides a rich and informative state distribution, that can be leveraged to guide the agent during training. At the start of each episode, a state can be sampled from the reference motion, and used to initialize the state of the agent. We will refer to this strategy as *reference state initialization* (RSI). Similar strategies have been previously used for planar bipedal walking [Sharon and van de Panne 2005] and manipulation [Nair et al. 2017; Rajeswaran et al. 2017]. By sampling initial

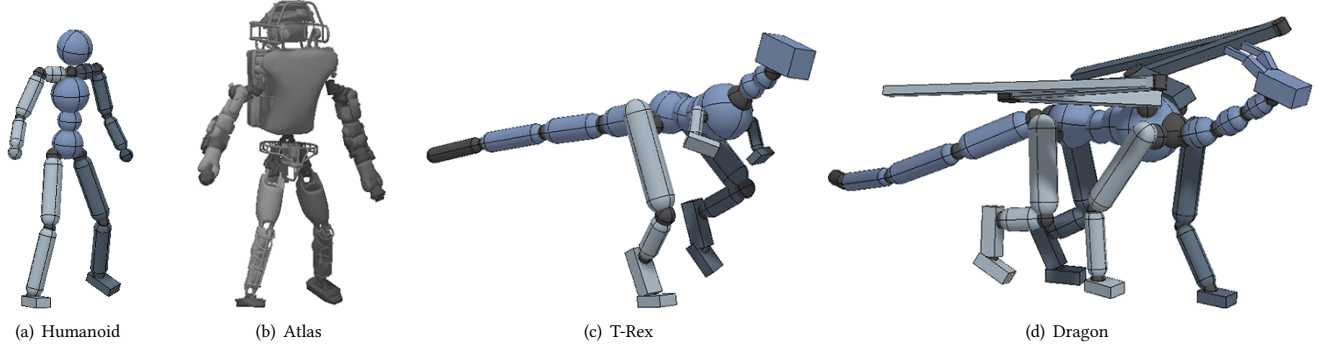


Fig. 3. 3D simulated characters. Our framework is able to train policies for a wide range of character morphologies.

states from the reference motion, the agent encounters desirable states along the motion, even before the policy has acquired the proficiency needed to reach those states. For example, consider the challenge of learning to perform a backflip. With a fixed initial state, in order for the character to discover that performing a full rotation mid-air will result in high returns, it must first learn to perform a carefully coordinated jump. However, for the character to be motivated to perform such a jump, it must be aware that the jump will lead to states that yield higher rewards. Since the motion is highly sensitive to the initial conditions at takeoff, many strategies will result in failure. Thus the agent is unlikely to encounter states from a successful flip, and never discover such high reward states. With RSI, the agent immediately encounters such promising states during the early stages of training. Instead of accessing information from the reference motion only through the reward function, RSI can be interpreted as an additional channel through which the agent can access information from the reference motion in the form of a more informative initial state distribution.

6.2 Early Termination

For cyclic skills, the task can be modeled as an infinite horizon MDP. But during training, each episode is simulated for a finite horizon. An episode terminates either after a fixed period of time, or when certain termination conditions have been triggered. For locomotion, a common condition for early termination (ET) is the detection of a fall, characterized by the character's torso making contact with the ground [Peng et al. 2016] or certain links falling below a height threshold [Heess et al. 2016]. While these strategies are prevalent, they are often mentioned in passing and their impact on performance has not been well evaluated. In this work, we will use a similar termination condition as [Peng et al. 2016], where an episode is terminated whenever certain links, such as the torso or head, makes contact with the ground. Once early termination has been triggered, the character is left with zero reward for the remainder of the episode. This instantiation of early termination provides another means of shaping the reward function to discourage undesirable behaviors. Another advantages of early termination is that it can function as a curating mechanism that biases the data distribution in favor of samples that may be more relevant for a task. In the case of skills such as walking and flips, once the character has fallen, it can be challenging for it to recover and return to its nominal trajectory.

Table 1. Properties of the characters.

Property	Humanoid	Atlas	T-Rex	Dragon
Links	13	12	20	32
Total Mass (kg)	45	169.8	54.5	72.5
Height (m)	1.62	1.82	1.66	1.83
Degrees of Freedom	34	31	55	79
State Features	197	184	262	418
Action Parameters	36	32	64	94

Without early termination, data collected during the early stages of training will be dominated by samples of the character struggling on the ground in vain, and much of the capacity of the network will be devoted to modeling such futile states. This phenomena is analogous to the class imbalance problem encountered by other methodologies such as supervised learning. By terminating the episode whenever such failure states are encountered, this imbalance can be mitigated.

7 MULTI-SKILL INTEGRATION

The discussion thus far has been focused on imitating individual motion clips. But the ability to compose and sequence multiple clips is vital for performing more complex tasks. In this section, we propose several methods for doing this, which are each suited for different applications. First, we need not be restricted to a single reference clip to define a desired motion style, and can instead choose to use a richer and more flexible *multi-clip* reward. Second, we can further provide the user with control over which behavior to trigger, by training a *skill selector* policy that takes in a user-specified one-hot clip-selection input. Third, we can avoid training new policies for every clip combination by instead constructing a *composite policy* out of existing single-clip policies. In this setup, multiple policies are learned independently and, at runtime, their value functions are used to determine which policy should be activated.

Multi-Clip Reward: To utilize multiple reference motion clips during training, we define a composite imitation objective calculated simply as the max over the previously introduced imitation objective applied to each of the k motion clips:

$$r_t^I = \max_{j=1, \dots, k} r_t^j,$$

where r_t^j is the imitation objective with respect to the j th clip. We will show that this simple composite objective is sufficient to integrate multiple clips into the learning process. Unlike [Peng et al. 2017a], which required a manually crafted kinematic planner to select a clip for each walking step, our objective provides the policy with the flexibility to select the most appropriately clip for a given situation, and the ability to switch between clips whenever appropriate, without the need to design a kinematic planner.

Skill Selector: Besides simply providing the policy with multiple clips to use as needed to accomplish a goal, we can also provide the user with control over which clip to use at any given time. In this approach, we can train a policy that simultaneously learns to imitate a set of diverse skills and, once trained, is able to execute arbitrary sequences of skills on demand. The policy is provided with a goal g_t represented by a one-hot vector, where each entry $g_{t,i}$ corresponds to the motion that should be executed. The character's goal then is to perform the motion corresponding to the nonzero entry of g_t . There is no additional task objective r_t^G , and the character is trained only to optimize the imitation objective r_t^I , which is computed based on the currently selected motion $r_t^I = r_t^i$, where $g_{t,i} = 1$ and $g_{t,j} = 0$ for $j \neq i$. During training, a random g_t is sampled at the start of each cycle. The policy is therefore required to learn to transition between all skills within the set of clips.

Composite Policy: The previously described methods both learn a single policy for a collection of clips. But requiring a network to learn multiple skills jointly can be challenging as the number of skills increases, and can result in the policy failing to learn any of the skills adequately. An alternative is to adopt a divide-and-conquer strategy, where separate policies are trained to perform different skills, and then integrated together into a composite policy. Since the value function provides an estimate of a policy's expected performance in a particular state, the value functions can be leveraged to determine the most appropriate skill to execute in a given state. Given a set of policies and their value functions $\{V^i(s), \pi^i(a|s)\}_{i=1}^k$, a composite policy $\Pi(a|s)$ can be constructed using a Boltzmann distribution

$$\Pi(a|s) = \sum_{i=1}^k p^i(s) \pi^i(a|s), \quad p^i(s) = \frac{\exp[V^i(s)/\mathcal{T}]}{\sum_{j=1}^k \exp[V^j(s)/\mathcal{T}]},$$

where $\mathcal{T}$ is a temperature parameter. Policies with larger expected values at a given state will therefore be more likely to be selected. By repeatedly sampling from the composite policy, the character is then able to perform sequences of skills from a library of diverse motions without requiring any additional training. The composite policy resembles the mixture of actor-critics experts model (MACE) proposed by [Peng et al. 2016], although it is even simpler as each sub-policy is trained independently for a specific skill.

8 CHARACTERS

Our characters include a 3D humanoid, an Atlas robot model, a T-Rex, and a dragon. Illustrations of the characters are available in Figure 3, and Table 1 details the properties of each character. All characters are modeled as articulated rigid bodies, with each link attached to its parent link via a 3 degree-of-freedom spherical joint, except for the knees and elbows, which are attached via 1

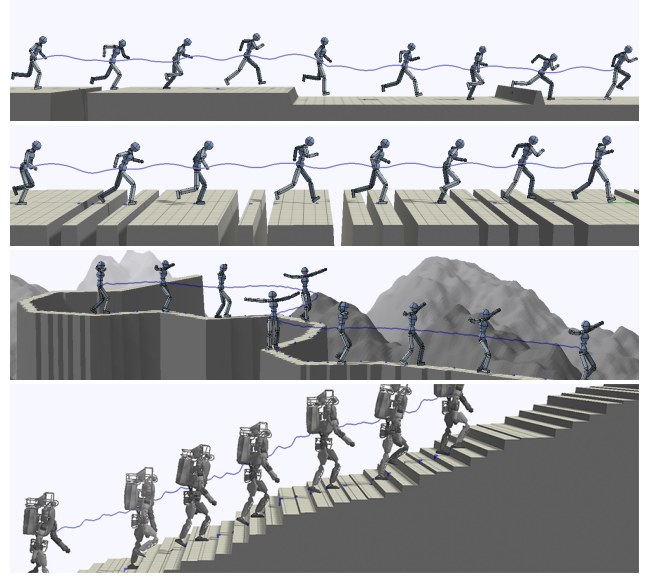


Fig. 4. Characters traversing randomly generated terrains. **Top-to-bottom:** mixed obstacles, dense gaps, winding balance beam, stairs. The blue line traces the trajectory of the character's center-of-mass.

degree-of-freedom revolute joints. PD controllers are positioned at each joint, with manually specified gains that are kept constant across all tasks. Both the humanoid and Atlas share similar body structures, but their morphology (e.g., mass distribution) and actuators (e.g., PD gains and torque limits) differ significantly, with the Atlas being almost four times the mass of the humanoid. The T-Rex and dragon provide examples of learning behaviors for characters from keyframed animation when no mocap data is available, and illustrate that our method can be readily applied to non-bipedal characters. The humanoid character has a 197D state space and a 36D action space. Our most complex character, the dragon, has a 418D state space and 94D action space. Compared to standard continuous control benchmarks for RL [Brockman et al. 2016b], which typically have action spaces varying between 3D to 17D, our characters have significantly higher-dimensional action spaces.

9 TASKS

In addition to imitating a set of motion clips, the policies can be trained to perform a variety of tasks while preserving the style prescribed by the reference motions. The task-specific behaviors are encoded into the task objective r_t^G . We describe the tasks evaluated in our experiments below.

Target Heading: Steerable controllers can be trained by introducing an objective that encourages the character to travel in a target direction d_t^* , represented as a 2D unit vector in the horizontal plane. The reward for this task is given by

$$r_t^G = \exp \left[-2.5 \max(0, v^* - v_t^T d_t^*)^2 \right],$$

where v^* specifies the desired speed along the target direction d_t^* , and v_t represents the center-of-mass velocity of the simulated character. The objective therefore penalizes the character for traveling

Table 2. Performance statistics of imitating various skills. All skills are performed by the humanoid unless stated otherwise. Policies are trained only to imitate a reference motion without additional task objectives. T_{cycle} is the length of the clip. $N_{samples}$ specifies the number of samples collected to train the final policy. NR represents the normalized return of the final policy averaged over 32 episodes, with 0 being the minimum possible return per episode, and 1 the maximum return. For cyclic skills, each episode has a horizon of 20s. For acyclic skills, the horizon is specified by T_{cycle} .

Skill	T_{cycle} (s)	$N_{samples}$ (10^6)	NR
Backflip	1.75	72	0.729
Balance Beam	0.73	96	0.783
Baseball Pitch	2.47	57	0.785
Cartwheel	2.72	51	0.804
Crawl	2.93	68	0.932
Dance A	1.62	67	0.863
Dance B	2.53	79	0.822
Frontflip	1.65	81	0.485
Getup Face-Down	3.28	49	0.885
Getup Face-Up	4.02	66	0.838
Headspin	1.92	112	0.640
Jog	0.80	51	0.951
Jump	1.77	86	0.947
Kick	1.53	50	0.854
Landing	2.83	66	0.590
Punch	2.13	60	0.812
Roll	2.02	81	0.735
Run	0.80	53	0.951
Sideflip	2.44	64	0.805
Spin	4.42	191	0.664
Spinkick	1.28	67	0.748
Vault 1-Handed	1.53	41	0.695
Vault 2-Handed	1.90	87	0.757
Walk	1.26	61	0.985
Atlas: Backflip	1.75	63	0.630
Atlas: Run	0.80	48	0.846
Atlas: Spinkick	1.28	66	0.477
Atlas: Walk	1.26	44	0.988
T-Rex: Walk	2.00	140	0.979
Dragon: Walk	1.50	139	0.990

slower than the desired speed along the target direction, but does not penalize it for exceeding the requested speed. The target direction is provided as the input goal $g_t = d_t^*$ to the policy. During training, the target direction is randomly varied throughout an episode. At runtime, d_t^* can be manually specified to steer the character.

Strike: In this task, the character's goal is to strike a randomly placed spherical target using specific links, such as the feet. The reward is given by

$$r_t^G = \begin{cases} 1, & \text{target has been hit} \\ \exp[-4||p_t^{tar} - p_t^e||^2], & \text{otherwise} \end{cases}$$

p_t^{tar} specifies the location of the target, and p_t^e represents the position of the link used to hit the target. The target is marked as being hit if the center of the link is within 0.2m of the target location. The goal $g_t = (p_t^{tar}, h)$ consists of the target location p_t^{tar}

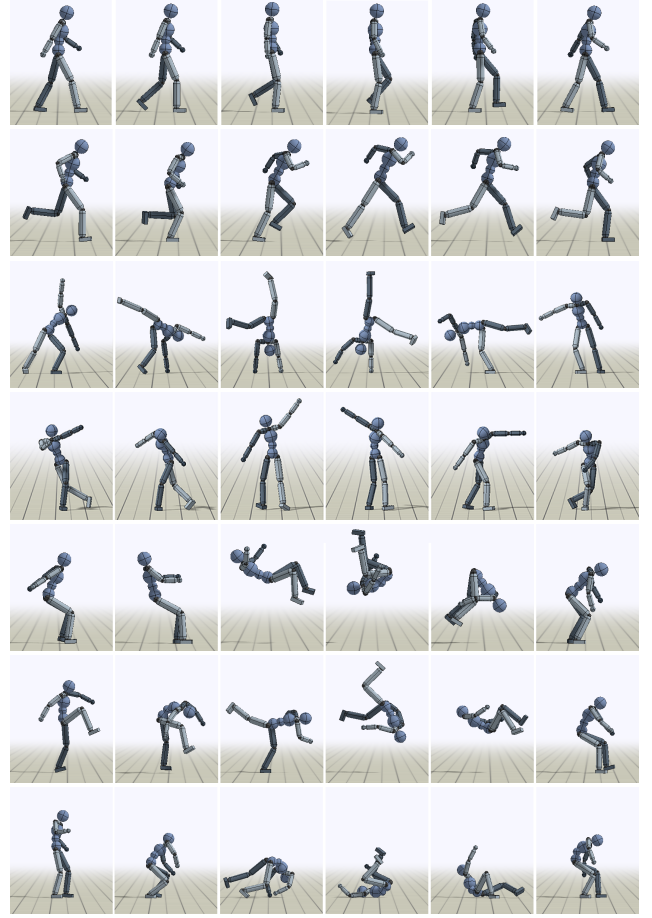


Fig. 5. Snapshots of motions from the trained policies. **Top-to-bottom:** walk, run, cartwheel, dance A, backflip, frontflip, roll.

and a binary variable h that indicates if the target has been hit in a previous timestep. As we are using feedforward networks for all policies, h acts as a memory for the state of the target. The target is randomly placed within a distance of $[0.6, 0.8]m$ from the character, the height is sampled randomly between between $[0.8, 1.25]m$, and the initial direction from the character to the target varies by $2rad$. The target location and h are reset at the start of each cycle. The memory state h can be removed by training a recurrent policy, but our simple solution avoids the complexities of training recurrent networks while still attaining good performance.

Throw: This task is a variant of the strike task, where instead of hitting a target with one of the character's links, the character is tasked with throwing a ball to the target. At the start of an episode, the ball is attached to the character's hand via a spherical joint. The joint is released at a fixed point in time during the episode. The goal g_t and reward r_t^G is the same as the strike task, but the character state s_t is augmented with the position, rotation, linear and angular velocity of the ball. The distance of the target varies between $[2.5, 3.5]m$, with height between $[1, 1.25]m$, and direction between $[0.7, 0.9]rad$.

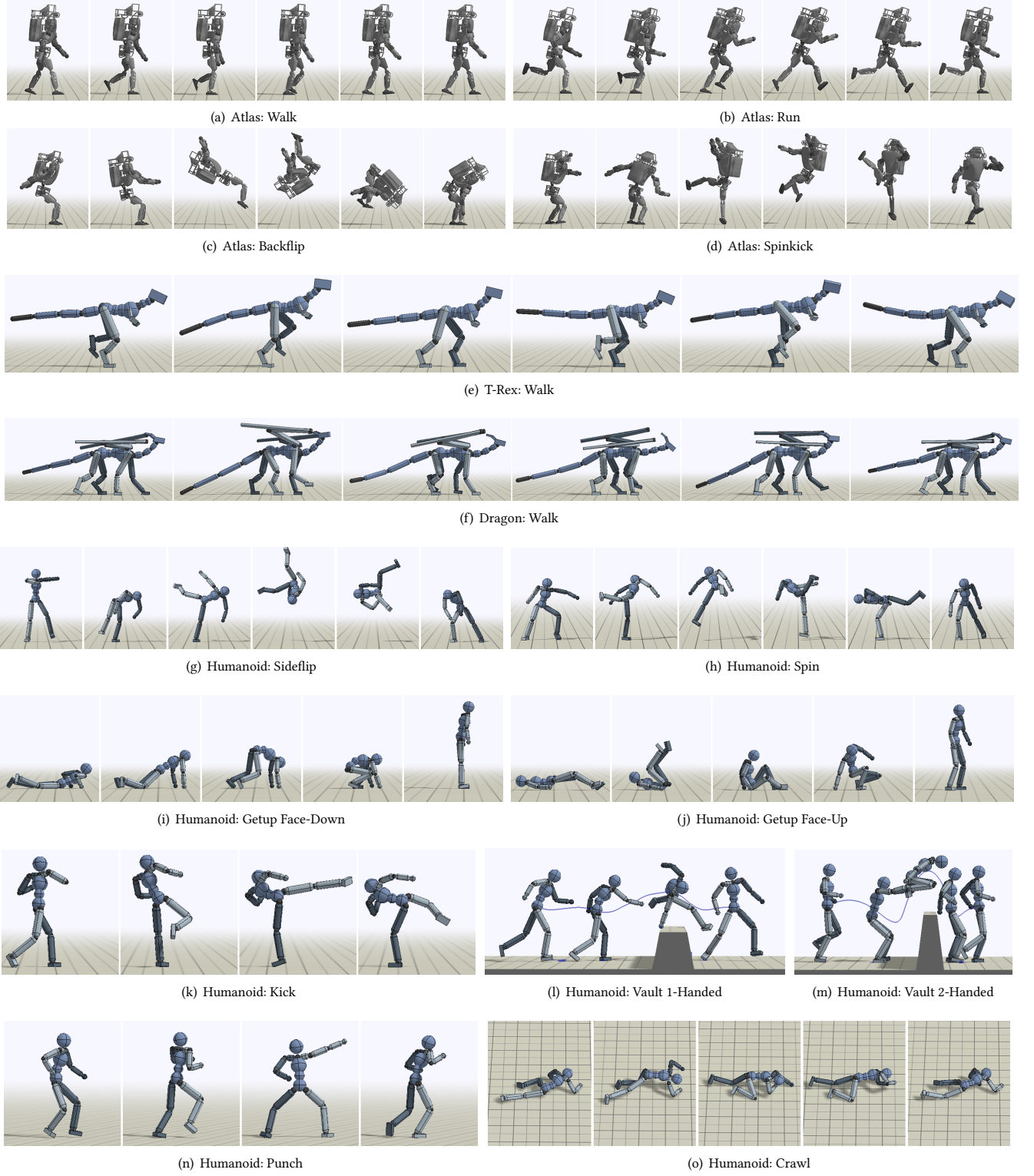


Fig. 6. Simulated characters performing various skills. Our framework is able to train policies for a broad range of characters, skills, and environments.

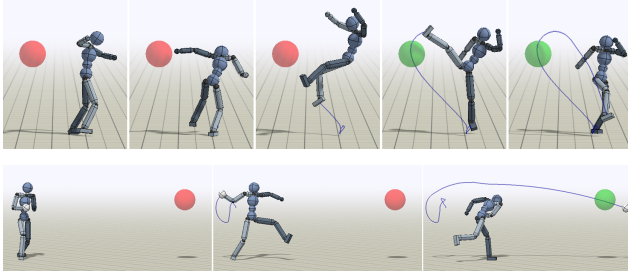


Fig. 7. **Top:** Spinkick policy trained to strike a target with the character’s right foot. **Bottom:** Baseball pitch policy trained to throw a ball to a target.

Terrain Traversal: In this task, the character is trained to traverse obstacle-filled environments. The goal g_t and task-objective r_t^G are similar to those of the target heading task, except the target heading is fixed along the direction of forward progress.

We consider four environments consisting of mixed obstacles, dense gaps, a winding balance beam, and stairs. Figure 4 illustrates examples of the environments. The mixed obstacles environment is composed of gap, step, and wall obstacles similar to those presented in [Peng et al. 2016]. Each gap has a width between $[0.2, 1]m$, the height of each wall varies between $[0.25, 0.4]m$, and each step has a height between $[0.35, -0.35]m$. The obstacles are interleaved with flat stretches of terrain between $[5, 8]m$ in length. The next environment consists of sequences of densely packed gaps, where each sequence consists of 1 to 4 gaps. The gaps are $[0.1, 0.3]m$ in width, with $[0.2, 0.4]m$ of separation between adjacent gaps. Sequences of gaps are separate by $[1, 2]m$ of flat terrain. The winding balance beam environment sports a narrow winding path carved into irregular terrain. The width of the path is approximately $0.4m$. Finally, we constructed a stairs environment, where the character is to climb up irregular steps with height varying between $[0.01, 0.2]m$ and a depth of $0.28m$.

To facilitate faster training, we adopt a progressive learning approach, where the standard fully-connected networks (i.e., without the input heightmap and convolutional layers) are first trained to imitate their respective motions on flat terrain. Next, the networks are augmented with an input heightmap and corresponding convolutional layers, then trained in the irregular environments. Since the mixed obstacles and dense gaps environments follow a linear layout, the heightmaps are represented by a 1D heightfield with 100 samples spanning $10m$. In the winding balance beam environment, a 32×32 heightmap is used, covering a $3.5 \times 3.5m$ area.

10 RESULTS

The motions from the trained policies are best seen in the supplemental videos. Snapshots of the skills performed by the simulated characters are available in Figure 5, 6, and 7. The policies are executed at 30Hz. Physics simulation is performed at 1.2kHz using the Bullet physics engine [Bullet 2015]. All neural networks are built and trained using TensorFlow. The characters’ motions are driven by torques computed using stable PD controllers [Tan et al. 2011]. Results for the humanoid are demonstrated for a large collection of locomotion, acrobatic, and martial arts skills, while the results for the dragon and T-Rex are demonstrated for locomotion. Each skill

Table 3. Performance statistics of imitating motion clips while also fulfilling additional task objectives.

Environment	$N_{samples} (10^6)$	NR
Humanoid: Walk - Target Heading	85	0.911
Humanoid: Jog - Target Heading	108	0.876
Humanoid: Run - Target Heading	40	0.637
Humanoid: Spinkick - Strike	85	0.601
Humanoid: Baseball Pitch - Throw	221	0.675
Humanoid: Run - Mixed Obstacles	466	0.285
Humanoid: Run - Dense Gaps	265	0.650
Humanoid: Winding Balance Beam	124	0.439
Atlas: Walk - Stairs	174	0.808

is learned from approximately 0.5-5s of mocap data collected from <http://mocap.cs.cmu.edu> and <http://mocap.cs.sfu.ca>. For characters such as the T-Rex and dragon, where mocap data is not available, we demonstrate that our framework is also capable of learning skills from artist-authored keyframes. Before being used for training, the clips are manually processed and retargeted to their respective characters. A comprehensive list of the learned skills and performance statistics is available in Table 2. Learning curves for all policies are available in the supplemental material. Each environment is denoted by “Character: Skill - Task”. The task is left unspecified for policies that are trained solely to imitate a reference motion without additional task objectives. Performance is measured by the average return normalized by the minimum and maximum possible return per episode. Note that the maximum return may not be achievable. For example, for the throwing task, the maximum return requires moving the ball instantaneously to the target. When evaluating the performance of the policies, early termination is applied and the state of the character at the start of each episode is initialized via RSI. The weights for the imitation and task objectives are set to $\omega^I = 0.7$ and $\omega^G = 0.3$ for all tasks. More details regarding hyperparameter settings are available in the supplemental material.

By encouraging the policies to imitate motion capture data from human subjects, our system is able to learn policies for a rich repertoire of skills. For locomotion skills such as walking and running, our policies produce natural gaits that avoid many of the artifacts exhibited by previous deep RL methods [Merel et al. 2017; Schulman et al. 2015b]. The humanoid is able to learn a variety of acrobatic skills with long flight phases, such as backflips and spinkicks, which are particularly challenging since the character needs to learn to coordinate its motion in mid-air. The system is also able to reproduce contact-rich motions, such as crawling and rolling, as well as motions that require coordinated interaction with the environment, such as the vaulting skills shown in Figure 6. The learned policies are robust to significant external perturbation and generate plausible recovery behaviors. The policies trained for the T-Rex and dragon demonstrate that the system can also learn from artist generated keyframes when mocap is not available and scale to much more complex characters than those that have been demonstrated by previous work using deep RL.

10.1 Tasks

In addition to imitating reference motions, the policies can also adapt the motions as needed to satisfy additional task objectives,

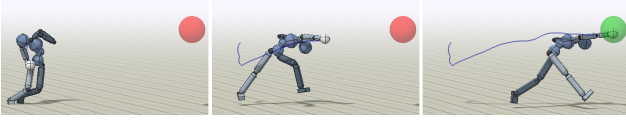


Fig. 8. Policy trained for the throw task without a reference motion. Instead of throwing the ball, the character learns to run towards the target.

Table 4. Success rate of policies trained with the imitation or task objectives disabled. Each policy is evaluated over 100 trials. Simply imitating the reference motions proves insufficient for fulfilling the task objectives. Training without a reference motion produces policies that develop awkward, but functional, strategies for satisfying the task objectives.

Environment	$r^I + r^G$	r^I	r^G
Humanoid: Strike - Spinkick	99%	19%	55%
Humanoid: Baseball Pitch - Throw	75%	5%	93%

such as following a target heading and throwing a ball to a randomly placed target. Performance statistics for each task are available in Table 3. To investigate the extent to which the motions are adapted for a particular task, we compared the performance of policies trained to optimize both the imitation objective r^I and the task objective r^G to policies trained only with the imitation objective. Table 4 summarizes the success rates of the different policies. For the throwing task, the policy trained with both objectives is able to hit the target with a success rate of 75%, while the policy trained only to imitate the baseball pitch motion is successful only in 5% of the trials. Similarly, for the strike task, the policy trained with both objectives successfully hits 99% of the targets, while the policy trained only to imitate the reference motion has a success rate of 19%. These results suggest that simply imitating the reference motions is not sufficient to successfully perform the tasks. The policies trained with the task objective are able to deviate from the original reference motion and developing additional strategies to satisfy their respective goals. We further trained policies to optimize only the task objective, without imitating a reference motion. The resulting policies are able to fulfill the task objectives, but without a reference motion, the policies develop unnatural behaviors, similar to those produced by prior deep RL methods. For the throw task, instead of throwing the ball, the policy adopts an awkward but functional strategy of running towards the target with the ball. Figure 8 illustrates this behavior.

10.2 Multi-Skill Integration

Our method can construct policies for reproducing a wide range of individual motion clips. However, many more complex skills require choosing from among a set of potential behaviors. In this section, we discuss several methods by which our approach can be extended to combine multiple clips into a single compound skill.

Multi-Clip Reward: To evaluate the multi-clip reward, we constructed an imitation objective from 5 different walking and turning clips. Using this reward, a humanoid policy is then trained to walk while following a desired heading. The resulting policy learns to utilize a variety of agile stepping behaviours in order to follow the target heading. To determine if the policy is indeed learning to imitate multiple clips, we recorded the ID of the clip that best matches the motion of the simulated character at each timestep as it follows

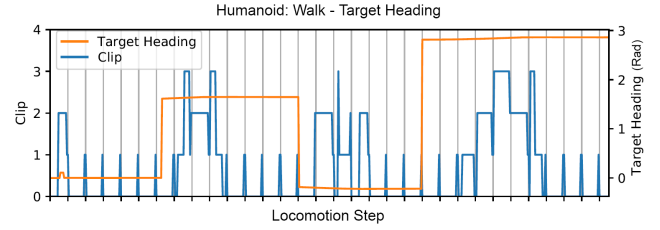


Fig. 9. Visualization of the walking reference motion clip that most closely matches the simulated character's motion as the character follows a changing target heading. Clip 0 is a forward walking motion, and clip 1-4 are turning motions. The policy is trained with the multi-clip reward.

a changing target heading. The best matching clip is designated as the clip that yields the maximum reward at the current timestep. Figure 9 records the best matching clips and the target heading over a 20s episode. Clip 0 corresponds to a forward walking motion, and clips 1-4 are different turning motions. When the target heading is constant, the character's motion primarily matches the forward walk. When the target heading changes, the character's motion becomes more closely matched with the turning motions. After the character has realigned with the target heading, the forward walking clip once again becomes the best match. This suggests that the multi-clip reward function does indeed enable the character to learn from multiple clips of different walking motions. When using the multi-clip reward, we found that the clips should generally be from a similar type of motion (e.g. different kinds of walks and turns). Mixing very different motions, like a sidestep and a frontflip together, can result in the policy imitating only a subset of the clips. For more diverse clips, we found the composite policy to be a more effective integration method.

Skill Selector: Using the one-hot vector representation, we trained a policy to perform various flips, and another to jump in different directions. The flip policy is trained to perform a frontflip, backflip, left sidestep, and right sidestep. The jump policy is trained to perform a forward, backward, left, and right jump. The first and last frame of each clip are duplicated to ensure that all clips within each set have the same cycle period. Once trained, the policies are able to perform arbitrary sequences of skills from their respective repertoires. The one-hot vector also provides a convenient interface for users to direct the character in real-time. Footage of the humanoid executing sequences of skills specified by a user is available in the supplemental video.

Composite Policy: While the multi-clip objective can be effective for integrating multiple clips that correspond to the same category (e.g., walking), we found it to be less effective for integrating clips of more diverse skills. To integrate a more diverse corpus of skills, we constructed a composite policy from the backflip, frontflip, sidestep, cartwheel, spinkick, roll, getup face-down, and getup face-up policies. The output of the value functions are normalized to be between $[-1, 1]$ and the temperature is set to $\mathcal{T} = 0.3$. A new skill is sampled from the composite policy at the start of each cycle, and the selected skill is executed for a full cycle before selecting a new skill. To prevent the character from repeatedly executing the same skill, the policy is restricted to never sample the same skill in consecutive

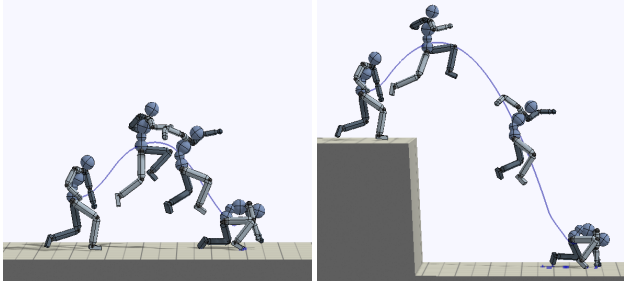


Fig. 10. **Left:** Original landing motion on flat terrain. **Right:** Policy trained to imitating landing motion while jumping down from a 2m ledge. Despite being provided only with a motion recorded on flat terrain, the policy is able to adapt the skill to jump down from a tall ledge.

cycles. Unlike the skill selector policies, the individual policies in the composite policy are never explicitly trained to transition between different skills. By using the value functions to guide the selection of skills, the character is nonetheless able to successfully transition between the various skills. When the character falls, the composite policy activates the appropriate getup policy without requiring any manual scripting, as shown in the supplemental video.

10.3 Retargeting

Due to modeling discrepancies between simulation and the real world, the dynamics under which a motion capture clip was recorded can differ dramatically from the dynamics of the simulated environments. Furthermore, keyframed motions may not be physically correct at all. To evaluate our framework's robustness to these discrepancies, we trained policies to perform similar skills with different character models, environments, and physics.

Character Retargeting: To demonstrate the system's capabilities in retargeting motions to different characters, we trained policies for walking, running, backflips and spinkicks on a simulated model of the Atlas robot from <http://www.mujoco.org/forum/index.php?resources/atlas-v5.16/>. The Atlas has a total mass of 169.8kg, almost four times the mass of the humanoid, as well as a different mass distribution. The serial 1D revolute joints in the original Atlas model are aggregated into 3D spherical joints to facilitate simpler retargeting of the reference motions. To retarget the motion clips, we simply copied the local joint rotations from the humanoid to the Atlas, without any further modification. New policies are then trained for the Atlas to imitate the retargeted clips. Despite the starkly different character morphologies, our system is able to train policies that successfully reproduce the various skills with the Atlas model. Performance statistics of the Atlas policies are available in Table 2. The performance achieved by the Atlas policies are comparable to those achieved by the humanoid. Qualitatively, the motions generated by the Atlas character closely resemble the reference clips. To further highlight the differences in the dynamics of the characters, we evaluated the performance of directly applying policies trained for the humanoid to the Atlas. The humanoid policies, when applied to the Atlas, fail to reproduce any of the skills, achieving a normalized return of 0.013 and 0.014 for the run and backflip, compared to 0.846

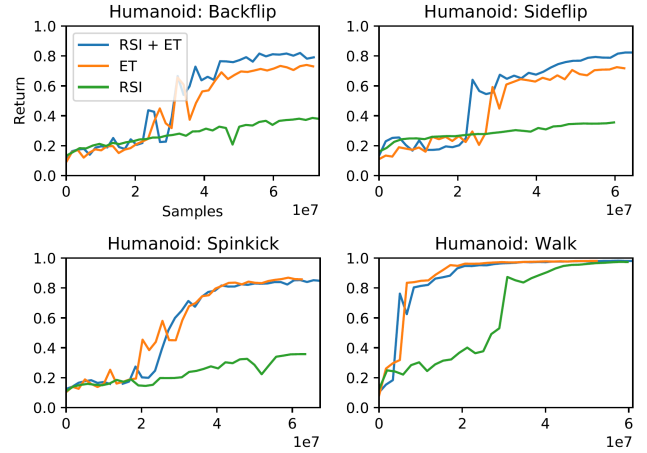


Fig. 11. Learning curves for policies trained with and without reference state initialization (RSI) and early termination (ET).

Table 5. Normalized average returns of policies trained with and without reference state initialization (RSI) and early termination (ET).

Skill	RSI + ET	ET	RSI
Backflip	0.791	0.730	0.379
Sideflip	0.823	0.717	0.355
Spinkick	0.848	0.858	0.358
Walk	0.980	0.981	0.974

and 0.630 achieved by policies that were trained specifically for the Atlas but using the same reference clips.

Environment Retargeting: While most of the reference motions were recorded on flat terrain, we show that the policies can be trained to adapt the motions to irregular environments. First, we consider the case of retargeting a landing motion, where the original reference motion is of a human subject jumping and landing on flat ground. From this reference motion, we trained a character to reproduce the motion while jumping down from a 2m ledge. Figure 10 illustrates the motion from the final policy. The system was able to adapt the reference motion to a new environment that is significantly different from that of the original clip.

Next, we explore vision-based locomotion in more complex procedurally generated environments. By augmenting the networks with a heightmap input, we are able to train the humanoid to run across terrains consisting of random obstacles. Examples of the environments are available in Figure 4. Over the course of training, the policies are able to adapt a single clip of a forward running motion into a variety of strategies for traversing across the different classes of obstacles. Furthermore, by training a policy to imitate a balance beam walk, the character learns to follow a narrow winding path using only a heightmap for pathfinding. The balance beam policy was trained with only a forward walking clip, but is nonetheless able to develop turning motions to follow the winding path. In addition to the humanoid, we also trained a policy for the Atlas to climb up stairs with irregular step heights. The policy was able to adapt the original walking clip on flat terrain to climb the steps, although the resulting motion still exhibits an awkward gait. We suspect that

the problem is partly related to the walking reference motion being ill-suited for the stairs environment.

Physics Retargeting: To further evaluate the framework’s robustness to discrepancies between the dynamics of the motion capture data and simulation, we trained policies to perform a spinkick and cartwheel under moon gravity ($1.622m/s^2$). Despite the difference in gravity, the policies were able to adapt the motions to the new dynamics, achieving a return of 0.792 for the spinkick and 0.688 for the cartwheel.

10.4 Ablations

To evaluate the impact of our design decisions, we compare our full method against alternative training schemes that disable some of the components. We found that the reference state initialization and early termination are two of the most important components of our training procedure. The comparisons include training with reference state initialization and with a fixed initial state, as well as training with early termination and without early termination. Without early termination, each episode is simulated for the full 20s. Figure 11 compares the learning curves with the different configurations and Table 5 summarizes the performance of the final policies. During evaluation, early termination is disabled and the character is initialized to a fixed state at the start of each episode. Due to the time needed to train each policy, the majority of performance statistics are collected from one run of the training process. However, we have observed that the behaviors are consistent across multiple runs. Early termination proves to be crucial for reproducing many of the skills. By heavily penalizing the character for making undesirable contacts with the ground, early termination helps to eliminate local optima, such as those where the character falls and mimes the motions as it lies on the ground. RSI also appears vital for more dynamic skills that have significant flight phases, such as the backflip. While the policies trained without RSI appear to achieve a similar return to those trained with RSI, an inspection of the resulting motions show that, without RSI, the character often fails to reproduce the desired behaviours. For the backflip, without RSI, the policy never learns to perform a full mid-air flip. Instead, it performs a small backwards hop while remaining upright.

10.5 Robustness

To determine the policies’ robustness to external perturbations, we subjected the trained policies to perturbation forces and recorded the maximum force the character can tolerate before falling. The perturbation forces are applied halfway through a motion cycle to the character’s pelvis for 0.2s. The magnitude of the force is increased by 10N until the character falls. This procedure is repeated for forces applied along the forward direction (x-axis) and sideways direction (z-axis). Table 6 summarizes the results from the experiments on the humanoid. The learned policies show comparable-or-better robustness than figures reported for SAMCON [Liu et al. 2016]. The run policy is able to recover from $720N \times 0.2s$ forward pushes, while the spin-kick policy is able to survive $600N \times 0.2s$ perturbations in both directions. Note that no external perturbations are applied during the training process; we suspect that the policies’ robustness is in large part due to the exploration noise applied by the stochastic policy used during training.

Table 6. Maximum forwards and sideways push each policy can tolerate before falling. Each push is applied to the character’s pelvis for 0.2s.

Skill	Forward (N)	Sideway (N)
Backflip	440	100
Cartwheel	200	470
Run	720	300
Spinkick	690	600
Walk	240	300

11 DISCUSSION AND LIMITATIONS

We presented a data-driven deep reinforcement learning framework for training control policies for simulated characters. We show that our method can produce a broad range of challenging skills. The resulting policies are highly robust and produce natural motions that are nearly indistinguishable from the original motion capture data in the absence of perturbations. Our framework is able to re-target skills to a variety of characters, environments, and tasks, and multiple policies can be combined into composite policies capable of executing multiple skills.

Although our experiments illustrate the flexibility of this approach, there are still numerous limitations to be addressed in future work. First, our policies require a phase variable to be synchronized with the reference motion, which advances linearly with time. This limits the ability of the policy to adjust the timing of the motion, and lifting this limitation could produce more natural and flexible perturbation recoveries. Our multi-clip integration approach works well for small numbers of clips, but has not yet been demonstrated on large motion libraries. The PD controllers used as the low-level servos for the characters still require some insight to set properly for each individual character morphology. The learning process itself is also quite time consuming, often requiring several days per skill, and is performed independently for each policy. Although we use the same imitation reward across all motions, this is still currently based on a manually defined state-similarity metric. The weighting of the reward terms also needs to be defined with some care.

We believe this work nevertheless opens many exciting directions for exploration. In future work, we wish to understand how the policies might be deployed on robotic systems, as applied to locomotion, dexterous manipulation, and other tasks. It would be interesting to understand the learned control strategies and compare them to the equivalent human strategies. We wish to integrate diverse skills that would enable a character to perform more challenging tasks and more complex interactions with their environments. Incorporating hierarchical structure is likely to be beneficial towards this goal.

ACKNOWLEDGEMENTS

We thank KangKang Yin, Libin Liu, and Ziva Dynamics for providing reference motion data for this project, the anonymous reviewers for their helpful feedback, and AWS for providing computational resources. This research was funded by an NSERC Postgraduate Scholarship, a Berkeley Fellowship for Graduate Study, and ONR PECASE N000141612723.

REFERENCES

Shailen Agrawal, Shuo Shen, and Michiel van de Panne. 2013. Diverse Motion Variations for Physics-based Character Animation. *Symposium on Computer Animation* (2013).

SUPPLEMENTARY MATERIAL

A MULTI-STEP RETURNS

We will refer to the return $R_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$ as the Monte-Carlo return. R_t provides an unbiased sample of the expected return at a given state, but due to stochasticity from the dynamics of the environment and policy, each reward r_t can be a random variable, the sum of which can result in a high variance estimator of the expected return. Alternatively, an n -step return can be used to provide a lower-variance estimator at the cost of introducing some bias. The n -step return can be computed by truncating the sum of returns after n steps, and approximating the return from the remaining steps using a value function $V(s)$:

$$R_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}).$$

$R_t^{(1)} = r_t + \gamma V(s_{t+1})$ results in the 1-step return commonly used in Q -learning [Mnih et al. 2015] and $R_t^{(\infty)} = \sum_{l=0}^{T-t} \gamma^l r_{t+l} = R_t$ recovers the original Monte-Carlo return, given that all rewards after the horizon T are 0. While $R_t^{(\infty)}$ provides an unbiased but high variance estimator, $R_t^{(1)}$ provides a biased but low variance estimator. Therefore, n acts as a trade-off between bias and variance for the value estimator.

Another method to trade off between the bias and variance of the estimator is to use a λ -return [Sutton and Barto 1998], calculated as an exponentially-weighted average of n -step returns with decay parameter λ :

$$R_t(\lambda) = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} R_t^n.$$

Assuming all rewards after step T are 0, such that $R_t^n = R_t^{T-t}$ for all $n \geq T - t$, the infinite sum can be calculated according to

$$R_t(\lambda) = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} R_t^n + \lambda^{T-t-1} R_t^{T-t}$$

$\lambda = 0$ reduces to the single-step return $R_t^{(1)}$, and $\lambda = 1$ recovers the Monte-Carlo return R_t . Intermediate values of $\lambda \in (0, 1)$ produces interpolants that can be used to balance the bias and variance of the value estimator.

Updating the value function using the temporal difference computed with the λ -return results in the TD(λ) algorithm [Sutton and Barto 1998]. Similarly, estimating the advantage with the λ -return, yields the generalized advantage estimator GAE(λ) [Schulman et al. 2015b]:

$$\hat{\mathcal{A}}_t = R_t(\lambda) - V(s_t).$$

Empirically, the λ -return has been shown to produce better performance than simply using an n -step return [Schulman et al. 2015b; Sutton and Barto 1998].

B OFF-POLICY LEARNING

In its previously stated form, the gradient of the expected return $\nabla_{\theta} J(\theta)$ is estimated with respect to the current policy parameters θ . As such, data collected from the current policy can be justified for use only in performing a single gradient step, after which a new

batch of data is required to estimate the gradient with respect to the updated parameters. Data efficiency can be improved by introducing importance sampling, which provides an unbiased estimate of the policy gradient using only off-policy samples from an older policy $\pi_{\theta_{old}}$:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s_t \sim d_{\theta_{old}}(s_t), a_t \sim \pi_{\theta_{old}}(a_t | s_t)} [w_t(\theta) \nabla_{\theta} \log(\pi_{\theta}(a_t | s_t)) \mathcal{A}_t]$$

$$w_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

The importance-sampled policy gradient can be interpreted as optimizing the surrogate objective

$$L^{IS}(\theta) = \mathbb{E}_{s_t \sim d_{\theta_{old}}(s_t), a_t \sim \pi_{\theta_{old}}(a_t | s_t)} [w_t(\theta) \mathcal{A}_t].$$

With L^{IS} , the same batch of data can be used to perform multiple update steps for θ . While in theory importance sampling allows for the policy gradient to be estimated with data collected from any policy with sufficient support, we will consider only the case where the data is collected from a previous set of policy parameters.

C PROXIMAL POLICY OPTIMIZATION

In practice, the policy gradient estimator discussed thus far suffers from high variance, often leading to instability during learning, where the policy's performance fluctuates drastically between iterations. While problems due to noisy gradients can be mitigated by using large batches of data per update, policy gradient algorithms can still be extremely unstable. Trust region methods have been proposed as a technique for improving the stability of policy gradient algorithms [Schulman et al. 2015a]. Trust Region Policy Optimization (TRPO) optimizes the same objective $J(\theta)$ but includes an additional KL-divergence constraint to prevent the behaviour of the current policy π_{θ} from deviating too far from the previous policy $\pi_{\theta_{old}}$.

$$\begin{aligned} \max_{\theta} \quad & J(\theta) \\ \text{s.t.} \quad & \mathbb{E}_{s_t \sim d_{\theta}(s_t)} \left[KL \left(\pi_{\theta_{old}}(\cdot | s_t) \middle| \pi_{\theta}(\cdot | s_t) \right) \right] \leq \delta_{KL} \end{aligned}$$

δ_{KL} is a hyper parameters that defines the trust region. TRPO has been successfully applied to solve a wide variety of challenging RL problems [Duan et al. 2016; Ho and Ermon 2016; Rajeswaran et al. 2016]. However, ensuring that the constraint is satisfied can be difficult, and it is often instead enforced approximately using the conjugate gradient algorithm with an adaptive stepsize selected by a line search to ensure the approximate constraint is satisfied [Schulman et al. 2015a].

Proximal Policy Optimization (PPO) is a variant of TRPO, where the hard constraint is replaced by optimizing a surrogate loss [Schulman et al. 2017]. In this work, we will be concerned mainly with the clipped surrogate loss $L^{CLIP}(\theta)$ defined according to

$$L^{CLIP}(\theta) = \mathbb{E}_{s_t, a_t} [\min(w_t(\theta) \mathcal{A}_t, \text{clip}(w_t(\theta), 1 - \epsilon, 1 + \epsilon) \mathcal{A}_t)].$$

When $\theta = \theta_{old}$, $w_t(\theta) = 1$, but as θ is changed, the likelihood ratio

moves away from 1. Therefore, the likelihood ratio can be interpreted as a measure of the similarity between two policies. To discourage the policies from deviating too far apart, $\text{clip}(w_t(\theta), 1 - \epsilon, 1 + \epsilon)$ sets the gradient to zero whenever the ratio is more than ϵ away from 1. This term therefore serves a similar function to the KL-divergence constraint in TRPO. The minimum is then taken between the clipped and unclipped advantages to create a lower bound of $L(\theta)$.

D LEARNING ALGORITHM

Algorithm 1 summarizes the common learning procedure used to train all policies. Policy updates are performed after a batch of $m = 4096$ samples has been collected. Minibatches of size $n = 256$ are then sampled from the data for each gradient step. A discount factor $\gamma = 0.95$ is used for all motions. $\lambda = 0.95$ is used for both TD(λ) and GAE(λ). The likelihood ratio clipping threshold is set to $\epsilon = 0.2$. A stepsize of $\alpha_v = 10^{-2}$ is used for the value function. A policy step size of $\alpha_\pi = 5 \times 10^{-5}$ is used for the humanoid and Atlas, and $\alpha_\pi = 2 \times 10^{-5}$ for the dragon and T-Rex. Once gradients have been computed, the network parameters are updated using stochastic gradient descent with momentum 0.9. The same hyperparameter settings are used for all characters and skills, with the exception of the step size. Humanoid policies for imitating individual skills typically require about 60 million samples to train, requiring about 2 days on an 8-core machine. All simulation and network updates are performed on the CPU and no GPU acceleration is used.

ALGORITHM 1: Proximal Policy Optimization

```

1:  $\theta \leftarrow$  random weights
2:  $\psi \leftarrow$  random weights
3: while not done do
4:    $s_0 \leftarrow$  sample initial state from reference motion
5:   Initialize character to state  $s_0$ 
6:   for step = 1, ...,  $m$  do
7:      $s \leftarrow$  start state
8:      $a \sim \pi_\theta(a|s)$ 
9:     Apply  $a$  and simulate forward one step
10:     $s' \leftarrow$  end state
11:     $r \leftarrow$  reward
12:    record  $(s, a, r, s')$  into memory  $D$ 
13:   end for
14:    $\theta_{old} \leftarrow \theta$ 
15:   for each update step do
16:     Sample minibatch of  $n$  samples  $\{(s_i, a_i, r_i, s'_i)\}$  from  $D$ 
17:     Update value function:
18:     for each  $(s_i, a_i, r_i, s'_i)$  do
19:        $y_i \leftarrow$  compute target values using TD( $\lambda$ )
20:     end for
21:      $\psi \leftarrow \psi + \alpha_v \left( \frac{1}{n} \sum_i \nabla_{\psi} V_{\psi}(s_i)(y_i - V(s_i)) \right)$ 
22:     Update policy:
23:     for each  $(s_i, a_i, r_i, s'_i)$  do
24:        $\mathcal{A}_i \leftarrow$  compute advantage using  $V_{\psi}$  and GAE
25:        $w_i(\theta) \leftarrow \frac{\pi_\theta(a_i|s_i)}{\pi_{\theta_{old}}(a_i|s_i)}$ 
26:     end for
27:      $\theta \leftarrow$ 
28:        $\theta + \alpha_\pi \frac{1}{n} \sum_i \nabla_{\theta} \min(w_i(\theta)\mathcal{A}_i, \text{clip}(w_i(\theta), 1 - \epsilon, 1 + \epsilon)\mathcal{A}_i)$ 
29:   end for
30: end while

```

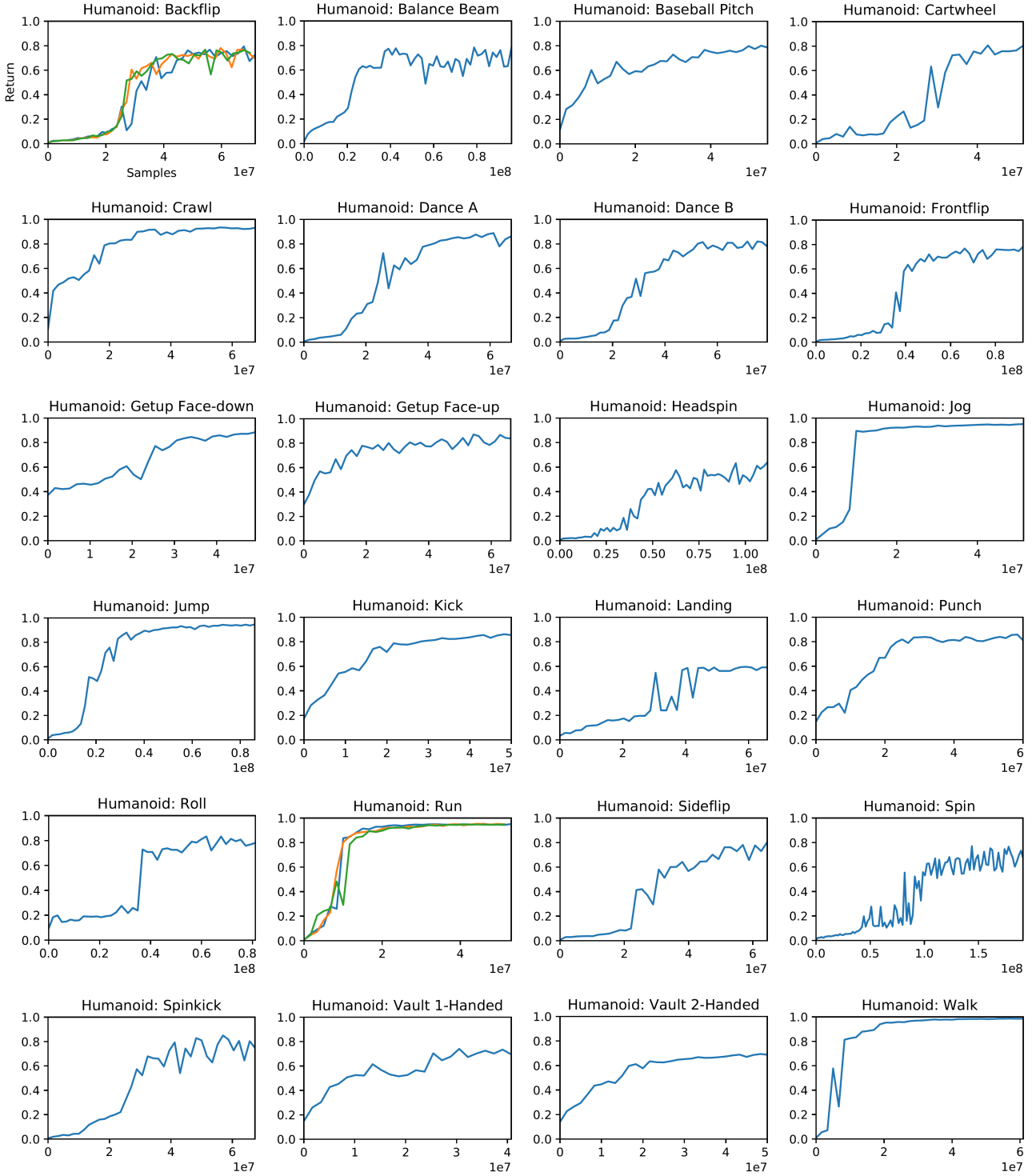


Fig. 12. Learning curves of the humanoid imitating individual motion clips. Performance is calculated as the mean return over 32 episodes. The returns are normalized by the minimum and maximum possible return per episode. Due to the time needed to train each policy, performance statistics are collected only from one run of the training process for the majority of skills. For the backflip and run policies, we collected statistics from 3 training runs using different random seeds. Performance appears consistent across multiple runs, and we have observed similar behaviours for many of the other skills.

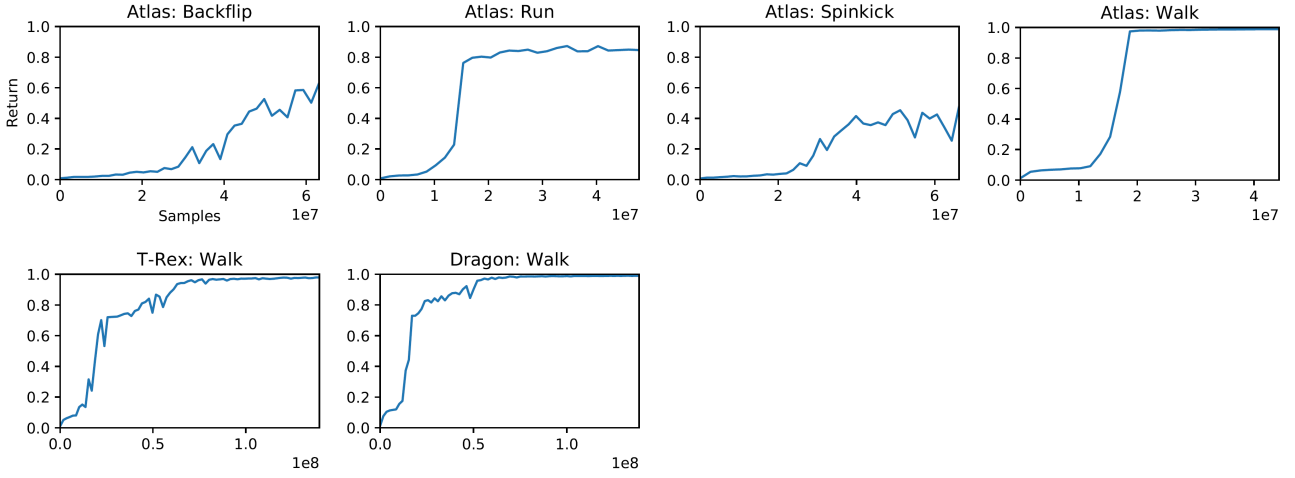


Fig. 13. Learning curves of the Atlas, T-Rex, and dragon imitating individual motion clips.

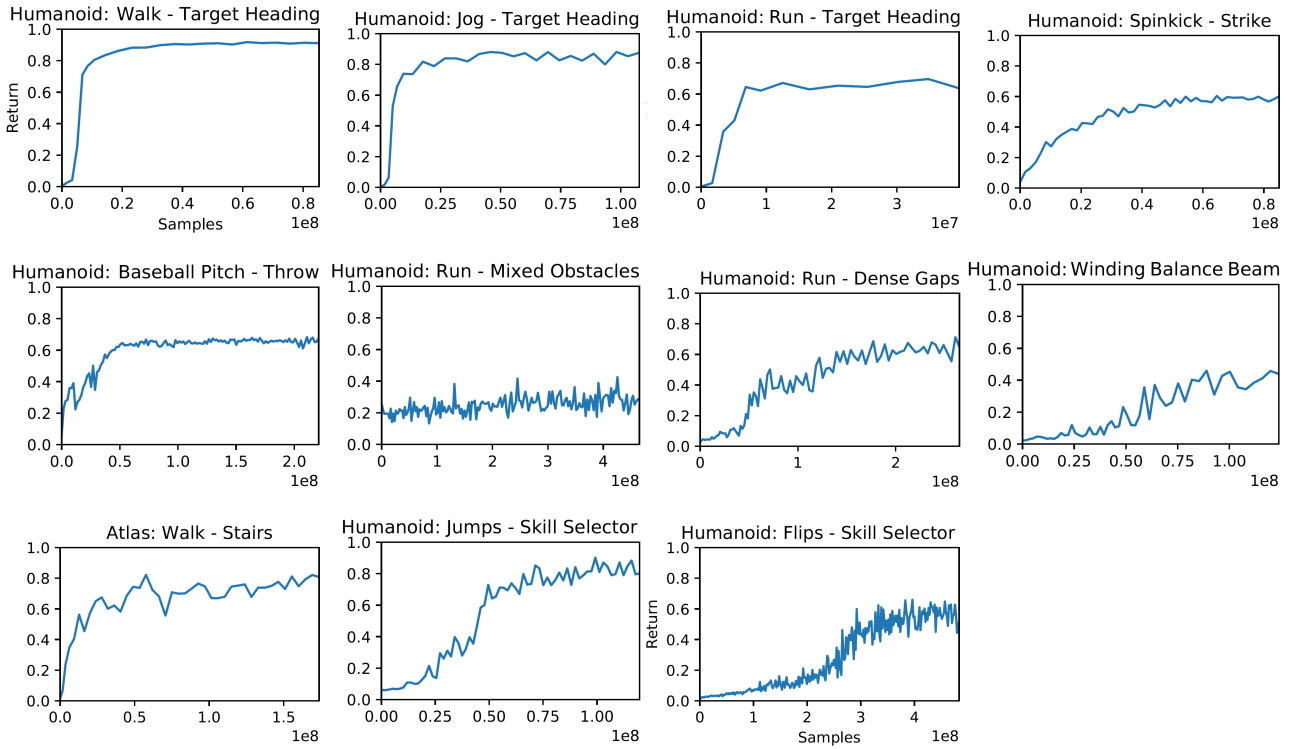


Fig. 14. Learning curves of policies trained to imitate motion clips while fulfilling additional task objectives.



A (Long) Peek into Reinforcement Learning

Date: February 19, 2018 | Estimated Reading Time: 31 min | Author: Lilian Weng

► Table of Contents

[Updated on 2020-09-03: Updated the algorithm of [SARSA](#) and [Q-learning](#) so that the difference is more pronounced.

[Updated on 2021-09-19: Thanks to [38 36 39 38 的 11 11](#), we have this post in [Chinese](#)].

A couple of exciting news in Artificial Intelligence (AI) has just happened in recent years. AlphaGo defeated the best professional human player in the game of Go. Very soon the extended algorithm AlphaGo Zero beat AlphaGo by 100-0 without supervised learning on human knowledge. Top professional game players lost to the bot developed by OpenAI on DOTA2 1v1 competition. After knowing these, it is pretty hard not to be curious about the magic behind these algorithms — Reinforcement Learning (RL). I'm writing this post to briefly go over the field. We will first introduce several fundamental concepts and then dive into classic approaches to solving RL problems. Hopefully, this post could be a good starting point for newbies, bridging the future study on the cutting-edge research.

What is Reinforcement Learning?

Say, we have an agent in an unknown environment and this agent can obtain some rewards by interacting with the environment. The agent ought to take actions so as to maximize cumulative rewards. In reality, the scenario could be a bot playing a game to achieve high scores, or a robot trying to complete physical tasks with physical items; and not just limited to these.

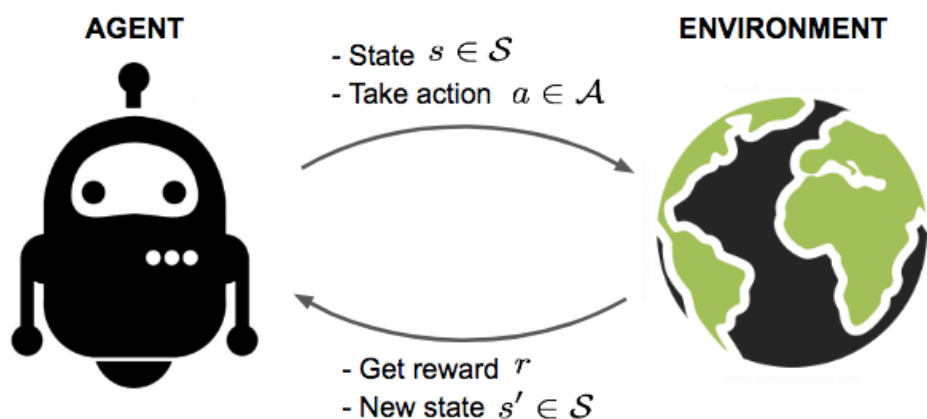




Fig. 1. An agent interacts with the environment, trying to take smart actions to maximize cumulative rewards.

The goal of Reinforcement Learning (RL) is to learn a good strategy for the agent from experimental trials and relative simple feedback received. With the optimal strategy, the agent is capable to actively adapt to the environment to maximize future rewards.

Key Concepts

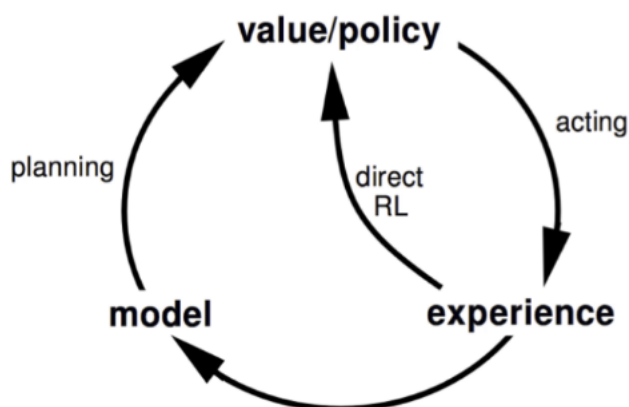
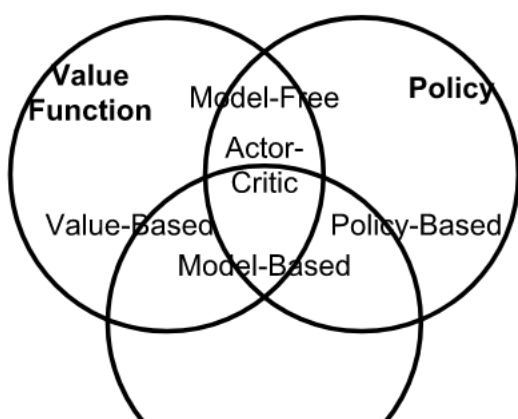
Now Let's formally define a set of key concepts in RL.

The agent is acting in an **environment**. How the environment reacts to certain actions is defined by a **model** which we may or may not know. The agent can stay in one of many **states** ($s \in \mathcal{S}$) of the environment, and choose to take one of many **actions** ($a \in \mathcal{A}$) to switch from one state to another. Which state the agent will arrive in is decided by transition probabilities between states (P). Once an action is taken, the environment delivers a **reward** ($r \in \mathcal{R}$) as feedback.

The model defines the reward function and transition probabilities. We may or may not know how the model works and this differentiate two circumstances:

- **Know the model:** planning with perfect information; do model-based RL. When we fully know the environment, we can find the optimal solution by Dynamic Programming (DP). Do you still remember "longest increasing subsequence" or "traveling salesmen problem" from your Algorithms 101 class? LOL. This is not the focus of this post though.
- **Does not know the model:** learning with incomplete information; do model-free RL or try to learn the model explicitly as part of the algorithm. Most of the following content serves the scenarios when the model is unknown.

The agent's **policy** $\pi(s)$ provides the guideline on what is the optimal action to take in a certain state with **the goal to maximize the total rewards**. Each state is associated with a **value** function $V(s)$ predicting the expected amount of future rewards we are able to receive in this state by acting the corresponding policy. In other words, the value function quantifies how good a state is. Both policy and value functions are what we try to learn in reinforcement learning.





model
learning

Fig. 2. Summary of approaches in RL based on whether we want to model the value, policy, or the environment. (Image source: reproduced from David Silver's RL course [lecture 1](#).)

The interaction between the agent and the environment involves a sequence of actions and observed rewards in time, $t = 1, 2, \dots, T$. During the process, the agent accumulates the knowledge about the environment, learns the optimal policy, and makes decisions on which action to take next so as to efficiently learn the best policy. Let's label the state, action, and reward at time step t as S_t , A_t , and R_t , respectively. Thus the interaction sequence is fully described by one **episode** (also known as "trial" or "trajectory") and the sequence ends at the terminal state S_T :

$$S_1, A_1, R_2, S_2, A_2, \dots, S_T$$

Terms you will encounter a lot when diving into different categories of RL algorithms:

- **Model-based**: Rely on the model of the environment; either the model is known or the algorithm learns it explicitly.
- **Model-free**: No dependency on the model during learning.
- **On-policy**: Use the deterministic outcomes or samples from the target policy to train the algorithm.
- **Off-policy**: Training on a distribution of transitions or episodes produced by a different behavior policy rather than that produced by the target policy.

Model: Transition and Reward

The model is a descriptor of the environment. With the model, we can learn or infer how the environment would interact with and provide feedback to the agent. The model has two major parts, transition probability function P and reward function R .

Let's say when we are in state s , we decide to take action a to arrive in the next state s' and obtain reward r . This is known as one **transition** step, represented by a tuple (s, a, s', r) .

The transition function P records the probability of transitioning from state s to s' after taking action a while obtaining reward r . We use $\mathbb{P}$ as a symbol of "probability".

$$P(s', r | s, a) = \mathbb{P}[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a]$$

Thus the state-transition function can be defined as a function of $P(s', r | s, a)$:

$$P_{ss'}^a = P(s' | s, a) = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} P(s', r | s, a)$$



The reward function R predicts the next reward triggered by one action:

$$R(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} P(s', r | s, a)$$

Policy

Policy, as the agent's behavior function π , tells us which action to take in state s . It is a mapping from state s to action a and can be either deterministic or stochastic:

- Deterministic: $\pi(s) = a$.
- Stochastic: $\pi(a|s) = \mathbb{P}_\pi[A = a | S = s]$.

Value Function

Value function measures the goodness of a state or how rewarding a state or an action is by a prediction of future reward. The future reward, also known as **return**, is a total sum of discounted rewards going forward. Let's compute the return G_t starting from time t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The discounting factor $\gamma \in [0, 1]$ penalize the rewards in the future, because:

- The future rewards may have higher uncertainty; i.e. stock market.
- The future rewards do not provide immediate benefits; i.e. As human beings, we might prefer to have fun today rather than 5 years later ;).
- Discounting provides mathematical convenience; i.e., we don't need to track future steps forever to compute return.
- We don't need to worry about the infinite loops in the state transition graph.

The **state-value** of a state s is the expected return if we are in this state at time t , $S_t = s$:

$$V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

Similarly, we define the **action-value** ("Q-value"; Q as "Quality" I believe?) of a state-action pair as:

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

Additionally, since we follow the target policy π , we can make use of the probability distribution over possible actions and the Q-values to recover the state-value:

$$V_\pi(s) = \sum_{a \in \mathcal{A}} Q_\pi(s, a) \pi(a|s)$$

The difference between action-value and state-value is the action **advantage** function ("A-value"):



$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$$

Optimal Value and Policy

The optimal value function produces the maximum return:

$$V_*(s) = \max_{\pi} V_{\pi}(s), Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$$

The optimal policy achieves optimal value functions:

$$\pi_* = \arg \max_{\pi} V_{\pi}(s), \pi_* = \arg \max_{\pi} Q_{\pi}(s, a)$$

And of course, we have $V_{\pi_*}(s) = V_*(s)$ and $Q_{\pi_*}(s, a) = Q_*(s, a)$.

Markov Decision Processes

In more formal terms, almost all the RL problems can be framed as **Markov Decision Processes** (MDPs). All states in MDP has "Markov" property, referring to the fact that the future only depends on the current state, not the history:

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_1, \dots, S_t]$$

Or in other words, the future and the past are **conditionally independent** given the present, as the current state encapsulates all the statistics we need to decide the future.

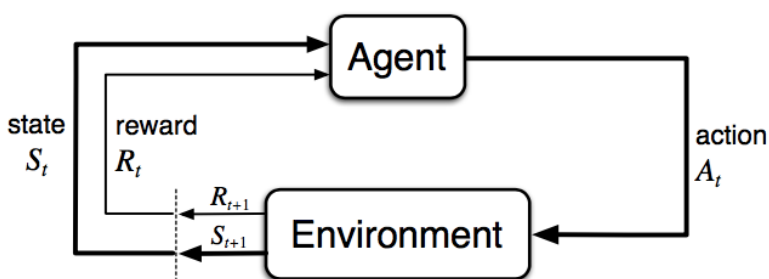


Fig. 3. The agent-environment interaction in a Markov decision process. (Image source: Sec. 3.1 Sutton & Barto (2017).)

A Markov decision process consists of five elements $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where the symbols carry the same meanings as key concepts in the previous section, well aligned with RL problem settings:

- $\mathcal{S}$ - a set of states;
- $\mathcal{A}$ - a set of actions;
- P - transition probability function;
- R - reward function;
- γ - discounting factor for future rewards. In an unknown environment, we do not have perfect knowledge about P and R .

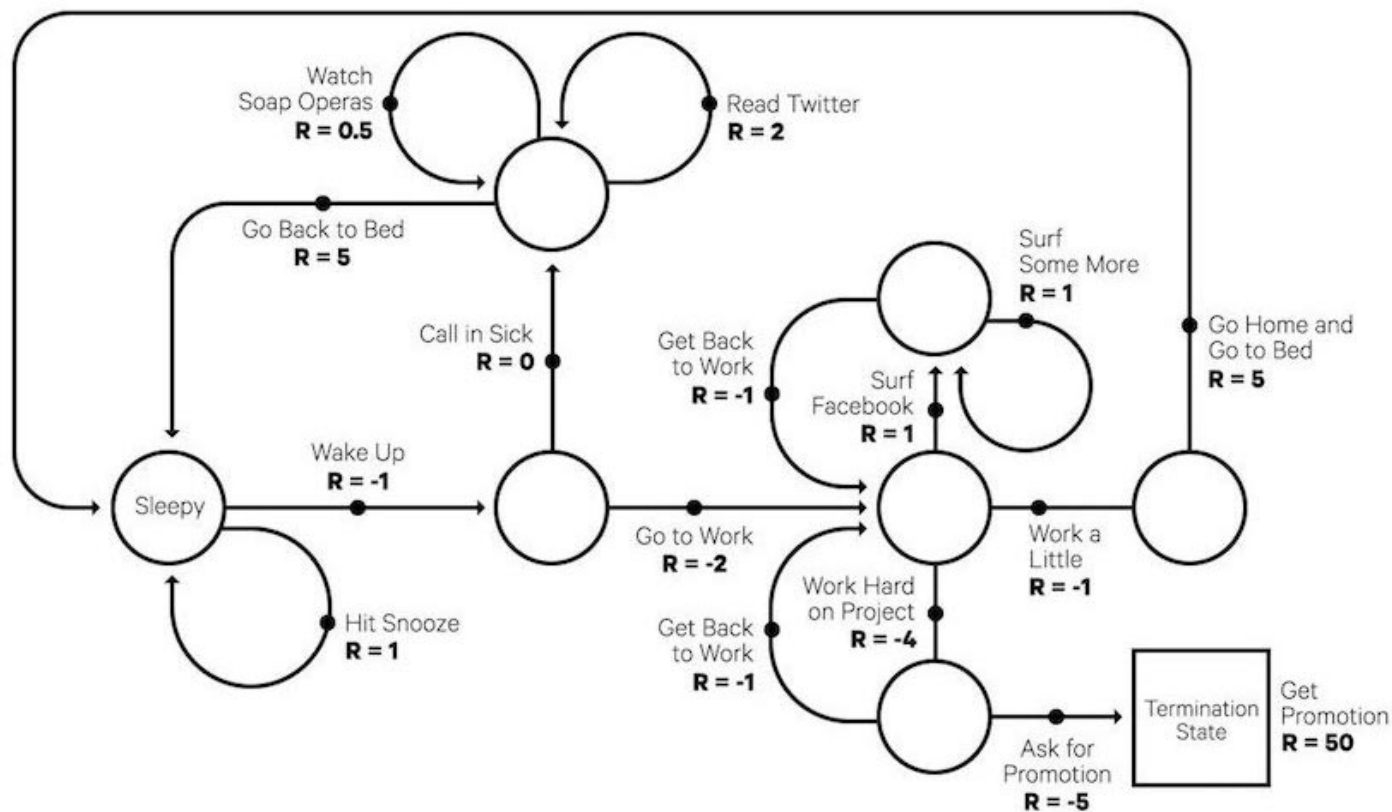


Fig. 4. A fun example of Markov decision process: a typical work day. (Image source: randomant.net/reinforcement-learning-concepts)

Bellman Equations

Bellman equations refer to a set of equations that decompose the value function into the immediate reward plus the discounted future values.

$$\begin{aligned}
 V(s) &= \mathbb{E}[G_t | S_t = s] \\
 &= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s] \\
 &= \mathbb{E}[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) | S_t = s] \\
 &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s] \\
 &= \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) | S_t = s]
 \end{aligned}$$

Similarly for Q-value,

$$\begin{aligned}
 Q(s, a) &= \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) | S_t = s, A_t = a] \\
 &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}_{a' \sim \pi} Q(S_{t+1}, a') | S_t = s, A_t = a]
 \end{aligned}$$

Bellman Expectation Equations

The recursive update process can be further decomposed to be equations built on both state-value and action-value functions. As we go further in future action steps, we extend V and Q alternatively by following the policy π .

$$\begin{aligned}
 & \mathbf{a}' \\
 & \mathbf{a}' \cap (\mathbf{a}' \cap \mathbf{a}')
 \end{aligned}$$

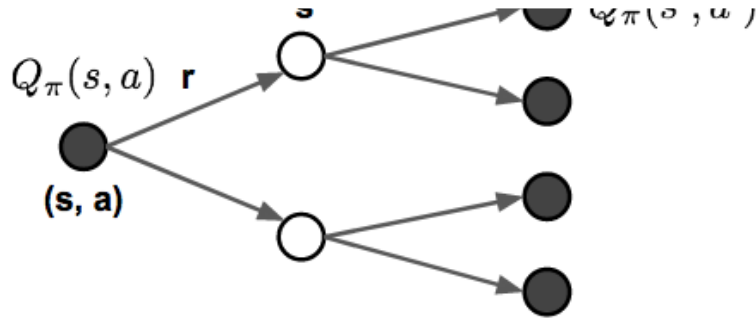


Fig. 5. Illustration of how Bellman expectation equations update state-value and action-value functions.

$$\begin{aligned}
 V_{\pi}(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) Q_{\pi}(s, a) \\
 Q_{\pi}(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_{\pi}(s') \\
 V_{\pi}(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_{\pi}(s') \right) \\
 Q_{\pi}(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') Q_{\pi}(s', a')
 \end{aligned}$$

Bellman Optimality Equations

If we are only interested in the optimal values, rather than computing the expectation following a policy, we could jump right into the maximum returns during the alternative updates without using a policy. RECAP: the optimal values V_* and Q_* are the best returns we can obtain, defined [here](#).

$$\begin{aligned}
 V_*(s) &= \max_{a \in \mathcal{A}} Q_*(s, a) \\
 Q_*(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s') \\
 V_*(s) &= \max_{a \in \mathcal{A}} \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s') \right) \\
 Q_*(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \max_{a' \in \mathcal{A}} Q_*(s', a')
 \end{aligned}$$

Unsurprisingly they look very similar to Bellman expectation equations.

If we have complete information of the environment, this turns into a planning problem, solvable by DP. Unfortunately, in most scenarios, we do not know $P_{ss'}^a$ or $R(s, a)$, so we cannot solve MDPs by directly applying Bellmen equations, but it lays the theoretical foundation for many RL algorithms.

Common Approaches

Now it is the time to go through the major approaches and classic algorithms for solving RL problems. In future posts, I plan to dive into each approach further.



Dynamic Programming

When the model is fully known, following Bellman equations, we can use Dynamic Programming (DP) to iteratively evaluate value functions and improve policy.

Policy Evaluation

Policy Evaluation is to compute the state-value V_π for a given policy π :

$$V_{t+1}(s) = \mathbb{E}_\pi[r + \gamma V_t(s') | S_t = s] = \sum_a \pi(a|s) \sum_{s', r} P(s', r | s, a) (r + \gamma V_t(s'))$$

Policy Improvement

Based on the value functions, Policy Improvement generates a better policy $\pi' \geq \pi$ by acting greedily.

$$Q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = s, A_t = a] = \sum_{s', r} P(s', r | s, a) (r + \gamma V_\pi(s'))$$

Policy Iteration

The *Generalized Policy Iteration (GPI)* algorithm refers to an iterative procedure to improve the policy when combining policy evaluation and improvement.

$$\pi_0 \xrightarrow{\text{evaluation}} V_{\pi_0} \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{evaluation}} V_{\pi_1} \xrightarrow{\text{improve}} \pi_2 \xrightarrow{\text{evaluation}} \dots \xrightarrow{\text{improve}} \pi_* \xrightarrow{\text{evaluation}} V_*$$

In GPI, the value function is approximated repeatedly to be closer to the true value of the current policy and in the meantime, the policy is improved repeatedly to approach optimality. This policy iteration process works and always converges to the optimality, but why this is the case?

Say, we have a policy π and then generate an improved version π' by greedily taking actions, $\pi'(s) = \arg \max_{a \in \mathcal{A}} Q_\pi(s, a)$. The value of this improved π' is guaranteed to be better because:

$$\begin{aligned} Q_\pi(s, \pi'(s)) &= Q_\pi(s, \arg \max_{a \in \mathcal{A}} Q_\pi(s, a)) \\ &= \max_{a \in \mathcal{A}} Q_\pi(s, a) \geq Q_\pi(s, \pi(s)) = V_\pi(s) \end{aligned}$$

Monte-Carlo Methods

First, let's recall that $V(s) = \mathbb{E}[G_t | S_t = s]$. Monte-Carlo (MC) methods uses a simple idea: It learns from episodes of raw experience without modeling the environmental dynamics and computes the observed mean return as an approximation of the expected return. To compute the empirical return G_t , MC methods need to learn from **complete** episodes $S_1, A_1, R_2, \dots, S_T$ to compute $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$ and all the episodes must eventually terminate.

The empirical mean return for state s is:

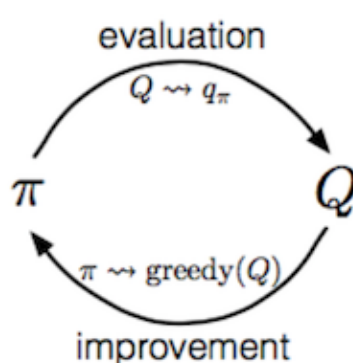


$$V(s) = \frac{\sum_{t=1}^T \mathbb{1}[S_t = s] G_t}{\sum_{t=1}^T \mathbb{1}[S_t = s]}$$

where $\mathbb{1}[S_t = s]$ is a binary indicator function. We may count the visit of state s every time so that there could exist multiple visits of one state in one episode ("every-visit"), or only count it the first time we encounter a state in one episode ("first-visit"). This way of approximation can be easily extended to action-value functions by counting (s, a) pair.

$$Q(s, a) = \frac{\sum_{t=1}^T \mathbb{1}[S_t = s, A_t = a] G_t}{\sum_{t=1}^T \mathbb{1}[S_t = s, A_t = a]}$$

To learn the optimal policy by MC, we iterate it by following a similar idea to [GPI](#).



1. Improve the policy greedily with respect to the current value function: $\pi(s) = \arg \max_{a \in \mathcal{A}} Q(s, a)$.
2. Generate a new episode with the new policy π (i.e. using algorithms like [ε-greedy](#) helps us balance between exploitation and exploration.)
3. Estimate Q using the new episode: $q_\pi(s, a) = \frac{\sum_{t=1}^T (\mathbb{1}[S_t=s, A_t=a] \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1})}{\sum_{t=1}^T \mathbb{1}[S_t=s, A_t=a]}$

Temporal-Difference Learning

Similar to Monte-Carlo methods, Temporal-Difference (TD) Learning is model-free and learns from episodes of experience. However, TD learning can learn from **incomplete** episodes and hence we don't need to track the episode up to termination. TD learning is so important that Sutton & Barto (2017) in their RL book describes it as "one idea ... central and novel to reinforcement learning".

Bootstrapping

TD learning methods update targets with regard to existing estimates rather than exclusively relying on actual rewards and complete returns as in MC methods. This approach is known as **bootstrapping**.

Value Estimation

The key idea in TD learning is to update the value function $V(S_t)$ towards an estimated return $R_{t+1} + \gamma V(S_{t+1})$ (known as "**TD target**"). To what extent we want to update the value function is

controlled by the learning rate hyperparameter α :

$$\begin{aligned}V(S_t) &\leftarrow (1 - \alpha)V(S_t) + \alpha G_t \\V(S_t) &\leftarrow V(S_t) + \alpha(G_t - V(S_t)) \\V(S_t) &\leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))\end{aligned}$$

Similarly, for action-value estimation:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

Next, let's dig into the fun part on how to learn optimal policy in TD learning (aka "TD control"). Be prepared, you are gonna see many famous names of classic algorithms in this section.

SARSA: On-Policy TD control

"SARSA" refers to the procedure of updating Q-value by following a sequence of $\dots, S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, \dots$. The idea follows the same route of GPI. Within one episode, it works as follows:

1. Initialize $t = 0$.
2. Start with S_0 and choose action $A_0 = \arg \max_{a \in \mathcal{A}} Q(S_0, a)$, where ϵ -greedy is commonly applied.
3. At time t , after applying action A_t , we observe reward R_{t+1} and get into the next state S_{t+1} .
4. Then pick the next action in the same way as in step 2: $A_{t+1} = \arg \max_{a \in \mathcal{A}} Q(S_{t+1}, a)$.
5. Update the Q-value function: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$.
6. Set $t = t + 1$ and repeat from step 3.

In each step of SARSA, we need to choose the *next* action according to the *current* policy.

Q-Learning: Off-policy TD control

The development of Q-learning (Watkins & Dayan, 1992) is a big breakout in the early days of Reinforcement Learning. Within one episode, it works as follows:

1. Initialize $t = 0$.
2. Starts with S_0 .
3. At time step t , we pick the action according to Q values, $A_t = \arg \max_{a \in \mathcal{A}} Q(S_t, a)$ and ϵ -greedy is commonly applied.
4. After applying action A_t , we observe reward R_{t+1} and get into the next state S_{t+1} .
5. Update the Q-value function:
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a) - Q(S_t, A_t)).$$
6. $t = t + 1$ and repeat from step 3.

The key difference from SARSA is that Q-learning does not follow the current policy to pick the second

action A_{t+1} . It estimates Q^* out of the best Q values, but which action (denoted as a^*) leads to this maximal Q does not matter and in the next step Q-learning may not follow a^* .

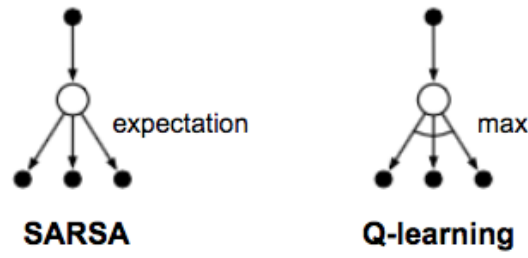


Fig. 6. The backup diagrams for Q-learning and SARSA. (Image source: Replotted based on Figure 6.5 in Sutton & Barto (2017))

Deep Q-Network

Theoretically, we can memorize $Q_*(.)$ for all state-action pairs in Q-learning, like in a gigantic table. However, it quickly becomes computationally infeasible when the state and action space are large. Thus people use functions (i.e. a machine learning model) to approximate Q values and this is called **function approximation**. For example, if we use a function with parameter θ to calculate Q values, we can label Q value function as $Q(s, a; \theta)$.

Unfortunately Q-learning may suffer from instability and divergence when combined with an nonlinear Q-value function approximation and bootstrapping (See Problems #2).

Deep Q-Network ("DQN"; Mnih et al. 2015) aims to greatly improve and stabilize the training procedure of Q-learning by two innovative mechanisms:

- **Experience Replay:** All the episode steps $e_t = (S_t, A_t, R_t, S_{t+1})$ are stored in one replay memory $D_t = \{e_1, \dots, e_t\}$. D_t has experience tuples over many episodes. During Q-learning updates, samples are drawn at random from the replay memory and thus one sample could be used multiple times. Experience replay improves data efficiency, removes correlations in the observation sequences, and smooths over changes in the data distribution.
- **Periodically Updated Target:** Q is optimized towards target values that are only periodically updated. The Q network is cloned and kept frozen as the optimization target every C steps (C is a hyperparameter). This modification makes the training more stable as it overcomes the short-term oscillations.

The loss function looks like this:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

where $U(D)$ is a uniform distribution over the replay memory D; θ^- is the parameters of the frozen target Q-network.

In addition, it is also found to be helpful to clip the error term to be between $[-1, 1]$. (I always get mixed feeling with parameter clipping, as many studies have shown that it works empirically but it makes the math much less pretty. :/)

```

Initialize replay memory  $D$  to capacity  $N$ 
Initialize action-value function  $Q$  with random weights  $\theta$ 
Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
For episode = 1,  $M$  do
    Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
    For  $t = 1, T$  do
        With probability  $\varepsilon$  select a random action  $a_t$ 
        otherwise select  $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$ 
        Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
        Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
        Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
        Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
        Set  $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$ 
        Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  with respect to the network parameters  $\theta$ 
        Every  $C$  steps reset  $\hat{Q} = Q$ 
    End For
End For

```

Fig. 7. Algorithm for DQN with experience replay and occasionally frozen optimization target. The preprocessed sequence is the output of some processes running on the input images of Atari games. Don't worry too much about it; just consider them as input feature vectors. (Image source: Mnih et al. 2015)

There are many extensions of DQN to improve the original design, such as DQN with dueling architecture (Wang et al. 2016) which estimates state-value function $V(s)$ and advantage function $A(s, a)$ with shared network parameters.

Combining TD and MC Learning

In the previous [section](#) on value estimation in TD learning, we only trace one step further down the action chain when calculating the TD target. One can easily extend it to take multiple steps to estimate the return.

Let's label the estimated return following n steps as $G_t^{(n)}, n = 1, \dots, \infty$, then:

n	G_t	Notes
$n = 1$	$G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$	TD learning
$n = 2$	$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$	

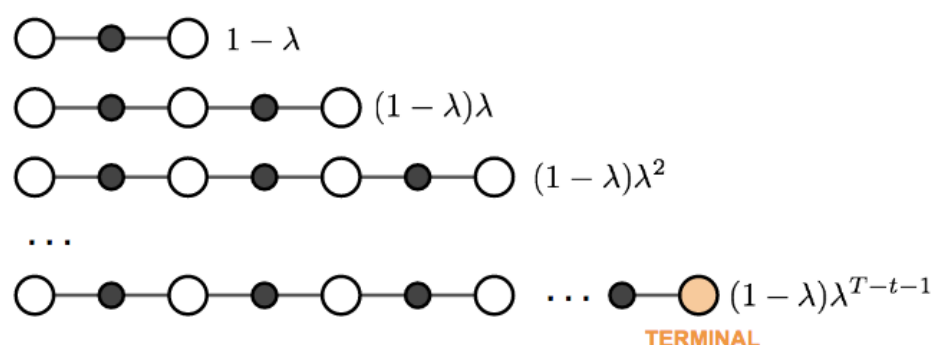


n	G_t	Notes
...		
$n = n$	$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$	
...		
$n = \infty$	$G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T + \gamma^{T-t} V(S_T)$	MC estimation

The generalized n-step TD learning still has the same form for updating the value function:

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t^{(n)} - V(S_t))$$

At time step t , $TD(\lambda)$



We are free to pick any n in TD learning as we like. Now the question becomes what is the best n ? Which $G_t^{(n)}$ gives us the best return approximation? A common yet smart solution is to apply a weighted sum of all possible n -step TD targets rather than to pick a single best n . The weights decay by a factor λ with n , λ^{n-1} ; the intuition is similar to why we want to discount future rewards when computing the return: the more future we look into the less confident we would be. To make all the weight ($n \rightarrow \infty$) sum up to 1, we multiply every weight by $(1-\lambda)$, because:

$$\begin{aligned} \text{let } S &= 1 + \lambda + \lambda^2 + \dots \\ S &= 1 + \lambda(1 + \lambda + \lambda^2 + \dots) \\ S &= 1 + \lambda S \\ S &= 1/(1 - \lambda) \end{aligned}$$

This weighted sum of many n -step returns is called λ -return $G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$. TD learning that adopts λ -return for value updating is labeled as **TD(λ)**. The original version we introduced above is equivalent to **TD(0)**.

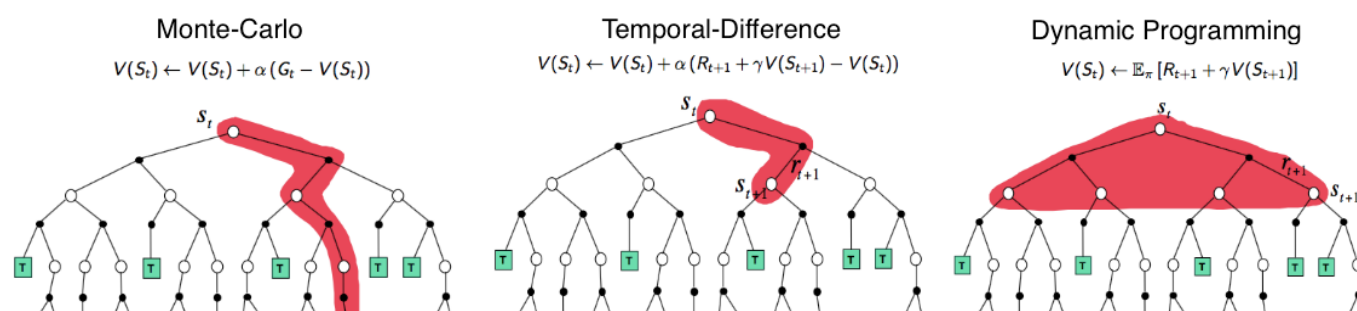




Fig. 8. Comparison of the backup diagrams of Monte-Carlo, Temporal-Difference learning, and Dynamic Programming for state value functions. (Image source: David Silver's RL course [Lecture 4: "Model-Free Prediction"](#))

Policy Gradient

All the methods we have introduced above aim to learn the state/action value function and then to select actions accordingly. Policy Gradient methods instead learn the policy directly with a parameterized function respect to θ , $\pi(a|s; \theta)$. Let's define the reward function (opposite of loss function) as *the expected return* and train the algorithm with the goal to maximize the reward function. My [next post](#) described why the policy gradient theorem works (proof) and introduced a number of policy gradient algorithms.

In discrete space:

$$\mathcal{J}(\theta) = V_{\pi_{\theta}}(S_1) = \mathbb{E}_{\pi_{\theta}}[V_1]$$

where S_1 is the initial starting state.

Or in continuous space:

$$\mathcal{J}(\theta) = \sum_{s \in \mathcal{S}} d_{\pi_{\theta}}(s) V_{\pi_{\theta}}(s) = \sum_{s \in \mathcal{S}} \left(d_{\pi_{\theta}}(s) \sum_{a \in \mathcal{A}} \pi(a|s, \theta) Q_{\pi}(s, a) \right)$$

where $d_{\pi_{\theta}}(s)$ is stationary distribution of Markov chain for π_{θ} . If you are unfamiliar with the definition of a "stationary distribution," please check this [reference](#).

Using *gradient ascent* we can find the best θ that produces the highest return. It is natural to expect policy-based methods are more useful in continuous space, because there is an infinite number of actions and/or states to estimate the values for in continuous space and hence value-based approaches are computationally much more expensive.

Policy Gradient Theorem

Computing the gradient *numerically* can be done by perturbing θ by a small amount ϵ in the k -th dimension. It works even when $J(\theta)$ is not differentiable (nice!), but unsurprisingly very slow.

$$\frac{\partial \mathcal{J}(\theta)}{\partial \theta_k} \approx \frac{\mathcal{J}(\theta + \epsilon u_k) - \mathcal{J}(\theta)}{\epsilon}$$

Or *analytically*,

$$\mathcal{J}(\theta) = \mathbb{E}_{\pi_{\theta}}[r] = \sum_{s \in \mathcal{S}} d_{\pi_{\theta}}(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) R(s, a)$$

Actually we have nice theoretical support for (replacing $d(\cdot)$ with $d_\pi(\cdot)$):

$$\mathcal{J}(\theta) = \sum_{s \in \mathcal{S}} d_{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) Q_\pi(s, a) \propto \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) Q_\pi(s, a)$$

Check Sec 13.1 in Sutton & Barto (2017) for why this is the case.

Then,

$$\begin{aligned} \mathcal{J}(\theta) &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) Q_\pi(s, a) \\ \nabla \mathcal{J}(\theta) &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \nabla \pi(a|s; \theta) Q_\pi(s, a) \\ &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) \frac{\nabla \pi(a|s; \theta)}{\pi(a|s; \theta)} Q_\pi(s, a) \\ &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) \nabla \ln \pi(a|s; \theta) Q_\pi(s, a) \\ &= \mathbb{E}_{\pi_\theta} [\nabla \ln \pi(a|s; \theta) Q_\pi(s, a)] \end{aligned}$$

This result is named "Policy Gradient Theorem" which lays the theoretical foundation for various policy gradient algorithms:

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\pi_\theta} [\nabla \ln \pi(a|s, \theta) Q_\pi(s, a)]$$

REINFORCE

REINFORCE, also known as Monte-Carlo policy gradient, relies on $Q_\pi(s, a)$, an estimated return by MC methods using episode samples, to update the policy parameter θ .

A commonly used variation of REINFORCE is to subtract a baseline value from the return G_t to reduce the variance of gradient estimation while keeping the bias unchanged. For example, a common baseline is state-value, and if applied, we would use $A(s, a) = Q(s, a) - V(s)$ in the gradient ascent update.

1. Initialize θ at random
2. Generate one episode $S_1, A_1, R_2, S_2, A_2, \dots, S_T$
3. For $t=1, 2, \dots, T$:
 1. Estimate the the return G_t since the time step t .
 2. $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla \ln \pi(A_t|S_t, \theta)$.

Actor-Critic

If the value function is learned in addition to the policy, we would get Actor-Critic algorithm.

- **Critic:** updates value function parameters w and depending on the algorithm it could be action-

value $Q(a|s; w)$ or state-value $V(s; w)$.

- **Actor:** updates policy parameters θ , in the direction suggested by the critic, $\pi(a|s; \theta)$.

Let's see how it works in an action-value actor-critic algorithm.

1. Initialize s, θ, w at random; sample $a \sim \pi(a|s; \theta)$.
2. For $t = 1 \dots T$:
 1. Sample reward $r_t \sim R(s, a)$ and next state $s' \sim P(s'|s, a)$.
 2. Then sample the next action $a' \sim \pi(s', a'; \theta)$.
 3. Update policy parameters: $\theta \leftarrow \theta + \alpha_\theta Q(s, a; w) \nabla_\theta \ln \pi(a|s; \theta)$.
 4. Compute the correction for action-value at time t :
$$G_{t:t+1} = r_t + \gamma Q(s', a'; w) - Q(s, a; w)$$
and use it to update value function parameters:
$$w \leftarrow w + \alpha_w G_{t:t+1} \nabla_w Q(s, a; w).$$
 5. Update $a \leftarrow a'$ and $s \leftarrow s'$.

α_θ and α_w are two learning rates for policy and value function parameter updates, respectively.

A3C

Asynchronous Advantage Actor-Critic (Mnih et al., 2016), short for A3C, is a classic policy gradient method with the special focus on parallel training.

In A3C, the critics learn the state-value function, $V(s; w)$, while multiple actors are trained in parallel and get synced with global parameters from time to time. Hence, A3C is good for parallel training by default, i.e. on one machine with multi-core CPU.

The loss function for state-value is to minimize the mean squared error, $\mathcal{J}_v(w) = (G_t - V(s; w))^2$ and we use gradient descent to find the optimal w . This state-value function is used as the baseline in the policy gradient update.

Here is the algorithm outline:

1. We have global parameters, θ and w ; similar thread-specific parameters, θ' and w' .
2. Initialize the time step $t = 1$
3. While $T \leq T_MAX$:
 1. Reset gradient: $d\theta = 0$ and $dw = 0$.
 2. Synchronize thread-specific parameters with global ones: $\theta' = \theta$ and $w' = w$.
 3. $t_{start} = t$ and get s_t .
 4. While $(s_t \neq \text{TERMINAL})$ and $(t - t_{start} \leq t_{max})$:



1. Pick the action $a_t \sim \pi(a_t|s_t; \theta')$ and receive a new reward r_t and a new state s_{t+1} .
2. Update $t = t + 1$ and $T = T + 1$.

5. Initialize the variable that holds the return estimation

$$R = \begin{cases} 0 & \text{if } s_t \text{ is TERMINAL} \\ V(s_t; w') & \text{otherwise} \end{cases}$$

.

6. For $i = t - 1, \dots, t_{\text{start}}$:

1. $R \leftarrow r_i + \gamma R$; here R is a MC measure of G_i .
2. Accumulate gradients w.r.t. θ' : $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i; \theta')(R - V(s_i; w'))$;
Accumulate gradients w.r.t. w' : $dw \leftarrow dw + \nabla_{w'}(R - V(s_i; w'))^2$.

7. Update synchronously θ using $d\theta$, and w using dw .

A3C enables the parallelism in multiple agent training. The gradient accumulation step (6.2) can be considered as a reformation of minibatch-based stochastic gradient update: the values of w or θ get corrected by a little bit in the direction of each training thread independently.

Evolution Strategies

Evolution Strategies (ES) is a type of model-agnostic optimization approach. It learns the optimal solution by imitating Darwin's theory of the evolution of species by natural selection. Two prerequisites for applying ES: (1) our solutions can freely interact with the environment and see whether they can solve the problem; (2) we are able to compute a **fitness** score of how good each solution is. We don't have to know the environment configuration to solve the problem.

Say, we start with a population of random solutions. All of them are capable of interacting with the environment and only candidates with high fitness scores can survive (*only the fittest can survive in a competition for limited resources*). A new generation is then created by recombining the settings (*gene mutation*) of high-fitness survivors. This process is repeated until the new solutions are good enough.

Very different from the popular MDP-based approaches as what we have introduced above, ES aims to learn the policy parameter θ without value approximation. Let's assume the distribution over the parameter θ is an isotropic multivariate Gaussian with mean μ and fixed covariance $\sigma^2 I$. The gradient of $F(\theta)$ is calculated:



$$\begin{aligned}
& \nabla_{\theta} \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} F(\theta) \\
&= \nabla_{\theta} \int_{\theta} F(\theta) \Pr(\theta) \\
&= \int_{\theta} F(\theta) \Pr(\theta) \frac{\nabla_{\theta} \Pr(\theta)}{\Pr(\theta)} \\
&= \int_{\theta} F(\theta) \Pr(\theta) \nabla_{\theta} \log \Pr(\theta) \\
&= \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} [F(\theta) \nabla_{\theta} \log \Pr(\theta)] \\
&= \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} \left[F(\theta) \nabla_{\theta} \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(\theta-\mu)^2}{2\sigma^2}} \right) \right] \\
&= \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} \left[F(\theta) \nabla_{\theta} \left(-\log \sqrt{2\pi\sigma^2} - \frac{(\theta-\mu)^2}{2\sigma^2} \right) \right] \\
&= \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} \left[F(\theta) \frac{\theta - \mu}{\sigma^2} \right]
\end{aligned}$$

$\Pr(\cdot)$ is the Gaussian density function.

Similar to how we do policy gradient update.

We can rewrite this formula in terms of a “mean” parameter θ (different from the θ above; this θ is the base gene for further mutation), $\epsilon \sim N(0, I)$ and therefore $\theta + \epsilon \sigma \sim N(\theta, \sigma^2)$. ϵ controls how much Gaussian noises should be added to create mutation:

$$\nabla_{\theta} \mathbb{E}_{\epsilon \sim N(0, I)} F(\theta + \sigma \epsilon) = \frac{1}{\sigma} \mathbb{E}_{\epsilon \sim N(0, I)} [F(\theta + \sigma \epsilon) \epsilon]$$

Algorithm 2 Parallelized Evolution Strategies

```

1: Input: Learning rate  $\alpha$ , noise standard deviation  $\sigma$ , initial policy parameters  $\theta_0$ 
2: Initialize:  $n$  workers with known random seeds, and initial parameters  $\theta_0$ 
3: for  $t = 0, 1, 2, \dots$  do
4:   for each worker  $i = 1, \dots, n$  do
5:     Sample  $\epsilon_i \sim \mathcal{N}(0, I)$ 
6:     Compute returns  $F_i = F(\theta_t + \sigma \epsilon_i)$ 
7:   end for
8:   Send all scalar returns  $F_i$  from each worker to every other worker
9:   for each worker  $i = 1, \dots, n$  do
10:    Reconstruct all perturbations  $\epsilon_j$  for  $j = 1, \dots, n$  using known random seeds
11:    Set  $\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{n\sigma} \sum_{j=1}^n F_j \epsilon_j$ 
12:   end for
13: end for

```

Fig. 9. A simple parallel evolution-strategies-based RL algorithm. Parallel workers share the random seeds so that they can reconstruct the Gaussian noises with tiny communication bandwidth. (Image source: Salimans et al. 2017.)

ES, as a black-box optimization algorithm, is another approach to RL problems (*In my original writing, I used the phrase “a nice alternative”; Seita pointed me to this discussion and thus I updated my wording.*). It has a couple of good characteristics (Salimans et al., 2017) keeping it fast and easy to train:

- ES does not need value function approximation;



- ES does not perform gradient back-propagation;
- ES is invariant to delayed or long-term rewards;
- ES is highly parallelizable with very little data communication.

Known Problems

Exploration-Exploitation Dilemma

The problem of exploration vs exploitation dilemma has been discussed in my previous [post](#). When the RL problem faces an unknown environment, this issue is especially a key to finding a good solution: without enough exploration, we cannot learn the environment well enough; without enough exploitation, we cannot complete our reward optimization task.

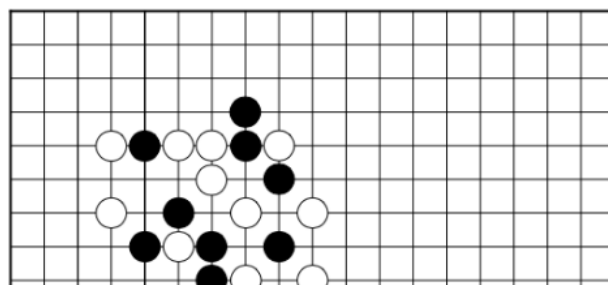
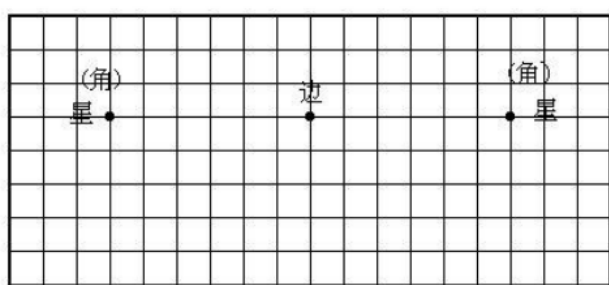
Different RL algorithms balance between exploration and exploitation in different ways. In [MC](#) methods, [Q-learning](#) or many on-policy algorithms, the exploration is commonly implemented by ϵ -greedy; In [ES](#), the exploration is captured by the policy parameter perturbation. Please keep this into consideration when developing a new RL algorithm.

Deadly Triad Issue

We do seek the efficiency and flexibility of TD methods that involve bootstrapping. However, when off-policy, nonlinear function approximation, and bootstrapping are combined in one RL algorithm, the training could be unstable and hard to converge. This issue is known as the **deadly triad** (Sutton & Barto, 2017). Many architectures using deep learning models were proposed to resolve the problem, including DQN to stabilize the training with experience replay and occasionally frozen target network.

Case Study: AlphaGo Zero

The game of [Go](#) has been an extremely hard problem in the field of Artificial Intelligence for decades until recent years. AlphaGo and AlphaGo Zero are two programs developed by a team at DeepMind. Both involve deep Convolutional Neural Networks ([CNN](#)) and Monte Carlo Tree Search (MCTS) and both have been approved to achieve the level of professional human Go players. Different from AlphaGo that relied on supervised learning from expert human moves, AlphaGo Zero used only reinforcement learning and self-play without human knowledge beyond the basic rules.



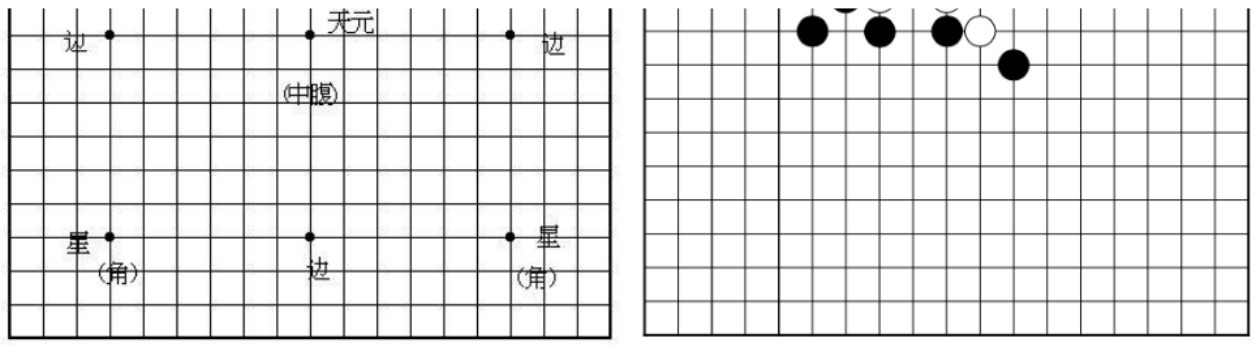


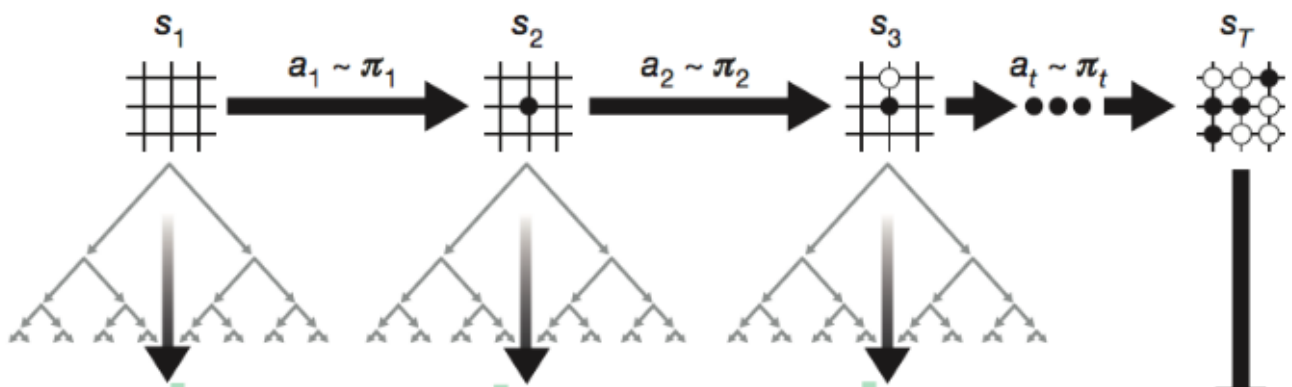
Fig. 10. The board of Go. Two players play black and white stones alternatively on the vacant intersections of a board with 19 x 19 lines. A group of stones must have at least one open point (an intersection, called a "liberty") to remain on the board and must have at least two or more enclosed liberties (called "eyes") to stay "alive". No stone shall repeat a previous position.

With all the knowledge of RL above, let's take a look at how AlphaGo Zero works. The main component is a deep CNN over the game board configuration (precisely, a ResNet with batch normalization and ReLU). This network outputs two values:

$$(p, v) = f_{\theta}(s)$$

- s : the game board configuration, 19 x 19 x 17 stacked feature planes; 17 features for each position, 8 past configurations (including current) for the current player + 8 past configurations for the opponent + 1 feature indicating the color (1=black, 0=white). We need to code the color specifically because the network is playing with itself and the colors of current player and opponents are switching between steps.
- p : the probability of selecting a move over $19^2 + 1$ candidates (19^2 positions on the board, in addition to passing).
- v : the winning probability given the current setting.

During self-play, MCTS further improves the action probability distribution $\pi \sim p(\cdot)$ and then the action a_t is sampled from this improved policy. The reward z_t is a binary value indicating whether the current player *eventually* wins the game. Each move generates an episode tuple (s_t, π_t, z_t) and it is saved into the replay memory. The details on MCTS are skipped for the sake of space in this post; please read the original [paper](#) if you are interested.



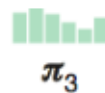
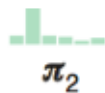


Fig. 11. AlphaGo Zero is trained by self-play while MCTS improves the output policy further in every step. (Image source: Figure 1a in Silver et al., 2017).

The network is trained with the samples in the replay memory to minimize the loss:

$$\mathcal{L} = (z - v)^2 - \pi^\top \log p + c \|\theta\|^2$$

where c is a hyperparameter controlling the intensity of L2 penalty to avoid overfitting.

AlphaGo Zero simplified AlphaGo by removing supervised learning and merging separated policy and value networks into one. It turns out that AlphaGo Zero achieved largely improved performance with a much shorter training time! I strongly recommend reading these [two papers](#) side by side and compare the difference, super fun.

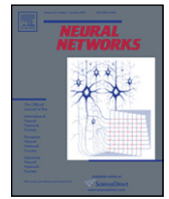
I know this is a long read, but hopefully worth it. *If you notice mistakes and errors in this post, don't hesitate to contact me at [lilian dot wengweng at gmail dot com].* See you in the next post! :)

Cited as:

```
@article{weng2018bandit,  
  title   = "A (Long) Peek into Reinforcement Learning",  
  author  = "Weng, Lilian",  
  journal = "lilianweng.github.io",  
  year    = "2018",  
  url     = "https://lilianweng.github.io/posts/2018-02-19-rl-overview/"  
}
```

References

- [1] Yuxi Li. [Deep reinforcement learning: An overview](#). arXiv preprint arXiv:1701.07274. 2017.
- [2] Richard S. Sutton and Andrew G. Barto. [Reinforcement Learning: An Introduction; 2nd Edition](#). 2017.
- [3] Volodymyr Mnih, et al. [Asynchronous methods for deep reinforcement learning](#). ICML. 2016.
- [4] Tim Salimans, et al. [Evolution strategies as a scalable alternative to reinforcement learning](#). arXiv preprint arXiv:1703.03864 (2017).
- [5] David Silver, et al. [Mastering the game of go without human knowledge](#). Nature 550.7676 (2017):



Real-time reinforcement learning by sequential Actor–Critics and experience replay

Paweł Wawrzyński

Warsaw University of Technology, Institute of Control and Computation Engineering, Nowowiejska 15/19, 00-665 Warsaw, Poland

ARTICLE INFO

Article history:

Received 4 December 2008

Revised and accepted 25 May 2009

Keywords:

Direct adaptive control

Machine learning

Reinforcement learning

Experience replay

ABSTRACT

Actor–Critics constitute an important class of reinforcement learning algorithms that can deal with continuous actions and states in an easy and natural way. This paper shows how these algorithms can be augmented by the technique of experience replay without degrading their convergence properties, by appropriately estimating the policy change direction. This is achieved by truncated importance sampling applied to the recorded past experiences. It is formally shown that the resulting estimation bias is bounded and asymptotically vanishes, which allows the experience replay-augmented algorithm to preserve the convergence properties of the original algorithm. The technique of experience replay makes it possible to utilize the available computational power to reduce the required number of interactions with the environment considerably, which is essential for real-world applications. Experimental results are presented that demonstrate that the combination of experience replay and Actor–Critics yields extremely fast learning algorithms that achieve successful policies for non-trivial control tasks in considerably short time. Namely, the policies for the cart-pole swing-up [Doya, K. (2000). Reinforcement learning in continuous time and space. *Neural Computation*, 12(1), 219–245] are obtained after as little as 20 min of the cart-pole time and the policy for Half-Cheetah (a walking 6-degree-of-freedom robot) is obtained after four hours of Half-Cheetah time.

© 2009 Elsevier Ltd. All rights reserved.

1. Introduction

Reinforcement learning (RL) addresses the problem of an agent that optimizes its reactive policy in a poorly structured and initially unknown environment (Sutton & Barto, 1998). Algorithms developed in this area can be viewed as computational processes that transform observations of states, actions and rewards into policy parameters. Several important RL algorithms, such as Q-Learning (Watkins & Dayan, 1992) and Actor–Critic methods (Barto, Sutton, & Anderson, 1983; Bhatnagar, Sutton, Ghavamzadeh, & Lee, 2008; Kimura & Kobayashi, 1998; Konda & Tsitsiklis, 2003), process the data sequentially. Each single observation is used for adjusting the algorithms' parameters and then becomes unavailable for further use. We shall call such methods *sequential*. They are based on a common assumption that RL applications to real-world learning control problems require large amounts of data which cannot be kept in a limited amount of memory assigned to the algorithm.

Sequential algorithms do not exploit all the information contained in the data and are known to require a large number of environment steps to obtain a satisfactory policy. Usually, this number is large enough to make the learning process detrimental

to any real device whose control policy we would hope to optimize by means of RL. However, there are other, *non-sequential* methods that require much fewer environment steps to obtain a policy of the same quality. They achieve this at the cost of collecting data and some extensive processing thereof. This distinction between sequential and non-sequential algorithms should not be confused with the distinction between online and offline algorithms. Here, we are interested *only* in online algorithms which improve the policy as the agent–environment interaction proceeds. Online sequential and non-sequential algorithms differ in the way of using the available computational power during the interaction.

The most obvious way to design the computations for a non-sequential algorithm is to estimate the model of the environment and use it in some variant of dynamic programming. This idea appears to have been applied for the first time in the DYNA architecture (Sutton, 1990). Further research in this area has brought the family of E^3 algorithms (Abbeel & Ng, 2005) applicable not only to finite-space problems but also to those with linear models. Recent work extends this methodology to problems without regular structure (Abbeel, Quigley, & Ng, 2006).

Another possible approach to non-sequential algorithms can be summarized as follows. Assume there exists a quality index $J : \mathcal{R}^{n_\theta} \mapsto \mathcal{R}$ that assigns to each policy parameter $\theta \in \mathcal{R}^{n_\theta}$ a value $J(\theta)$ that evaluates the policy based on θ . Let us denote by $\hat{J}_t(\theta)$ an estimator of $J(\theta)$ based on the course of the Markov Decision

E-mail addresses: p.wawrzyński@elka.pw.edu.pl, p.wawrzyński@gmail.com.

Process (MDP) before time t . The objective of data processing can be to optimize the policy by maximizing $\hat{J}_t(\theta)$ with respect to θ . Peshkin and Shelton (2002) and Shelton (2001) present methods of policy optimization in Partially Observable Markov Decision Processes that are based on the history of agent–environment interactions and importance sampling estimators. Properties of estimators and bounds for the uniform convergence of estimates are discussed by Peshkin and Mukherjee (2001) and a way is suggested of using these bounds to select a policy class among candidate classes of varying complexity, similar to structural risk minimization.

In the third approach, a non-sequential algorithm is constructed by adding some non-sequential data processing to a sequential algorithm. It collects historical experiences (observations of states, actions and rewards) and applies to them the operations of the original sequential algorithm as if they have just taken place. This idea of repeating many similar operations to the same event, called *experience replay* (Cichosz, 1999; Lin, 1992; Lin & Mitchell, 1992; Mahadevan & Connell, 1992), was popular a few years ago but has received little attention recently. Unfortunately, experience replay is not automatically applicable to an arbitrary RL algorithm. In particular, it cannot be directly combined with on-policy methods, i.e., those based on the assumption that the actions producing data for policy improvements are drawn from the current policy. Consequently, the same experience cannot be applied many times to adjust a continuously changing policy.

The issue of policy optimization on the basis of past experience can be reduced to the problem of estimation of the expected value of a random vector with one distribution using samples from different distribution(s). That can be done with the use of the importance sampling (IS) estimators (Rubinstein, 1981). Various types of IS estimators have been applied in RL several times. Shelton (2001) and Peshkin and Shelton (2002) used them to define an estimator of a quality index of a given policy; and Frank, Mannor, and Precup (2008) Precup, Sutton, and Dasgupta (2001) applied IS estimators to determine a value function approximation of a given policy on the basis of realization of a decision process driven by another policy; Uchibe and Doya (2004) applied a certain truncated IS estimator to optimize several policies of various complexities on the basis of a single decision process realization.

1.1. Contribution

In this paper we show how to apply experience replay to a family of RL algorithms called Actor–Critics (Barto et al., 1983; Kimura & Kobayashi, 1998; Konda & Tsitsiklis, 2003). These are on-policy algorithms that have proved successful in a number of tasks, particularly with continuous actions which cannot be directly dealt with by the more popular Q-learning and related algorithms. The combination of Actor–Critics and experience replay has been attempted before with limited success (Lin, 1992), probably due to the above-mentioned difficulty resulting from the discrepancy between the current policy and the policy that generated a past experience. To overcome this difficulty, a novel variant of the experience replay technique is proposed in which the policy adjustments based on the data coming from a different policy are reconstructed to compensate for the discrepancy. The contribution of this paper may be summarized in the following points:

- It is shown here how to apply experience replay to any sequential Actor–Critic algorithm in such a way that (i) convergence properties and robustness of the algorithm are preserved and (ii) the speed of learning of the algorithm measured in terms of length of agent–environment interaction is increased, at the cost of additional background computations.
 - The methodology presented here is based on improvement direction estimators computed with the use of data on past experience collected in a database. The randomized-truncated importance sampling estimators are introduced here for that purpose. Their properties are proved: bounded variance and asymptotic unbiasedness.
 - An experimental study is presented that demonstrates the application of the introduced methodology to challenging control problems, namely the cart–pole swing-up task (Doya, 2000) and Half-Cheetah (Wawrzyński, 2007).
- The idea presented in this paper is partially inspired by the significant evidence in neuropsychology suggesting that the human brain does not learn “sequentially”. Instead, it stores extracts of percepts in the short-time memory and uses them in a certain process that updates synaptic weights. Those results suggest that RL algorithms should be “non-sequential”, or “batch”, to be efficient. The design we propose supports this intuition, which appears to be somewhat neglected in the RL literature.
- The paper is organized as follows. In Section 2 the problem of our interest is defined along with the class of algorithms that encompasses sequential Actor–Critics. It is shown that a given Actor–Critic method defines certain improvement directions of its parameter vectors. Section 3 shows how to estimate these directions using the data from the preceding state transition and to accelerate a sequential algorithm by combining these estimators with experience replay. Conditions for asymptotic unbiasedness of these estimators are established in Section 4 that enable the algorithm with experience replay to inherit the limit properties of the original sequential method. The experimental study on the introduced methodology is presented in Section 5, and Section 6 concludes.

2. Problem formulation

We will consider the standard RL setup (Sutton & Barto, 1998). A Markov Decision Process (MDP) defines a problem of an agent that observes its state s_t in discrete time $t = 1, 2, 3, \dots$, performs actions a_t , receives rewards r_t and moves to other states s_{t+1} . A particular MDP is a tuple $\langle \mathcal{S}, \mathcal{A}, P_s, r \rangle$, where $\mathcal{S}$ and $\mathcal{A}$ are the state and action spaces, respectively; $\{P_s(\cdot|s, a) : s \in \mathcal{S}, a \in \mathcal{A}\}$ is a set of state transition distributions; we write $s_{t+1} \sim P_s(\cdot|s_t, a_t)$ and assume that each P_s is a density. Each state transition generates a reward, $r_t \in \mathcal{R}$. Here we assume that each reward depends deterministically on the current action and the next state, $r_t = r(a_t, s_{t+1})$.

Actions are generated according to a policy, π , which is a family of distributions parameterized by the state and a policy vector $\theta \in \mathcal{R}^{n_\theta}$, namely $a_t \sim \pi(\cdot|s_t, \theta)$. The objective of reinforcement learning is, in general, to make the policy maximize future rewards by optimizing θ . The strict meaning of this goal may be specified in various ways. We may require the policy to maximize the average reward or to maximize the sum of future discounted rewards expected in each state.

We are interested in applications of the MDP framework to learning control tasks. We thus require the learning algorithm to be fast, i.e., to obtain a satisfactory policy after as few agent steps as possible. The control efficiency should be maximized within a short period of learning to keep the controlled machine from being damaged by too many wrong actions. We assume the availability of a limited database that keeps a recent part of the agent–environment interaction history. This history is available between consecutive agent steps for a limited number of operations aiming at policy optimization.

Below we analyze a large class of the existing sequential RL methods suitable for policy determination in simulations and propose a way of their acceleration based on a more extensive data processing. Our intention is to design methods that obtain a satisfactory policy after a much smaller amount of agent time but not necessarily after a smaller amount of computation.

2.1. Sequential Actor–Critic algorithms

Actor–Critics constitute probably the most efficient and the most theoretically developed class of reinforcement learning algorithms. The actor–critic structure alone appears to have been presented for the first time by Barto et al. (1983). An important development was made by Gullapalli (1990), who introduced the actor adjustment scheme based on expectation gradient estimators computed with the use of likelihood ratios. These estimators were invented in the generic form earlier by Williams (1989). Efficient actor and critic training requires so-called TD(λ)-estimators of future rewards. These were introduced into the critic training by Sutton (1988) and into the actor training by Kimura and Kobayashi (1998). The first step toward a proof of convergence of an approximation-based actor–critic algorithm

Algorithm 1 The Basic Actor–Critic. $\gamma \in (0, 1)$ is a discount factor, $\lambda \in [0, 1]$, $\bar{V}$ is the value function approximator (the critic) parameterized by vector v . β_t^θ and β_t^v are the step-sizes.

```

0: Set  $y_v = 0$ ,  $y_\theta = 0$ ,  $t := 1$ . Initialize  $\theta$  and  $v$ .
1: Draw the action,  $a_t \sim \pi(\cdot; s_t, \theta)$ .
2: Execute  $a_t$ , evaluate the next state  $s_{t+1}$  and the reward  $r_t$ .
3: Calculate the temporal difference of the form
4:    $d_t(v) = r_t + \gamma \bar{V}(s_{t+1}; v) - \bar{V}(s_t; v)$ .
5: Adjust the actor:
6:    $y_\theta := (\gamma \lambda) y_\theta + \beta_t^\theta \nabla_\theta \ln \pi(a_t; s_t, \theta)$ 
7:    $\theta := \theta + y_\theta d_t(v)$ 
8: Adjust the critic:
9:    $y_v := (\gamma \lambda) y_v + \beta_t^v \nabla_v \bar{V}(s_t; v)$ 
10:   $v := v + y_v d_t(v)$ .
11: Set  $t := t + 1$  and repeat from Point 1.

```

was made by Sutton, McAllester, Singh, and Mansour (2000), who designed a critic that was in a special way compatible with the actor. The proof of convergence of an approximation-based algorithm with the actor–critic structure was given by Konda and Tsitsiklis (2003). Up to that time, actor–critic algorithms had been adjusting the policy vector along the policy gradient estimators. However, Peters and Schaal (2008) and Peters, Vijayakumar, and Schaal (2003) introduced a very efficient actor training scheme based on adjusting the policy vector along estimators of the natural policy gradient (Kakade, 2002). The original algorithm presented by Peters et al. (2003) is not purely sequential as it trains the critic with the use of LSTD-Q(λ) (Nedić & Bertsekas, 2003) which is a method of batch mean-square approximation of the action-value function. However, Bhatnagar et al. (2008) and Bhatnagar, Sutton, Ghavamzadeh, and Lee (in press) introduced algorithms based on natural policy gradient and fully sequential Actor and Critic adjusting.

2.1.1. The Basic Actor–Critic

Let us analyze the first example of methods that will be of interest for us: the algorithm presented by Kimura and Kobayashi (1998), called here the Basic Actor–Critic (Algorithm 1). The actor is represented here, as usual, by the parameterized policy π . The critic is represented by the approximator $\bar{V}(s; v)$ parameterized by the critic vector $v \in \mathbb{R}^n$. The critic approximates the value function V^π , that is for a given policy, π , equal to the sum of future discounted rewards expected in a given state, namely

$$V^\pi(s) = E \left(\sum_{i \geq 0} \gamma^i r_{t+i} | s_t = s, \pi \right)$$

where $\gamma \in (0, 1)$ is a discount factor. The values β_t^θ and β_t^v are the step-sizes: they are positive reals decreasing with growing t .

Also, they should satisfy the standard stochastic approximation conditions (Kushner & Yin, 1997), namely

$$\sum_{t \geq 1} \beta_t = \infty, \quad \sum_{t \geq 1} \beta_t^2 < \infty.$$

Below, we show how this algorithm works: After each instant t , it performs a sequence of adjustments of θ and v that modify these vectors along estimators of certain quality indexes associated with instant t . Before we explain the mechanics of Actor and Critic training, we have to present the fundamental concepts of TD(λ) Sutton (1988) and incremental expectation maximization based on likelihood ratio (Williams, 1989).

2.1.1.1. TD(λ)-estimators of future rewards. Let us consider the following sum:

$$R_t^k = r_t + \gamma r_{t+1} + \dots + \gamma^{k-1} r_{t+k} + \gamma^k \bar{V}(s_{t+k+1}).$$

We can see that this is an estimator of the expected value of the sum of discounted rewards when starting from state s_t with action a_t and then following the current policy. The larger k , the smaller is the bias of the estimator (the less it relies on the non-precise approximator $\bar{V}$) and the larger is its variance. Then, we define

$$R_t^{(\lambda)} = \begin{cases} \sum_{k \geq 0} (1 - \lambda) \lambda^k R_t^k & \text{for } \lambda \in [0, 1) \\ \lim_{\lambda \rightarrow 1} \sum_{k \geq 0} (1 - \lambda) \lambda^k R_t^k = R_t^\infty & \text{for } \lambda = 1. \end{cases}$$

Hence, $R_t^{(\lambda)}$ is a weighted average of R_t^k . For small λ it weights more those R_t^k with small k (more relying on the approximation $\bar{V}$) and for large λ , it relies more on those R_t^k with large k (more relying on rewards actually received). One can verify the equalities (Sutton, 1988)

$$R_t^{(\lambda)} = r_t + \bar{V}(s_{t+1}; v) + \sum_{k \geq 1} (\gamma \lambda)^k \times (r_{t+i} + \gamma \bar{V}(s_{t+i+1}; v) - \bar{V}(s_{t+i}; v)) \quad (1)$$

$$R_t^{(\lambda)} - \bar{V}(s_t; v) = \sum_{k \geq 0} (\gamma \lambda)^k (r_{t+i} + \gamma \bar{V}(s_{t+i+1}; v) - \bar{V}(s_{t+i}; v)) \quad (2)$$

which express $R_t^{(\lambda)}$ as a sum of diminishing elements and play a crucial role in the Actor–Critic design.

2.1.1.2. Incremental expectation maximization with the use of likelihood ratio. Suppose $R_t^{(\lambda)}$ is influenced by θ only through a_t . Then, we could exploit the following equation (Williams, 1989) to adjust θ in order to maximize the expected $R_t^{(\lambda)}$:

$$\nabla_\theta E_\theta (R_t^{(\lambda)}) = E_\theta (\nabla_\theta \ln \pi(a_t; s, \theta) (R_t^{(\lambda)} - c)) \quad (3)$$

where $c \in \mathbb{R}$ is a non-random baseline and $\nabla_\theta \ln \pi$ is the likelihood ratio. Eq. (3) states that the term $\nabla_\theta \ln \pi(a_t; s, \theta) (R_t^{(\lambda)} - c)$ is an unbiased estimator of the gradient of the quality index $E_\theta R_t^{(\lambda)}$. In particular, for $c = \bar{V}(s_t; v)$, we obtain a random vector

$$\nabla_\theta \ln \pi(a_t; s, \theta) (R_t^{(\lambda)} - \bar{V}(s_t; v)) \quad (4)$$

which is an unbiased estimator of the gradient of the expected $R_t^{(\lambda)}$.

2.1.1.3. The actor training. Let us consider the total adjustment of the policy vector θ during the work of the algorithm. To this end, we analyze the value of y_θ by the end of the algorithm's loop. Since y_θ results from summation in Point 6, at instant t it becomes equal to

$$y_\theta = \sum_{k=0}^{t-1} (\gamma\lambda)^k \beta_{t-k}^\theta \nabla_\theta \ln \pi(a_{t-k}; s_{t-k}, \theta).$$

Therefore, the total adjustment is equal to

$$\Delta\theta = \sum_{t>0} d_t(v) \sum_{k=0}^{t-1} (\gamma\lambda)^k \beta_{t-k}^\theta \nabla_\theta \ln \pi(a_{t-k}; s_{t-k}, \theta).$$

By changing the summation order we obtain

$$\Delta\theta = \sum_{t>0} \beta_t^\theta \nabla_\theta \ln \pi(a_t; s_t, \theta) \sum_{k \geq 0} (\gamma\lambda)^k d_{t+k}(v). \quad (5)$$

We can see that an element of the above sum, namely

$$\beta_t^\theta \nabla_\theta \ln \pi(a_t; s_t, \theta) \sum_{k \geq 0} (\gamma\lambda)^k d_{t+k}(v), \quad (6)$$

attributes to the state s_t the total adjustment of the policy vector that the visit in this state induces. By combining Eqs. (2), (4) and (6) we can see that the total adjustment induced at time t is equal to

$$\beta_t^\theta \nabla_\theta \ln \pi(a_t; s_t, \theta) \left(R_t^{(\lambda)} - \bar{V}(s_t; v) \right),$$

that is, a product of the step-size and an estimator of the gradient of the expected value of $R_t^{(\lambda)}$. Therefore, what the algorithm is trying to do is maximization of the sum of future discounted rewards expected in each state.

2.1.1.4. The critic training. A similar analysis reveals the compact form of the total adjustment of the critic vector. Namely, at the end of the algorithm's loop, the vector y_v is equal to

$$y_v = \sum_{k=0}^{t-1} (\gamma\lambda)^k \nabla_v \bar{V}(s_{t-k}; v).$$

Therefore, the total adjustment of the critic vector is equal to

$$\Delta v = \sum_{t>0} \beta_t^v d_t(v) \sum_{k=0}^{t-1} (\gamma\lambda)^k \nabla_v \bar{V}(s_{t-k}; v).$$

By changing the summation order we obtain

$$\Delta v = \sum_{t>0} \beta_t^v \nabla_v \bar{V}(s_t; v) \sum_{k \geq 0} (\gamma\lambda)^k d_{t+k}(v).$$

From Eq. (2) we also know that

$$\Delta v = \sum_{t>0} \beta_t^v \nabla_v \bar{V}(s_t; v) \left(R_t^{(\lambda)} - \bar{V}(s_t; v) \right).$$

We can see that an element of the above sum attributes to the state s_t the total adjustment of the critic vector that the visit in this state induces. In order to understand the character of this adjustment, suppose for a while that $R_t^{(\lambda)}$ does not depend on v . Then, it is easy to verify that

$$\nabla_v E_v \frac{1}{2} \left(R_t^{(\lambda)} - \bar{V}(s_t; v) \right)^2 = E_v \nabla_v \bar{V}(s_t; v) \left(\bar{V}(s_t; v) - R_t^{(\lambda)} \right).$$

Hence, the critic training consists in minimization of the quality index

$$E_v \frac{1}{2} \left(R_t^{(\lambda)} - \bar{V}(s_t; v) \right)^2$$

that represents the mean quadratic discrepancy between $\bar{V}(s_t; v)$ and $R_t^{(\lambda)}$. The minimization is performed by adjusting the critic vector along estimators of the gradient of this quality index.

2.1.2. The Natural Actor–Critic

Another example of methods that will be of interest for us is quoted in Algorithm 2. The algorithm is called, for the purposes of this paper, the Natural Actor–Critic. It is based on actor adjustments along the natural policy gradient estimators [Peters and Schaal \(2008\)](#), but is fully sequential. It is similar to Algorithm 3 of [Bhatnagar et al. \(2008\)](#) with some minor modifications: (i) it maximizes discounted rewards rather than the average reward, (ii) it is based on TD(λ)-estimators of future rewards rather than TD(0)-estimators, (iii) the algorithm applies “forgetting” of the critic vector (v'' , Step 10) in a slightly simplified way: it avoids computing large matrices, (iv) the critic here need not be linear.

Algorithm 2 The Natural Actor–Critic. $\gamma \in (0, 1)$ is a discount factor, $\lambda \in [0, 1]$, $\bar{Q}$ is the action-value function approximator (the critic) parameterized by vectors v and θ . β_t^θ and β_t^v are the step-sizes.

- 0: Set $y_v = 0$, $t := 1$. Initialize θ and v . Draw $a_1 \sim \pi(\cdot; s_1, \theta)$.
- 1: Execute a_t , evaluate the next state s_{t+1} and the reward r_t .
- 2: Draw the next action, $a_{t+1} \sim \pi(\cdot; s_{t+1}, \theta)$.
- 3: Calculate the temporal difference of the form
- 4: $e_t(\theta, v) = r_t + \gamma \bar{V}(a_{t+1}, s_{t+1}; v') - \bar{Q}(a_t, s_t; v, \theta)$
- 5: Adjust the critic:
- 6: $y_v := (\gamma\lambda)y_v + \beta_t^v \nabla_v \bar{Q}(a_t, s_t; v, \theta)$
- 7: $v := v + y_v e_t(\theta, v)$
- 8: Adjust the actor:
- 9: $\theta := \theta + \beta_t^\theta v''$
- 10: $v'' := (1 - \beta_t^\theta) v''$
- 11: Set $t := t + 1$ and repeat from Step 1.

The actor is represented, as usual, by the parameterized policy π . The critic is represented by the approximator $\bar{Q}(a; s; v, \theta)$ parameterized by $v = \begin{bmatrix} v' \\ v'' \end{bmatrix} \in \mathbb{R}^{n_v}$, and the policy vector θ . The critic approximates the action-value function Q^π , that is for a given policy, π , equal to the sum of future discounted rewards expected in a given state, provided the given initial action, namely

$$\bar{Q}(a; s; v, \theta) \approx Q^\pi(a, s) = E \left(\sum_{i \geq 0} \gamma^i r_{t+i} \mid s_t = s, a_t = a, \pi \right).$$

The approximator $\bar{Q}$ is of the form

$$\bar{Q}(a; s; v, \theta) = \bar{V}(s; v') + \nabla_\theta^\top \ln \pi(a; s, \theta) v'' \quad (7)$$

where $\bar{V}$ is the value-function approximator linearly parameterized in v' . Because the critic is here partially defined by the actor, π , they are called *compatible* actor and critic.

The values β_t^θ and β_t^v are the step-sizes. They satisfy the standard stochastic approximation conditions and, furthermore, there exists $d > 0$ such that

$$\sum_{t>0} \left(\frac{\beta_t^\theta}{\beta_t^v} \right)^d < \infty.$$

This means that β_t^θ decreases “faster” than β_t^v ; see [Konda and Tsitsiklis \(2003\)](#).

The scheme of the critic training in the Natural Actor–Critic remains the same as in the basic algorithm. The only difference lies in the subject of training: it is the action-value function approximator rather than the value function approximator. A similar argument to the one applied to the Basic Actor–Critic shows that the present algorithm minimizes the mean-quadratic

discrepancy between $\bar{Q}(a_t, s_t; v, \theta)$ and $\bar{R}_t^{(\lambda)}$, where $\bar{R}_t^{(\lambda)}$ is defined as

$$\bar{R}_t^{(\lambda)} = r_t + \gamma \bar{V}(s_{t+1}; v) + \sum_{i \geq 1} (\gamma \lambda)^i (r_{t+i} + \gamma \bar{V}(s_{t+i+1}; v) - \bar{Q}(a_{t+i}, s_{t+i}; v, \theta)).$$

See Eq. (1) to compare $\bar{R}_t^{(\lambda)}$ with $\bar{R}_t^{(\lambda)}$.

The actor training is designed to make the policy parameter θ follow the direction of the natural policy gradient (Kakade, 2002). Below we provide some intuition why the actor training in this algorithm may work. For more detailed discussion we refer an interested reader to Peters and Schaal (2008). Suppose $\bar{Q}$ is a good approximation of the true action-value function. What happens with the value $\bar{Q}(a, s; v, \theta)$ expected in a given state s when we adjust the actor vector by $\delta v''$ for a small $\delta > 0$?

$$\begin{aligned} E_{s, \theta + \delta v''} \bar{Q}(a, s; v, \theta) &= \int (\bar{V}(s; v') + \nabla_{\theta}^T \ln \pi(a; s, \theta) v'') \pi(a; s, \theta + \delta v'') da \\ &\approx \int (\bar{V}(s; v') + \nabla_{\theta}^T \ln \pi(a; s, \theta) v'') (\pi(a; s, \theta) + \nabla_{\theta}^T \pi(a; s, \theta) \delta v'') da \\ &= \int (\bar{V}(s; v') + \nabla_{\theta}^T \ln \pi(a; s, \theta) v'') \pi(a; s, \theta) da \\ &\quad \times (1 + \nabla_{\theta}^T \ln \pi(a; s, \theta) \delta v'') \\ &= \int (\bar{V}(s; v') + \nabla_{\theta}^T \ln \pi(a; s, \theta) v'') \pi(a; s, \theta) da \\ &\quad + \bar{V}(s; v') E_{s, \theta} (\ln \pi(a; s, \theta)) \delta v'' \\ &\quad + \int \nabla_{\theta}^T \ln \pi(a; s, \theta) v'' \nabla_{\theta}^T \ln \pi(a; s, \theta) \delta v'' \pi(a; s, \theta) da \\ &= E_{s, \theta} \bar{Q}(a, s; v, \theta) + 0 + \delta \|v''\|^2 \\ &\quad \times \int \|\nabla_{\theta} \ln \pi(a; s, \theta)\|^2 \pi(a; s, \theta) da. \end{aligned}$$

We can see that the expected value of $\bar{Q}$ increases. The vector v'' is applied to adjust the policy in Step 9 of the algorithm and is shortened afterwards. The use of v'' in policy improvement makes the value of this vector obsolete. Its influence on further policy improvement is hence reduced, similarly as is done in the original algorithm presented by Peters et al. (2003).

2.1.3. Generalization

In general, we will consider sequential actor-critic-type algorithms characterized by the following features:

1. Actions are generated by a stationary policy (actor) i.e., a distribution π parameterized by state s_t and the policy vector $\theta \in \mathbb{R}^{n_{\theta}}$, namely

$$a_t \sim \pi(\cdot; s_t, \theta).$$

2. A visit in state s_t causes a modification of the policy vector θ by a product

$$\beta_t^{\theta} \hat{\phi}_t.$$

$\hat{\phi}_t$ on average indicates the direction in which θ assures larger future rewards expected in state s_t whereas $(\beta_t^{\theta}, t = 1, 2, \dots)$ is a vanishing sequence of step-sizes.

3. The algorithm may compute $\hat{\phi}_t$ with the use of an auxiliary parameter $v \in \mathbb{R}^{n_v}$. A visit in state s_t results in a modification of v by a vector

$$\beta_t^v \hat{\psi}_t.$$

$\hat{\psi}_t$ on average points into the direction where v assures better quality of $\hat{\phi}_t$ whereas $(\beta_t^v, t = 1, 2, \dots)$ is a vanishing sequence of step-sizes.

4. The vectors $\hat{\phi}_t$ and $\hat{\psi}_t$, while different from one another, are of the same form,

$$G_t(\theta, v) \sum_{k \geq 0} (\alpha \rho)^k z_{t,k}(\theta, v) \quad (8)$$

where G_t is a vector defined by s_t and a_t , $\alpha \in [0, 1)$, $\rho \in [0, 1)$, and $z_{t,k} \in \mathbb{R}$ is defined by s_{t+k} , a_{t+k} , r_{t+k} , s_{t+k+1} , and possibly a_{t+k+1} .

A number of RL algorithms fit into the above schema. In the Basic Actor-Critic algorithm mentioned above we have

$$\hat{\phi}_t = \nabla_{\theta} \ln \pi(a_t; s_t, \theta) \sum_{k \geq 0} (\gamma \lambda)^k d_{t+k}(v),$$

$$\hat{\psi}_t = \nabla_v \bar{V}(s_t; v) \sum_{k \geq 0} (\gamma \lambda)^k d_{t+k}(v),$$

which means that both $\hat{\phi}_t$ and $\hat{\psi}_t$ are of the form (8) with $\gamma \lambda = \alpha \rho$, and

$$G_t(\theta, v) = \nabla_{\theta} \ln \pi(a_t; s_t, \theta), \quad z_{t,k}(\theta, v) = d_{t+k}(v)$$

for the actor, and

$$G_t(\theta, v) = \nabla_v \bar{V}(s_t; v), \quad z_{t,k}(\theta, v) = d_{t+k}(v)$$

for the critic.

In the Natural Actor-Critic algorithm we have

$$\hat{\phi}_t = v'',$$

$$\hat{\psi}_t = \nabla_v \bar{Q}(a_t, s_t; v, \theta) \sum_{k \geq 0} (\gamma \lambda)^k e_{t+k}(v, \theta),$$

which means that both $\hat{\phi}_t$ and $\hat{\psi}_t$ are of the form (8) with $\gamma \lambda = \alpha \rho$, and

$$G_t(\theta, v) = v'', \quad z_{t,k}(\theta, v) = \begin{cases} 1 & \text{for } k = 0, \\ 0 & \text{for } k > 0 \end{cases}$$

for the actor, and

$$G_t(\theta, v) = \nabla_v \bar{Q}(a_t, s_t; v, \theta), \quad z_{t,k}(\theta, v) = e_{t+k}(v, \theta)$$

for the critic.

Important algorithms that also fit into the discussed schema are the actor-critic presented by Konda and Tsitsiklis (2003) and OLPOMDP (Bartlett & Baxter, 2000), although OLPOMDP does not employ any auxiliary vector v .

Analysis of RL methods usually assumes that, for each policy parameter θ , the sequence of states $\{s_t, t = 1, 2, \dots\}$ forms an aperiodic and irreducible Markov chain with a stationary distribution. In our presentation of the mechanics of the sequential RL algorithms, we will also assume that the states visited from the policy parameter θ is applied are drawn independently from a stationary distribution. A reader interested in a deeper analysis and proofs of convergence is referred to Bartlett and Baxter (2000); Konda and Tsitsiklis (2003).

Let us analyze the average direction of $\hat{\phi}_t$ and $\hat{\psi}_t$. Namely, let ϕ be a function defined as

$$\phi(s, \theta, v) = E_{\theta, v, \beta} (\hat{\phi}_t | s_t = s). \quad (9)$$

The definition of ϕ is based on the assumption that θ , v , and the step-sizes remain constant when $\hat{\phi}_t$ is calculated. In fact, they slightly vary and each $\hat{\phi}_t$ is in fact a biased estimator of $\phi(s_t, \theta, v)$ for θ and v used at time t . However, this bias is small, and since the dynamics of the parameters decreases in time, the bias asymptotically vanishes. The average $\phi(s, \theta, v)$ weighted by the

steady-state distribution defines the direction of the drift of the policy vector.

The drift of v may be analyzed in a similar way. Namely, let ψ be a function defined as

$$\psi(s, \theta, v) = E_{\theta, v, \beta}(\hat{\psi}_t | s_t = s). \quad (10)$$

As above, the definition of ψ requires that θ , v , and the step-sizes remain constant during the time when $\hat{\psi}_t$ is computed. The drift of v is defined by the average $\psi(s, \theta, v)$ weighted by the steady-state distribution. The usual role of the drift of the auxiliary parameter is to move it toward the point $v^*(\theta)$ such that the average $\phi(s, \theta, v^*(\theta))$ approximates either a policy gradient or a natural policy gradient. Hence, adjustments of θ ultimately lead to policy improvement.

3. Experience replay

The main idea analyzed in this paper is to apply to the agent's experience the same processing as a sequential actor-critic-type algorithm would, yet more intensively. A generic algorithm augmented by experience replay is presented in Algorithm 3.

After each instant t , the original sequential algorithm estimates $\phi(s_t, \theta, v)$, i.e., the direction of policy improvement, and adjusts the policy vector θ along the estimate. Within each instant t , the modified algorithm repeatedly draws one of the recently visited states, s_i , estimates $\phi(s_i, \theta, v)$, and modifies the policy vector along the estimate. Essentially both algorithms achieve the same goal but (i) the modified one does it more intensively and (ii) it employs experience gathered after visiting state s_i to adjust various policies characterized by different policy vectors. The auxiliary vector v undergoes similar operations in both algorithms.

Because the policy vector is constantly changing, each time its adjustment is performed, it is computed on the basis of the new values of θ and v . The intensity of replaying, $v(t)$, must be bounded for the sake of correctness of the algorithm. It is also limited by the computation power available during the agent–environment interaction. $v(t)$ should be additionally limited for small t to prevent many recalculations of few tuples in the database and to avoid overtraining.

Algorithm 3 Actor–Critic with Experience Replay. Estimators mentioned in Steps 6 and 7 are based on the data in a database.

-
- 0: $t := 1$. Initialize θ and v .
 - 1: Draw and execute an action, $a_t \sim \pi(\cdot; s_t, \theta)$.
 - 2: Register the tuple $\langle s_t, a_t, \theta, r_t, s_{t+1} \rangle$ in the database.
 - 3: Make sure only N most recent tuples remain in the database.
 - 4: Repeat $v(t)$ times:
 - 5: Draw $i \in \{t - N + 1, t - N + 2, \dots, t\}$.
 - 6: Adjust θ along an estimator of $\phi(s_i, \theta, v)$:
 - 7: $\theta := \theta + \beta_t^\theta \hat{\phi}_i^r(\theta, v)$.
 - 8: Adjust v along an estimator of $\psi(s_i, \theta, v)$:
 - 9: $v := v + \beta_t^v \hat{\psi}_i^r(\theta, v)$.
 - 10: Assign $t := t + 1$ and repeat from Step 1.
-

Designing the estimators of ϕ and ψ for Steps 7 and 9 of Algorithm 3 we have to guarantee that their variance is bounded and their bias asymptotically vanishes. This is the only way for the algorithm to preserve the limit properties of the original sequential method. In the following section we prove that both the requirements are met if the estimators are of the same generic form defined below.

Let $b > 1$, θ_{i+j} be the policy vector applied to generate a_{i+j} , and K be drawn independently from $Geom(\rho)$, the geometric

distribution¹ with parameter $\rho \in [0, 1)$. Also, let χ be equal to 0 if $z_{t,k}$ is not explicitly defined by a_{t+k+1} (as in the Basic Actor–Critic), and to 1 otherwise (as in Actor–Critic of Konda and Tsitsiklis (2003)). We introduce the *randomized-truncated estimators* $\hat{\phi}_i^r(\theta, v)$ and $\hat{\psi}_i^r(\theta, v)$ of the same generic form

$$\sum_{k=0}^K G_i(\theta, v) \alpha^k z_{i,k}(\theta, v) \min \left\{ \prod_{j=0}^{k+\chi} \frac{\pi(a_{i+j}; s_{i+j}, \theta)}{\pi(a_{i+j}; s_{i+j}, \theta_{i+j})}, b \right\} \quad (11)$$

where $\hat{\phi}_i^r$ is defined by those G and z defining $\hat{\phi}_t$, and $\hat{\psi}_i^r$ is defined by those G and z that define $\hat{\psi}_t$. We can see that (11) closely resembles the original form (8) of $\hat{\phi}_t$ and $\hat{\psi}_t$. However, there are two important differences. First, the infinite sum is replaced by the finite one with the appropriately designed random limit. Second, truncated density ratios are introduced in order to compensate for the fact that the current policy is different from the one that generated the actions $a_t, a_{t-1}, \dots$ that are present in the database.

4. Properties of the randomized-truncated estimator

The purpose of this section is to establish conditions under which estimator (11) is asymptotically unbiased and of bounded variance. To this end, we will first discuss the estimation technique called *importance sampling* in Section 4.1. This technique, in its pure form, suffers from the problem of excessive variance of estimators. We cope this problem by truncating the density ratios. This simple trick may be successful if it is applied to a certain class of families of probabilistic distributions analyzed in Section 4.2. Sufficient conditions for bounded variance and asymptotic unbiasedness of (11) are finally established in 4.3.

4.1. Importance sampling

Let $g(\cdot; \theta)$ be the density of random elements a in $\mathcal{A}$ parameterized by a vector $\theta \in \mathcal{R}^n$. Suppose for a given parameter θ_0 an action a_0 is drawn from $g(\cdot; \theta_0)$. The action $a_0 \in \mathcal{A}$ yields a certain vector $d(a_0)$, where $d: \mathcal{A} \mapsto \mathcal{R}^n$. We are interested in estimation of the expected $d(a)$ for actions drawn with the use of an arbitrary parameter θ , namely

$$D(\theta) = E_\theta d(a) = \int_{\mathcal{A}} d(a) g(a; \theta) da. \quad (12)$$

In order to construct an estimator of $D(\theta)$, we may apply the estimation technique called *importance sampling* (Rubinstein, 1981) frequently applied in RL (Frank et al., 2008; Peshkin & Shelton, 2002; Uchibe & Doya, 2004). Namely, given θ_0 and θ , the statistic

$$\hat{D}(\theta) = d(a_0) g(a_0; \theta) / g(a_0; \theta_0) \quad (13)$$

is an unbiased estimator of $D(\theta)$ since

$$E_{\theta_0} \hat{D}(\theta) = \int d(a) \frac{g(a; \theta)}{g(a; \theta_0)} g(a; \theta_0) da = D(\theta). \quad (14)$$

A well-known drawback of estimator (13) is its excessive variance when distributions $g(\cdot; \theta_0)$ and $g(\cdot; \theta)$ are far from one another (Wawrzyński & Pacut, 2006). A simple yet practically satisfactory way of fighting the variance problem is to truncate the density ratio in (13). Namely, the modified estimator has the form

$$\hat{D}^b(\theta) = d(a_0) \min \left\{ \frac{g(a_0; \theta)}{g(a_0; \theta_0)}, b \right\} \quad (15)$$

¹ That is, the random variable K of values in $\{0, 1, 2, \dots\}$ has distribution $Geom(\rho)$, iff $P(K = m) = (1 - \rho)\rho^m$ for non-negative integer m .

for constant $b > 1$. The above *truncated IS estimator* has also been applied in Uchibe and Doya (2004). Let us analyze its properties. Obviously if $\|d\|$ is bounded, then the variance of (15) is also bounded. Importantly, its bias happens to be bounded as well if only g satisfies certain regularity conditions and θ is close enough to θ_0 . We specify those conditions below.

4.2. Regular distributions

It seems reasonable to expect that the bias of the truncated estimator (15) is small for θ close to θ_0 . It is proved below that in principle it is true if only g is regular in a certain sense. Then, for small $\|\theta - \theta_0\|$, the densities ratio is rarely truncated as it rarely exceeds b . For the purpose of this discussion, the family of densities $g(\cdot; \theta)$ parameterized by θ will be called (M_1, M_2) -regular if it has the following features:

- (i) for every $a \in \mathcal{A}$ the mapping $\theta \mapsto g(a; \theta)$ is continuous and differentiable,
- (ii) the trace of the Fisher information, i.e., $I(\theta) = E_\theta \frac{\partial \ln g(a; \theta)}{\partial \theta} \frac{\partial \ln g(a; \theta)}{\partial \theta^T}$, is bounded by M_1 ,
- (iii) absolute eigenvalues of the Hessian $\frac{\partial^2 \ln g(a; \theta)}{\partial \theta^T \partial \theta}$ are bounded by M_2 .

The conditions above are typically satisfied by distributions that we could think of. For instance, the normal distribution $N(\theta, C)$ with non-singular C is (M_1, M_2) -regular for $M_1 = \text{tr } C^{-1}$ and $M_2 = \max \text{eig } C^{-1}$. In the experimental study presented in Section 5 a policy is applied in the form of the normal distributions with constant variance and mean value being a certain function of θ . Let us denote this function by f . We thus have

$$g(a; \theta) = \frac{1}{\sqrt{2\pi}^n |C|} \exp(-0.5(a - f(\theta))^T C^{-1} (a - f(\theta))).$$

It is easy to verify that this distribution is also (M_1, M_2) -regular if only C is non-singular and the f function and its derivatives are uniformly continuous and differentiable.

The propositions below apply the definition of (M_1, M_2) -regular distribution to establish bounds of the bias of the truncated importance sampling estimator (15). The first one relates this bias to the probability that the density ratio is truncated.

Proposition 1. For all θ_0, θ , it is true that $\widehat{D}^b(\theta)$ defined by (15) satisfies

$$\|E_{\theta_0} \widehat{D}^b(\theta) - D(\theta)\| \leq MP_\theta \left(\frac{g(a; \theta)}{g(a; \theta_0)} > b \right),$$

where $M = \sup_{a \in \mathcal{A}} \|d(a)\|$.

Proof. We split $E_{\theta_0} \widehat{D}^b(\theta)$ and $D(\theta)$ into pairs of integrals. For the first element of each pair the density ratio does not reach its upper bound and for the second one it does. The first elements cancel each other out and we derive a bound for the second ones. Namely, let us split $\mathcal{A}$ into two parts:

$$A_{\theta, \theta_0}^b = \left\{ a : \frac{g(a; \theta)}{g(a; \theta_0)} \leq b \right\},$$

$$\bar{A}_{\theta, \theta_0}^b = \left\{ a : \frac{g(a; \theta)}{g(a; \theta_0)} > b \right\} = \mathcal{A} - A_{\theta, \theta_0}^b.$$

Then, we have

$$\|E_{\theta_0} \widehat{D}^b(\theta) - D(\theta)\| = \left\| \int_{A_{\theta, \theta_0}^b \cup \bar{A}_{\theta, \theta_0}^b} d(\alpha) \min \left\{ \frac{g(\alpha; \theta)}{g(\alpha; \theta_0)}, b \right\} g(\alpha; \theta_0) d\alpha \right.$$

$$\begin{aligned} & \left. - \int_{A_{\theta, \theta_0}^b \cup \bar{A}_{\theta, \theta_0}^b} d(\alpha) g(\alpha; \theta) d\alpha \right\| \\ &= \left\| \int_{A_{\theta, \theta_0}^b} \left[d(\alpha) \frac{g(\alpha; \theta)}{g(\alpha; \theta_0)} g(\alpha; \theta_0) - d(\alpha) g(\alpha; \theta) \right] d\alpha \right. \\ & \quad \left. + \int_{\bar{A}_{\theta, \theta_0}^b} \left[d(\alpha) b g(\alpha; \theta_0) - d(\alpha) g(\alpha; \theta) \right] d\alpha \right\| \\ &= \left\| 0 + \int_{\bar{A}_{\theta, \theta_0}^b} d(\alpha) \left[b \frac{g(\alpha; \theta_0)}{g(\alpha; \theta)} - 1 \right] g(\alpha; \theta) d\alpha \right\|. \end{aligned}$$

Because for $\alpha \in \bar{A}_{\theta, \theta_0}^b$ we have

$$b \frac{g(\alpha; \theta_0)}{g(\alpha; \theta)} \leq 1,$$

we finally obtain

$$\begin{aligned} \|E_{\theta_0} \widehat{D}^b(\theta) - D(\theta)\| &\leq \int_{\bar{A}_{\theta, \theta_0}^b} \|d(\alpha)\| g(\alpha; \theta) d\alpha \leq MP_\theta (a \in \bar{A}_{\theta, \theta_0}^b) \\ &= MP_\theta \left(\frac{g(a; \theta)}{g(a; \theta_0)} > b \right). \quad \blacksquare \end{aligned}$$

Note that P_θ is the probability defined by the condition that the random value a is drawn from the density parameterized by θ . Another proposition defines a bound of P_θ .

Proposition 2. For each $M_1, M_2 > 0$, and $b > 1$ there exists $M > 0$ such that, if the density g is (M_1, M_2) -regular, then for all θ_0, θ

$$P_\theta (g(a; \theta)/g(a; \theta_0) > b) \leq M \|\theta - \theta_0\|^2.$$

In the proof of the above proposition, the following notation is applied that “converts” predicates into numeric values:

$$[\text{predicate}] = \begin{cases} 1 & \text{if the predicate is true} \\ 0 & \text{otherwise.} \end{cases} \quad (16)$$

For example, $|x| = x[x > 0] - x[x < 0]$. This notation will also be applied in the experimental study (Section 5).

Proof of Proposition 2. We have

$$\begin{aligned} g(a; \theta)/g(a; \theta_0) > b &\Leftrightarrow g(a; \theta) > b g(a; \theta_0) \Leftrightarrow \frac{g(a; \theta)}{g(a; \theta_0)} > b \\ &\Leftrightarrow \ln \frac{g(a; \theta)}{g(a; \theta_0)} > \ln b \Leftrightarrow (\ln b)^{-1} (\ln g(a; \theta) - \ln g(a; \theta_0)) > 1. \end{aligned}$$

Consequently

$$\begin{aligned} P_\theta (g(a; \theta)/g(a; \theta_0) > b) \\ &= P_\theta ((\ln b)^{-1} (\ln g(a; \theta) - \ln g(a; \theta_0)) > 1). \end{aligned}$$

Then, we obtain

$$\begin{aligned} P_\theta (g(a; \theta)/g(a; \theta_0) > b) \\ &= \int [(\ln b)^{-1} (\ln g(\alpha; \theta) - \ln g(\alpha; \theta_0)) > 1] g(\alpha; \theta) d\alpha \\ &= \int [(\ln b)^{-2} (\ln g(\alpha; \theta) - \ln g(\alpha; \theta_0))^2 > 1] g(\alpha; \theta) d\alpha \\ &\leq \int (\ln b)^{-2} (\ln g(\alpha; \theta) - \ln g(\alpha; \theta_0))^2 g(\alpha; \theta) d\alpha. \end{aligned}$$

The last inequality results from the fact that $[x > 1] \leq x$ if only $x \geq 0$. Because g is (M_1, M_2) -regular, there exists M_2 such that $(\forall \alpha, \theta) \|\nabla_\theta^2 \ln g(\alpha; \theta)\| \leq M_2$, and we have

$$P_\theta(g(a; \theta)/g(a; \theta_0) > b) \leq (\ln b)^{-2} \int (\|\nabla_\theta \ln g(\alpha; \theta)\| \|\theta_0 - \theta\| + M_2 \|\theta_0 - \theta\|^2) g(\alpha; \theta) d\alpha \quad (17)$$

$$\leq \|\theta_0 - \theta\|^2 (\ln b)^{-2} \int \|\nabla_\theta \ln g(\alpha; \theta)\|^2 g(\alpha; \theta) d\alpha \quad (18)$$

$$+ \|\theta_0 - \theta\|^3 2M_2 (\ln b)^{-2} \int \|\nabla_\theta \ln g(\alpha; \theta)\| g(\alpha; \theta) d\alpha \quad (19)$$

$$+ \|\theta_0 - \theta\|^4 M_2^2 (\ln b)^{-2} \int g(\alpha; \theta) d\alpha \quad (20)$$

$$\leq \|\theta_0 - \theta\|^2 (\ln b)^{-2} (M_1 + 2\|\theta_0 - \theta\| M_2 \sqrt{M_1} + \|\theta_0 - \theta\|^2 M_2^2). \quad (21)$$

Inequality (17) results from the second-order Taylor expansion of $\ln g(\alpha; \theta + \Delta\theta)$ and the bound of Hessian eigenvalues. The bounds of (18) and (19) result from Property (ii) of (M_1, M_2) -regular distributions. The bound of (19) results also from the fact that $E|f(a)| \leq (E|f(a)|^p)^{1/p}$ for $p > 1$. We now exploit the fact that the required probability is bounded by 1. We search for M in the form

$$M = (\ln b)^{-2} M_1 + M_3 + M_4$$

such that

$$\min\{\|\theta_0 - \theta\|^3 (\ln b)^{-2} 2M_2 \sqrt{M_1}, 1\} \leq M_3 \|\theta_0 - \theta\|^2,$$

$$\min\{\|\theta_0 - \theta\|^4 (\ln b)^{-2} M_2^2, 1\} \leq M_4 \|\theta_0 - \theta\|^2.$$

It is easy to verify that the smallest such M_3 and M_4 have the values

$$M_3 = (\ln b)^{-4/3} 2^{2/3} M_2^{2/3} M_1^{1/3},$$

$$M_4 = (\ln b)^{-1} M_2.$$

We thus obtain

$$\begin{aligned} P_\theta(g(a; \theta)/g(a; \theta_0) > b) &\leq \min \left\{ \|\theta_0 - \theta\|^2 (\ln b)^{-2} (M_1 + 2\|\theta_0 - \theta\| M_2 \sqrt{M_1} + \|\theta_0 - \theta\|^2 M_2^2), 1 \right\} \\ &\leq \min\{\|\theta_0 - \theta\|^2 (\ln b)^{-2} M_1, 1\} \\ &\quad + \min\{2\|\theta_0 - \theta\|^3 (\ln b)^{-2} M_2 \sqrt{M_1}, 1\} \\ &\quad + \min\{\|\theta_0 - \theta\|^4 (\ln b)^{-2} M_2^2, 1\} \\ &\leq M \|\theta_0 - \theta\|^2 \\ &\leq ((\ln b)^{-2} M_1 + (\ln b)^{-4/3} 2^{2/3} M_2^{2/3} M_1^{1/3} + (\ln b)^{-1} M_2) \|\theta_0 - \theta\|^2. \quad \blacksquare \end{aligned}$$

In the present work a special case of the analyzed problem is discussed in which d is a function of a finite sequence of independent random elements. In fact, both the above propositions apply to this case because a sequence of random vectors is also a random vector. However, a joint density of a random sequence need not be (M_1, M_2) -regular even if densities of all its elements are. This causes difficulties in application of Proposition 2. The proposition below treats this issue.

Proposition 3. For each $M_1, M_2 > 0$, and $b > 1$ there exists $M > 0$ such that if all densities in the sequence $g_1, \dots, g_k$ are (M_1, M_2) -regular, then for each set of vectors $(\theta_1, \dots, \theta_k, \theta)$,

$$P_\theta \left(\prod_{i=1}^k \frac{g_i(a_i; \theta)}{g_i(a_i; \theta_i)} > b \right) \leq \frac{k^2 M}{(\ln b)^2} \sum_{i=1}^k \|\theta - \theta_i\|^2.$$

provided random elements $a_i, i = 1, \dots, k$ are drawn independently from $g_i(\cdot; \theta_i)$.

Proof. Let us denote

$$A = \left\{ \prod_{i=1}^k \frac{g_i(a_i; \theta)}{g_i(a_i; \theta_i)} > b \right\}, \quad A_i = \left\{ \frac{g_i(a_i; \theta)}{g_i(a_i; \theta_i)} > \sqrt[k]{b} \right\},$$

for $i = 1, \dots, k$. We have

$$\bigcap_{i=1}^k (\Omega - A_i) \subseteq \Omega - A$$

and

$$\prod_{i=1}^k (1 - P_\theta(A_i)) \leq 1 - P_\theta(A).$$

A simple induction shows that

$$1 - \sum_{i=1}^k P_\theta(A_i) \leq \prod_{i=1}^k (1 - P_\theta(A_i)).$$

A combination of the above two inequalities yields

$$P_\theta(A) \leq \sum_{i=1}^k P_\theta(A_i).$$

Application of Proposition 2 completes the proof. $\blacksquare$

4.3. Bounds for variance and bias of the randomized-truncated estimator

We are now ready to derive properties of estimator (11). Because ϕ and ψ have the same form, further in this section we will focus on ϕ , understanding that exactly the same can be said about ψ . Because of truncation, it is easy to verify that variance of $\hat{\phi}_i^r$ is bounded if only G and z are bounded. However, also because of truncation, the estimator is biased. Fortunately, the bias vanishes when the parameters θ_{i+j} of distributions that generated actions get closer to the analyzed value θ . The following proposition clarifies this issue. In order to prove that the variance of $\hat{\phi}_i^r(\theta, \nu)$ is bounded, it is enough to establish a bound of the trace of the variance. This is what the following proposition does.

Proposition 4. Suppose G_i and $z_{i,k}$ are uniformly bounded such that $\|G_i z_{i,k}\| \leq M$. Then

$$\text{tr } \mathcal{V}_{\hat{\phi}_i^r}(\theta, \nu) \leq \frac{b^2 M^2}{(1 - \alpha)^2}.$$

Proof. We have

$$\begin{aligned} \text{tr } \mathcal{V}_{\hat{\phi}_i^r}(\theta, \nu) &\leq E (\hat{\phi}_i^r(\theta, \nu)^\top \hat{\phi}_i^r(\theta, \nu)) \leq \left(\sum_{j \geq 0} \alpha^j M b \right)^2 \\ &\leq \frac{b^2 M^2}{(1 - \alpha)^2}. \quad \blacksquare \end{aligned}$$

Another proposition establishes the bounds of bias of (11).

Proposition 5. Suppose G_i and $z_{i,k}$ are uniformly bounded and distributions $\pi(\cdot; s_{i+j}, \cdot)$ are (M_1, M_2) -regular for certain common M_1, M_2 and all $j \geq 0$. Then, there exists c such that

$$\|\widehat{E\phi}_i^r(\theta, \nu) - \phi(s_i, \theta, \nu)\| \leq c\delta^2$$

for all i, θ, ν and

$$\delta = \sup_{j \geq 0} \|\theta - \theta_{i+j}\|.$$

Proof. ϕ_t (8) is a weighted, by $(\alpha\rho)^k$, sum of components of the form $G_i(\theta, \nu)z_{i,k}(\theta, \nu)$. Each of them is a function of the random elements $a_t, s_{t+1}, \dots, a_{t+k}, s_{t+k+1}$, and possibly a_{t+k+1} . On the other hand, each element of ϕ_i^r (11) is a truncated importance-sampling estimator of $E_{\theta, \nu}(G_i(\theta, \nu)z_{i,k}(\theta, \nu)|s_i)$, scaled by α^k . Propositions 1 and 3 define a bound of the bias of these estimators, which is of the form

$$\frac{M(k+2)^2}{(\ln b)^2} \sum_{j=0}^{k+1} \|\theta - \theta_{i+j}\|^2.$$

Because the sum in the above formula is no greater than $(k+2)\delta^2$, and because $P(K \geq m) = \rho^m$ (which is a simple property of $\text{Geom}(\rho)$), we obtain

$$\begin{aligned} \|\widehat{E\phi}_i^r(\theta, \nu) - \phi(s_i, \theta, \nu)\| &\leq E \left(\sum_{k=0}^K \alpha^k \frac{M(k+2)^3 \delta^2}{(\ln b)^2} \right) \\ &= \sum_{k \geq 0} (\rho\alpha)^k \frac{M(k+2)^3 \delta^2}{(\ln b)^2} = \delta^2 \frac{M}{(\ln b)^2} \sum_{k \geq 0} (\rho\alpha)^k (k+2)^3. \end{aligned}$$

Because $\rho\alpha \in (0, 1)$, the value of the last sum is bounded, which completes the proof. ■

Let us now see how the estimator (11) is asymptotically unbiased. Because only a certain constant number, N , of recent events are kept in the database, at the moment t the indexes i are drawn randomly from the set $\{t-N+1, t-N+2, \dots, t\}$. Let us analyze δ_t as the upper bound of $\|\theta - \theta_{i+j}\|$ for i drawn at moment t . Because the step-sizes β_t^θ vanish in time, the changes of the policy vector, θ , also vanish. Therefore, the value δ_t must vanish as well, the conditions of Proposition 5 are met, and the estimators $\widehat{\phi}_i^r$ and $\widehat{\psi}_i^r$ become asymptotically unbiased.

Another consideration is associated with the changes of steady-state distribution that follow the adjustments of the policy vector. Our objective is to change θ along the average $\phi(s, \theta, \nu)$ and ν along the average $\psi(s, \theta, \nu)$, both weighted by the steady-state distribution. Even if $\widehat{\phi}_i^r(\theta, \nu)$ is a good estimator of $\phi(s_i, \theta, \nu)$, this is not enough for it to be a good estimator of the average ϕ . However, this would be enough if s_i could be regarded as drawn from the steady-state distribution. Once again, we exploit the facts that only N events are kept in the database and that, for growing t , the changes of θ get smaller. Then, whenever $\widehat{\phi}_i^r(\theta, \nu)$ is computed at moment t , state s_i has been visited when the policy in use has been similar to the one with parameter θ .

5. Experimental study

This section reports experiments on application of the presented methodology to challenging learning-control tasks. First, the Basic Actor–Critic augmented by experience replay is applied to the cart-pole swing-up problem (Doya, 2000). Second, the Natural Actor–Critic with experience replay is confronted with Half-Cheetah (Wawrzyński, 2007). In all the experiments the controlled system is emulated, i.e., simulated in real time. A quantum of the system's real time is equal to a quantum of the corresponding computer time, which means that the computer has a lot of spare time

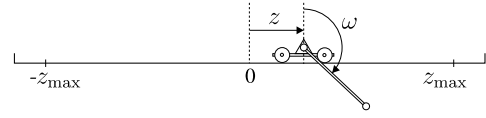


Fig. 1. The cart-pole swing-up problem.

that can be devoted to parallel computations. The setup of our experiments is designed to closely resemble a situation when control of a physical machine is to be optimized in real time by means of learning. The purpose of this study is to verify the presented methodology in those circumstances.

5.1. The Basic Actor–Critic for cart-pole swing-up

In this section we report how experience replay can be applied to one of the simplest RL algorithms, namely the Basic Actor–Critic. To continue the discussion in Section 2, we base our experiments on the version of the algorithm presented by Kimura and Kobayashi (1998).

The cart-pole swing-up task (Doya, 2000) is used for the experiments. It is an issue of control of a cart moving along a track with a freely hanging pole (see Fig. 1). The system is controlled by a force applied to the cart. The objective is to avoid hitting the track bounds, swing the pole, turn it up, and stabilize upwards.

Algorithm 4 The Basic Actor–Critic with Experience Replay (Replaying BAC).

- 0: $t := 1$. Initialize θ and ν .
- 1: Draw and execute an action, $a_t \sim \pi(\cdot; s_t, \theta)$.
- 2: Register the tuple $\langle s_t, a_t, \theta, r_t, s_{t+1} \rangle$ in the database.
- 3: Make sure only N most recent tuples remain in the database.
- 4: Repeat $\nu(t)$ times:
 - 5: Draw $i \in \{t-N+1, t-N+2, \dots, t\}$.
Draw $K \sim \text{Geom}(\rho)$.
Calculate SUM equal to $\sum_{k=0}^K \alpha^k d_{i+k} \min \left\{ \prod_{j=0}^k \frac{\pi(a_{i+j}; s_{i+j}, \theta)}{\pi(a_{i+j}; s_{i+j}, \theta_{i+j})}, b \right\}$
for $d_{i+k} = r_{i+k} + \gamma \bar{V}(s_{i+k+1}; \nu) - \bar{V}(s_{i+k}; \nu)$.
 - 6: Adjust θ along an estimator of $\phi(s_i, \theta, \nu)$:
 $\theta := \theta + \beta_t^\theta \nabla_\theta \ln \pi(a_i; s_i, \theta) SUM$.
 - 7: Adjust ν along an estimator of $\psi(s_i, \theta, \nu)$:
 $\nu := \nu + \beta_t^\nu \nabla_\nu \bar{V}(s_i; \nu) SUM$.
 - 10: Assign $t := t + 1$ and repeat from Step 1.

In the experiments the actor, π , is comprised of two parts: a neural network and a normal distribution. The input of the network is the state. The network has a hidden layer with 20 sigmoidal (arctan) neurons and a linear output neuron. The output becomes a mean value of the normal distribution with variance $\sigma^2 = 4$. The distribution generates actions. The critic is a neural network of the same structure as above with 50 neurons in its hidden layer. All neurons in the networks have a constant input (bias). The initial weights in the hidden layers are drawn randomly from the normal distribution $N(0, 1)$ and weights of the output layers are initially set to zero.

Implementation of the Basic Actor–Critic augmented by experience replay (Replaying BAC, in short) follows the discussion in Section 3. We use ν in the form

$$\nu(t) = \min\{c_1 t, c_0\} \quad (22)$$

that defines the replying intensity. It is characterized by c_0 and c_1 . The computer speed, together with the period length between two actions for the emulated system, bounds the maximal value of the ultimate intensity c_0 of the experience replay.

There are four state variables: position of the cart z , the pole angle ω , and their time derivatives $\dot{z}$, $\dot{\omega}$. The force F , applied to the cart every 0.1 s, is a single variable. A typical reward used in this problem is equal to the elevation of the pole tip. Initially, the waving pole hovers and the rewards are close to -1 . When the goal is reached and the pole is stabilized upwards, the rewards are close to 1. Physical restrictions make the state of the cart-pole *acceptable* if the cart position is within the track limits $[-2.4, 2.4]$. The action a_t is equal to the force and is limited to an interval $[-10, 10]$, i.e., $F_t = \min\{\max\{-10, a_t\}, 10\}$. The reward is calculated as

$$r_t = r_t^0 - 0.2|F_t - a_t| \quad \text{with} \\ r_t^0 = \begin{cases} \cos \omega & \text{if } |z| \leq 1.2 \wedge |\dot{\omega}| < 2\pi, \\ -1 & \text{if } |z| \in (1.2, 2.4] \vee |\dot{\omega}| \geq 2\pi, \\ -30 & \text{if } |z| > 2.4, \end{cases}$$

for z , ω , and $\dot{\omega}$ measured at $t + 1$. The state vector s_t feeding the approximators is normalized, namely

$$s_t = \left[\frac{z}{2}, \frac{\dot{z}}{3}, \frac{\sin \omega}{0.8}, \frac{\cos \omega}{0.8}, \frac{\dot{\omega}}{4} \right]^T.$$

The training consists of a sequence of *trials*. A trial ends when the pendulum's state becomes unacceptable. Otherwise, the trial lasts for a random real time drawn from the exponential distribution with the expected value equal to 20 s (each moment has equal probability of forcing the trial to end). At the beginning of each trial, the state is reset by drawing z and ω from the uniform distributions $U(-2.4, 2.4)$ and $U(0, 2\pi)$, respectively, and setting $\dot{z}$, $\dot{\omega}$ to zero.

The parameters of the Basic Actor-Critic taken in our simulations are $\gamma = 0.95$, $\lambda = 0.5$, $\beta_t^\theta = \beta_t^v \equiv 0.003$, and $\lambda = 0.5$. The algorithm usually obtains a satisfactory behavior after 2000 trials (about 6 h of the agent's real time); see the top part of Fig. 2.

The Replaying Actor-Critic (see the middle part of Fig. 2) exhibits a satisfactory behavior after just 90 trials (about 20 min of the cart-pole's time). The parameters used in the Replaying Actor-Critic are $\beta_t^\theta = \beta_t^v \equiv 0.001$, $b = 2$, $c_0 = 1000$, $c_1 = 0.3$, $N = 10^5$, $\alpha = \gamma = 0.95$ and $\rho = \lambda = 0.5$. Note that the difference in efficiency of the Basic Actor-Critic and the Replaying Actor-Critic is so large that the results must be presented in different figures.

The bottom part of Fig. 2 presents learning curves for the Replaying Actor-Critic with various intensity of experience replay, i.e., the parameter c_0 . We can see that the larger the intensity, the faster the convergence. However, intensity above 1000 does not yield further improvement. It appears that the value $c_0 = 1000$ is sufficient to utilize online all the information from the learning process for policy determination.

5.2. The Natural Actor-Critic for Half-Cheetah

We are interested in applications of the MDP framework to learning reactive policies of machines. In this section we analyze a challenging problem of this type, namely learning to run a planar model of a large cat. The cat robot, called Half-Cheetah, is presented in Fig. 3. It is a planar kinematic chain of nine links, eight joints, and two "paws". Because the fifth joint is fixed at 180° , and its adjacent links have the same length, joints 4 and 6 are always at the same position; therefore, the object does not look like a chain. The angles of the fourth and fifth joint are fixed; all the others are controllable. Consequently, Half-Cheetah is a model of a 6-degree-of-freedom walking robot.

The torque τ_i applied at the i -th joint is calculated as

$$\tau_i = T_i \min \{ \max \{ -1, \tau_i^0 + a_i^0 \}, 1 \}$$

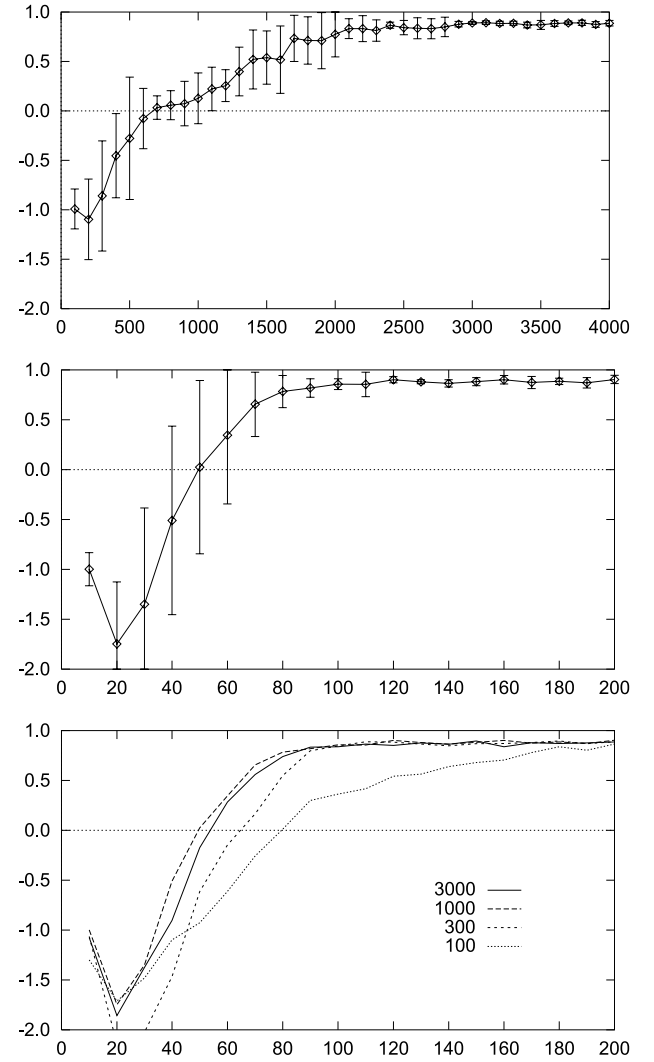


Fig. 2. Actor-Critics for cart-pole swing-up: The average reward vs. trial number. The average duration of a trial is 20 s (200 time instants). Each curve averages 10 runs. *Top:* The Basic Actor-Critic. Each point averages 100 consecutive trials. The one-sigma limits are calculated to assess run-to-run variability of trial averages. *Middle:* The Basic Actor-Critic with Experience Replay (ER). Note that the number of trials in this figure is about 20 times smaller than that of the top figure for the basic method. Each point averages only 10 consecutive trials. *Bottom:* The Basic Actor-Critic with ER for various replaying intensity, c_0 .

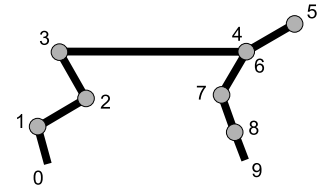


Fig. 3. The initial position of Half-Cheetah. It consists of nine links, eight joints, of which two are fixed (4, 5), and two paws (0, 9).

where τ_i^0 is a "spontaneous" torque at the i -th joint, a_i^0 is the output of the learning controller, and T_i expresses the "strength" of i -th joint. The spontaneous torque τ_i^0 is implemented as a PD controller with saturation. It roughly stabilizes the i -th joint at its initial angle. We follow a typical setting of control system design. While it is usually relatively easy to provide a controller that stabilizes a system around a certain state (e.g. by using PD controllers), it is much more difficult to design a controller that makes the system perform a certain non-trivial activity. In our paper, the controller is to *learn* to make Half-Cheetah run.

Table 1

Variables of Half-Cheetah state. $j = 0, 1, 2, 3$ (behind the neck) and $k = 0, 1, 2$ (in front of the neck).

$s_{t,1} = (y_0 - 0.1)/0.1$	$s_{t,6} = \dot{y}_3/0.7$	$s_{t,11+3j} = \cos \omega_j^l/0.7$
$s_{t,2} = \dot{y}_0$	$s_{t,7} = \dot{y}_4/0.7$	$s_{t,12+3j} = \sin \omega_j^l/0.7$
$s_{t,3} = ([y_0 = 0] - 0.5)/0.5$	$s_{t,8} = (y_9 - 0.1)/0.1$	$s_{t,13+3j} = \dot{\omega}_j^l/6$
$s_{t,4} = ((y_3 + y_4)/2 - 0.6)/0.2$	$s_{t,9} = \dot{y}_9$	$s_{t,23+3k} = \cos \omega_{6+k}^l/0.7$
$s_{t,5} = (\dot{x}_3 + \dot{x}_4)/4 - 1/2$	$s_{t,10} = ([y_9 = 0] - 0.5)/0.5$	$s_{t,24+3k} = \sin \omega_{6+k}^l/0.7$
		$s_{t,25+3k} = \dot{\omega}_{6+k}^l/6$

The fact that Half-Cheetah is designed as a kinematic chain allows one to efficiently emulate its dynamics, e.g. using the Newton–Euler method (Walker & Orin, 1982). The mass of the object is concentrated in its joints, which weigh

1, 1, 1, 4, 1, 2, 1, 1, 1, and 1

kilos, respectively. The consecutive links of Half-Cheetah are of lengths

$$\frac{0.2}{\cos \frac{\pi}{12}}, \frac{0.15}{\cos \frac{\pi}{3}}, \frac{0.25}{\cos \frac{\pi}{6}}, 1, 0.3, 0.3, \frac{0.25}{\cos \frac{\pi}{9}}, \frac{0.2}{\cos \frac{\pi}{9}}, \frac{0.15}{\cos \frac{\pi}{9}}$$

meters. The initial angle between the first link and the perpendicular is $-\pi/12$. The initial angles of the joints, ω_i^0 , for $i = 1, \dots, 8$, are

$$-\frac{5}{12}\pi, \frac{1}{2}\pi, -\frac{2}{3}\pi, \frac{1}{6}\pi, -\pi, \frac{5}{6}\pi, \frac{5}{18}\pi, \text{ and } 0.$$

The movement of joints are limited to intervals:

$$\left[-\frac{5}{6}\pi, -\frac{1}{6}\pi\right], \left[\frac{1}{4}\pi, \frac{3}{4}\pi\right], \left[-\frac{5}{6}\pi, -\frac{1}{3}\pi\right], \left[\frac{1}{6}\pi, \frac{1}{6}\pi\right], \\ [-\pi, -\pi], \left[\frac{1}{18}\pi, \frac{1}{2}\pi\right], \left[0, \frac{8}{9}\pi\right], \left[-\frac{2}{3}\pi, \frac{1}{9}\pi\right].$$

The object is controlled by applying torques at the joints. The torque τ_i at the i -th joint is computed in two phases: the variable

$$\tau_i^0 = 2\pi^{-1} \arctan(-2(\omega_i - \omega_i^0) - 0.05\dot{\omega}_i)$$

that expresses a “spontaneous” torque at the i -th joint implied by its angle, ω_i , and angular velocity, $\dot{\omega}_i$, is added to the signal a_i^0 coming from the learning controller, projected on the interval $[-1, 1]$, and multiplied by the “strength”, T_i , of i -th joint, namely

$$\tau_i = T_i \min\{\max\{-1, \tau_i^0 + a_i^0\}, 1\}.$$

The “strengths” in consecutive joints are as follows

60, 90, 120, –, –, 90, 60, 30,

where “–” corresponds to undefined strengths of fixed joints.

The objective of control is to make Half-Cheetah run forward as fast as possible. Notice that, without the external control (i.e. for $a_i^0 \equiv 0$), the torque $T_i \tau_i^0$ implements a PD control (a differentiator with proportional gain) with saturation. It stabilizes the joint angle close to its initial value which makes Half-Cheetah stand still.

5.2.0.1. State and reward

In order to apply the reinforcement learning to Half-Cheetah, one must define the states and the rewards the learning algorithm has access to. In the specification below, the joints are indexed from 0 to 9, whereas the links have indexes from 0 to 8. The two-dimensional position of the i -th joint is denoted by (x_i, y_i) and its speed by $(\dot{x}_i, \dot{y}_i)$. The angle between the i -th link and the horizontal line is denoted by ω_i^l , and the corresponding angular speed is $\dot{\omega}_i^l$. Each state variable is normalized to roughly cover the interval $[-1, 1]$. We apply the notation (16). For instance, the term $[y_0 = 0]$ is equal to 1 if the hind paw stands on the ground and 0 otherwise.

All 31 variables describing the state of Half-Cheetah are depicted in Table 1.

In our definition of reward we apply a *soft-step function* of the form

$$f(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ 2x^2 & \text{for } x \in (0, 0.5] \\ 1 - 2(x - 1)^2 & \text{for } x \in (0.5, 1] \\ 1 & \text{for } x > 1. \end{cases}$$

It is mainly applied to penalize Half-Cheetah for touching the ground with heels, knees, etc. Note that f may be approximated to the ordinary step function by multiplying its argument by a large positive number. The reward is calculated as a sum of basic rewards that play various roles in different stages of learning. Its definition is provided below for $I_j = \{1, 2, 3, 6, 7, 8\}$,

$$r = 0.5(\dot{x}_3 + \dot{x}_4) \quad (23)$$

$$- 0.05 \sum_{i \in I_j} \max\{|\tau_i^0 + a_i^0| - 1, 0\} \quad (24)$$

$$- 0.1 \sum_{i \in I_j} \min\{|\tau_i^w|, 50\} \quad (25)$$

$$+ (\max\{\min\{\dot{y}_3, 1\}, f(0.5[y_0 > 0] + 5y_0), \\ \min\{\dot{y}_4, 1\}, f(0.5[y_9 > 0] + 5y_9)\} - 1) \quad (26)$$

$$\times (1 - f(\dot{x}_3/2 + \dot{x}_4/2)) \\ + (f(9y_1) - 1) + (f(5y_2) - 1) + (f(2y_5) - 1). \quad (27)$$

The meaning of particular elements of the reward is as follows:

- (23) the reward for moving ahead with the speed as large as possible,
- (24) a penalty for an attempt to apply torque from outside of the permissible interval,
- (25) a penalty for the internal force that keeps the joint angle within its bounds; τ_i^w is a torque applied by the “tendon” (if the i -th joint angle not equal to either of its bounds, then $\tau_i^w = 0$),
- (26) a penalty for not moving the trunk up and keeping the paws on the ground when the animal is not moving forward,
- (27) penalties for touching the ground with the heel, the knee, and the head, respectively.

Although the reward is quite complex, its main part is the Half-Cheetah speed [m/s].

5.2.0.2. The learning algorithm

In order to make Half-Cheetah run, we combine the Natural Actor–Critic and the idea of experience replay. The algorithm we apply (Replaying NAC, in short) is specified below.

The policy applied to Half-Cheetah is similar to the one applied to the cart-pole swing-up problem. Namely, it is comprised of two parts: a neural network and a normal distribution. The input of the network is the state. The output becomes a mean value of the normal distribution with covariance matrix $C = 5^2 I$. The distribution generates actions. The elements of the six-dimensional action a are transformed into the control stimuli a^0 as

$$a_i^0 = a_j/30$$

where the indexes $i = 1, 2, 3, 6, 7, 8$ correspond to $j = 1, 2, 3, 4, 5, 6$, respectively. The constant $1/30$ results from the fact that the control signals a_i^0 cover the interval $[-1, 1]$ while we would prefer the outputs of the network to be of larger scale, in this case to roughly cover the interval $[-30, 30]$.

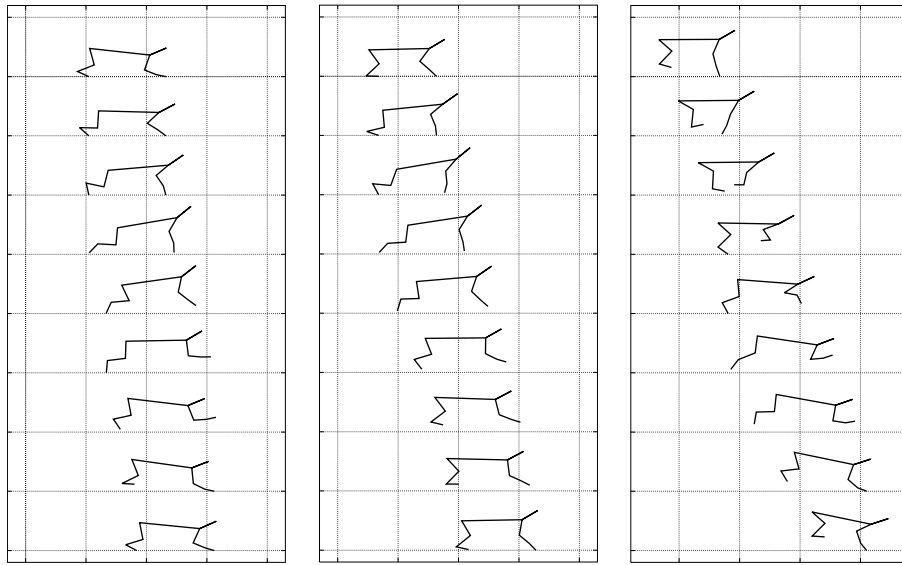


Fig. 4. Typical sequences of Half-Cheetah states at various stages of learning by the Replaying NAC. *Left:* Awkward walk after 1 h of training. *Middle:* Trot after 2.5 h of training. *Right:* Nimble run after 7 h of training.

Algorithm 5 The Natural Actor–Critic with Experience Replay (Replaying NAC).

- 0: $t := 1$. Initialize θ and v .
- 1: Draw and execute an action, $a_t \sim \pi(\cdot; s_t, \theta)$.
- 2: Register the tuple $s_t, a_t, \theta, r_t, s_{t+1}$ in the database.
- 3: Make sure only N most recent tuples remain in the database.
- 4: Repeat $v(t)$ times:
 - 5: Draw $i \in \{t - N + 1, t - N + 2, \dots, t\}$.
 Draw $K \sim \text{Geom}(\rho)$.
 Calculate SUM equal to

$$\sum_{k=0}^K \alpha^k e_{i+k} \min \left\{ \prod_{j=0}^k \frac{\pi(a_{i+j}; s_{i+j}, \theta)}{\pi(a_{i+j}; s_{i+j}, \theta_{i+j})}, b \right\}$$

 for e_{i+k} equal to

$$r_{i+k} + \gamma V(s_{i+k+1}; v) - \bar{Q}(a_{i+k}, s_{i+k}; v, \theta).$$
 - 6: Adjust θ along an estimator of $\phi(s_i, \theta, v)$:
 - 7: $\theta := \theta + \beta_t^\theta v''$.
 - 8: Adjust v along an estimator of $\psi(s_i, \theta, v)$:
 - 9: $v := v + \beta_t^v \nabla_v \bar{Q}(a_i, s_i; v, \theta) SUM$.
 $v'' := (1 - \beta_t^\theta) v''$.
- 10: Assign $t := t + 1$ and repeat from Step 1.

The second approximator used by the learning algorithm is the critic, i.e. the approximation of the action-value function (7). The critic output is a sum of two elements. The first one results from the actor and the second one, $\bar{V}$, is implemented by a feedforward neural network. Both the critic network and the actor network have the form of a two-layer perceptron with a linear output layer. Their hidden layers consist of M^A (the actor) and M^C (the critic) sigmoidal (arctan) elements. Each neuron has a constant input (bias). The initial weights of the hidden layers are drawn randomly from the normal distribution $N(0, 1)$ and the initial weights of the output layers are set to zero.

The parameters of the Natural Actor–Critic are as follows: (the actor) $M^A = 80$, $C = 5^2 I$, (the critic) $M^C = 160$, (step-sizes) $\beta_t^\theta \equiv 10^{-3}$, $\beta_t^v \equiv 3.10^{-5}$, (estimation) $\gamma = 0.99$, $\lambda = 0.9$. The resulting learning curves are depicted on the top part of Fig. 5.

The parameters of the Replaying NAC are as follows: (the actor) $M^A = 80$, $C = 5^2 I$, (the critic) $M^C = 160$, (step-sizes) $\beta_t^\theta \equiv 10^{-3}$, $\beta_t^v \equiv 2.10^{-5}$, (database) $N = 3.10^4$, (estimation) $\gamma = \alpha = 0.99$, $\lambda = \rho = 0.9$, (computational effort) $c_0 = 30$, $c_1 = 0.3$. With a computer equipped with Intel Quad™ Q9300, the simulations of Half-Cheetah were carried out in real time.

5.2.0.3. Experiments

Learning curves for the setting discussed above are shown in Fig. 5. A single trial lasts, on the average, for 5 s. The top part of Fig. 5 reports experiments with the Natural Actor–Critic applied to Half-Cheetah. It is seen that the algorithm learns to control Half-Cheetah in about 3000 trials, which is about 42 h of Half-Cheetah time. The middle, and the bottom part of Fig. 5 present the averaged learning curve for the Replaying NAC applied to the same problem. The curve reports about 7 h of learning. The algorithm learns to control Half-Cheetah in about 4500 trials, which is about 6 h of Half-Cheetah time. It is interesting to observe that Half-Cheetah learned policy at various stages of learning (Fig. 4). In about 1000 trials (1 h of computer, or real, time) Half-Cheetah walks without significant difficulties, albeit awkwardly. After about 2000 trials (2.5 h) it trots. The movement obtained in 4500 trials (6 h) is a fairly nimble run, very similar to what one might expect from an animal constructed that way.

It is interesting to watch Half-Cheetah learning. At first it only stands shaking because of the noise in the control system. Thanks to the main part of the reward (23), it quickly learns to lean ahead. But then its start to fall on its head, which it quickly starts to avoid thanks to component (27) of the reward. All the time Half-Cheetah learns to jump, thanks to component (26) of the reward. After some time, it can move ahead by means of small jumps. Then, these jumps are smoothed to become a nimble run.

Let us now analyze whether the concepts introduced in the paper indeed improve the quality and the speed of learning. The bottom part of Fig. 5 demonstrates how the intensity of the computation process translates into the speed of learning. It is seen that the larger the intensity, the faster the convergence. Plausibly, for a certain large c_0 , further increase of this parameter does not yield learning speed improvement. However, it is quite time-consuming to investigate high values of c_0 . In fact, for $n > 30$ the computations are too slow to take place in real time of Half-Cheetah. The computer time of a single run is then proportional to c_0 , and for $c_0 = 100$ it is around 31 h. Obviously, it is only a matter of computer power. With a fast enough computer, the processing for $c_0 = 100$ could be performed in real time of Half-Cheetah. A satisfactory policy could be then obtained after 3200 trails, which is about 4 h of Half-Cheetah time.

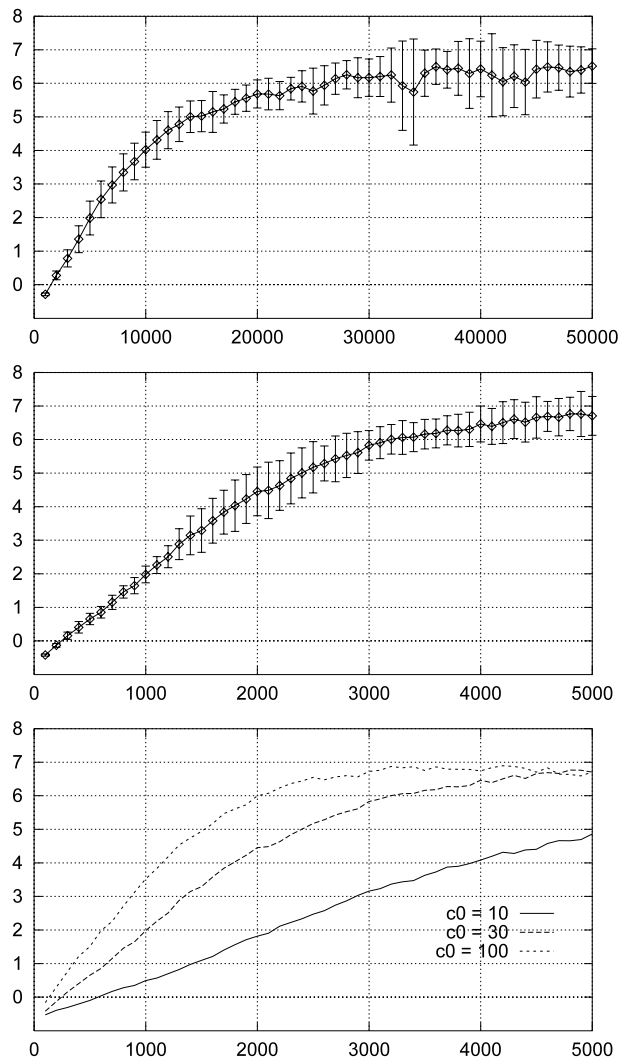


Fig. 5. Actor-Critics for Half-Cheetah: The average reward vs. trial number. *Top:* The Natural Actor-Critic. Each point averages 1000 consecutive trials. The curve averages 10 runs. The one-sigma limits are calculated to assess run-to-run variability of trial averages. *Middle:* The Natural Actor-Critic with Experience Replay (ER). Note that the number of trials in this figure is about 10 times smaller than that of the top figure for the basic method. The curve averages 10 runs and each point averages only 100 consecutive trials. *Bottom:* The Natural Actor-Critic with ER for various replaying intensity, c_0 . Each curve averages 10 runs. It is seen that the larger the intensity, the faster the convergence.

5.3. Discussion

The step-sizes applied in all the presented experiments were set invariable in time in order to make equal terms for comparison of various methods. While convergence of all analyzed algorithms requires the step-sizes to be decreasing in time, in this study we were interested more in the speed of learning of various methods than in their limit properties analyzed earlier. In the experiments with the Basic Actor-Critic and the Natural Actor-Critic, the step-sizes were of the largest values still giving good ultimate policies but not causing instability of the learning process. The following heuristics was applied, rather successfully, to determine step-sizes for the algorithms with experience replay. When introducing experience replay to a given Actor-Critic algorithm working with a given problem, one should decrease the step-sizes about 3 times. The only exception is the actor step-size in the Natural Actor-Critic; it should be kept unchanged. This is because the actor vector in this method is not adjusted by a function of actions; therefore, it can be adjusted in the algorithm with experience

replay in the same way as in the original one, rather than by means of a randomized-truncated estimator.

The experiments suggest that the algorithms augmented by experience replay learn policies of the same quality in much shorter time in comparison with the original methods. Growing intensity of the experience replay generally increases the speed of learning. Obviously there are some limits to this growth. These limits were possible to determine in the case of the Replaying BAC and cart-pole swing-up. Namely, more than 1000 replays per time-step does not yield any further increase in the speed of learning. Experiments with the Replaying NAC and Half-Cheetah did not detect these limits: the more computational power that was engaged, the larger the speed of learning that was obtained. In both cases, application of experience replay caused an increase in the speed of learning of sequential Actor-Critic algorithms up to 20 times.

All implementations of Actor-Critic algorithms presented in this study applied nonlinear critics taught incrementally by means of the $TD(\lambda)$ estimators of future rewards. In fact this method is proved to be convergent only for linear approximators (Tsitsiklis & Van Roy, 1997) and it surely does not converge for certain nonlinear structures. However, our experiments are based on the assumption that this method is also convergent for a large class of nonlinear approximators that encompasses feedforward neural networks. Available empirical results confirm that assumption.

Introduction of experience replay into an actor-critic algorithm requires providing a few additional coefficients. One of these is the threshold for the density ratios truncating, denoted above by b . Our experience suggests that a value around 2 or 3 gives satisfactory behavior. A smaller value gives poor ultimate performance (large bias of the improvement direction estimators) and larger ones result in instability of the learning process (large variance of the estimators). However, it is unknown whether this experience is representative. Another unobvious parameter is the size of the database of registered events, denoted above by N . The following rule for defining its value has been applied quite successfully in the experiments. The size of the database should be (i) at least an order of magnitude larger than the sum of dimensions of the actor vector and the critic vector, and (ii) large enough for the database to contain a representative history of an agent-environment interaction.

6. Conclusions

In this paper we combined the technique of experience replay for reinforcement learning speedup with sequential Actor-Critic-like algorithms. These algorithms deserve serious attention since they represent the most successful approach to applying reinforcement learning to realistic control tasks with continuous state and action spaces. Unlike the Q-learning algorithm for which experience replay was used in prior work, they are on-policy algorithms that require the actions producing observations for policy improvements to be drawn from the current, actually followed policy. Therefore, to make this combination correct, it was necessary to ensure that the discrepancy between current policy and the policy used at the time when the replayed experience was generated is compensated appropriately. This is achieved by a novel variant of experience replay, using importance sampling for the estimation of the direction of policy modification. The analysis of the corresponding estimators established bounds of their variance and bias, and the latter was shown to asymptotically vanish when the step-size was reduced. This form of experience replay can therefore be used to augment Actor-Critic reinforcement learning algorithms without degrading their convergence properties.

As has been verified experimentally, experience replay gives a radical learning speedup by allocating the available computational power to processing past observations. The required number of interactions with the environment, which is critical for the applicability of reinforcement learning to real-world tasks, can be considerably reduced. For a moderately difficult cart-pole swing-up task we observed a speedup factor of 20, allowing a satisfactory policy to be reached after as little as 20 min of the cart-pole time, compared to about 6 h required by the Basic Actor–Critic and 2 h obtained by the method of Doya (2000). For the fairly difficult Half-Cheetah task we observed a speedup factor of 10, allowing a satisfactory policy to be reached after as little as 4 h of Half-Cheetah time (assuming availability of very large computation power), compared to about 42 h required by the natural Actor–Critic. This is definitely an important step toward enhancing the application potential of reinforcement learning to realistic, challenging control tasks.

References

- Abbeel, P., & Ng, A. Y. (2005). Exploration and apprenticeship learning in reinforcement learning. In *Proceedings of the 22nd international conference on machine learning* (pp. 1–8). New York, NY, USA: ACM.
- Abbeel, P., Quigley, M., & Ng, A. Y. (2006). Using inaccurate models in reinforcement learning. In *Proceedings of the 23rd international conference on machine learning* (pp. 1–8). New York, NY, USA: ACM.
- Bartlett, P. L., & Baxter, J. (2000). Stochastic optimization of controlled partially observable Markov decision processes. In *Proceedings of the 39th IEEE conference on decision and control* (pp. 124–129). Vol. 1.
- Barto, A. G., Sutton, R. S., & Anderson, C. W. (1983). Neuronlike adaptive elements that can learn difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 13, 834–846.
- Bhatnagar, S., Sutton, R. S., Ghavamzadeh, M., & Lee, M. (2008). Incremental natural actor–critic algorithms. In *Advances in neural information processing systems: Vol. 21*.
- Bhatnagar, S., Sutton, R. S., Ghavamzadeh, M., & Lee, M. (2009). Natural actor–critic algorithms. *Automatica* (in press).
- Cichosz, P. (1999). An analysis of experience replay in temporal difference learning. *Cybernetics and Systems*, 30, 341–363.
- Doya, K. (2000). Reinforcement learning in continuous time and space. *Neural Computation*, 12(1), 219–245.
- Frank, J., Mannor, S., & Precup, D. (2008). Reinforcement learning in the presence of rare events. In *Proceedings of the 25th international conference on machine learning* (pp. 580–587).
- Gullapalli, V. (1990). A stochastic reinforcement learning algorithm for learning real-valued functions. *Neural Networks*, 3(6), 671–692.
- Kakade, S. (2002). A natural policy gradient. In *Advances in neural information processing systems: Vol. 14* (pp. 1531–1538). MIT Press.
- Kimura, H., & Kobayashi, S. (1998). An analysis of actor/critic algorithm using eligibility traces: Reinforcement learning with imperfect value functions. In *Proceedings of the 15th international conference on machine learning* (pp. 278–286).
- Konda, V., & Tsitsiklis, J. (2003). Actor–critic algorithms. *SIAM Journal on Control and Optimization*, 42(4), 1143–1166.
- Kushner, H. J., & Yin, G. (1997). *Stochastic approximation algorithms and applications*. New York: Springer-Verlag.
- Lin, L.-J. (1992). Reinforcement learning for robots using neural networks. *Ph.D. thesis*. Carnegie Mellon University, Pittsburgh, PA, USA.
- Lin, L.-J., & Mitchell, T. M. (1992). Memory approaches to reinforcement learning in non-Markovian domains. *Tech. Rep. CMU-CS-92-138*. Carnegie Mellon University, Pittsburgh, PA 15213.
- Mahadevan, S., & Connell, J. (1992). Automatic programming of behavior-based robots using reinforcement learning. *Artificial Intelligence*, 55(2–3), 311–365.
- Nedić, A., & Bertsekas, D. P. (2003). Least squares policy evaluation algorithms with linear function approximation. *Discrete Event Dynamic Systems*, 13(1–2), 79–110.
- Peshkin, L., & Mukherjee, S. (2001). Bounds on sample size for policy evaluation in Markov environments. In *Proceedings of the 14th annual conference on computational learning theory* (pp. 616–629). Vol. 2111 Springer, Berlin.
- Peshkin, L., & Shelton, C. (2002). Learning from scarce experience. In *Proceedings of the 19th international conference on machine learning* (pp. 498–505).
- Peters, J., & Schaal, S. (2008). Natural actor–critic. *Neurocomputing*, 71(7–9), 1180–1190.
- Peters, J., Vijayakumar, S., & Schaal, S. (2003). Reinforcement learning for humanoid robotics. In *Humanoids2003, Third IEEE-RAS international conference on humanoid robots*.
- Precup, D., Sutton, R. S., & Dasgupta, S. (2001). Off-policy temporal-difference learning with function approximation. In *Proceedings of the 18th international conference on machine learning* (pp. 417–424). San Francisco, CA: Morgan Kaufmann.
- Rubinstein, R. Y. (1981). *Simulation and the Monte Carlo method*. New York, USA: John Wiley & Sons, Inc.
- Shelton, C. R. (2001). Policy improvement for POMDPs using normalized importance sampling. In *Proceedings of the international conference on uncertainty in artificial intelligence* (pp. 496–503).
- Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. *Machine Learning*, 3, 9–44.
- Sutton, R. S. (1990). Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the 7th international conference on machine learning* (pp. 216–224). Morgan Kaufmann.
- Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning: An introduction*. Cambridge, MA: MIT Press.
- Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. In *Advances in neural information processing systems: Vol. 12* (pp. 1057–1063). MIT Press.
- Tsitsiklis, J. N., & Van Roy, B. (1997). An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, 42(5), 674–690.
- Uchibe, E., & Doya, K. (2004). Competitive-cooperative-concurrent reinforcement learning with importance sampling. In *Proceedings of international conference on simulation of adaptive behavior: From animals and animats* (pp. 287–296).
- Walker, M. W., & Orin, D. E. (1982). Efficient dynamic computer simulation of robotic mechanisms. *Journal of Dynamic Systems, Measurement and Control*, 104, 205–211.
- Watkins, C., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292.
- Wawrzyński, P. (2007). Learning to control a 6-degree-of-freedom walking robot. In *Proceedings of EUROCON 2007* (pp. 698–705). IEEE.
- Wawrzyński, P., & Pacut, A. (2006). Balanced importance sampling estimation. In *Proceedings of the international conference on information processing and management of uncertainty in knowledge-based systems* (pp. 66–73).
- Williams, R. J. (1989). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(23).

An Analysis of Actor/Critic Algorithms using Eligibility Traces: Reinforcement Learning with Imperfect Value Functions

Hajime Kimura*
Tokyo Institute of Technology
gen@fe.dis.titech.ac.jp

Shigenobu Kobayashi
Tokyo Institute of Technology
kobayasi@dis.titech.ac.jp

Abstract

We present an analysis of actor/critic algorithms, in which the actor updates its policy using eligibility traces of the policy parameters. Most of the theoretical results for eligibility traces have been for only critic's value iteration algorithms. This paper investigates what the actor's eligibility trace does. The results show that the algorithm is an extension of Williams' REINFORCE algorithms for infinite horizon reinforcement tasks, and then the critic provides an appropriate reinforcement baseline for the actor. Thanks to the actor's eligibility trace, the actor improves its policy by using a gradient of actual return, not by using a gradient of the estimated return in the critic. It enables the agent to learn a fairly good policy under the condition that the approximated value function in the critic is hopelessly inaccurate for conventional actor/critic algorithms. Also, if an accurate value function is estimated by the critic, the actor's learning is dramatically accelerated in our test cases. The behavior of the algorithm is demonstrated through simulations of a linear quadratic control problem and a pole balancing problem.

1 Introduction

Actor/critic architecture is an adaptive version of policy iteration [Kaelbling et al.96]. In general, policy iteration alternates two phases: a policy evaluation phase and a policy improvement phase. The actor implements a *stochastic policy* that maps from a representation of a state to a probability distribution over

actions. The critic attempts to estimate the evaluation function for the current policy. The actor improves its control policy using critic's *temporal difference (TD)* as an effective reinforcement. In many cases, the policy improvement is executed concurrently with the policy evaluation, because it is not feasible to wait for the policy evaluation to converge.

The actor/critic algorithms have been successfully applied to a variety of delayed reinforcement tasks; ASE/ACE architecture for a pole balancing [Barto et al. 83] [Gullapalli 92], RFALCON for a pole balancing and for control of a ball-beam system [Lin et al. 96], a cart-pole swing-up task [Doya 96]. Although convergence proofs for the actor/critic algorithms (e.g. [Williams et al. 90] and [Gullapalli 92]) are less than value-iteration based algorithms such as Q-learning [Watkins et al. 92], the actor/critic algorithms have the following practical advantages.

- It is easy to implement multidimensional continuous action, that is often mixed with discrete action [Gullapalli 92]. Because the actor selects action by its stochastic policy, therefore problems of action selection like as Q-learning does not exist. The Q-learning needs to estimate returns for all state-action pairs, but the critic would estimate only the return of each state.
- Memory-less stochastic policies can be considerably better than memory-less deterministic policies in the case of partially observable Markov decision processes (POMDPs) [Singh 94] [Jaakkola 94] or multi-player games [Littman 94].
- It is easy to incorporate an expert's knowledge into the learning system by applying conventional supervised learning techniques to the actor [Clouse et al. 92].

Eligibility traces are a fundamental mechanism that has been widely used to handle delayed reward [Singh 96]. Also the traces are often used to overcome non-Markovian effects [Sutton 95],

* Interdisciplinary Graduate School of Science and Engineering, Tokyo Institute of Technology, 4259 Nagatsuta Midori-ku Yokohama 226-8502 JAPAN.

[Pendrith et al. 96]. In Barto, Sutton and Anderson's ASE/ACE architecture, both the critic and the actor make use of the eligibility trace. Theoretical results of eligibility traces in the context of TD(λ) [Sutton 88] have been obtained. But, in actor/critic algorithms, the effect of the actor's trace has not been investigated. This paper presents an analysis of an actor/critic algorithm, in which the actor improves its policy using eligibility traces of the policy parameters. This may be the first analysis of the actor's eligibility traces.

2 Discounted Reward Criteria

At each discrete time t , the agent observes x_t containing information about its current state, select action a_t , and then receives an instantaneous reward r_t resulting from state transition in the environment. In general, the reward and the next state may be random, but their probability distributions are assumed to depend only on x_t and a_t in Markov decision processes (MDPs), in which many reinforcement learning algorithms are studied. The objective of reinforcement learning is to construct a policy that maximizes the agent's performance. A natural performance measure for infinite horizon tasks is the cumulative discounted reward:

$$V_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} , \quad (1)$$

where the discount factor, $0 \leq \gamma < 1$ specifies the importance of future rewards. V_t is called the *actual return*, that specifies how good the reward sequence after time t is. By this notation, the goal of the learning is to maximize the *expected return*. In MDPs, the expected return can be defined for all states as:

$$V^{\pi}(x) = E_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_k | x_0 = x \right] , \quad (2)$$

where E_{π} denotes the expectation assuming the agent always uses stationary policy π . $V^{\pi}(x)$ is called the *value function*, that specifies how good the given state x is. In MDPs, the goal of the learning is to find an optimal policy that maximizes the value of each state x defined by Equation 2. Although similar value functions can be given in POMDPs, difficulties to define the optimum have pointed out in [Singh 94].

3 Actor/Critic Algorithms

Figure 1 and 2 give an overview of actor/critic algorithms [Sutton 90] [Crites et al. 94]. There are many ways to implement the policy and its updating scheme in the actor. The algorithms for the critic are mostly TD methods. We should notice the following two points; one is the actor implements stochastic policy,

the other is the actor improves its policy using TD-error. This paper especially investigates an algorithm for the actor.

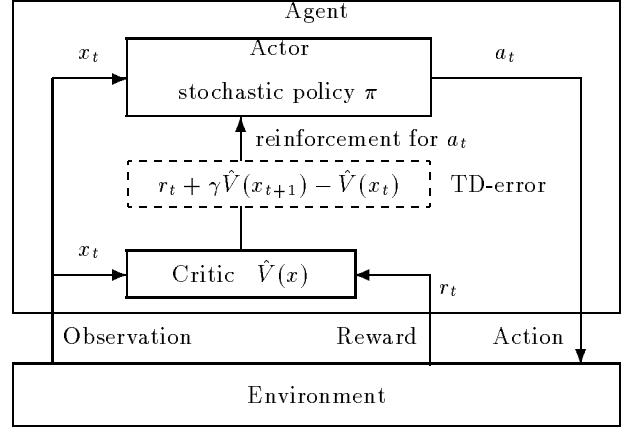


Figure 1: A generic actor/critic framework.

1. The agent observes x_t in the environment, and the actor executes action a_t according to the current stochastic policy π .
2. The critic receives the immediate reward r_t , and then observes the resulting next state x_{t+1} . The critic provides TD error as a useful reinforcement feedback to the actor, according to

$$(\text{TD-error}) = [r_t + \gamma \hat{V}(x_{t+1})] - \hat{V}(x_t) ,$$

where $0 \leq \gamma < 1$ is the discount factor, $\hat{V}(x)$ is an estimated value function by the critic.

3. The actor updates the stochastic policy using the TD-error. If (TD-error) > 0 , action a_t performed relatively good and its probability should be increased. If (TD-error) < 0 , action a_t performed relatively poorly and its probability should be decreased.
4. The critic updates estimated value function $\hat{V}(x)$ according to TD methods. e.g., TD(0) algorithm adjusts $\hat{V}(x_t) \leftarrow \hat{V}(x_t) + \alpha (\text{TD-error})$, where α is the learning rate.
5. Go to step 1.

Figure 2: Main loop of the generic actor/critic algorithm.

4 Adding Eligibility Trace to the Actor

4.1 Function Approximation for Stochastic Policies

In this paper, $\pi(a, W, x)$ denotes probability of selecting action a under the policy π in the observation x . The $\pi(a, W, X)$ is taken to be a probability density function when the set of possible action is continuous. The policy is represented by a parametric function approximator using the internal variable vector W . The agent can improve the policy π by modifying W . For example, W corresponds to synaptic weights where the action selecting probability is represented by neural networks, or W means weight of rules in classifier systems. The advantage of using the notation of the parametric function $\pi()$ is that computational restriction and mechanisms of the agent can be specified simply by a form of the function, and then we can provide a sound theory of learning algorithms for arbitrary types of the actor.

4.2 Details of the Algorithm

Figure 3 specifies the actor/critic algorithm that uses the eligibility trace in the actor. The ASE/ACE system configured for pole-balancing [Barto et al. 83] is just an instance of this algorithm. The actor's eligibility in step 3 is the same variable defined in Williams' REINFORCE algorithms [Williams 92]. The eligibility $e_i(t)$ specifies a correlation between the associated policy parameter w_i and the executed action a_t . The eligibility trace $D_i(t)$ is a discounted running average of eligibility. It accumulates the agent's history. When a positive reinforcement is given, the actor updates W so that the probability of actions recorded in the history is increased. It means the TD-error at the time t affects not only the action a_t but also $a_{t-1}, a_{t-2}, \dots$. At first glance, this idea is senseless for improving the policy, but it has very interesting features given in detail later. Note that the algorithm shown in Figure 3 is identical to a stochastic gradient ascent for discounted reward [Kimura et al. 97] when the actor's discount factor $\beta = \gamma$ and the $\hat{V}(x)$ in the critic equals a constant b for all observations.

The actor requires a memory to implement W for the policy and to implement D_i for the eligibility trace. The amount of the memory for D_i is equal to W 's.

4.3 An Analysis of the Algorithm

Assume that the actor's discount factor β equals γ , and for all $t < 0$, $D_i(t) = 0$, then the algorithm shown

1. The agent observes x_t , and the actor executes action a_t with probability $\pi(a_t, W, x_t)$.
2. The critic receives the immediate reward r_t , and then observes the resulting next state x_{t+1} . The critic provides TD error to the actor according to

$$(\text{TD-error}) = [r_t + \gamma \hat{V}(x_{t+1})] - \hat{V}(x_t), \quad (3)$$

where $0 \leq \gamma < 1$ is the discount factor, $\hat{V}(x)$ is an estimated value function by the critic.

3. The actor updates the stochastic policy using the TD-error according to:

$$\begin{aligned} \text{Eligibility:} \quad e_i(t) &= \frac{\partial}{\partial w_i} \ln(\pi(a_t, W, x_t)) \\ \text{Eligibility Trace:} \quad D_i(t) &= e_i(t) + \beta D_i(t-1), \\ \Delta w_i(t) &= (\text{TD-error}) D_i(t) \\ W &\leftarrow W + \alpha_p \Delta W(t), \end{aligned}$$

where w_i denotes the i^{th} component of W , e_i and D_i are the associated eligibility and eligibility trace respectively, β ($0 \leq \beta < 1$) is a discount factor for the eligibility trace, α_p is the learning rate for the actor.

4. The critic updates estimated value function $\hat{V}(x)$ according to TD methods. e.g., TD(0) algorithm adjusts $\hat{V}(x) \leftarrow \hat{V}(x) + \alpha (\text{TD-error})$, where α is the learning rate.
5. Go to step 1.

Figure 3: The actor/critic algorithm adding the eligibility trace to the actor.

in Figure 3 updates the policy parameters as:

$$\begin{aligned} & \sum_{t=0}^{\infty} \Delta w_i(t) \\ &= \sum_{t=0}^{\infty} (r_t + \gamma \hat{V}(x_{t+1}) - \hat{V}(x_t)) D_i(t) \\ &= \sum_{t=0}^{\infty} (r_t + \gamma \hat{V}(x_{t+1}) - \hat{V}(x_t)) \left(\sum_{\tau=0}^t \gamma^{t-\tau} e_i(\tau) \right) \\ &= \sum_{t=0}^{\infty} e_i(t) \left(\sum_{\tau=t}^{\infty} \gamma^{\tau-t} (r_{\tau} + \gamma \hat{V}(x_{\tau+1}) - \hat{V}(x_{\tau})) \right) \\ &= \sum_{t=0}^{\infty} e_i(t) \left(\left(\sum_{\tau=t}^{\infty} \gamma^{\tau-t} r_{\tau} \right) - \hat{V}(x_t) \right) \quad (4) \\ &= \sum_{t=0}^{\infty} e_i(t) (V_t - \hat{V}(x_t)) \quad (5) \end{aligned}$$

Equation 5 is given by Equation 1 and 4. Here we assume that the statistics of the random variable V_t depends only on the current policy parameter. It means $E\{V_t\}$ is a deterministic function of W , where E de-

notes the expectation operator. This assumption may be right if the policy is converged to an equilibrium point. The critic's estimation $\hat{V}(x_t)$ is obviously independent of the action at the time t . From the theory of Williams' REINFORCE algorithm [Williams 92], the value V_t and $\hat{V}(x_t)$ in Equation 5 can be seen as a reinforcement signal and a reinforcement baseline respectively, then we have $E\{e_i(t)(V_t - \hat{V}(x_t))\} = (\partial/\partial w_i)E\{V_t\}$. It says that the algorithm updates policy parameters statistically in a direction for increasing the actual return V_t , not in a direction of a gradient of estimated value function in the critic. Also It can be seen as an extension of reinforcement comparison methods [Sutton et al. 98], then $\hat{V}(x_t)$ corresponds to the *reference reward*.

From the above analysis and Figure 3, we can explain what the actor's eligibility trace does. At the time t , the algorithm reinforces a_t using TD error $r_t + \hat{V}(x_{t+1}) - \hat{V}(x_t)$ as a temporary expedient, thereafter the actor's eligibility trace replaces $\hat{V}(x_{t+1})$ with the actual return $(r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \dots)$ in order.

The critic does not affect the direction of the average update vector, because the critic works as a reinforcement baseline. Therefore, the actor can improve its policy, whether the critic is able to learn the value function or not. If the critic approximates the value function well, the actor's learning would be accelerated.

The above results are under the special condition $\beta = \gamma$. If $\beta = 0$, the actor updates W in the direction of the gradient of the approximated value function in the critic. The β ($0 < \beta < \gamma$) interpolates between the above two limiting cases. The characteristics of the β are similar to the λ in TD(λ) [Sutton 88] and Q(λ)-learning [Peng et al. 94].

5 Preliminary Experiments

This section demonstrates the performance of the algorithm applying to a simple linear control problem.

5.1 A Linear Quadratic Regulator (LQR)

The following linear control problem can serve as a benchmark of delayed reinforcement tasks [Baird 94]. At a given discrete-time t , the state of the environment is the real value x_t . The agent chooses a control action a_t that is also real value. The dynamics of the environment is:

$$x_{t+1} = x_t + a_t + \text{noise} , \quad (6)$$

where the *noise* is the normal distribution that follows the standard deviation $\sigma_{\text{noise}} = 0.5$. The immediate

reward is given by

$$r_t = -x_t^2 - a_t^2 . \quad (7)$$

The goal is to maximize the total discounted reward, defined by Equation 1 or 2 for all x . Because the task is a linear quadratic regulator (LQR) problem, it is possible to calculate the optimal control rule. From the discrete-time Riccati equation, the optimum regulator is given by

$$a_t = -k_1 x_t , \text{ where } k_1 = 1 - \frac{2}{1 + 2\gamma + \sqrt{4\gamma^2 + 1}} . \quad (8)$$

The optimum value function is given by $V^*(x_t) = -k_2 x_t^2$, where k_2 is a some positive constant. In this experiment, the set of possible states is constrained to lie in the range $[-4, 4]$. When the state transition given by Equation 6 does not result in the range $[-4, 4]$, the x_t is truncated. When the agent chooses an action that is not lie in the range $[-4, 4]$, the action executed in the environment is also truncated.

5.2 Implementation for the LQR Problem

5.2.1 The Actor

Remember the policy $\pi(a, W, X)$ is a probability density function when the set of possible action is continuous. The normal distribution is a simple multiparameter distribution for a continuous random variable. It has two parameters, the mean μ and the standard deviation σ . When the policy function π is given by the equation 9, the eligibility of μ and σ are

$$\pi(a, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(a - \mu)^2}{2\sigma^2}\right) \quad (9)$$

$$e_\mu = \frac{a_t - \mu}{\sigma^2} \quad (10)$$

$$e_\sigma = \frac{(a_t - \mu)^2 - \sigma^2}{\sigma^3} . \quad (11)$$

One useful feature of such a *Gaussian unit* [Williams 92] is that the agent has a potential to control its degree of exploratory behavior. We must draw attention to the fact that the eligibility is to divergent when σ goes close to 0, because the parameter σ is occupying the denominators of Equation 10 and 11. The divergence of the eligibility has a bad influence on the algorithm. One way to overcome this problem is to control the step size of the update parameter vector using σ . It is obtained by setting the learning rate parameter proportional to σ^2 , then the eligibility can be seen as

$$e_\mu = a_t - \mu , e_\sigma = \frac{(a_t - \mu)^2 - \sigma^2}{\sigma} . \quad (12)$$

The actor would first compute μ and σ deterministically and then draw its output from the normal distribution that follows mean equal to μ and standard

deviation equal to σ . The actor has two internal variables, w_1 and w_2 , and computes the values of μ and σ according to

$$\mu = w_1 x_t \quad , \quad \sigma = \frac{1}{1 + \exp(-w_2)} . \quad (13)$$

Then, w_1 can be seen as a feedback gain. The reason for this calculation of σ is to guarantee the σ to keep positive. The e_1 and e_2 are the characteristic eligibilities of w_1 and w_2 respectively. From Equation 12, e_1 and e_2 are given by

$$e_1 = e_\mu \frac{\partial}{\partial w_1} \mu = (a_t - \mu) x_t , \quad (14)$$

$$e_2 = e_\sigma \frac{\partial}{\partial w_2} \sigma = ((a_t - \mu)^2 - \sigma^2)(1 - \sigma) . \quad (15)$$

The w_1 is initialized to 0.35 ± 0.15 , and $w_2 = 0$, i.e., $\sigma = 0.5$. The learning rate α_p is fixed to 0.001.

5.2.2 The Critic

The critic quantizes the continuous state-space ($-4 \leq x \leq 4$) into an array of boxes. We have tried two types of the quantizing: one is discretizing x evenly into 3 boxes, the other is 10 boxes. And the critic attempts to store in each box a prediction of the value $\hat{V}$ by using TD(0) [Sutton 88]. The learning rate α for TD(0) is fixed to 0.2.

5.3 Simulation Results

Figure 4, 5, 6, 7 and 8 show the performance of 100 trials in the LQR problem with the discount rate $\gamma = 0.9$.

Figure 4 shows the performance of the algorithm, in which the critic uses 3 boxes, the actor does not use eligibility traces, i.e., $\beta = 0$. Figure 6 shows the performance where the critic uses 10 boxes, the actor does not use the traces. The algorithm in Figure 6 converged close to the optimum feedback gain. In contrast, Figure 4 didn't. The reason for this is that the ability of the function approximation (3 boxes) is insufficient for learning policy without the trace.

Figure 5 shows the performance where the critic uses 3 boxes, the actor uses the trace, $\beta = \gamma = 0.9$. It achieved much better results in terms of both the learning efficiency and the quality of the mean value of the converged policy than the algorithm in Figure 4 or 5. Obviously, the actor's eligibility trace relates these two advantages. The reason for the learning efficiency in this case may be that the actor's trace accelerates propagating information. The better quality of the policy is clearly owing to the property that the actor improves its policy by using a gradient of actual return, shown in Section 4.3. Therefore, the algorithm

using the trace was not influenced by the critic's ability in terms of the quality of the mean of the policy. We can also see this property in Figure 8, but its deviation is considerably large. Figure 9 shows the value function that is defined by Equation 1 and 7 over the parameter space μ and σ . The value of performance is fairly flat around the optimal solution. This is the reason that the deviation of the policy is large in Figure 8. This example makes it clear that the critic controls step-size of the actor's backups so that the step-size is taken to be smaller around the local maximum.

The algorithm in Figure 7 achieved best results in terms of both the mean and the deviation of the policy. The reason for this may be owing to the critic's perfect value estimation.

In this preliminary experiment, we can see that the algorithm using the actor's eligibility trace performed better than the algorithm without using the trace in the same computational resources.

Here we presented the results of the actor-critic that use only TD(0) in the critic, but we have also experimented on TD(λ) where $0 < \lambda \leq 1$. Roughly speaking, we have poor performance when the λ approaches close to 1. It follows from this that the eligibility trace in the critic cannot make up for the critic's poor ability of function approximation. The details of the experiments using TD(λ) will appear in other papers.

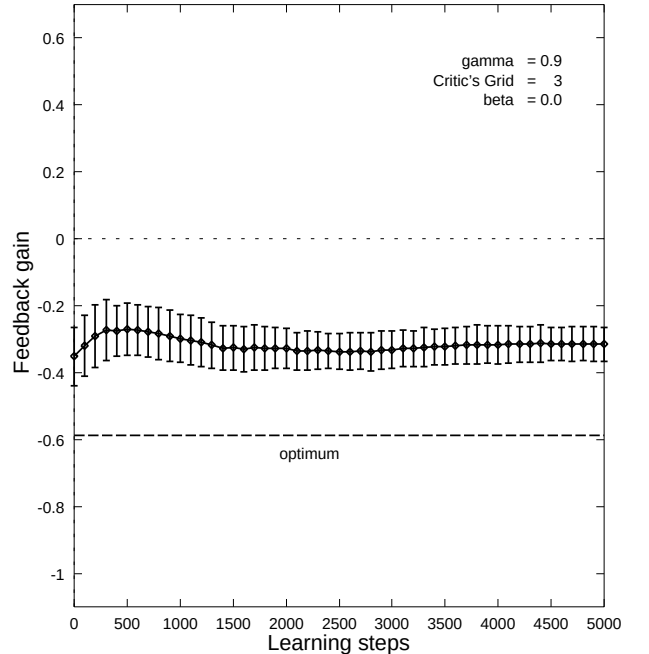


Figure 4: The average performance of 100 trials without the actor's eligibility trace ($\beta = 0$). The critic uses 3 boxes.

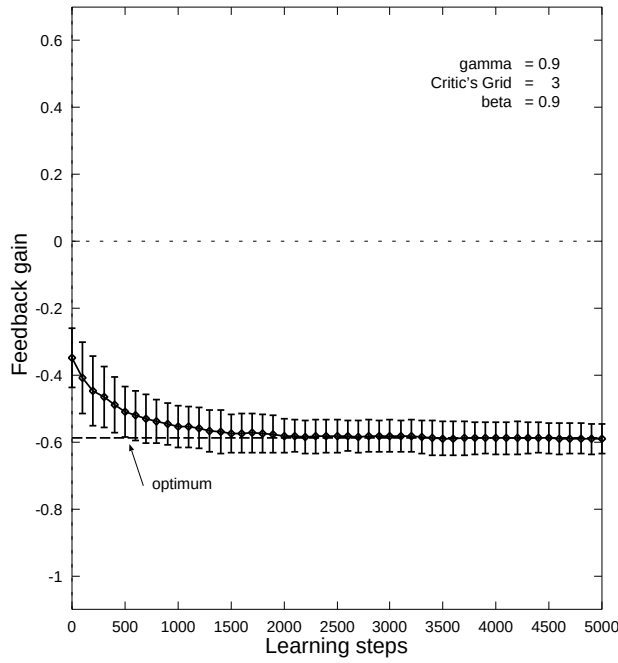


Figure 5: The average performance of 100 trials using the actor's trace $\beta = 0.9$. The critic uses 3 boxes.

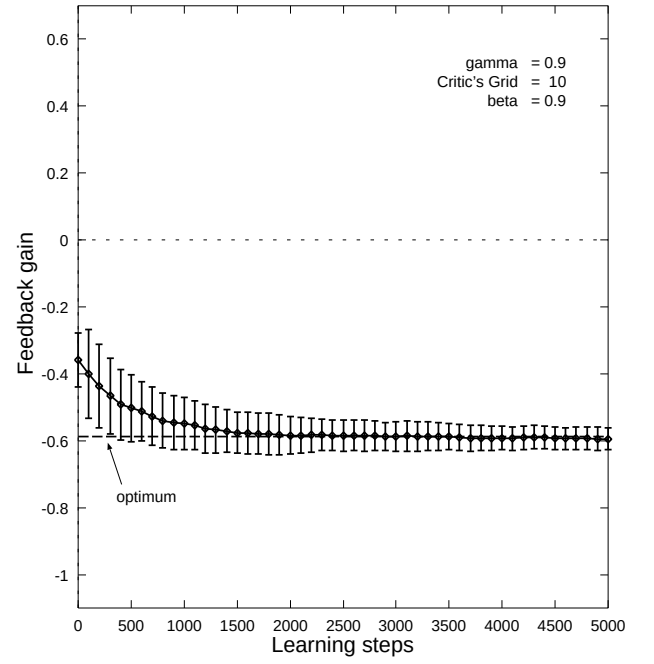


Figure 7: The average performance of 100 trials using the actor's trace $\beta = 0.9$. The critic uses 10 boxes.

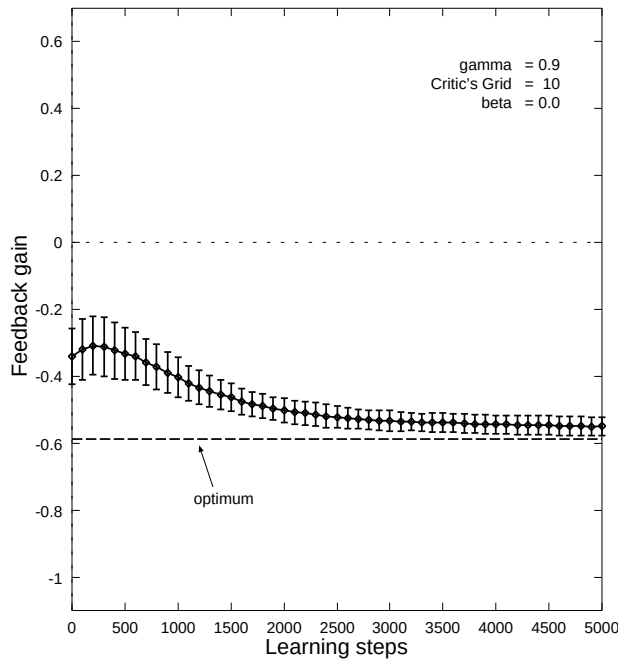


Figure 6: The average performance of 100 trials without the actor's trace ($\beta = 0$). The critic uses 10 boxes.

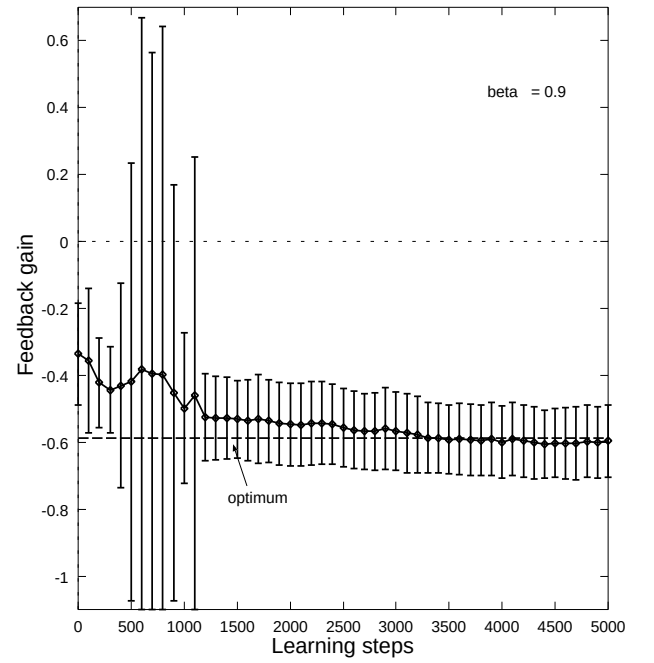


Figure 8: The average performance of 100 trials. $\beta = 0.9$. The agent learns without the critic, i.e., the critic provides $\hat{V}(x) = 0$ for all x .

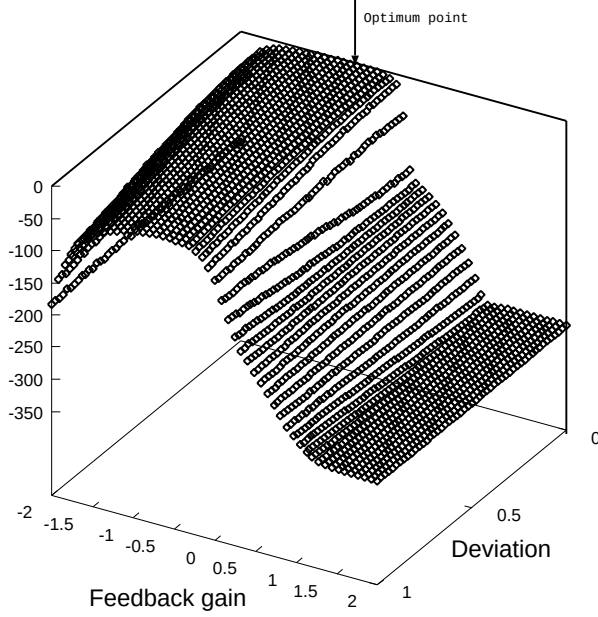


Figure 9: Value function over the parameter space in the LQR problem, where $\gamma = 0.9$. It is fairly flat around the optimum: $\mu = -0.5884$, $\sigma = 0$.

6 Applying to a Cart-Pole Problem

The behavior of this algorithm is demonstrated through a computer simulation of a cart-pole control task, that is a multi-dimensional nonlinear non-quadratic problem. We modified the cart-pole problem described in [Barto et al. 83] so that the action is taken to be continuous.

6.1 Problem Formulation

The dynamics of the cart-pole system is modeled by

$$\ddot{\theta} = \frac{g \sin \theta + \cos \theta \left(\frac{-F - m\ell\dot{\theta}^2 \sin \theta + \mu_c s g n(\dot{x})}{M+m} \right) - \frac{\mu_p \dot{\theta}}{m\ell}}{\ell \left(\frac{4}{3} - \frac{m \cos^2 \theta}{M+m} \right)},$$

$$\ddot{x} = \frac{F + m\ell \left(\dot{\theta}^2 \sin \theta - \ddot{\theta} \cos \theta \right) - \mu_c s g n(\dot{x})}{M+m},$$

where $M = 1.0$ (kg) denotes mass of the cart, $m = 0.1$ (kg) is mass of the pole, $2\ell = 1$ (m) is a length of the pole, $g = 9.8$ (m/sec^2) is the acceleration of gravity, F (N) denotes the force applied to cart's center of mass, $\mu_c = 0.0005$ is a coefficient of friction of cart, $\mu_p = 0.000002$ is a coefficient of friction of pole. In this simulation, we use discrete-time system to approximate these equations, where $\Delta t = 0.02$ sec. At each discrete time step, the agent observes $(x, \dot{x}, \theta, \dot{\theta})$, and controls the force F . The agent can execute action in

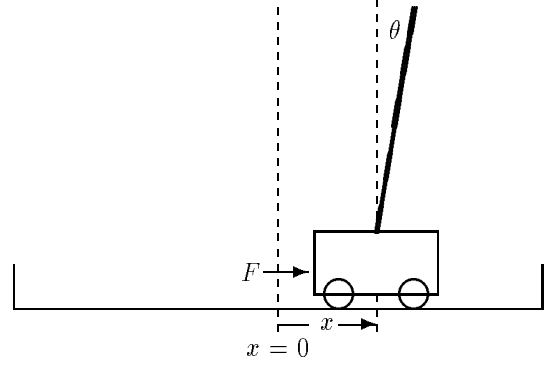


Figure 10: The cart-pole problem.

arbitrary range, but the possible action in the cart-pole system is constrained to lie in the range $[-20, 20]$ (N). When the agent chooses an action which is not lie in that range, the action executed in the system is truncated. The system begins with $(x, \dot{x}, \theta, \dot{\theta}) = (0, 0, 0, 0)$. The system fails and receives a reward (penalty) signal of -1 when the pole falls over ± 12 degrees or the cart runs over the bounds of its track ($-2.4 \leq x \leq 2.4$), then the cart-pole system is reset to the initial state.

6.2 Details of the Agent

In this experiment, the actor adopts similar implementation shown in Equation 9 and 12. The state space is constrained in the range $(x, \dot{x}, \theta, \dot{\theta}) = (\pm 2.4 \text{ m}, \pm 2 \text{ m/sec}, \pm \pi \times 12/180 \text{ rad}, \pm 1.5 \text{ rad/sec})$. The actor has five internal variables $w_1 \dots w_5$, and computes the μ and σ according to

$$\mu = w_1 \frac{x_t}{2.4} + w_2 \frac{\dot{x}_t}{2} + w_3 \frac{\theta_t}{12\pi/180} + w_4 \frac{\dot{\theta}_t}{1.5},$$

$$\sigma = 0.1 + \frac{1}{1 + \exp(-w_5)}. \quad (16)$$

Similarly to Equation 14 and 15, the eligibilities $e_1 \dots e_5$ are given by

$$e_1 = (a_t - \mu) x_t, \quad e_2 = (a_t - \mu) \dot{x}_t$$

$$e_3 = (a_t - \mu) \theta_t, \quad e_4 = (a_t - \mu) \dot{\theta}_t$$

$$e_5 = ((a_t - \mu)^2 - \sigma^2)(1 + 0.1 - \sigma).$$

The critic discretizes the normalized state space evenly into $3 \times 3 \times 3 \times 3 = 81$ boxes, and attempts to store in each box $\hat{V}$ by using TD(0) algorithm [Sutton 88]. The parameters are set to $\gamma = 0.95$, $\alpha = 0.5$, $\alpha_p = 0.001$.

6.3 Simulation Results

Figure 11 shows the performance of three learning algorithms in which the policy representation is the

same. The actor/critic algorithm using the actor's trace achieved best results. In contrast, the algorithm without using the trace couldn't learn the control policy because of the poor ability of function approximation in the critic.

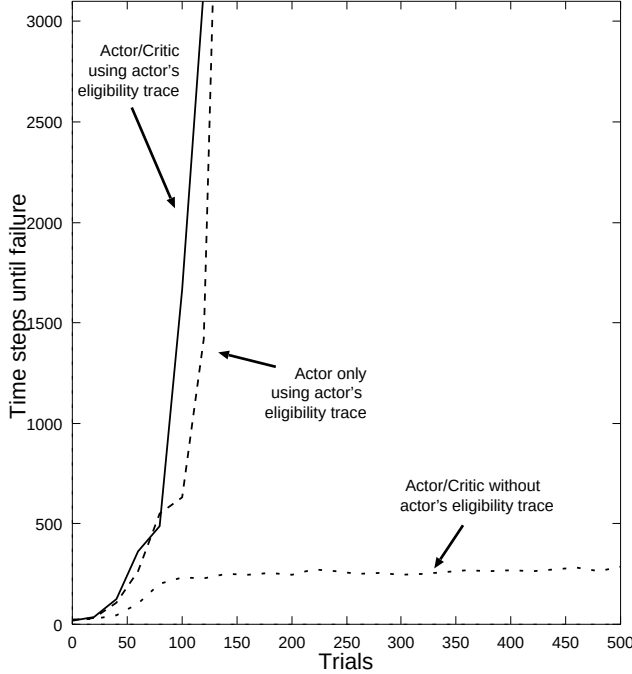


Figure 11: The average performance of three algorithms on 100 trials. The critic uses $3 \times 3 \times 3$ boxes. A trial means an attempt from initial state to a failure.

7 Discussion

Representation of Policies: First of all, actor/critic algorithms should have sufficient ability to approximate policies. If it is satisfied, use of the actor's eligibility trace ($\beta = \gamma$) enables to learn an acceptable policy with less cost rather than increasing the critic's ability of function approximation in our test cases. The reason is that the policy function representation would require less memory than the representation of the state-action value function in many cases.

Controlling Step-Size of Backups: It is analytically shown in Section 4.3 that the critic provides an appropriate reinforcement baseline to the actor. The adaptive baseline controls step-size of the actor's backups so that the step-size is taken to be smaller around the local maximum. This property would contribute the better learning efficiency and the suppression of harmful drift of the policy that are shown in the experiments.

To Overcome non-Markovian:

There are many ways to implement the critic's learning scheme. [Peng et al. 94] and [Sutton 95] pointed out that increasing λ makes $TD(\lambda)$ less sensitive to non-Markovian effect. The actor's eligibility traces are also useful in getting over non-Markovian problems [Kimura et al. 97]. Therefore, the combination of $TD(\lambda)$ and the actor's eligibility trace will be robust in non-Markovian problems.

Combining with Efficient DP-based Methods:

If the hidden state is relatively small in the state space, the agent may perform good in which efficient DP-based algorithms are adopted for the critic. The DP-based algorithms accelerate the actor's learning in completely observable states, and the actor's stochastic policy and its trace ($\beta = \gamma$) would make up for the non-Markovian effects owing to the hidden state or function approximation.

8 Conclusions

This paper presented an analysis of actor/critic algorithms in which the actor updates its policy using the eligibility trace of the policy parameters. The results show that when the discount rate of the value function equals the discount factor of the actor's trace, the actor improves its policy by using a gradient of actual return, not by using a gradient of the estimated return in the critic. Then, the critic provides an adaptive reinforcement baseline to the actor controlling the step-size of the actor's backups. It enables the agent to learn a fairly good policy under the condition that the approximated value function in the critic is hopelessly imperfect. The behavior is demonstrated through simulations showing that the trace contributes the learning efficiency and the suppression of undesirable drifts of the policy. Analysis of the algorithm in non-Markovian environments is a future work.

Acknowledgements

We would like to thank Andrew Barto, Jing Peng, Jeff Schneider, Satinder Singh, Richard Sutton, and reviewers for many helpful comments and suggestions.

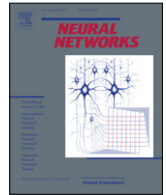
References

- [Baird 94] Baird, L. C.: Reinforcement Learning in Continuous Time: Advantage Updating, *Proceedings of IEEE International Conference on Neural Networks*, Vol. IV, pp. 2448-2453 (1994).
- [Barto et al. 83] Barto, A. G., Sutton, R. S. and Anderson, C. W.: Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems, *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no.5, September/October 1983, pp. 834-846.



Contents lists available at SciVerse ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

2012 Special Issue

Autonomous reinforcement learning with experience replay

Paweł Wawrzyński^{a,*}, Ajay Kumar Tanwani^b^a Warsaw University of Technology, Institute of Control and Computation Engineering, Poland^b École Polytechnique Fédérale De Lausanne, Switzerland

ARTICLE INFO

Keywords:

Reinforcement learning
Autonomous learning
Step-size estimation
Actor–critic

ABSTRACT

This paper considers the issues of efficiency and autonomy that are required to make reinforcement learning suitable for real-life control tasks. A real-time reinforcement learning algorithm is presented that repeatedly adjusts the control policy with the use of previously collected samples, and autonomously estimates the appropriate step-sizes for the learning updates. The algorithm is based on the actor–critic with experience replay whose step-sizes are determined on-line by an enhanced fixed point algorithm for on-line neural network training. An experimental study with simulated octopus arm and half-cheetah demonstrates the feasibility of the proposed algorithm to solve difficult learning control problems in an autonomous way within reasonably short time.

© 2012 Elsevier Ltd. All rights reserved.

1. Introduction

Reinforcement learning (RL) is a potentially useful methodology for control optimization. It may be the only feasible approach to many problems. However, its widespread use is limited by the total time many RL algorithms require to solve a given problem. This time encompasses duration of a single run in collecting appropriate number of samples multiplied by the number of runs necessary to tune problem dependent parameters. Selection of appropriate parameter setting requires learning to start over again, thereby, making the previously collected data useless. The cost of collecting data from control process and the elapsed time to handcraft the parameters makes learning infeasible for many real world tasks. Ideally, the learner should interact with the environment in a ‘plug-and-play’ manner and optimize a control policy in a single trial with minimum number of data samples. The main focus of this research is to make reinforcement learning suitable for real control applications and minimize the associated implementation costs.

A broad class of RL algorithms that encompass Actor–Critics (Bhatnagar, Sutton, Ghavamzadeh, & Lee, 2009; Konda & Tsitsiklis, 2003; Peters, Vijayakumar, & Schaal, 2005) and policy gradient methods (Sutton, McAllester, Singh, & Mansour, 2000; Williams, 1992) optimizes a control policy on the basis of stochastic gradient estimates. In these algorithms, the control policy is parameterized by a vector that is optimized incrementally on the basis of the data coming from the control process. The policy vector adjustment is

local in the sense that the algorithm moves a small step in the estimate of the improvement direction; its length is defined by the step-size of the learning process. The appropriate length of the step-size depends on the stage of the learning process and setting it wrong can make the process unstable or prohibitively slow. In this paper, we present a real-time reinforcement learning algorithm that is considerably efficient and autonomous due to: (1) *experience replay*—it stores the agent–environment interaction samples in a database and repeatedly draws previous samples—simultaneous to the interaction—to estimate the improvement direction of policy vector and speed up learning (Wawrzyński, 2009), and (2) *step-size estimation*—it autonomously determines the appropriate step-size to optimize the learning updates and eliminate the overhead of its manual tuning (Wawrzyński, 2010).

Optimization by means of improvement direction estimates is a tool commonly used in RL but it is not unique to this field. It is the main principle of a wide class of stochastic approximation algorithms (Kushner & Yin, 1997). In all these methods the step-size plays a key role in controlling their speed and stability. Despite significance of the problem of step-size estimation, its generally applicable solution has not been found.

1.1. Related work

A lot of effort is made in the reinforcement learning community to speed up the learning methods by using the previously collected samples. The concept of reusing samples evolved from recomputing previous experience by means of dynamic programming (Sutton, 1990). Application of the *importance sampling* technique (Rubinstein, 1981) made it possible to optimize a parametric control policy with the use of experience gained with other policies

* Corresponding author.

E-mail addresses: pwawrzyński@gmail.com, p.wawrzyński@elka.pw.edu.pl (P. Wawrzyński), ajay.tanwani@epfl.ch (A.K. Tanwani).

(Hachiya, Peters, & Sugiyama, 2011; Peshkin & Mukherjee, 2001). Kober and Peters (2011) proposed an algorithm called PoWER—Policy Learning by Weighting Exploration with the Returns—that combines the previous experience by per-decision importance weighting technique. The approach to sample reuse developed here is experience replay (Adam, Busoniu, & Babuska, 2012; Cichosz, 1999) applied to actor-critics with the use of the randomized truncated estimators (Wawrzyński, 2009).

Real world applications of reinforcement learning require a number of parameters to be defined and empirically tuned, e.g., reward function, policy structure, step-size, exploration noise, discount factor, and others specific to the learning algorithm in use. Despite several promising approaches to provide leverage for these overheads such as the use of apprenticeship learning to find reward function (Abbeel & Ng, 2005), dynamic evolution of policy parameters to get its appropriate size and representation (Kormushev, Ugurlu, Calinon, Tsagarakis, & Caldwell, 2011), the parameter that remains difficult to determine autonomously in reinforcement learning is the step-size of the learning process. The work (Sutton, 1992a, 1992b) established *delta-bar-delta* algorithm, introduced earlier by Jacobs (1988) as a generally applicable approach to step-size estimation in RL. The method requires the initial step-size to be specified and collapses if this value is too large, which delimits its autonomous use. Consequently, the solution to the problem of step-size determination in reinforcement learning is still a subject of active research (George & Powell, 2006; Noda, 2009; Schraudolph, Yu, & Aberdeen, 2006).

Many approaches to step-size estimation have been designed as tools for online neural network training. Algorithms of this type include *delta-delta* (Silva & Almeida, 1990), aforementioned *delta-bar-delta*, and stochastic meta-descent (Schraudolph & Giannakopoulos, 2000). Among others, step-sizes can be estimated to ensure Lyapunov-stability of the process (Behera, Kumar, & Patnaik, 2006; Kathirvalavakumar & Subavathi, 2009).

Most methods of autonomous or adaptive step-size estimation are designed as recursions that are updated from one sample to another (George & Powell, 2006). This design is often based on some problem-dependent parameters and may be sensitive to wrong initialization or nonstationarity. In this work we build upon the fixed-point method of step-size estimation (Wawrzyński, 2010) which has a different design to remedy the aforementioned problems. By computing gradient estimates always at two points it captures global characteristics of the process, thereby, obtaining robustness and independence from any parameters. In Wawrzyński and Papis (2011) this approach was specialized to neural-network on-line training, e.g., each weight of the network was assigned a separate step-size. Here the original algorithm is enhanced and adopted to provide autonomy to RL with experience replay.

1.2. Contributions

The purpose of this paper is to present a reinforcement learning framework that is fast and autonomous enough for on-line optimization of control of systems with complex dynamics and multidimensional, possibly continuous, state and input spaces. The contribution of this paper may be summarized in the following points:

- An enhanced fixed point method of step-size estimation (Wawrzyński, 2010) is presented.
- The above method of step-size estimation is combined with experience replay for actor-critic class of RL algorithms (Wawrzyński, 2009).
- The proposed approach is implemented on two simulated robot-control problems, both with complex dynamics and rich state and action spaces, namely *octopus arm* and *half-cheetah*.

The rest of the paper is organized as follows: Section 2 formulates the reinforcement learning problem followed by the description of actor-critic algorithm with experience replay in Section 3. In Section 4, we describe the fixed point method of step-size adaptation in a generic stochastic descent procedure. The resulting actor-critic reinforcement learning algorithm with experience replay and step-size estimation is proposed in Section 5. Experimental study is presented in Section 6 on challenging learning control problems. The last section concludes the paper with an outlook to future work.

2. Problem formulation

We consider the standard reinforcement learning setup under the Markov Decision Process (MDP) framework (Sutton & Barto, 1998). The problem concerns an agent that observes the state of its environment, s_t , in discrete time, $t = 1, 2, 3, \dots$, performs an action, a_t , which moves the environment to the next state, s_{t+1} , and gives the agent a reward, $r_t \in \mathcal{R}$. The environment is in general stochastic which means that the consecutive state, s_{t+1} , is a result of sampling from the transition distribution conditioned on the preceding state, s_t , and the action, a_t . Mathematically,

$$s_{t+1} \sim P_s(\cdot | s_t, a_t).$$

The reward may depend deterministically on the current action and the next state, $r_t = r(a_t, s_{t+1})$. A particular MDP is a tuple $\langle \mathcal{S}, \mathcal{A}, P_s, r \rangle$ where $\mathcal{S}$ and $\mathcal{A}$ are the state and action spaces, respectively; $\{P_s(\cdot | s, a) : s \in \mathcal{S}, a \in \mathcal{A}\}$ is a set of state transition distributions and r is the reward function. The transition distributions, P_s , and the reward function, r , are initially unknown to the agent. The goal of learning is to determine a stochastic control policy π that assigns actions to states such that in each state the agent may expect the highest rewards in the future. Note that the control policy maps each state into a distribution of actions rather than a single action. Performing different actions in each state, the agent is able to differentiate good actions from inferior ones in each state. The control policy π is parameterized by the policy vector, θ , e.g., a set of weights in a neural network. It can be represented as

$$a \sim \pi(\cdot | s, \theta).$$

Let us denote by π_θ the policy of the form π with the fixed parameter θ . The quality of this policy is measured by the *value-function* V^{π_θ} defined as the expected value of the discounted sum of future rewards the agent may gain in the future, starting from the given state, s , at a certain time, t , and the policy π_θ :

$$V^{\pi_\theta}(s) = E \left(\sum_{i \geq 0} \gamma^i r_{t+i} | s_t = s, \text{ policy} = \pi_\theta \right). \quad (1)$$

The parameter $\gamma \in [0, 1)$ is the discount factor that defines the weight of distant rewards in relation to those obtained sooner. The quality of a policy may also be evaluated using the *action-value function* Q^{π_θ} , defined as

$$Q^{\pi_\theta}(s, a) = E \left(\sum_{i \geq 0} \gamma^i r_{t+i} | s_t = s, a_t = a, \text{ policy} = \pi_\theta \right). \quad (2)$$

It is the expected value of the same sum, but here it also depends on the first action to be performed, that is the action performed in the given state. This function is crucial for the policy optimization as it tells good actions from inferior ones in the same state.

We seek a learning algorithm that can optimize the control policy (i) without parameters determined experimentally, (ii) within short time of agent–environment interaction, but not necessarily after small amount of computation.

Algorithm 1. The classic actor–critic.

- 0: Set $y_\theta = 0, y_v = 0, t = 1$. Initialize θ and v
- 1: Choose the action: $a_t \sim \pi(\cdot; s_t, \theta)$
- 2: Execute a_t , evaluate s_{t+1} and r_t
- 3: Calculate the temporal difference $d_t(v)$
- 4: $d_t(v) = r_t + \gamma \bar{V}(s_{t+1}; v) - \bar{V}(s_t; v)$
- 5: Update the actor:
- 6: $y_\theta := (\gamma \lambda) y_\theta + \beta_t^\theta \nabla_\theta \ln \pi(a_t; s_t, \theta)$
- 7: $\theta := \theta + y_\theta d_t$
- 8: Update the critic:
- 9: $y_v := (\gamma \lambda) y_v + \beta_t^v \nabla_v \bar{V}(s_t; v)$
- 10: $v := v + y_v d_t$
- 11: Assign $t := t + 1$ and repeat from Point 1

3. Actor–critic with experience replay

This section presents actor–critic algorithms (Barto, Sutton, & Anderson, 1983; Bhatnagar et al., 2009; Kimura & Kobayashi, 1998; Konda & Tsitsiklis, 2003) and the way of its speeding-up by experience replay (Adam et al., 2012; Cichosz, 1999; Wawrzyński, 2009). Particular attention is paid to the ‘classic’ actor–critic in the form presented by Kimura and Kobayashi (1998) because this algorithm is conceptually the simplest among its family to which the ideas presented in further sections are directly applicable.

3.1. Classic Actor–Critic

The classic actor–critic algorithm employs two data structures. The first one is an actor, $\pi(a; s, \theta)$, that represents a stochastic control policy to generate random actions on the basis of the given state and the policy vector $\theta \in \mathbb{R}^{n_\theta}$. The second one is a critic, $\bar{V}(s; v)$, that represents an approximator of the value function (1) parameterized by the critic vector $v \in \mathbb{R}^{n_v}$. The algorithm is based on a special estimator of the action-value function $Q^{\pi_\theta}(s_t, a_t)$ (Eq. (2)). This estimator has the form:

$$R_t^\lambda = r_t + \gamma \bar{V}(s_{t+1}; v) + \sum_{i \geq 1} (\gamma \lambda)^i (r_{t+i} + \gamma \bar{V}(s_{t+i+1}; v) - \bar{V}(s_{t+i}; v)). \quad (3)$$

The parameter $\lambda \in [0, 1]$ defines how much R_t^λ is influenced by the value-function approximator, $\bar{V}$, and the true rewards, r . For $\lambda = 1$ the estimator is based on true rewards only and thus unbiased, but its variance may be very large. For $\lambda = 0$ variance of the estimator is the smallest, but it is biased by the inaccuracy of the value-function approximator. Moderate values of λ balance the bias and the variance of the estimator.

The classic actor–critic is sketched in Algorithm 1. It can be verified that after a visit in state s_t , for $t = 1, 2, \dots$ the algorithm modifies the policy vector θ by the product $\beta_t^\theta \hat{\phi}_t$ where β_t^θ is a step-size (small positive number) and $\hat{\phi}_t$ is the policy vector improvement direction estimate given by

$$\hat{\phi}_t = (R_t^\lambda - \bar{V}(s_t; v)) \nabla_\theta \ln \pi(a_t; s_t, \theta), \quad (4)$$

where ∇_θ means gradient with respect to θ . (For $\lambda = 0$, the modification takes place in step t , for $\lambda > 0$ it is also partially distributed over further steps). Consequently, if the action a_t turns out to bring rewards, R_t^λ , larger than $\bar{V}(s_t; v)$, i.e., the rewards expected in state s_t , then the probability of action a_t in state s_t is being increased. If, conversely, the action turns out to bring rewards smaller than expected, then its probability is being decreased.

The purpose of the critic, $\bar{V}(s; v)$ is to approximate the expected value of R_t^λ . Therefore, a visit in state s_t induces a modification of the critic vector v by the product $\beta_t^v \hat{\psi}_t$ where β_t^v is a step-size and

Algorithm 2. Actor–Critic with Experience Replay. Estimators mentioned in Steps 6 and 7 are based on the samples in a database.

- 1: $t := 1$, Initialize θ and v
- 2: Choose and execute an action, $a_t \sim \pi(\cdot; s_t, \theta)$
- 3: Assign $\pi_t = \pi(a_t; s_t, \theta)$
- 4: Repeat $v(t)$ times, begin
- 5: Draw $i \in \{t - N + 1, t - N + 2, \dots, t\}$
- 6: Adjust θ along an estimator of $\phi(s_i, \theta, v)$:
- 7: $\theta := \theta + \beta_t^\theta \hat{\phi}_i^r(\theta, v)$
- 8: Adjust v along an estimator of $\psi(s_i, \theta, v)$:
- 9: $v := v + \beta_t^v \hat{\psi}_i^r(\theta, v)$
- 10: End
- 11: Register the tuple $\langle s_t, a_t, \pi_t, r_t, s_{t+1} \rangle$ in the database
- 12: Make sure only N most recent tuples remain in the database.
- 13: Assign $t := t + 1$ and repeat from Line 2

$\hat{\psi}_t$ is the critic vector improvement direction estimate:

$$\hat{\psi}_t = (R_t^\lambda - \bar{V}(s_t; v)) \nabla_v \bar{V}(s_t; v). \quad (5)$$

3.2. Classic actor–critic with experience replay

The idea of experience replay is to repeatedly recall previous pieces of experience and apply them to update the current policy as a sequential RL algorithm (like the classic actor–critic) would if they have just happened. This idea was applied to actor–critics in Wawrzyński (2009). Namely, let us denote by

$$\phi(s, \theta, v) = E(\hat{\phi}_t | s_t = s, \theta, v) \quad (6)$$

a direction in which θ is on average modified to increase the rewards expected in state s . Also, let us denote by

$$\psi(s, \theta, v) = E(\hat{\psi}_t | s_t = s, \theta, v) \quad (7)$$

a direction in which v is on average modified to increase the accuracy of the value function approximation in state s .

After each instant t , the classic actor–critic estimates $\phi(s_t, \theta, v)$, i.e., the direction of policy improvement, and adjusts the policy vector θ along the estimate. Before the next instant t , the modified algorithm repeatedly chooses one of the recently visited states at random, s_i , estimates $\phi(s_i, \theta, v)$, and modifies the policy vector along the estimate. Essentially both algorithms achieve the same goal, but the modified one improves the current actor and critic with the use of the whole gathered experience rather than only the present event, like the classic method. Due to more exhaustive exploitation of information experience replay leads to faster learning at the cost of additional computation.

In Wawrzyński (2009), the *randomized-truncated estimators* are introduced for estimating $\phi(s_i, \theta, v)$ and $\psi(s_i, \theta, v)$. To define them, let b be the upper limit of the randomized truncated estimators with $b > 1$, θ_{i+j} be the policy vector applied to generate a_{i+j} , $\alpha \in [0, 1]$ and K be drawn independently from $Geom(\rho)$ the geometric distribution¹ with parameter $\rho \in [0, 1]$. The randomized-truncated estimators, $\hat{\phi}_i^r(\theta, v)$ and $\hat{\psi}_i^r(\theta, v)$ have a generic form that in the case of classic actor–critic is reduced to

$$\times \sum_{k=0}^K \alpha^k d_{i+k}(v) \min \left\{ \prod_{j=0}^k \frac{\pi(a_{i+j}; s_{i+j}, \theta)}{\pi_{i+j}}, b \right\}, \quad (8)$$

¹ That is, random variable K of values in $\{0, 1, 2, \dots\}$ has distribution $Geom(\rho)$, iff $P(K = m) = (1 - \rho)\rho^m$ for nonnegative integer m .

$$\hat{\psi}_i^r(\theta, v) = \nabla_v \bar{V}(s_i; v) \times \sum_{k=0}^K \alpha^k d_{i+k}(v) \min \left\{ \prod_{j=0}^k \frac{\pi(a_{i+j}; s_{i+j}, \theta)}{\pi_{i+j}}, b \right\} \quad (9)$$

where

$$d_i(v) = r_i + \gamma \bar{V}(s_{i+1}; v) - \bar{V}(s_i; v),$$

is the temporal difference and π_{i+j} is the probability of the action taken at instant $i+j$ (or it is the density if $\mathcal{A}$ is continuous). Remembering the definition of R_t^k (Eq. (3)) makes it easy to notice the similarity between $\hat{\phi}_t$ (Eq. (4)) and $\hat{\phi}_i^r$ (Eq. (8)), and between $\hat{\psi}_t$ (Eq. (5)) and $\hat{\psi}_i^r$ (Eq. (9)). Though, there are two important differences. First, the infinite sum in Eq. (3) is replaced by the finite one with appropriately designed random limit in Eqs. (8) and (9). Second, truncated density ratios are introduced in order to compensate for the difference between the current policy and the ones that generated the actions $a_t, a_{t-1}, \dots$ present in the database.

The actor-critic with experience replay is presented in Algorithm 2. The number of computation steps after t -th control step is defined by the function $\nu(t)$. Since computation is carried out on-line, this function is bounded from above. It should also be limited for small t to prevent overtraining with too few samples in the database. (In Section 5 this function is discussed in more detail.)

3.3. Generalized actor-critic with experience replay

Experience replay may be applied to the classic actor-critic to increase its learning speed, but also to a wide class of sequential actor-critic algorithms. As shown in Wawrzyński (2009) the sufficient condition is that each step in the environment prompts the original algorithm to do modifications (possibly distributed in time) of the policy vector and the critic vector with estimates, respectively, $\hat{\phi}_t$ and $\hat{\psi}_t$ of the generic form

$$G_t(\theta, v) \sum_{k \geq 0} (\alpha \rho)^k z_{t,k}(\theta, v)$$

where G_t is a vector defined by s_t and a_t , $\alpha \in [0, 1)$, $\rho \in [0, 1)$, and $z_{t,k} \in \mathcal{R}$ is defined by $s_{t+k}, a_{t+k}, r_{t+k}, s_{t+k+1}$, and possibly a_{t+k+1} .

4. Fixed point method of step-size estimation

The actor-critic algorithm with experience replay repeatedly samples data and uses it to compute a certain vector with which it updates parameters θ and v . This pattern of operation places it in a wide class of optimization algorithms based on stochastic gradient descent (Kushner & Yin, 1997). Each algorithm of that type defines a sequence

$$\theta_{t+1} = \theta_t - \beta_t g(\theta_t; \xi_t), \quad t = 1, 2, \dots \quad (10)$$

in which $\theta \in \mathcal{R}^{n_\theta}$ is a parameter to be optimized, $\beta_t, t = 1, 2, \dots$ is a sequence of step-sizes, $\xi_t, t = 1, 2, \dots$ is a sequence of data samples and g is a function such that for random ξ the expected value of $-g(\theta, \xi)$ defines the direction towards the local minimum of a certain quality index, $J : \mathcal{R}^{n_\theta} \mapsto \mathcal{R}$. Typically this direction is the opposite of the gradient

$$Eg(\theta, \xi) = \nabla J(\theta).$$

For example, suppose a feedforward neural network is trained to approximate a function from given samples. Then θ is the vector of neural weights, J is the quality index, ξ represents an input-output pair, and g is the gradient computed by means of e.g., error backpropagation. In order to train the network, the input-output pairs may be sampled repeatedly, and its weights may be adjusted according to Eq. (10), which is exactly how the

well established back-propagation algorithm works (Rumelhart, Hinton, & Williams, 1988).

For the sequence (10) to have convergence guarantees, a set of requirements needs to be fulfilled (Kushner & Yin, 1997). Among others, the sequence of step-sizes has to be vanishing at an appropriate pace:

$$\begin{cases} \sum_{t \geq 1} \beta_t = +\infty, \\ \sum_{t \geq 1} \beta_t^2 < +\infty. \end{cases} \quad (11)$$

Unfortunately the practical value of the above conditions is limited. Even if they are satisfied, the step-sizes may still be too large and hence make the convergence process unstable, or too small and hence yield a very slow convergence.

The fixed point method of step-size estimation addresses this issue by autonomously setting the step-sizes (Wawrzyński, 2010). The method is based on the following principles:

1. The process (10) is divided into parts such that within each part the step-size remains constant. The length, n , of a part starting at moment t is kept large enough to enable estimation of the expected value of $g(\theta_t, \xi)$ with sufficient accuracy.
2. The sums $G_{t,n}$ and $G_{t,n}^*$ are computed for each part beginning at time t and ending at time $t+n$

$$G_{t,n} = \sum_{i=0}^{n-1} g(\theta_{t+i}, \xi_{t+i}), \quad G_{t,n}^* = \sum_{i=0}^{n-1} g(\theta_t, \xi_{t+i}). \quad (12)$$

where the vector $G_{t,n}$ is proportional to the displacement of the point θ between instant t and $t+n$, and the vector $G_{t,n}^*$ is proportional to the displacement of the point θ if all the gradient estimators g are calculated using θ_t .

3. At the end of each part the discrepancy between $G_{t,n}^*$ and $G_{t,n}$ is registered. On average, it increases with the ratio of the step-size used and the optimal one giving fastest convergence. Therefore, if the discrepancy is too small, the step-size is being increased. If it is too large, it is decreased. If it is radically too large, then instability of the process is detected, the step-size is radically decreased, and θ moves back to the point where the part began.

This last principle is illustrated in Fig. 1. For a very small β , the vector θ_{t+i} moves through a flat slope of the function J . This results in small differences between $g(\theta_t, \xi_{t+i})$ and $g(\theta_{t+i}, \xi_{t+i})$, thereby, making the discrepancy between $G_{t,n}$ and $G_{t,n}^*$ small as well. The speed of the local minimum search can be increased with larger values of β . However, if β is too large, θ_{t+i} jumps over the minimum of the function to be optimized. This creates large differences between $g(\theta_t, \xi_{t+i})$ and $g(\theta_{t+i}, \xi_{t+i})$ manifested by large discrepancy between $G_{t,n}$ and $G_{t,n}^*$.

The papers (Wawrzyński, 2010; Wawrzyński & Papis, 2011) present a statistical analysis to formalize the above intuitive principles in a simple, unidimensional model. Since each $g(\theta_t, \xi_{t+i})$ in (12) is an unbiased estimate of $\nabla J(\theta_t)$, the value $\frac{1}{n} G_{t,n}^*$ is also an unbiased estimator of $\nabla J(\theta_t)$ and its quality increases with n . The model considers n that ensures signal-to-noise ratio of 1 in $\frac{1}{n} G_{t,n}^*$, namely

$$n = \frac{E_{\theta_t} \|g(\theta_t, \xi)\|^2}{\|\nabla J(\theta_t)\|^2}. \quad (13)$$

For n of that size, and constant $\alpha_0 \cong 0.15$ the difference

$$\alpha_0 \|G_{t,n}^*\|^2 - \|G_{t,n}^* - G_{t,n}\|^2 \quad (14)$$

is on average equal to 0 if the step-size is optimal for fast convergence of (10). If the step-size is smaller than optimal, the

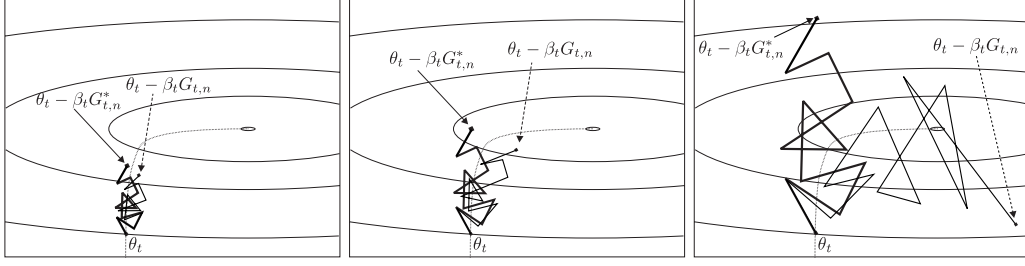


Fig. 1. The sketches depict contour lines of three different functions J with equal $\nabla J(\theta_t)$, β , and $g(\theta_t, \xi_{t+i})$ for $i \geq 0$. In n steps starting from t , the parameter in the J domain evolves from θ_t to $\theta_{t+n} = \theta_t - \beta_t G_{t,n}$. Left: Small curvature of J in relation to β , causes small difference of $-\beta_t G_{t,n}$ and $-\beta_t G_{t,n}^*$. Middle: Moderate curvature of J causes moderate difference between $-\beta_t G_{t,n}$ and $-\beta_t G_{t,n}^*$. Right: Very large curvature of J relatively to β causes instability of the optimization process which is manifested by divergence of $-\beta_t G_{t,n}$ and $-\beta_t G_{t,n}^*$ that grows with n .

first term of (14) is larger than the latter, and for too large step-size, the relation is opposite. Therefore, the difference (14) on average indicates the direction in which the step-size should be modified, i.e., increased or decreased. In other words, the step-size is stabilized at the best level when on average the discrepancy $\|G_{t,n}^* - G_{t,n}\|^2$ is stabilized at the value of $\alpha_0 \|G_{t,n}^*\|^2$. Also, if the term (14) with larger α_0 is applied to adjust the step-size, then the step-size is being stabilized at a higher level than one that is best for the purpose of static optimization (10). This last property will become useful in the next section.

In order to apply the same principle to multidimensional θ , we will consider only the direction of gradient estimate, that is we will project $G_{t,n}$ on $G_{t,n}^*$ and consider the discrepancy between $G_{t,n}^*$ and the projection. The indicator of step-size adjustment (14) will thus take the form

$$\alpha_0 \|G_{t,n}^*\|^2 - \|G_{t,n}^* - \text{proj}(G_{t,n}, G_{t,n}^*)\|^2, \quad (15)$$

for

$$\text{proj}(G, G^*) = G^* G^T G / (\|G^*\|^2 + \epsilon) \quad (16)$$

and ϵ being a small number that prevents division by zero.

The discussion above leads to Algorithm 3 for autonomously setting the step-sizes. It divides the learning process into parts or *estimation periods* in which the step-size β remains fixed. In order to compute n according to (13) the fraction (L/M) estimates

$$E_{\theta_t} \|g(\theta_t, \xi)\|^2$$

such that L adds $\|g(\theta_t, \xi_{t+i})\|^2$ and M counts them. The fraction (L'/M') estimates $\|\nabla J(\theta_t)\|^2$ on the basis of the following property. Let ξ and ξ' be sampled independently from the same distribution. Then we have

$$\begin{aligned} E_{\theta_t} g(\theta_t, \xi)^T g(\theta_t, \xi') &= E_{\theta_t} g(\theta_t, \xi)^T E_{\theta_t} g(\theta_t, \xi') \\ &= \|\nabla J(\theta_t)\|^2. \end{aligned}$$

The estimator of $\|\nabla J(\theta_t)\|^2$ can thus take the form of the statistics

$$\frac{\sum_{i=1}^n \sum_{i'=0}^{i-1} g(\theta_t, \xi_{t+i})^T g(\theta_t, \xi_{t+i'})}{\sum_{i=1}^n i} = \frac{\sum_{i=1}^n g(\theta_t, \xi_{t+i})^T G_{t,i}^*}{\sum_{i=1}^n i}, \quad (17)$$

whose numerator and denominator are denoted by L' and M' , respectively.

We now explain the algorithm in a step-wise manner for better understanding. In Line 3, it receives a new training example. In Lines 6–8, it initializes variables of the fixed point algorithm. In Lines 9–12, it updates estimators L/M and L'/M' . In Lines 13–16 it updates the values $G_{t,n}$, θ_{t+n} , $G_{t,n}^*$, and

$$h = \max_{i=1, \dots, n} \|G_{t,i}^*\|^2,$$

respectively.

In Line 19, the algorithm checks whether the learning process is stable. Its instability is manifested by $\|G_{t,n} - G_{t,n}^*\|^2$ being larger than $h = \max_{1 \leq i \leq n} \|G_{t,i}^*\|^2$. This condition is additionally tightened in the k_0 initial periods. If instability is detected, the estimation period is reinitialized with halved step-size.

In Lines 21–22, the algorithm checks if the present estimation period has come to its end. It happens when

$$n \geq \frac{L/M}{L'/M'},$$

that is when the estimator of $Eg(\theta_t, \xi)$ has appropriate quality (13). The period is also finished when

$$n \geq \frac{3}{2} n',$$

where n' is the duration of the previous period. This prevents the algorithm from getting stuck in local minima ($Eg(\theta, \xi) = 0$ in a minimum and appropriate duration of an estimation period becomes infinite).

In the end of an estimation period, the step-size is modified (Line 25) and a new estimation period is started. Namely, the step-size is incremented proportionally to the statistics (15). The increment is scaled with the variable $\bar{h}$, that is determined in Line 26 as the maximal value of previous $\|G_{t,n}^*\|^2 / (1 - \alpha_1)$. Line 24 prevents very small values of the scaling factor $\bar{h}$, for instance in the first period.

Every time a new estimation period begins, the estimators (L/M) and (L'/M') become obsolete and the amount of information they carry is decreased. In Lines 27–28, the algorithm decreases proportionally the weight of previous samples in comparison to forthcoming ones.

The algorithm updates the step-size after each estimation period to optimize the process (10). One such period is defined by the number of steps sufficient to estimate the direction of movement of θ with unity information-noise ratio. While various processes may differ substantially, the algorithm divides them to similar sequences of periods. Coefficients of the algorithm are chosen to make these sequences converge fast for all underlying processes (10) at its all stages.

In comparison to its version presented in Wawrzyński (2010), the algorithm is improved in two directions. First, it estimates $\|Eg(\theta_t, \xi)\|^2$ better by means of the statistics (17). Second, after each of first k_0 estimation periods it moves the process back to its origin; at this time only data is collected to estimate $\|Eg(\theta_0, \xi)\|^2$ and $E\|g(\theta_0, \xi)\|^2$. The goal of these modifications is to make the algorithm exploit data better and be robust to too large initial step-size.

5. Actor-critic with experience replay and the fixed point method of step-size estimation

This section discusses combination of the actor-critic algorithm with experience replay (ac&er) and the fixed point method of

Algorithm 3. The fixed point method of step-size estimation. The coefficients applied: $\alpha_1 = 0.9$, $\alpha_2 = 0.5$, $k_0 = 10$, initial $\beta = 1$.
FIXED_POINT

```

1: FP_INITIALIZE( $\theta_0$ )
2: Repeat
3:   Get  $\xi_{t+n}$ 
4:   FP_LOOP( $g(\theta_t, \xi_{t+n}), g(\theta_{t+n}, \xi_{t+n})$ )
5: Until some-stopping-condition

```

FP_INITIALIZE(θ_0)

```

6: Assign  $t := n := 0, k := 1, L = L' = M = M' := 0$ 
7: Assign  $G_{0,0} = G_{0,0}^* := 0, h = \bar{h} := 0, n' := +\infty$ 
8: Initialize  $\theta_0$ , and  $\beta$ 

```

FP_LOOP($g(\theta_t, \xi_{t+n}), g(\theta_{t+n}, \xi_{t+n})$)

```

9:  $L := L + \|g(\theta_t, \xi_{t+n})\|^2$ 
10:  $M := M + 1$ 
11:  $L' := \max\{L' + g(\theta_t, \xi_{t+n})^T G_{t,n}^*, 0\}$ 
12:  $M' := M' + n$ 
13:  $G_{t,n+1} := G_{t,n} + g(\theta_{t+n}, \xi_{t+n})$ 
14:  $\theta_{t+n+1} := \theta_{t+n} - \beta g(\theta_{t+n}, \xi_{t+n})$ 
15:  $G_{t,n+1}^* := G_{t,n}^* + g(\theta_t, \xi_{t+n})$ 
16:  $h := \max\{h, \|G_{t,n+1}^*\|^2\}$ 
17:  $n := n + 1$ 
18:  $newperiod := false$ 
19: If  $\|G_{t,n} - G_{t,n}^*\|^2 > h \vee (k < k_0 \wedge \|G_{t,n} - G_{t,n}^*\|^2 > 0.3h)$  then
20:    $\beta := \beta/2, newperiod := true$ 
21: If  $\neg newperiod \wedge n \geq 3 \wedge L' > 0$ 
22:    $\wedge n \geq \min\{(L/M)/(L'/M'), \frac{3}{2}n'\}$  then begin
23:   If  $k \geq k_0$  then begin
24:      $\bar{h} := \max\{\bar{h}, \alpha_0 \|G_{t,n}^*\|^2, \|proj(G_{t,n}, G_{t,n}^*) - G_{t,n}^*\|^2\}$ 
25:      $\beta := \beta \exp((\alpha_0 \|G_{t,n}^*\|^2 - \|proj(G_{t,n}, G_{t,n}^*) - G_{t,n}^*\|^2)/\bar{h})$ 
26:      $\bar{h} := \max\{\alpha_1 \bar{h}, (L/M)n/(1 - \alpha_1)\}$ 
27:      $L := \alpha_1 L, M := \alpha_1 M$ 
28:      $L' := \alpha_2 L', M' := \alpha_2 M'$ 
29:      $n' := n$ 
30:   end
31:    $t := t + n$ 
32:    $k := k + 1$ 
33:    $newperiod := true$ 
34: end
35: If  $newperiod$  then
36:    $n := 0, G_{t,0} = G_{t,0}^* := 0, h := h/2$ 

```

step-size adaptation. The original ac&er repeatedly selects data samples and computes the improvement direction estimators $\hat{\phi}_i^r$ and $\hat{\psi}_i^r$ to adjust the parameters of the actor and critic respectively. The combined algorithm optimizes the actor and critic parameters with separate modules that realize the fixed point method of step-size estimation (separate objects, as we would say in terms of object-oriented programming). Below, the generic terms of Section 4 are translated to the components of the ac&er algorithm. For the actor step-size optimization:

- ξ translates into the sequence of data that is required to compute $\hat{\phi}_i^r$.
- the vector that undergoes optimization is the actor parameter, θ ,
- its fixed value will be denoted by θ^* ,
- $g(\theta, \xi)$ represents $-\hat{\phi}_i^r(\theta, v)$; the minus results from the fact, that originally the fixed point method was meant for

minimization while here the direction of optimization is opposite,
– $g(\theta^*, \xi)$ represents $-\hat{\phi}_i^r(\theta^*, v)$.

For the critic step-size estimation we have:

- as previously, ξ translates into the sequence of data that is required to compute $\hat{\psi}_i^r$.
- the vector being optimized is now the critic parameter, v ,
- its fixed value will be denoted by v^* ,
- $g(\theta, \xi)$ represents $-\hat{\psi}_i^r(\theta, v)$; notice that now θ in the first term corresponds to v in the second while θ in the second is only one of the entities that define this random vector,
- $g(\theta^*, \xi)$ represents $-\hat{\psi}_i^r(\theta^*, v^*)$.

The above explanation suffices for straightforward combination of ac&er and the fixed point method of Section 4. This combination may be further enhanced by addressing the issues of overtraining, wrong fixed point initialization, and mutual adjustments of the actor and critic.

Overtraining. In general, a model is overtrained when it fits the data very well, but because the data is not sufficiently representative, the model does not describe the actual system that has generated the data. In the case of ac&er, the actor and critic may become overtrained when they are optimized on the basis of a short history of experiences. For instance, suppose the experience contains only several state-action pairs among which there are pairs with the same state, and the ac&er has enough resources for full optimization of the policy on the basis of these pairs. Then, it would readily make the better action in the same state infinitely more probable than the worse one. Consequently, the resulting policy would not try the worse actions at all which could lead to inability to explore certain area of the action space.

A way to overcome the above difficulty is to introduce regularization. Let

$$l : \mathcal{S} \times \mathcal{R}^{n_\theta} \mapsto \mathcal{R}_+$$

be a loss function that is continuous and assigns, for each state, zero to a bounded set of policy vectors that ensure exploration in a state, and an increasing positive score ($\in \mathcal{R}_+$) to the policy vectors with growing distance from this set. Then, the policy vector adjustment takes the form

$$\theta := \theta + \beta^\theta (\hat{\phi}_i^r(\theta, v) - \nabla l_\theta(s_i, \theta)).$$

As a result, even if the unrepresentative data pushes the policy vector outside the feasible exploration region, the loss function pulls it back.

Wrong fixed point initialization. In the initial period of the fixed point method, the algorithm only collects data and estimates $\|Eg(\theta, \xi)\|^2$ and $E\|g(\theta, \xi)\|^2$. It is assumed then that the g vectors are independently sampled from the appropriate distribution. If there are very few samples in the database, then this assumption is obviously violated and the estimator of $E\|g(\theta, \xi)\|^2$ becomes positively biased. As a result, the length of the estimation periods gets smaller. In order to stabilize the discrepancy between G and G^* for too short estimation periods, the step-sizes are being increased. This may lead to oscillations in the early stages of learning process.

A simple way to overcome this problem is to delay learning until a certain number of samples are collected in the database. To this end, we use the v function to regulate the intensity of replaying experience, of the form

$$v(t) = \begin{cases} 0 & \iff t \leq \text{min_capacity} \\ \text{comp_steps} & \iff t > \text{min_capacity} \end{cases} \quad (18)$$

for $\text{min_capacity} > \text{comp_steps} > 0$, i.e., $v(t)$ is a piecewise constant function that triggers the intensity of replaying experience given by comp_steps only after a certain time has elapsed denoted

Algorithm 4. Actor–Critic with experience replay and step-size estimation based on the fixed point method.

```

1:  $t := 1$ , Initialize  $\theta$  and  $v$ 
2: Actor.FP_INITIALIZE( $\theta_0$ )
3: Critic.FP_INITIALIZE( $v_0$ )
4: Draw and execute an action,  $a_t \sim \pi(\cdot; s_t, \theta)$ 
5: Assign  $\pi_t = \pi(a_t; s_t, \theta)$ 
6: Repeat  $v(t)$  times, begin
7:   Draw  $i \in \{t - N, t - N + 2, \dots, t - 1\}$ 
8:   Adjust  $\theta$  along an estimator of  $\phi(s_i, \theta, v)$ :
9:   Actor.FP_LOOP( $-\hat{\phi}_i^r(\theta^*, v) + \nabla_{\theta^*} l(s_i, \theta^*)$ ,
     $-\hat{\phi}_i^r(\theta, v) + \nabla_{\theta} l(s_i, \theta)$ )
10:  Adjust  $v$  along an estimator of  $\psi(s_i, \theta, v)$ :
11:  Critic.FP_LOOP( $-\hat{\psi}_i^r(\theta, v^*)$ ,  $-\hat{\psi}_i^r(\theta, v)$ )
12: End
13: Register the tuple  $\langle s_t, a_t, \pi_t, r_t, s_{t+1} \rangle$  in the database
14: Make sure only  $N$  most recent tuples remain in the
    database
15: Assign  $t := t + 1$  and repeat from Line 4

```

by *min_capacity*. While *comp_steps* is defined by available computation power, *min_capacity* should be set to exclude the use of too large computational power to optimize the actor and critic vectors on the basis of too few data samples.

Mutual adjustments of the actor and critic. The fixed point method of step-size estimation is originally designed as a tool for stochastic optimization. Actor–critic with experience replay may be understood as an algorithm of that type in which the actor parameter, θ , is optimized to adapt it to the current critic, and the critic parameter, v , is optimized to adapt it to the current actor. In both the cases, the ultimate goal of adaptation is constantly changing or running away. Consequently, the step-sizes should be estimated taking into account larger distance to the goal than currently perceived. The discrepancy between G and G^* in the fixed point method should be thus stabilized at a relatively higher level in ac&er as compared to stationary stochastic optimization. It is also inherent in Actor–Critics that the critic should adapt to the actor faster than *vice versa* (see e.g., Bhatnagar et al., 2009; Konda & Tsitsiklis, 2003).

The above remarks translate into the following guideline for the use of the fixed point method in ac&er: the discrepancies between G and G^* , regulated by the α_0 parameter, should be stabilized at relatively higher level for the critic than for the actor, and in both cases should be larger than optimal for stationary stochastic optimization.

The combined algorithm is presented in Algorithm 4. It defines the agent–environment interaction and the improvement directions for the actor and the critic. Their step-sizes and parameters are adjusted with the two fixed-point modules.

6. Experimental study

In the previous section, a reinforcement learning algorithm is introduced that combines experience replay with autonomous step-size estimation. In this section, it is confronted with two simulated control problems in multidimensional continuous spaces. The problems are selected to have general characteristics and the level of complexity corresponding to real-life robotic applications. The tasks analyzed are: (1) point reaching movement of octopus arm, and (2) cyclic running motion of half-cheetah.

6.1. Octopus arm

Octopus is well-known for exhibiting a high level of flexibility in controlling arm movements. The highly developed limbs

of octopus make it capable of bending, stretching, shortening and twisting its arm at any point and in any direction with virtually unlimited degrees of freedom. Artificial octopus arm has found its niche in continuum robotics to replace conventional serial manipulators with smooth, continuous and flexible links (McMahan et al., 2006). However, their use is severely limited by existing motion planning and control algorithms. Classical approaches of controlling redundant manipulators by applying functional constraints to the additional degrees of freedom with null-space projection do not scale to continuum robots due to added computational complexity (Chiaverini, Oriolo, & Walker, 2008). While the theoretical foundation of continuum kinematics is in its nascent stages, learning based approaches provide an interesting alternative to control such robots. To this end, Engel, Szabó, and Volkinshtein (2005) used the Gaussian process temporal difference based reinforcement learning algorithm to successfully demonstrate various learning tasks of a simulated octopus arm model. Because the original state and action space of an octopus arm has huge number of dimensions, it often serves as a test bed for solutions to the problem of dimensionality in RL (Koutnik, Gomez, & Schmidhuber, 2010; Woolley & Stanley, 2010). Here the proposed reinforcement learning algorithm is tested for solving the task of reaching a given goal by an octopus arm; the state and action spaces are reduced but still rich enough to represent current position and dynamics of the arm.

6.1.1. Control problem

We are interested in learning a reactive control policy for the octopus arm to reach an arbitrary goal point starting from some random position. The octopus arm is represented by a planar simulator model composed of 10 quadrilateral compartments with each segment containing a longitudinal or a transverse muscle, as described by Yekutieli et al. (2005). The mass of each compartment is distributed in its four corners as point masses which are connected by linear damped springs representing muscles. Each compartment follows the muscular hydrostat principle of maintaining a constant volume (area in this case) assuming the muscles to be incompressible. The activation pattern in muscles acts as input to the model and the resulting position of point masses is obtained on the basis of the net force acting on the arm. The forces modeled in the simulation are of four types: (1) internal force generated by muscles, (2) vertical force due to gravity and buoyancy of arm in seawater, (3) external force produced by water drag, and (4) constraint force induced by change in pressure to keep the area of each compartment constant. Moreover, the base is non-rotating in the model, i.e., the first spring is fixed to the ground. The planar dynamic model of the octopus arm provides a trade-off between its fidelity and real-time processing overheads. The model is simulated with the use of software made available for 2006 RL competition (Octopus-sources, 2006) originally written in Java and rewritten for the experiments presented here to C#. The simulator worked in its standard setting with 10 compartments, fixed base, each muscle 1 unit long, and action duration 0.1 s.

We now define the state and action spaces along with the associated reward function to formulate the control problem of octopus arm as MDP for reinforcement learning.

State space. In this study, we define three moving frames in polar coordinates that are used to localize the octopus arm movement towards its goal. The first frame (r_1, ϕ_1) is located at the center of the point masses of all the quadrilateral compartments, the second one (r_2, ϕ_2) is located at the center of the point masses of last 5 quadrilateral compartments, and the third one (r_3, ϕ_3) is located at the center of mass of the last compartment. The first two frames are movable with respect to the arm as they may or may not be located inside the model depending upon its configuration (see Fig. 2).

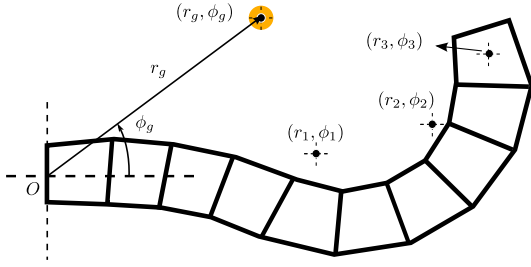


Fig. 2. Simulated octopus arm model with geometric description.

Table 1
State variables of octopus arm.

$s_0 = (r_g - 10)/3$	$s_4 = \dot{r}_1/0.01$	$s_8 = (r_3 - r_g)/3$
$s_1 = \phi_g$	$s_5 = \dot{\phi}_1/0.01$	$s_9 = \phi_3 - \phi_g$
$s_2 = (r_1 - r_g)/3$	$s_6 = (r_2 - r_g)/3$	$s_{10} = \text{sign}(r_3 - r_g)$
$s_3 = \phi_1 - \phi_g$	$s_7 = \phi_2 - \phi_g$	$s_{11} = \text{sign}(\phi_3 - \phi_g)$

Based on the three frames and the goal frame (r_g, ϕ_g) , the state space of our model consists of 12 state variables given in Table 1. Each state variable is normalized to roughly cover the interval $[-1, 1]$. Majority of the variables define the position of three frames with reference to the goal frame. Additionally, variables s_4 and s_5 express velocity of the center of mass, and discrete variables s_{10} and s_{11} describe whether the last compartment is 'in front of' or 'behind' the target. While these variables are redundant in defining arm configuration, they are quite useful inputs to the control policy.

Action space. The action space consists of a set of 6 actions each of which pre-defines the activation level in the arm's muscles, as used in Engel et al. (2005). The action space is shown in Fig. 3.

Episodes and reward function. The learning task is carried out in episodes. An episode terminates with success when the last compartment touches the goal (which is transparent for other compartments). If this goal is not reached within 500 steps, the episode terminates. A new target position is sampled in the beginning of every episode within the area described by a circular region:

$$r_g \in (7.5, 9.5)$$

$$\phi_g \in (-\pi/4, \pi/4).$$

The position of the arm is initialized such that it is set in its default position, an action is chosen randomly from the set {Action 1, Action 2}, and it is applied to the arm for a number of steps chosen randomly from the set $\{1, \dots, 100\}$.

The reward function is defined such that the controller is penalized with a negative score of -1 for all the learning steps in which the goal has not been reached. Moreover, the arm is rewarded for moving towards the goal in proportion to its speed. The reward function is given by:

$$r_t = \begin{cases} d_t - d_{t+1} - 1 & \text{if the target is not reached at } t + 1 \\ d_t & \text{if the target is reached at } t + 1. \end{cases} \quad (19)$$

The above term d_t denotes Euclidean distance between the tip of the arm and the goal position measured at instant t . The goal of the octopus arm is to maximize the reward function by reaching the goal position as quickly as possible. The total reward to be obtained within an episode is therefore equal to the difference between the initial distance of the last compartment to the goal and the number of steps in which the goal is reached.

6.1.2. Actor and critic structure

The actor and critic are based on feedforward neural networks, namely 2-layer perceptrons with sigmoidal neurons in their hidden layers and linear neurons in their output layers. The input of both the networks is the scaled state of the octopus arm. All neurons in the networks have a constant input (bias). The initial weights in the hidden layers are drawn randomly from the normal distribution $N(0, 1)$ and weights of the output layers are initially set to zero.

The actor is the combination of a neural network and a generator of discrete numbers. The network has N_A neurons in its hidden layer and 6 neurons in the output layer corresponding to the size of the discrete action set. Given state s and the observed neural network outputs

$$\mu_i(s, \theta), \quad i = 1 \dots 6,$$

the likelihood function for choosing a discrete action, Action i , is given by:

$$P(\text{Action } i | s) = \frac{\exp(\mu_i(s, \theta)/10)}{\sum_{j=1}^6 \exp(\mu_j(s, \theta)/10)}. \quad (20)$$

The exploration takes place as each action has a non-zero probability. This is ensured by the penalty function of the form

$$l(s, \theta) = \sum_{i=1}^6 \max\{0, |\mu_i(s, \theta)| - 20\}^2.$$

As a result, the policy is not penalized for the network's outputs in the range $[-20, 20]$, which means that the best action is not more than $e^4 \approx 54$ times more probable than the worst one.

The critic is a 2-layer perceptron with N_C neurons in its hidden layer and one neuron in the output layer.

6.1.3. Experiments and results

The experiments of the learning control problem of an octopus arm with the proposed algorithm are now presented. The parameter setting of the actor-critic reinforcement learning algorithm is as follows: $N_A = 50$, $N_C = 100$, $\gamma = 0.98$, $b = 2$, $\alpha = \gamma = 0.98$, and $\rho = \lambda = 0.8$. The experience database contained $M = 10^4$ events. The v function took the form (18) with $\min_capacity = 1000$ and $\text{comp_steps} = 100$. The fixed point algorithm parameters were: $\alpha_1 = 0.9$, $\alpha_2 = 0.5$, $\alpha_0 = 0.2$ for the actor and $\alpha_0 = 0.3$ for the critic.

Fig. 4 shows different stages of the octopus arm in reaching the desired goal. Figs. 5–9 present the result of experiments performed with the above parameter setting. The actor-critic with experience replay and various constant step-sizes is compared against those estimated by the fixed point method. Fig. 5 presents learning curves (average rewards vs. episode number), Fig. 6 presents efficiency (ratio of runs in which the goal was reached), Figs. 7 and 8 present average duration of episodes, and Fig. 9 presents the step-sizes estimated by the fixed point method.

What is not demonstrated in the figures is that the learning process is unstable for constant step-sizes greater or equal to 0.1 for both the actor and the critic. For $\beta^\theta = \beta^v = 10^{-2}$, the algorithm does not converge and produces poor performance. Smaller step-sizes lead to better performance but for $\beta^\theta \leq 10^{-4}$ or $\beta^v \leq 10^{-4}$ the learning process becomes very slow. The fixed point method of adjusting step-sizes enables at least as good performance of the learning algorithm, at any stage of its operation, as given by any constant step-size. The training is completed (no further improvement is observed) after 240 episodes which is equivalent to 80 min of Octopus time.

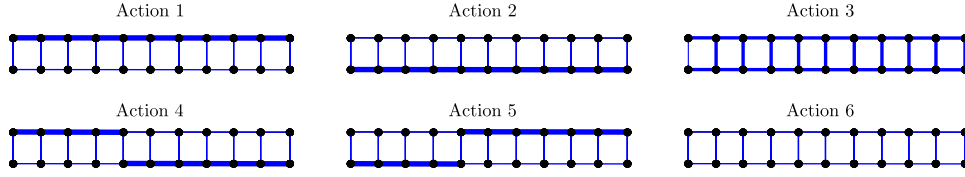


Fig. 3. Action set. The activated muscles are indicated by thick blue lines. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

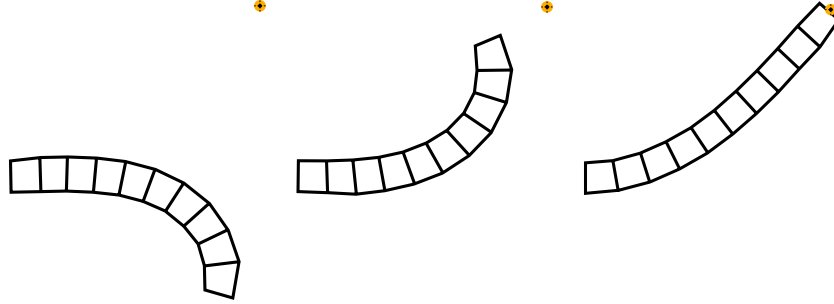


Fig. 4. Simulated goal reaching movement of octopus arm. *Left:* Arm position in the start of episode. *Middle:* Arm position trying to reach the goal. *Right:* Arm position when reached the goal. After learning, the arm reaches the goal in about 100 steps with maximum efficiency.

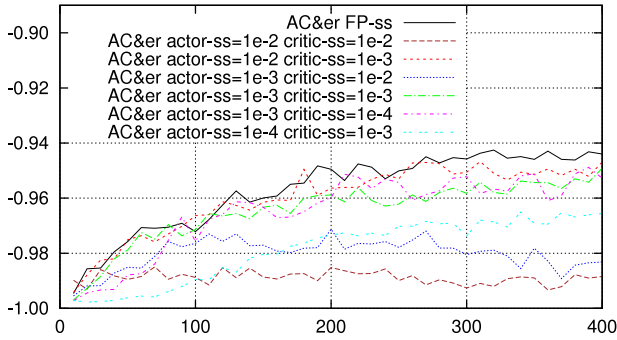


Fig. 5. Octopus arm: Average rewards vs. episode no. for actor-critic with experience replay and constant step-sizes against those estimated by the fixed point method. Each curve averages 10 runs.

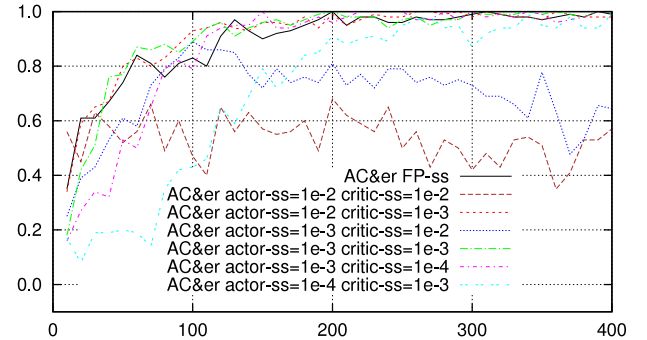


Fig. 6. Octopus arm: Efficiency vs. episode no. for actor-critic with experience replay, step-sizes estimated by the fixed point method, and constant step-sizes. Each curve averages 10 runs.

Fig. 9 shows the evolution of step-sizes driven by the fixed point method. They are averaged over runs and episodes according to the formula

$$\bar{\beta} = \exp \left(\frac{1}{N} \sum_i \ln \beta_i \right), \quad (21)$$

where N is the number of registered step-sizes. This way of averaging is better here than the normal one because it does not underweight very small step-sizes. It is seen that the actor step-size starts from a large initial value and then gradually decreases. The critic step-size initially decreases to a rather small value and then, after about 100 episodes, almost remains the same. This suggests that slowing down of the actor learning does not necessarily corresponds to slowing down of the critic learning.

6.2. Half-cheetah

Running in biological systems has largely inspired the development of legged robotics. Starting from the seminal spring-loaded hopping robots to the state-of-the-art quadrupeds and bipeds with compliant legs, a number of prototypes have been proposed to mimic versatility, speed and robustness in natural running. The dramatic progress is hindered by the lack of ability in robots to acquire these skills by online learning. In this section, we apply our proposed algorithm on a planar model of a large cat, called half-cheetah, to learn the complex skill of running.

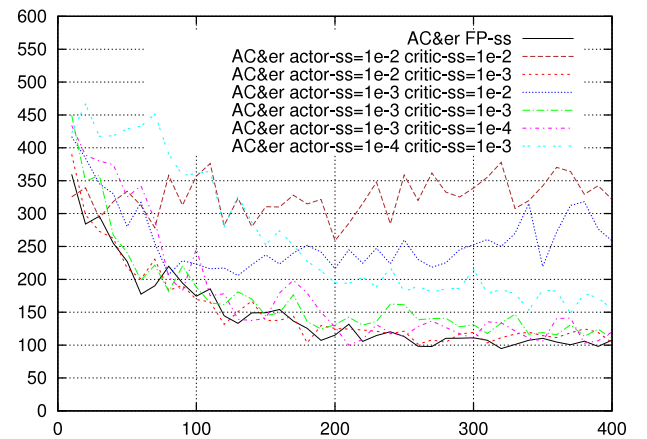


Fig. 7. Octopus arm: Duration in steps vs. episode no. for actor-critic with experience replay, step-sizes estimated by the fixed point method, and constant step-sizes. Each curve averages 10 runs.

6.2.1. Control problem

Half-Cheetah is a 6 degrees of freedom planar robot introduced in Wawrzyński (2009). It is composed of nine links, eight joints and two paws (see Fig. 10). The angles of the fourth and the fifth joint are fixed while others are controllable. The torque applied at each

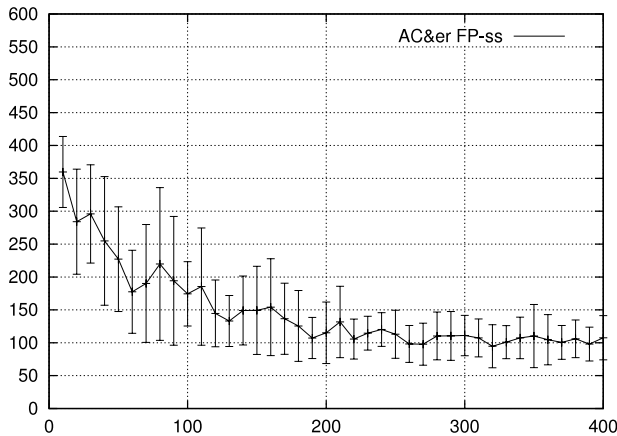


Fig. 8. Octopus arm: Duration in steps vs. episode no. for actor-critic with experience replay and step-sizes estimated by the fixed point method. The one-sigma limits are calculated to show run-to-run variability of trial averages.

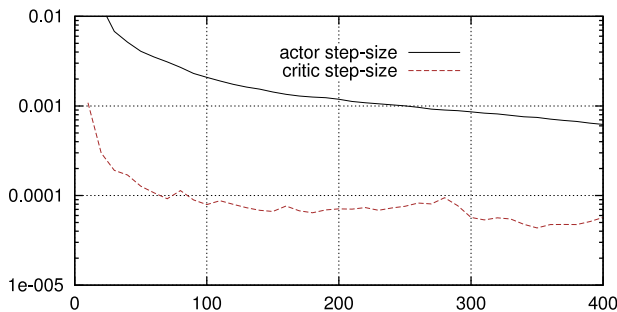


Fig. 9. Octopus arm: The fixed point method of step-size estimation vs. episode no. for actor-critic with experience replay. Each curve averages 10 runs.

joint acts as input to the model and the next position of the robot is obtained as output by integrating its dynamic equations of motion.

The control problem is to learn a reactive policy under the MDP framework to make half-cheetah run as fast as possible. State of half-cheetah is defined by 31 variables. The action space is continuous, contrary to that of octopus arm, with 6 dimensions each corresponding to one actuated joint independently. Learning is divided into episodes with an average duration of 250 steps, for 0.02 s duration of each step. The robot is mainly rewarded for its speed of moving forward. Other components are minor penalties for violating torque limits, joint limits, not moving the trunk in idle position and touching the ground with other body parts than paws. In this study, we use the simulator of half-cheetah, as reported in Wawrzyński (2009), with same experimental settings including its dynamics, state and action definition, and reward function.

6.2.2. Actor and critic structure

The actor, π , is composed of two parts: a neural network and a normal distribution. The input of the network is the state of half-cheetah. The network has a hidden layer with N_A sigmoidal neurons and six linear output neurons corresponding to the dimensionality of the action space. The output, $\mu(s; \theta)$, becomes a mean value of the normal distribution with covariance matrix equal to $I\sigma^2$. The distribution generates actions. Because each component of the action is projected on the interval $[-30, 30]$, the penalty function keeps each network output in this interval for a given state. It is of the form

$$l(s, \theta) = \sum_{i=1}^6 10^{-3} \max\{0, |\mu_i(s, \theta)| - 30\}^2.$$

The critic is a neural network of the same structure as the actor network with N_C neurons in its hidden layer. The initial

weights in the hidden layers are drawn randomly from the normal distribution $N(0, 1)$ and weights of the output layers are initially set to zero.

6.2.3. Experiments and results

The experiments to make half-cheetah run are configured with following parameters: $N_A = 160$, $N_C = 160$, $\sigma = 5$, $\gamma = 0.99$, $b = 2$, $\alpha = \gamma = 0.99$, $\rho = \lambda = 0.9$. The experience database contained $M = 10^5$ events. The v function took the form (18) with $\min_capacity = 100$ and $\text{comp_steps} = 30$. The fixed point algorithm parameters were kept same as for octopus arm with: $\alpha_1 = 0.9$, $\alpha_2 = 0.5$, $\alpha_0 = 0.2$ for the actor and $\alpha_0 = 0.3$ for the critic.

Fig. 10 shows various stages of the learned running gait of half-cheetah. The cat robot starts from a standing position and first learns to move forward with the added noise in the control system. The awkward walk transforms into a trot gait which at the end of training becomes a smooth nimble run. The use of experience replay speeds up the learning process of running in proportion to the intensity of replaying computations.

In this study, we are interested in analyzing the effect of the proposed fixed-point method of step-size estimation on the learned policy. To this end, Figs. 11 and 12 compare the learning curve of the adaptive step-size estimation algorithm with the ones obtained by setting various constant step-sizes. Among the learning curves with constant step-sizes, the learning algorithm produces optimum results for $\beta^{\theta} = \beta^v = 10^{-5}$. For larger values, the policy evolves with a reasonable speed in the initial period but the ultimate performance is worse. Smaller step-sizes slow down the learning speed significantly. The fixed point method of estimating the step-sizes makes the learning algorithm perform better at every stage of the learning process than with any manually selected step-sizes. The algorithm required 3500 episodes (about 5 h of half-cheetah time) to learn to receive average reward of 6, which corresponds to a really nimble run.

Fig. 13 presents evolution of step-sizes driven by the fixed point method. They are averaged over runs and episodes according to formula (21). It is seen that the actor and critic step-sizes start descending from the same value and then gradually converge to about $5 \cdot 10^{-6}$ and 10^{-6} respectively.

6.3. Discussion

The purpose of this experimental study is to evaluate the feasibility of autonomously determining the step-sizes for actor-critic class of reinforcement learning algorithms with experience replay. The results are promising: the augmented algorithm has been used to find control policies for two challenging problems similar to those encountered in robotics. The algorithm gave at least as good performance at any stage of the learning process as Actor-Critic with experience replay and constant step-sizes.

In all the experiments, the step-sizes of the fixed point method were initialized blindly, with a value of 1.0 (Algorithm 3, Line 8). As experiments with constant step-sizes showed, the value of 1.0 was very large and surely caused instability of the learning, which was identified by the algorithm (Line 19), and the step-sizes were halved (Line 20) until they ensured normal learning. Therefore, despite their large initial value, improper for the problems at hand, the step-sizes are always smaller than 1.0 in Figs. 9 and 13. Further, the step-sizes were autonomously adjusted with time (Algorithm 3, Line 25), which usually (but not always) meant their slow decrease.

The fixed point method of estimating step-sizes gave at least as good performance as the manually defined constant step-sizes used in the experiments. We argue that constant step-sizes generally do not ensure good performance as even their

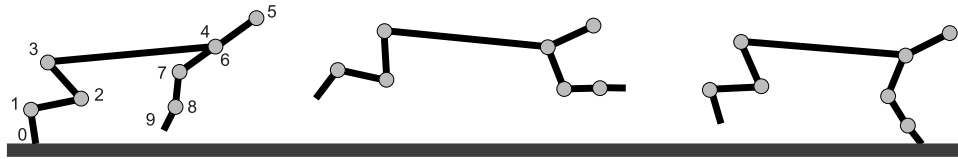


Fig. 10. Simulated run of Half-Cheetah. *Left:* Initial stance. *Middle:* Middle stance with feet in the air. *Right:* Landing stance. After learning, half-cheetah runs with a speed of about 6.5 m/s.

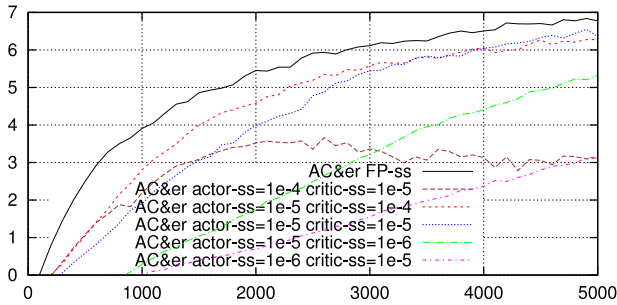


Fig. 11. Half-Cheetah: Average rewards vs. episode no. for actor-critic with experience replay and constant step-sizes against those estimated by the fixed point method. Each curve averages 10 runs.

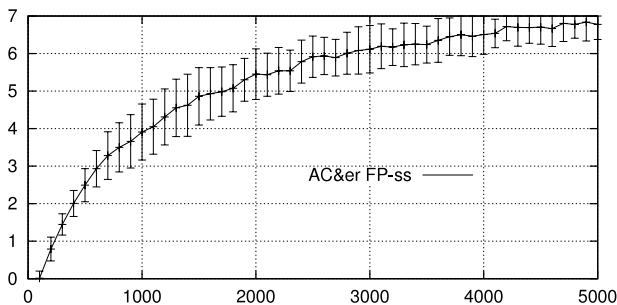


Fig. 12. Half-Cheetah: Average rewards vs. episode no. for actor-critic with experience replay and step-sizes estimated by the fixed point method. The one-sigma limits are calculated to show run-to-run variability of trial averages.

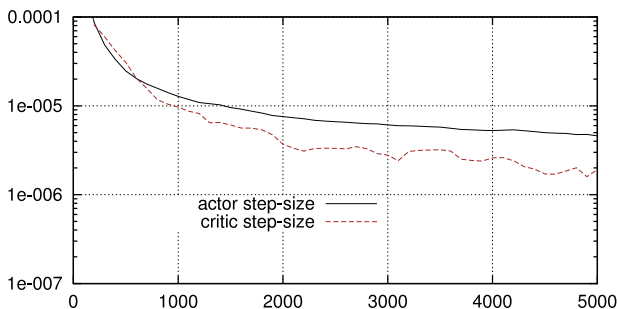


Fig. 13. Half-Cheetah: The fixed point method of step-size estimation vs. episode no. for actor-critic with experience replay. Each curve averages 10 runs.

handcrafted values yield slow learning in the early stages and fluctuations in the final stages. The fixed point method estimates large step-sizes initially and gradually decreases them during the course of learning. However, it has been observed in our supplementary experiments that the critic step-sizes that were twice larger than estimated by the fixed-point algorithm led to even better performance. Step-size estimation for the critic is an especially difficult problem as the critic learns mainly on the basis of values that it produces alone, thereby, going beyond the model of stochastic optimization for which the fixed-point algorithm along with most other methods of step-size estimation is designed. Still, the results presented here suggest the correctness of the principles that govern the fixed-point method.

Step-sizes are not the only input parameters of the learning algorithm. The rest of the parameters, however, either have the same values for both tasks or can be assigned by the experimenters based on their experience. The first category encompasses the parameters of the fixed-point method and parameter b of the randomized-truncated estimators (Eqs. (8) and (9)). The second category includes the parameters whose values are in principle problem-dependent, but the learning is not very sensitive to them and not much experience of an experimenter is required to set their proper values. Those are: (1) the sizes of the neural networks, N_A and N_C —the more difficult the control problem gets, the larger is the size of required networks, (2) the discount factor, γ , and the λ parameter—the more farsighted the control policy is required to be, the larger are the values of γ and λ , (3) the size, M , of the database with experience—it should be sufficiently large for this experience to be representative, (4) the intensity of experience replay, $comp_steps$, results from the available computer power.

In comparison to the parameters discussed above, the step-sizes have different nature: (1) the learning is quite sensitive to them, (2) their proper values differ orders of magnitude from one task to another, (3) for the same task they may differ substantially between stages of the learning. Their on-line estimation is therefore crucial for autonomy of learning.

7. Conclusions and future work

In this paper, an actor-critic reinforcement learning algorithm was introduced that combines experience replay (Wawrzyński, 2009) with the fixed point method of step-size estimation, improved upon its version presented in Wawrzyński (2010). Potentially problematic issues associated with this combination were addressed, namely overtraining, wrong fixed point initialization, and mutual adjustment of the actor and critic. The experimental study on octopus arm and half-cheetah revealed that the algorithm learned control policies in multidimensional continuous space within a short time. It is also autonomous in the sense that it achieved its goal essentially within a single run, without auxiliary experiments aiming at manual tuning of the problematic step-size parameters.

As the proposed approach has been designed for robotic applications, we plan its evaluation on a real humanoid walking robot. While the fixed point method of step-size estimation is justified by a simplified model and empirical performance, there is room for its further development and analysis of its convergence. This method has been applied here for optimization of the actor and critic step-sizes separately, while the learning of actor and of critic are coupled processes. The question whether the principles underlying the fixed point method can be formally extended to such coupling is another interesting subject of further research.

References

- Abbeel, P., & Ng, A. Y. (2005). Exploration and apprenticeship learning in reinforcement learning. In *Proc. of the 22nd ICML* (pp. 1–8). ACM.
- Adam, S., Busoniu, L., & Babuska, R. (2012). Experience replay for real-time reinforcement learning control. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, 42(2), 201–212.
- Barto, A. G., Sutton, R. S., & Anderson, C. W. (1983). Neuronlike adaptive elements that can learn difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 13, 834–846.

CONTINUOUS CONTROL WITH DEEP REINFORCEMENT LEARNING

Timothy P. Lillicrap*, Jonathan J. Hunt*, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver & Daan Wierstra

Google Deepmind

London, UK

{countzero, jjhunt, apritzel, heess, etom, tassa, davidsilver, wierstra} @ google.com

ABSTRACT

We adapt the ideas underlying the success of Deep Q-Learning to the continuous action domain. We present an actor-critic, model-free algorithm based on the deterministic policy gradient that can operate over continuous action spaces. Using the same learning algorithm, network architecture and hyper-parameters, our algorithm robustly solves more than 20 simulated physics tasks, including classic problems such as cartpole swing-up, dexterous manipulation, legged locomotion and car driving. Our algorithm is able to find policies whose performance is competitive with those found by a planning algorithm with full access to the dynamics of the domain and its derivatives. We further demonstrate that for many of the tasks the algorithm can learn policies “end-to-end”: directly from raw pixel inputs.

1 INTRODUCTION

One of the primary goals of the field of artificial intelligence is to solve complex tasks from unprocessed, high-dimensional, sensory input. Recently, significant progress has been made by combining advances in deep learning for sensory processing (Krizhevsky et al., 2012) with reinforcement learning, resulting in the “Deep Q Network” (DQN) algorithm (Mnih et al., 2015) that is capable of human level performance on many Atari video games using unprocessed pixels for input. To do so, deep neural network function approximators were used to estimate the action-value function.

However, while DQN solves problems with high-dimensional observation spaces, it can only handle discrete and low-dimensional action spaces. Many tasks of interest, most notably physical control tasks, have continuous (real valued) and high dimensional action spaces. DQN cannot be straightforwardly applied to continuous domains since it relies on finding the action that maximizes the action-value function, which in the continuous valued case requires an iterative optimization process at every step.

An obvious approach to adapting deep reinforcement learning methods such as DQN to continuous domains is to simply discretize the action space. However, this has many limitations, most notably the curse of dimensionality: the number of actions increases exponentially with the number of degrees of freedom. For example, a 7 degree of freedom system (as in the human arm) with the coarsest discretization $a_i \in \{-k, 0, k\}$ for each joint leads to an action space with dimensionality: $3^7 = 2187$. The situation is even worse for tasks that require fine control of actions as they require a correspondingly finer grained discretization, leading to an explosion of the number of discrete actions. Such large action spaces are difficult to explore efficiently, and thus successfully training DQN-like networks in this context is likely intractable. Additionally, naive discretization of action spaces needlessly throws away information about the structure of the action domain, which may be essential for solving many problems.

In this work we present a model-free, off-policy actor-critic algorithm using deep function approximators that can learn policies in high-dimensional, continuous action spaces. Our work is based

*These authors contributed equally.

on the deterministic policy gradient (DPG) algorithm (Silver et al., 2014) (itself similar to NFQCA (Hafner & Riedmiller, 2011)), and similar ideas can be found in (Prokhorov et al., 1997). However, as we show below, a naive application of this actor-critic method with neural function approximators is unstable for challenging problems.

Here we combine the actor-critic approach with insights from the recent success of Deep Q Network (DQN) (Mnih et al., 2013; 2015). Prior to DQN, it was generally believed that learning value functions using large, non-linear function approximators was difficult and unstable. DQN is able to learn value functions using such function approximators in a stable and robust way due to two innovations: 1. the network is trained off-policy with samples from a replay buffer to minimize correlations between samples; 2. the network is trained with a target Q network to give consistent targets during temporal difference backups. In this work we make use of the same ideas, along with batch normalization (Ioffe & Szegedy, 2015), a recent advance in deep learning.

In order to evaluate our method we constructed a variety of challenging physical control problems that involve complex multi-joint movements, unstable and rich contact dynamics, and gait behavior. Among these are classic problems such as the cartpole swing-up problem, as well as many new domains. A long-standing challenge of robotic control is to learn an action policy directly from raw sensory input such as video. Accordingly, we place a fixed viewpoint camera in the simulator and attempted all tasks using both low-dimensional observations (e.g. joint angles) and directly from pixels.

Our model-free approach which we call Deep DPG (DDPG) can learn competitive policies for all of our tasks using low-dimensional observations (e.g. cartesian coordinates or joint angles) using the same hyper-parameters and network structure. In many cases, we are also able to learn good policies directly from pixels, again keeping hyperparameters and network structure constant¹.

A key feature of the approach is its simplicity: it requires only a straightforward actor-critic architecture and learning algorithm with very few “moving parts”, making it easy to implement and scale to more difficult problems and larger networks. For the physical control problems we compare our results to a baseline computed by a planner (Tassa et al., 2012) that has full access to the underlying simulated dynamics and its derivatives (see supplementary information). Interestingly, DDPG can sometimes find policies that exceed the performance of the planner, in some cases even when learning from pixels (the planner always plans over the underlying low-dimensional state space).

2 BACKGROUND

We consider a standard reinforcement learning setup consisting of an agent interacting with an environment E in discrete timesteps. At each timestep t the agent receives an observation x_t , takes an action a_t and receives a scalar reward r_t . In all the environments considered here the actions are real-valued $a_t \in \mathbb{R}^N$. In general, the environment may be partially observed so that the entire history of the observation, action pairs $s_t = (x_1, a_1, \dots, a_{t-1}, x_t)$ may be required to describe the state. Here, we assumed the environment is fully-observed so $s_t = x_t$.

An agent’s behavior is defined by a policy, π , which maps states to a probability distribution over the actions $\pi: \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$. The environment, E , may also be stochastic. We model it as a Markov decision process with a state space $\mathcal{S}$, action space $\mathcal{A} = \mathbb{R}^N$, an initial state distribution $p(s_1)$, transition dynamics $p(s_{t+1}|s_t, a_t)$, and reward function $r(s_t, a_t)$.

The return from a state is defined as the sum of discounted future reward $R_t = \sum_{i=t}^T \gamma^{(i-t)} r(s_i, a_i)$ with a discounting factor $\gamma \in [0, 1]$. Note that the return depends on the actions chosen, and therefore on the policy π , and may be stochastic. The goal in reinforcement learning is to learn a policy which maximizes the expected return from the start distribution $J = \mathbb{E}_{r_i, s_i \sim E, a_i \sim \pi} [R_1]$. We denote the discounted state visitation distribution for a policy π as ρ^π .

The action-value function is used in many reinforcement learning algorithms. It describes the expected return after taking an action a_t in state s_t and thereafter following policy π :

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_{i \geq t}, s_{i > t} \sim E, a_{i > t} \sim \pi} [R_t | s_t, a_t] \quad (1)$$

¹You can view a movie of some of the learned policies at <https://goo.gl/J4PIAz>

Many approaches in reinforcement learning make use of the recursive relationship known as the Bellman equation:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})]] \quad (2)$$

If the target policy is deterministic we can describe it as a function $\mu : \mathcal{S} \leftarrow \mathcal{A}$ and avoid the inner expectation:

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))] \quad (3)$$

The expectation depends only on the environment. This means that it is possible to learn Q^μ off-policy, using transitions which are generated from a different stochastic behavior policy β .

Q-learning (Watkins & Dayan, 1992), a commonly used off-policy algorithm, uses the greedy policy $\mu(s) = \arg \max_a Q(s, a)$. We consider function approximators parameterized by θ^Q , which we optimize by minimizing the loss:

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} [(Q(s_t, a_t | \theta^Q) - y_t)^2] \quad (4)$$

where

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) | \theta^Q). \quad (5)$$

While y_t is also dependent on θ^Q , this is typically ignored.

The use of large, non-linear function approximators for learning value or action-value functions has often been avoided in the past since theoretical performance guarantees are impossible, and practically learning tends to be unstable. Recently, (Mnih et al., 2013; 2015) adapted the Q-learning algorithm in order to make effective use of large neural networks as function approximators. Their algorithm was able to learn to play Atari games from pixels. In order to scale Q-learning they introduced two major changes: the use of a *replay buffer*, and a separate *target network* for calculating y_t . We employ these in the context of DDPG and explain their implementation in the next section.

3 ALGORITHM

It is not possible to straightforwardly apply Q-learning to continuous action spaces, because in continuous spaces finding the greedy policy requires an optimization of a_t at every timestep; this optimization is too slow to be practical with large, unconstrained function approximators and nontrivial action spaces. Instead, here we used an actor-critic approach based on the DPG algorithm (Silver et al., 2014).

The DPG algorithm maintains a parameterized actor function $\mu(s | \theta^\mu)$ which specifies the current policy by deterministically mapping states to a specific action. The critic $Q(s, a)$ is learned using the Bellman equation as in Q-learning. The actor is updated by following the applying the chain rule to the expected return from the start distribution J with respect to the actor parameters:

$$\begin{aligned} \nabla_{\theta^\mu} J &\approx \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_{\theta^\mu} Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t | \theta^\mu)}] \\ &= \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_a Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) |_{s=s_t}] \end{aligned} \quad (6)$$

Silver et al. (2014) proved that this is the *policy gradient*, the gradient of the policy’s performance²

As with Q learning, introducing non-linear function approximators means that convergence is no longer guaranteed. However, such approximators appear essential in order to learn and generalize on large state spaces. NFQCA (Hafner & Riedmiller, 2011), which uses the same update rules as DPG but with neural network function approximators, uses batch learning for stability, which is intractable for large networks. A minibatch version of NFQCA which does not reset the policy at each update, as would be required to scale to large networks, is equivalent to the original DPG, which we compare to here. Our contribution here is to provide modifications to DPG, inspired by the success of DQN, which allow it to use neural network function approximators to learn in large state and action spaces online. We refer to our algorithm as Deep DPG (DDPG, Algorithm 1).

²In practice, as in commonly done in policy gradient implementations, we ignored the discount in the state-visitation distribution ρ^β .

One challenge when using neural networks for reinforcement learning is that most optimization algorithms assume that the samples are independently and identically distributed. Obviously, when the samples are generated from exploring sequentially in an environment this assumption no longer holds. Additionally, to make efficient use of hardware optimizations, it is essential to learn in minibatches, rather than online.

As in DQN, we used a replay buffer to address these issues. The replay buffer is a finite sized cache $\mathcal{R}$. Transitions were sampled from the environment according to the exploration policy and the tuple (s_t, a_t, r_t, s_{t+1}) was stored in the replay buffer. When the replay buffer was full the oldest samples were discarded. At each timestep the actor and critic are updated by sampling a minibatch uniformly from the buffer. Because DDPG is an off-policy algorithm, the replay buffer can be large, allowing the algorithm to benefit from learning across a set of uncorrelated transitions.

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using “soft” target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

When learning from low dimensional feature vector observations, the different components of the observation may have different physical units (for example, positions versus velocities) and the ranges may vary across environments. This can make it difficult for the network to learn effectively and may make it difficult to find hyper-parameters which generalise across environments with different scales of state values.

One approach to this problem is to manually scale the features so they are in similar ranges across environments and units. We address this issue by adapting a recent technique from deep learning called *batch normalization* (Ioffe & Szegedy, 2015). This technique normalizes each dimension across the samples in a minibatch to have unit mean and variance. In addition, it maintains a running average of the mean and variance to use for normalization during testing (in our case, during exploration or evaluation). In deep networks, it is used to minimize covariance shift during training, by ensuring that each layer receives whitened input. In the low-dimensional case, we used batch normalization on the state input and all layers of the μ network and all layers of the Q network prior to the action input (details of the networks are given in the supplementary material). With batch normalization, we were able to learn effectively across many different tasks with differing types of units, without needing to manually ensure the units were within a set range.

A major challenge of learning in continuous action spaces is exploration. An advantage of off-policies algorithms such as DDPG is that we can treat the problem of exploration independently from the learning algorithm. We constructed an exploration policy μ' by adding noise sampled from a noise process $\mathcal{N}$ to our actor policy

$$\mu'(s_t) = \mu(s_t|\theta_t^\mu) + \mathcal{N} \quad (7)$$

$\mathcal{N}$ can be chosen to suit the environment. As detailed in the supplementary materials we used an Ornstein-Uhlenbeck process (Uhlenbeck & Ornstein, 1930) to generate temporally correlated exploration for exploration efficiency in physical control problems with inertia (similar use of autocorrelated noise was introduced in (Wawrzynski, 2015)).

4 RESULTS

We constructed simulated physical environments of varying levels of difficulty to test our algorithm. This included classic reinforcement learning environments such as cartpole, as well as difficult,

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
 Initialize a random process $\mathcal{N}$ for action exploration
 Receive initial observation state s_1
 for $t = 1, T$ **do**
 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
 Execute action a_t and observe reward r_t and observe new state s_{t+1}
 Store transition (s_t, a_t, r_t, s_{t+1}) in R
 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))|\theta^{Q'}$
 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

 end for
end for

high dimensional tasks such as gripper, tasks involving contacts such as puck striking (*canada*) and locomotion tasks such as *cheetah* (Wawrzynski, 2009). In all domains but *cheetah* the actions were torques applied to the actuated joints. These environments were simulated using MuJoCo (Todorov et al., 2012). Figure 1 shows renderings of some of the environments used in the task (the supplementary contains details of the environments and you can view some of the learned policies at <https://goo.gl/J4PIAz>).

In all tasks, we ran experiments using both a low-dimensional state description (such as joint angles and positions) and high-dimensional renderings of the environment. As in DQN (Mnih et al., 2013, 2015), in order to make the problems approximately fully observable in the high dimensional environment we used action repeats. For each timestep of the agent, we step the simulation 3 timesteps, repeating the agent’s action and rendering each time. Thus the observation reported to the agent contains 9 feature maps (the RGB of each of the 3 renderings) which allows the agent to infer velocities using the differences between frames. The frames were downsampled to 64x64 pixels and the 8-bit RGB values were converted to floating point scaled to $[0, 1]$. See supplementary information for details of our network structure and hyperparameters.

We evaluated the policy periodically during training by testing it without exploration noise. Figure 2 shows the performance curve for a selection of environments. We also report results with components of our algorithm (i.e. the target network or batch normalization) removed. In order to perform well across all tasks, both of these additions are necessary. In particular, learning without a target network, as in the original work with DPG, is very poor in many environments.

Surprisingly, in some simpler tasks, learning policies from pixels is just as fast as learning using the low-dimensional state descriptor. This may be due to the action repeats making the problem simpler. It may also be that the convolutional layers provide an easily separable representation of state space, which is straightforward for the higher layers to learn on quickly.

Table 1 summarizes DDPG’s performance across all of the environments (results are averaged over 5 replicas). We normalized the scores using two baselines. The first baseline is the mean return from a naive policy which samples actions from a uniform distribution over the valid action space. The second baseline is iLQG (Todorov & Li, 2005), a planning based solver with full access to the

underlying physical model and its derivatives. We normalize scores so that the naive policy has a mean score of 0 and iLQG has a mean score of 1. DDPG is able to learn good policies on many of the tasks, and in many cases some of the replicas learn policies which are superior to those found by iLQG, even when learning directly from pixels.

It can be challenging to learn accurate value estimates. Q-learning, for example, is prone to over-estimating values (Hasselt, 2010). We examined DDPG’s estimates empirically by comparing the values estimated by Q after training with the true returns seen on test episodes. Figure 3 shows that in simple tasks DDPG estimates returns accurately without systematic biases. For harder tasks the Q estimates are worse, but DDPG is still able learn good policies.

To demonstrate the generality of our approach we also include Torcs, a racing game where the actions are acceleration, braking and steering. Torcs has previously been used as a testbed in other policy learning approaches (Koutník et al., 2014b). We used an identical network architecture and learning algorithm hyper-parameters to the physics tasks but altered the noise process for exploration because of the very different time scales involved. On both low-dimensional and from pixels, some replicas were able to learn reasonable policies that are able to complete a circuit around the track though other replicas failed to learn a sensible policy.

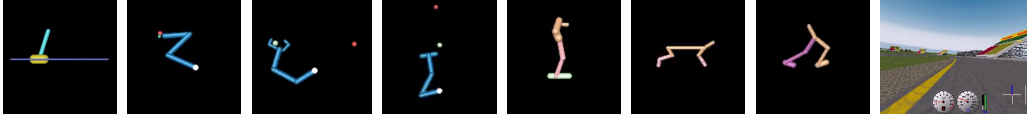


Figure 1: Example screenshots of a sample of environments we attempt to solve with DDPG. In order from the left: the cartpole swing-up task, a reaching task, a gasp and move task, a puck-hitting task, a monoped balancing task, two locomotion tasks and Torcs (driving simulator). We tackle all tasks using both low-dimensional feature vector and high-dimensional pixel inputs. Detailed descriptions of the environments are provided in the supplementary. Movies of some of the learned policies are available at <https://goo.gl/J4PIAz>.

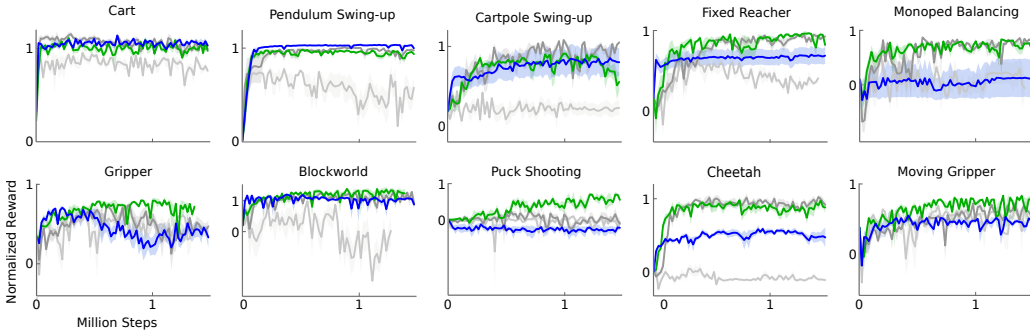


Figure 2: Performance curves for a selection of domains using variants of DPG: original DPG algorithm (minibatch NFQCA) with batch normalization (light grey), with target network (dark grey), with target networks and batch normalization (green), with target networks from pixel-only inputs (blue). Target networks are crucial.

5 RELATED WORK

The original DPG paper evaluated the algorithm with toy problems using tile-coding and linear function approximators. It demonstrated data efficiency advantages for off-policy DPG over both on- and off-policy stochastic actor critic. It also solved one more challenging task in which a multi-jointed octopus arm had to strike a target with any part of the limb. However, that paper did not demonstrate scaling the approach to large, high-dimensional observation spaces as we have here.

It has often been assumed that standard policy search methods such as those explored in the present work are simply too fragile to scale to difficult problems (Levine et al., 2015). Standard policy search

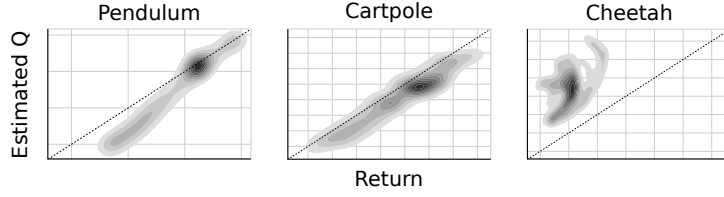


Figure 3: Density plot showing estimated Q values versus observed returns sampled from test episodes on 5 replicas. In simple domains such as pendulum and cartpole the Q values are quite accurate. In more complex tasks, the Q estimates are less accurate, but can still be used to learn competent policies. Dotted line indicates unity, units are arbitrary.

Table 1: Performance after training across all environments for at most 2.5 million steps. We report both the average and best observed (across 5 runs). All scores, except Torcs, are normalized so that a random agent receives 0 and a planning algorithm 1; for Torcs we present the raw reward score.

environment	$R_{av,lowd}$	$R_{best,lowd}$	$R_{av,pix}$	$R_{best,pix}$
blockworld1	1.156	1.511	0.466	1.299
blockworld3da	0.340	0.705	0.889	2.225
canada	0.303	1.735	0.176	0.688
canada2d	0.400	0.978	-0.285	0.119
cart	0.938	1.336	1.096	1.258
cartpole	0.844	1.115	0.482	1.138
cartpoleBalance	0.951	1.000	0.335	0.996
cartpoleParallelDouble	0.549	0.900	0.188	0.323
cartpoleSerialDouble	0.272	0.719	0.195	0.642
cartpoleSerialTriple	0.736	0.946	0.412	0.427
cheetah	0.903	1.206	0.457	0.792
fixedReacher	0.849	1.021	0.693	0.981
fixedReacherDouble	0.924	0.996	0.872	0.943
fixedReacherSingle	0.954	1.000	0.827	0.995
gripper	0.655	0.972	0.406	0.790
gripperRandom	0.618	0.937	0.082	0.791
hardCheetah	1.311	1.990	1.204	1.431
hopper	0.676	0.936	0.112	0.924
hyq	0.416	0.722	0.234	0.672
movingGripper	0.474	0.936	0.480	0.644
pendulum	0.946	1.021	0.663	1.055
reacher	0.720	0.987	0.194	0.878
reacher3daFixedTarget	0.585	0.943	0.453	0.922
reacher3daRandomTarget	0.467	0.739	0.374	0.735
reacherSingle	0.981	1.102	1.000	1.083
walker2d	0.705	1.573	0.944	1.476
torcs	-393.385	1840.036	-401.911	1876.284

is thought to be difficult because it deals simultaneously with complex environmental dynamics and a complex policy. Indeed, most past work with actor-critic and policy optimization approaches have had difficulty scaling up to more challenging problems (Deisenroth et al., 2013). Typically, this is due to instability in learning wherein progress on a problem is either destroyed by subsequent learning updates, or else learning is too slow to be practical.

Recent work with model-free policy search has demonstrated that it may not be as fragile as previously supposed. Wawrzyński (2009); Wawrzyński & Tanwani (2013) has trained stochastic policies in an actor-critic framework with a replay buffer. Concurrent with our work, Balduzzi & Ghifary (2015) extended the DPG algorithm with a “deviator” network which explicitly learns $\partial Q / \partial a$. However, they only train on two low-dimensional domains. Heess et al. (2015) introduced SVG(0) which also uses a Q-critic but learns a stochastic policy. DPG can be considered the deterministic limit of

SVG(0). The techniques we described here for scaling DPG are also applicable to stochastic policies by using the reparametrization trick (Heess et al., 2015; Schulman et al., 2015a).

Another approach, trust region policy optimization (TRPO) (Schulman et al., 2015b), directly constructs stochastic neural network policies without decomposing problems into optimal control and supervised phases. This method produces near monotonic improvements in return by making carefully chosen updates to the policy parameters, constraining updates to prevent the new policy from diverging too far from the existing policy. This approach does not require learning an action-value function, and (perhaps as a result) appears to be significantly less data efficient.

To combat the challenges of the actor-critic approach, recent work with guided policy search (GPS) algorithms (e.g., (Levine et al., 2015)) decomposes the problem into three phases that are relatively easy to solve: first, it uses full-state observations to create locally-linear approximations of the dynamics around one or more nominal trajectories, and then uses optimal control to find the locally-linear optimal policy along these trajectories; finally, it uses supervised learning to train a complex, non-linear policy (e.g. a deep neural network) to reproduce the state-to-action mapping of the optimized trajectories.

This approach has several benefits, including data efficiency, and has been applied successfully to a variety of real-world robotic manipulation tasks using vision. In these tasks GPS uses a similar convolutional policy network to ours with 2 notable exceptions: 1. it uses a spatial softmax to reduce the dimensionality of visual features into a single (x, y) coordinate for each feature map, and 2. the policy also receives direct low-dimensional state information about the configuration of the robot at the first fully connected layer in the network. Both likely increase the power and data efficiency of the algorithm and could easily be exploited within the DDPG framework.

PILCO (Deisenroth & Rasmussen, 2011) uses Gaussian processes to learn a non-parametric, probabilistic model of the dynamics. Using this learned model, PILCO calculates analytic policy gradients and achieves impressive data efficiency in a number of control problems. However, due to the high computational demand, PILCO is “impractical for high-dimensional problems” (Wahlström et al., 2015). It seems that deep function approximators are the most promising approach for scaling reinforcement learning to large, high-dimensional domains.

(Wahlström et al., 2015) used a deep dynamical model network along with model predictive control to solve the pendulum swing-up task from pixel input. They trained a differentiable forward model and encoded the goal state into the learned latent space. They use model-predictive control over the learned model to find a policy for reaching the target. However, this approach is only applicable to domains with goal states that can be demonstrated to the algorithm.

Recently, evolutionary approaches have been used to learn competitive policies for Torcs from pixels using compressed weight parametrizations (Koutník et al., 2014a) or unsupervised learning (Koutník et al., 2014b) to reduce the dimensionality of the evolved weights. It is unclear how well these approaches generalize to other problems.

6 CONCLUSION

The work combines insights from recent advances in deep learning and reinforcement learning, resulting in an algorithm that robustly solves challenging problems across a variety of domains with continuous action spaces, even when using raw pixels for observations. As with most reinforcement learning algorithms, the use of non-linear function approximators nullifies any convergence guarantees; however, our experimental results demonstrate that stable learning without the need for any modifications between environments.

Interestingly, all of our experiments used substantially fewer steps of experience than was used by DQN learning to find solutions in the Atari domain. Nearly all of the problems we looked at were solved within 2.5 million steps of experience (and usually far fewer), a factor of 20 fewer steps than DQN requires for good Atari solutions. This suggests that, given more simulation time, DDPG may solve even more difficult problems than those considered here.

A few limitations to our approach remain. Most notably, as with most model-free reinforcement approaches, DDPG requires a large number of training episodes to find solutions. However, we

believe that a robust model-free approach may be an important component of larger systems which may attack these limitations (Gläscher et al., 2010).

REFERENCES

- Balduzzi, David and Ghifary, Muhammad. Compatible value gradients for reinforcement learning of continuous deep policies. *arXiv preprint arXiv:1509.03005*, 2015.
- Deisenroth, Marc and Rasmussen, Carl E. Pilco: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on machine learning (ICML-11)*, pp. 465–472, 2011.
- Deisenroth, Marc Peter, Neumann, Gerhard, Peters, Jan, et al. A survey on policy search for robotics. *Foundations and Trends in Robotics*, 2(1-2):1–142, 2013.
- Gläscher, Jan, Daw, Nathaniel, Dayan, Peter, and O’Doherty, John P. States versus rewards: dissociable neural prediction error signals underlying model-based and model-free reinforcement learning. *Neuron*, 66(4):585–595, 2010.
- Glorot, Xavier, Bordes, Antoine, and Bengio, Yoshua. Deep sparse rectifier networks. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics. JMLR W&CP Volume*, volume 15, pp. 315–323, 2011.
- Hafner, Roland and Riedmiller, Martin. Reinforcement learning in feedback control. *Machine learning*, 84(1-2):137–169, 2011.
- Hasselt, Hado V. Double q-learning. In *Advances in Neural Information Processing Systems*, pp. 2613–2621, 2010.
- Heess, N., Hunt, J. J., Lillicrap, T. P., and Silver, D. Memory-based control with recurrent neural networks. *NIPS Deep Reinforcement Learning Workshop (arXiv:1512.04455)*, 2015.
- Heess, Nicolas, Wayne, Gregory, Silver, David, Lillicrap, Tim, Erez, Tom, and Tassa, Yuval. Learning continuous control policies by stochastic value gradients. In *Advances in Neural Information Processing Systems*, pp. 2926–2934, 2015.
- Ioffe, Sergey and Szegedy, Christian. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- Kingma, Diederik and Ba, Jimmy. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Koutník, Jan, Schmidhuber, Jürgen, and Gomez, Faustino. Evolving deep unsupervised convolutional networks for vision-based reinforcement learning. In *Proceedings of the 2014 conference on Genetic and evolutionary computation*, pp. 541–548. ACM, 2014a.
- Koutník, Jan, Schmidhuber, Jürgen, and Gomez, Faustino. Online evolution of deep convolutional network for vision-based reinforcement learning. In *From Animals to Animats 13*, pp. 260–269. Springer, 2014b.
- Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pp. 1097–1105, 2012.
- Levine, Sergey, Finn, Chelsea, Darrell, Trevor, and Abbeel, Pieter. End-to-end training of deep visuomotor policies. *arXiv preprint arXiv:1504.00702*, 2015.
- Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Graves, Alex, Antonoglou, Ioannis, Wierstra, Daan, and Riedmiller, Martin. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Rusu, Andrei A, Veness, Joel, Bellemare, Marc G, Graves, Alex, Riedmiller, Martin, Fidjeland, Andreas K, Ostrovski, Georg, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

Supplementary Information: Continuous control with deep reinforcement learning

7 EXPERIMENT DETAILS

We used Adam (Kingma & Ba, 2014) for learning the neural network parameters with a learning rate of 10^{-4} and 10^{-3} for the actor and critic respectively. For Q we included L_2 weight decay of 10^{-2} and used a discount factor of $\gamma = 0.99$. For the soft target updates we used $\tau = 0.001$. The neural networks used the rectified non-linearity (Glorot et al., 2011) for all hidden layers. The final output layer of the actor was a tanh layer, to bound the actions. The low-dimensional networks had 2 hidden layers with 400 and 300 units respectively ($\approx 130,000$ parameters). Actions were not included until the 2nd hidden layer of Q . When learning from pixels we used 3 convolutional layers (no pooling) with 32 filters at each layer. This was followed by two fully connected layers with 200 units ($\approx 430,000$ parameters). The final layer weights and biases of both the actor and critic were initialized from a uniform distribution $[-3 \times 10^{-3}, 3 \times 10^{-3}]$ and $[3 \times 10^{-4}, 3 \times 10^{-4}]$ for the low dimensional and pixel cases respectively. This was to ensure the initial outputs for the policy and value estimates were near zero. The other layers were initialized from uniform distributions $[-\frac{1}{\sqrt{f}}, \frac{1}{\sqrt{f}}]$ where f is the fan-in of the layer. The actions were not included until the fully-connected layers. We trained with minibatch sizes of 64 for the low dimensional problems and 16 on pixels. We used a replay buffer size of 10^6 .

For the exploration noise process we used temporally correlated noise in order to explore well in physical environments that have momentum. We used an Ornstein-Uhlenbeck process (Uhlenbeck & Ornstein, 1930) with $\theta = 0.15$ and $\sigma = 0.3$. The Ornstein-Uhlenbeck process models the velocity of a Brownian particle with friction, which results in temporally correlated values centered around 0.

8 PLANNING ALGORITHM

Our planner is implemented as a model-predictive controller (Tassa et al., 2012): at every time step we run a single iteration of trajectory optimization (using iLQG, (Todorov & Li, 2005)), starting from the true state of the system. Every single trajectory optimization is planned over a horizon between 250ms and 600ms, and this planning horizon recedes as the simulation of the world unfolds, as is the case in model-predictive control.

The iLQG iteration begins with an initial rollout of the previous policy, which determines the nominal trajectory. We use repeated samples of simulated dynamics to approximate a linear expansion of the dynamics around every step of the trajectory, as well as a quadratic expansion of the cost function. We use this sequence of locally-linear-quadratic models to integrate the value function backwards in time along the nominal trajectory. This *back-pass* results in a putative modification to the action sequence that will decrease the total cost. We perform a derivative-free line-search over this direction in the space of action sequences by integrating the dynamics forward (the *forward-pass*), and choose the best trajectory. We store this action sequence in order to warm-start the next iLQG iteration, and execute the first action in the simulator. This results in a new state, which is used as the initial state in the next iteration of trajectory optimization.

9 ENVIRONMENT DETAILS

9.1 TORCS ENVIRONMENT

For the Torcs environment we used a reward function which provides a positive reward at each step for the velocity of the car projected along the track direction and a penalty of -1 for collisions. Episodes were terminated if progress was not made along the track after 500 frames.

9.2 MUJoCo ENVIRONMENTS

For physical control tasks we used reward functions which provide feedback at every step. In all tasks, the reward contained a small action cost. For all tasks that have a static goal state (e.g. pendulum swingup and reaching) we provide a smoothly varying reward based on distance to a goal state, and in some cases an additional positive reward when within a small radius of the target state. For grasping and manipulation tasks we used a reward with a term which encourages movement towards the payload and a second component which encourages moving the payload to the target. In locomotion tasks we reward forward action and penalize hard impacts to encourage smooth rather than hopping gaits (Schulman et al., 2015b). In addition, we used a negative reward and early termination for falls which were determined by simple thresholds on the height and torso angle (in the case of walker2d).

Table 2 states the dimensionality of the problems and below is a summary of all the physics environments.

task name	dim(s)	dim(a)	dim(o)
blockworld1	18	5	43
blockworld3da	31	9	102
canada	22	7	62
canada2d	14	3	29
cart	2	1	3
cartpole	4	1	14
cartpoleBalance	4	1	14
cartpoleParallelDouble	6	1	16
cartpoleParallelTriple	8	1	23
cartpoleSerialDouble	6	1	14
cartpoleSerialTriple	8	1	23
cheetah	18	6	17
fixedReacher	10	3	23
fixedReacherDouble	8	2	18
fixedReacherSingle	6	1	13
gripper	18	5	43
gripperRandom	18	5	43
hardCheetah	18	6	17
hardCheetahNice	18	6	17
hopper	14	4	14
hyq	37	12	37
hyqKick	37	12	37
movingGripper	22	7	49
movingGripperRandom	22	7	49
pendulum	2	1	3
reacher	10	3	23
reacher3daFixedTarget	20	7	61
reacher3daRandomTarget	20	7	61
reacherDouble	6	1	13
reacherObstacle	18	5	38
reacherSingle	6	1	13
walker2d	18	6	41

Table 2: Dimensionality of the MuJoCo tasks: the dimensionality of the underlying physics model $\dim(s)$, number of action dimensions $\dim(a)$ and observation dimensions $\dim(o)$.

task name	Brief Description
blockworld1	Agent is required to use an arm with gripper constrained to the 2D plane to grab a falling block and lift it against gravity to a fixed target position.

blockworld3da	Agent is required to use a human-like arm with 7-DOF and a simple gripper to grab a block and lift it against gravity to a fixed target position.
canada	Agent is required to use a 7-DOF arm with hockey-stick like appendage to hit a ball to a target.
canada2d	Agent is required to use an arm with hockey-stick like appendage to hit a ball initialized to a random start location to a random target location.
cart	Agent must move a simple mass to rest at 0. The mass begins each trial in random positions and with random velocities.
cartpole	The classic cart-pole swing-up task. Agent must balance a pole attached to a cart by applying forces to the cart alone. The pole starts each episode hanging upside-down.
cartpoleBalance	The classic cart-pole balance task. Agent must balance a pole attached to a cart by applying forces to the cart alone. The pole starts in the upright positions at the beginning of each episode.
cartpoleParallelDouble	Variant on the classic cart-pole. Two poles, both attached to the cart, should be kept upright as much as possible.
cartpoleSerialDouble	Variant on the classic cart-pole. Two poles, one attached to the cart and the second attached to the end of the first, should be kept upright as much as possible.
cartpoleSerialTriple	Variant on the classic cart-pole. Three poles, one attached to the cart, the second attached to the end of the first, and the third attached to the end of the second, should be kept upright as much as possible.
cheetah	The agent should move forward as quickly as possible with a cheetah-like body that is constrained to the plane. This environment is based very closely on the one introduced by Wawrzyński (2009); Wawrzyński & Tanwani (2013).
fixedReacher	Agent is required to move a 3-DOF arm to a fixed target position.
fixedReacherDouble	Agent is required to move a 2-DOF arm to a fixed target position.
fixedReacherSingle	Agent is required to move a simple 1-DOF arm to a fixed target position.
gripper	Agent must use an arm with gripper appendage to grasp an object and maneuver the object to a fixed target.
gripperRandom	The same task as gripper except that the arm object and target position are initialized in random locations.
hardCheetah	The agent should move forward as quickly as possible with a cheetah-like body that is constrained to the plane. This environment is based very closely on the one introduced by Wawrzyński (2009); Wawrzyński & Tanwani (2013), but has been made much more difficult by removing the stabilizing joint stiffness from the model.
hopper	Agent must balance a multiple degree of freedom monopod to keep it from falling.
hyq	Agent is required to keep a quadroped model based on the hyq robot from falling.

movingGripper	Agent must use an arm with gripper attached to a moveable platform to grasp an object and move it to a fixed target.
movingGripperRandom	The same as the movingGripper environment except that the object position, target position, and arm state are initialized randomly.
pendulum	The classic pendulum swing-up problem. The pendulum should be brought to the upright position and balanced. Torque limits prevent the agent from swinging the pendulum up directly.
reacher3daFixedTarget	Agent is required to move a 7-DOF human-like arm to a fixed target position.
reacher3daRandomTarget	Agent is required to move a 7-DOF human-like arm from random starting locations to random target positions.
reacher	Agent is required to move a 3-DOF arm from random starting locations to random target positions.
reacherSingle	Agent is required to move a simple 1-DOF arm from random starting locations to random target positions.
reacherObstacle	Agent is required to move a 5-DOF arm around an obstacle to a randomized target position.
walker2d	Agent should move forward as quickly as possible with a bipedal walker constrained to the plane without falling down or pitching the torso too far forward or backward.

Stable-Baselines3: Reliable Reinforcement Learning Implementations

Antonin Raffin¹

Ashley Hill²

Adam Gleave³

Anssi Kanervisto⁴

Maximilian Ernestus⁵

Noah Dormann¹

ANTONIN.RAFFIN@DLR.DE

ASHLEY.HILL@CEA.FR

GLEAVE@BERKELEY.EDU

ANSSK@CS.UEF.FI

MAXIMILIAN@ERNESTUS.COM

NOAH.DORMANN@DLR.DE

¹ *Robotics and Mechatronics Center (RMC), German Aerospace Center (DLR), Weßling, Germany*

² *Interactive Robotics Laboratory, University Paris-Saclay, CEA, Palaiseau, France*

³ *Electrical Engineering and Computer Science, University of California, Berkeley, CA, USA*

⁴ *School of Computing, University of Eastern Finland, Joensuu, Finland*

⁵ *Kiteswarms GmbH, Freiburg, Germany*

Editor: Andreas Mueller

Abstract

STABLE-BASELINES3 provides open-source implementations of deep reinforcement learning (RL) algorithms in Python. The implementations have been benchmarked against reference codebases, and automated unit tests cover 95% of the code. The algorithms follow a consistent interface and are accompanied by extensive documentation, making it simple to train and compare different RL algorithms. Our documentation, examples, and source-code are available at <https://github.com/DLR-RM/stable-baselines3>.

Keywords: Reinforcement Learning, Baselines, Software, Open-Source, Python, PyTorch

1. Introduction

Deep reinforcement learning (RL) research has grown rapidly in recent years, yet results are often difficult to reproduce (Henderson et al., 2018). A major challenge is that small implementation details can have a substantial effect on performance – often greater than the difference between algorithms (Engstrom et al., 2020). It is particularly important that implementations used as experimental *baselines* are reliable; otherwise, novel algorithms compared to weak baselines lead to inflated estimates of performance improvements.

To address this challenge, we propose STABLE-BASELINES3 (SB3), an open-source framework implementing seven commonly used model-free deep RL algorithms (see Section 2). We take great care to adhere to software engineering best practices to achieve high-quality implementations that match prior results. Each algorithm has been benchmarked on common environments (Raffin and Stulp, 2020) and compared to prior implementations. Our test suite covers 95% of the code and, together with our active user base¹ scrutinizing changes, ensures that any implementation errors are minimized.

1. At the time of writing, SB3 had 800+ stars on GitHub, 100+ closed issues and 80+ merged pull requests

```

import gym
from stable_baselines3 import SAC
# Train an agent using Soft Actor-Critic on Pendulum-v0
env = gym.make("Pendulum-v0")
model = SAC("MlpPolicy", env).learn(total_timesteps=20000)
# Save the model
model.save("sac_pendulum")
# Load the trained model
model = SAC.load("sac_pendulum")
# Start a new episode
obs = env.reset()
# What action to take in state `obs`?
action, _ = model.predict(obs, deterministic=True)

```

Figure 1: Using STABLE-BASELINES3 to train, save, load, and infer an action from a policy.

STABLE-BASELINES3 builds on the experience gained from maintaining our previous implementation, STABLE-BASELINES2 (SB2; Hill et al., 2018)², that was forked from OpenAI Baselines (Dhariwal et al., 2017) and uses TensorFlow (Abadi et al., 2016). SB3 is a complete rewrite of the codebase implemented in PyTorch (Paszke et al., 2019), the framework preferred by a majority of our users in a survey (Raffin, 2020a). SB3 maintains a similar API, allowing a seamless upgrade pathway from SB2.³

Our main goal is to provide a user-friendly and reliable RL library. To keep SB3 simple to use and maintain, we focus on model-free, single-agent RL algorithms, and rely on external projects to extend the scope to imitation (Wang et al., 2020) and offline learning (Seno, 2020). We prioritize maintaining *stable* implementations over adding new features or algorithms, and avoid making breaking changes. We provide a consistent, clean and fully documented API, inspired by the `scikit-learn` API (Pedregosa et al., 2011). Our code is easily modifiable by users as we favour readability and simplicity over modularity, although we make use of object-oriented programming to reduce code duplication.

2. Features

Simple API. Figure 1 shows that training agents in STABLE-BASELINES3 takes just a few lines of code, after which the agent can be queried for actions. This allows researchers to easily use the baseline algorithms and components in their experiments (e.g. Klink et al. (2020); Nair et al. (2019); Gleave et al. (2020)), as well as apply RL to novel tasks and environments, like continual learning when attacking WiFi networks (Margaritelli, 2020) or dampening bridge vibrations (Berkowitz, 2019).

Documentation. SB3 comes with extensive documentation of the code API.⁴ We also include a user guide, covering both basic and more advanced usage with a collection of concrete examples. Moreover, we have developed a Colab notebook based RL tutorial,⁵ enabling users to demo the library directly in the browser. Additionally, we include common

2. SB2 has 650+ closed issues, 220+ merged pull requests and 200+ citations on Google Scholar.

3. Upgrade guide: <https://stable-baselines3.readthedocs.io/en/master/guide/migration.html>.

4. <https://stable-baselines3.readthedocs.io/en/master/>

5. <https://github.com/araffin/rl-tutorial-jnrr19>

tips for running RL experiments and a developer guide. We also pay close attention to questions and uncertainties from SB3 users, updating the documentation to address these.

High-Quality Implementations. Algorithms are verified against published results by comparing the agent learning curves⁶. Moreover, all functions are typed (parameter and return types) and documented with a consistent style, and most functions are covered by unit tests. Continuous integration checks that all changes pass unit tests and type check, as well as validating the code style and documentation.

Comprehensive. STABLE-BASELINES3 contains the following state-of-the-art on- and off-policy algorithms, commonly used as experimental baselines: A2C (Mnih et al., 2016), PPO (Schulman et al., 2017), DDPG (Lillicrap et al., 2016), SAC (Haarnoja et al., 2018), TD3 (Fujimoto et al., 2018), HER (Andrychowicz et al., 2017) and DQN (Mnih et al., 2015).

Moreover, SB3 provides various algorithm-independent features. We support logging to CSV files and TensorBoard. Users can log custom metrics and modify training via user-provided callbacks. To speed up training, we support parallel (or “vectorized”) environments. To simplify training, we implement common environment wrappers, like preprocessing Atari observations to match the original DQN experiments (Mnih et al., 2015).

Experimental Framework. RL Baselines Zoo (Raffin, 2018, 2020b) provides scripts to train and evaluate agents, tune hyperparameters, record videos, store experiment setup and visualize results. We also include a collection of pre-trained reinforcement learning agents together with tuned hyperparameters for simple control tasks, PyBullet environments (Coumans and Bai, 2016–2019) and Atari games, optimized using Optuna (Akiba et al., 2019). We follow best practices for training and evaluation (Henderson et al., 2018), such as evaluating in a separate environment, using deterministic evaluation where required (SAC) and storing all hyperparameters necessary to replicate the experiment.

Stable-Baselines3 Contrib. We implement experimental features in a separate contrib repository (Raffin et al., 2020). This allows SB3 to maintain a stable and compact core, while still providing the latest features, like Truncated Quantile Critics (Kuznetsov et al., 2020). Implementations in contrib need not be tightly integrated with the main SB3 codebase, but we maintain the same stringent review requirements to ensure users can trust the contrib implementations. Implementations from contrib that have stood the test of time may be integrated into the main repository.

3. Comparison to Related Software

Most libraries are targeted at experienced RL researchers, requiring expert knowledge to use (Weng et al., 2020; Hoffman et al., 2020; Fujita et al., 2021; Castro et al., 2018; Guadarrama et al., 2018; Gauci et al., 2018; Stooke and Abbeel, 2019; Kolesnikov, 2018). Only a few RL libraries offer more than a brief API documentation (garage contributors, 2019; Liang et al., 2018; Kuhnle et al., 2017; Guadarrama et al., 2018), and some are notoriously hard to understand.⁷ By contrast, STABLE-BASELINES3 is designed to be easy to use and comes with extensive documentation and tutorials.

6. For example, issue #48 or issue #49.

7. OpenAI Baselines (Dhariwal et al., 2017), see <https://www.reddit.com/r/MachineLearning/comments/95ft1j/>. This was a major starting point for STABLE-BASELINES2 (Hill et al., 2018)

The previous version of STABLE-BASELINES3, STABLE-BASELINES2, was created as a fork of OpenAI Baselines (Dhariwal et al., 2017) but the two codebases quickly diverged (see PR #481). SB3 is a complete rewrite of STABLE-BASELINES2 in PyTorch that keeps the major improvements and new algorithms from SB2 while going even further into improving code quality (e.g. cleaner codebase, better test coverage, type hints). More precisely, compared to Baselines, SB3 is fully documented, commented, tested, has 4 additional algorithms (SAC, TD3, QR-DQN, TQC⁸) and many additional features (e.g. dictionary observation support, callbacks, evaluation with multiple environments, environment checker). The only legacy features of OpenAI Baselines are the code structure (one folder per algorithm), the use of code-level optimizations and the environment tools which are greatly improved⁹ (additional features, bug fixes, comments, documentation and more testing).

Many libraries have a modular design (Caspi et al., 2017; Keng and Graesser, 2017; Hoffman et al., 2020; garage contributors, 2019). This allows them to quickly combine advances from different papers, but forces new users to understand the full code structure before being able to tweak the library. On the other extreme, educational implementations like Spinning Up (Achiam, 2018) are self-contained but hard to maintain due to code duplication. SB3 strives to strike a balance: factoring out widely used components like replay buffers, but minimizing the amount of code that needs to be understood to modify an algorithm.

As an exhaustive comparison to all RL libraries is not possible, in Table 1 we compare SB3 to a subset of other active or popular libraries, with a focus on quality of implementation and openness to new users.

RLlib (Liang et al., 2018) scores highly in the table, but is targeted at a different use-case from SB3. Whereas SB3 focuses on simplicity and reliability, RLlib (Liang et al., 2018) prioritizes scalability and support for distributed training. Additionally, RLlib includes both a PyTorch and TensorFlow backend, and includes support for multi-agent training. This versatility comes at a cost of a larger and more complex codebase.

Overall, we find SB3 compares favourably to other libraries in terms of documentation, testing and activity.

	SB3	OAI Baselines	PFRL	RLlib	Tianshou	Acme	Tensorforce
Backend	PyTorch	TF	PyTorch	PyTorch/TF	PyTorch	Jax/TF	TF
User Guide / Tutorials	✓/ ✓	✗/ —	—/ ✓	✓/ ✓	—/ ✓	—/ ✓	✓/ —
API Documentation	✓	✗	✓	✓	✓	✗	✓
Benchmark	✓	✓	✓	✓	—	—	—
Pretrained models	✓	✗	✓	✗	✗	✗	✗
Test Coverage	95%	49%	?	?	94%	74%	81%
Type Checking	✓	✗	✗	✓	✓	✓	✗
Issue / PR Template	✓	✗	✗	✓	✓	✗	✗
Last Commit (age)	< 1 week	> 6 months	< 1 month	< 1 week	< 1 month	< 1 week	< 1 month
Approved PRs (6 mo.)	75	0	13	222	85	5	7

Table 1: Comparison of SB3 to a representative subset of active or popular RL libraries. **Key:** — means that the feature is only partially present; OAI: OpenAI; TF: TensorFlow; PR: Pull Request.

8. QR-DQN and TQC are in the contrib repo.

9. As an example, one can compare “VecNormalize” in OAI Baselines vs SB3.

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI

{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a “surrogate” objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q -learning [Mni+15], “vanilla” policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q -learning (with function approximation) fails on many simple problems¹ and is poorly understood, vanilla policy gradient methods have poor data efficiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains the data efficiency and reliable performance of TRPO, while using only first-order optimization. We propose a novel objective with clipped probability ratios, which forms a pessimistic estimate (i.e., lower bound) of the performance of the policy. To optimize policies, we alternate between sampling data from the policy and performing several epochs of optimization on the sampled data.

Our experiments compare the performance of various different versions of the surrogate objective, and find that the version with the clipped probability ratios performs best. We also compare PPO to several previous algorithms from the literature. On continuous control tasks, it performs better than the algorithms we compare against. On Atari, it performs significantly better (in terms of sample complexity) than A2C and similarly to ACER though it is much simpler.

¹While DQN works well on game environments like the Arcade Learning Environment [Bel+15] with discrete action spaces, it has not been demonstrated to perform well on continuous control benchmarks such as those in OpenAI Gym [Bro+16] and described by Duan et al. [Dua+16].

2 Background: Policy Optimization

2.1 Policy Gradient Methods

Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm. The most commonly used gradient estimator has the form

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right] \quad (1)$$

where π_{θ} is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t . Here, the expectation $\hat{\mathbb{E}}_t[\dots]$ indicates the empirical average over a finite batch of samples, in an algorithm that alternates between sampling and optimization. Implementations that use automatic differentiation software work by constructing an objective function whose gradient is the policy gradient estimator; the estimator $\hat{g}$ is obtained by differentiating the objective

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]. \quad (2)$$

While it is appealing to perform multiple steps of optimization on this loss L^{PG} using the same trajectory, doing so is not well-justified, and empirically it often leads to destructively large policy updates (see Section 6.1; results are not shown but were similar or worse than the “no clipping or penalty” setting).

2.2 Trust Region Methods

In TRPO [Sch+15b], an objective function (the “surrogate” objective) is maximized subject to a constraint on the size of the policy update. Specifically,

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \quad (3)$$

$$\text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \quad (4)$$

Here, θ_{old} is the vector of policy parameters before the update. This problem can efficiently be approximately solved using the conjugate gradient algorithm, after making a linear approximation to the objective and a quadratic approximation to the constraint.

The theory justifying TRPO actually suggests using a penalty instead of a constraint, i.e., solving the unconstrained optimization problem

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right] \quad (5)$$

for some coefficient β . This follows from the fact that a certain surrogate objective (which computes the max KL over states instead of the mean) forms a lower bound (i.e., a pessimistic bound) on the performance of the policy π . TRPO uses a hard constraint rather than a penalty because it is hard to choose a single value of β that performs well across different problems—or even within a single problem, where the characteristics change over the course of learning. Hence, to achieve our goal of a first-order algorithm that emulates the monotonic improvement of TRPO, experiments show that it is not sufficient to simply choose a fixed penalty coefficient β and optimize the penalized objective Equation (5) with SGD; additional modifications are required.

3 Clipped Surrogate Objective

Let $r_t(\theta)$ denote the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$, so $r(\theta_{\text{old}}) = 1$. TRPO maximizes a “surrogate” objective

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] = \hat{\mathbb{E}}_t [r_t(\theta) \hat{A}_t]. \quad (6)$$

The superscript *CPI* refers to conservative policy iteration [KL02], where this objective was proposed. Without a constraint, maximization of L^{CPI} would lead to an excessively large policy update; hence, we now consider how to modify the objective, to penalize changes to the policy that move $r_t(\theta)$ away from 1.

The main objective we propose is the following:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (7)$$

where epsilon is a hyperparameter, say, $\epsilon = 0.2$. The motivation for this objective is as follows. The first term inside the min is L^{CPI} . The second term, $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t$, modifies the surrogate objective by clipping the probability ratio, which removes the incentive for moving r_t outside of the interval $[1 - \epsilon, 1 + \epsilon]$. Finally, we take the minimum of the clipped and unclipped objective, so the final objective is a lower bound (i.e., a pessimistic bound) on the unclipped objective. With this scheme, we only ignore the change in probability ratio when it would make the objective improve, and we include it when it makes the objective worse. Note that $L^{CLIP}(\theta) = L^{CPI}(\theta)$ to first order around θ_{old} (i.e., where $r = 1$), however, they become different as θ moves away from θ_{old} . Figure 1 plots a single term (i.e., a single t) in L^{CLIP} ; note that the probability ratio r is clipped at $1 - \epsilon$ or $1 + \epsilon$ depending on whether the advantage is positive or negative.

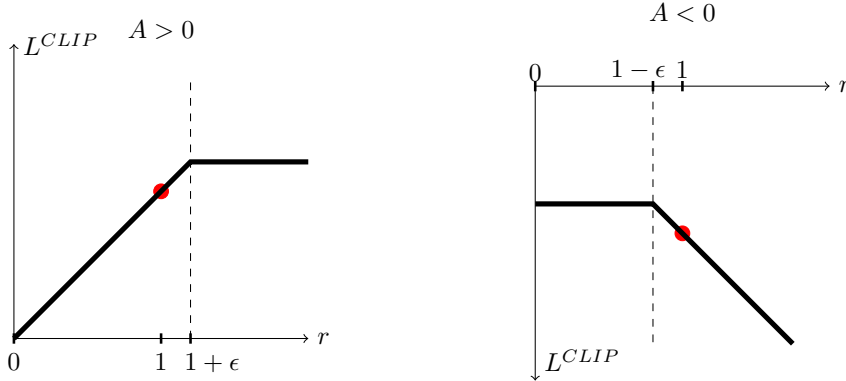


Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r , for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

Figure 2 provides another source of intuition about the surrogate objective L^{CLIP} . It shows how several objectives vary as we interpolate along the policy update direction, obtained by proximal policy optimization (the algorithm we will introduce shortly) on a continuous control problem. We can see that L^{CLIP} is a lower bound on L^{CPI} , with a penalty for having too large of a policy update.

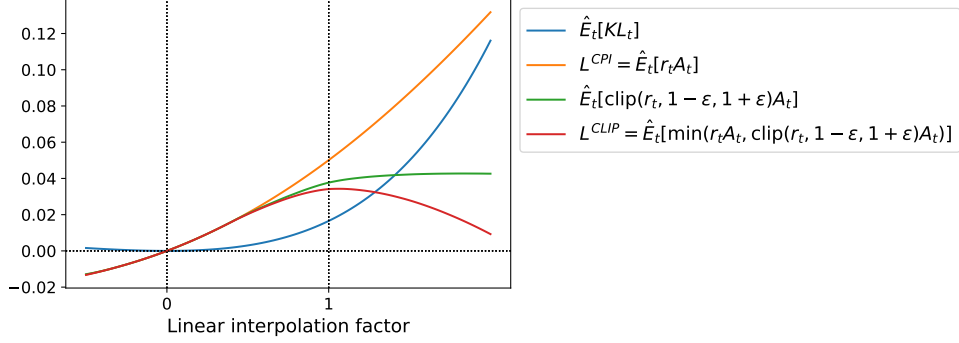


Figure 2: Surrogate objectives, as we interpolate between the initial policy parameter θ_{old} , and the updated policy parameter, which we compute after one iteration of PPO. The updated policy has a KL divergence of about 0.02 from the initial policy, and this is the point at which L^{CLIP} is maximal. This plot corresponds to the first policy update on the Hopper-v1 problem, using hyperparameters provided in Section 6.1.

4 Adaptive KL Penalty Coefficient

Another approach, which can be used as an alternative to the clipped surrogate objective, or in addition to it, is to use a penalty on KL divergence, and to adapt the penalty coefficient so that we achieve some target value of the KL divergence d_{targ} each policy update. In our experiments, we found that the KL penalty performed worse than the clipped surrogate objective, however, we’ve included it here because it’s an important baseline.

In the simplest instantiation of this algorithm, we perform the following steps in each policy update:

- Using several epochs of minibatch SGD, optimize the KL-penalized objective

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)] \right] \quad (8)$$

- Compute $d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)]]$
 - If $d < d_{\text{targ}}/1.5$, $\beta \leftarrow \beta/2$
 - If $d > d_{\text{targ}} \times 1.5$, $\beta \leftarrow \beta \times 2$

The updated β is used for the next policy update. With this scheme, we occasionally see policy updates where the KL divergence is significantly different from d_{targ} , however, these are rare, and β quickly adjusts. The parameters 1.5 and 2 above are chosen heuristically, but the algorithm is not very sensitive to them. The initial value of β is another hyperparameter but is not important in practice because the algorithm quickly adjusts it.

5 Algorithm

The surrogate losses from the previous sections can be computed and differentiated with a minor change to a typical policy gradient implementation. For implementations that use automatic differentiation, one simply constructs the loss L^{CLIP} or $L^{KL PEN}$ instead of L^{PG} , and one performs multiple steps of stochastic gradient ascent on this objective.

Most techniques for computing variance-reduced advantage-function estimators make use a learned state-value function $V(s)$; for example, generalized advantage estimation [Sch+15a], or the

finite-horizon estimators in [Mni+16]. If using a neural network architecture that shares parameters between the policy and value function, we must use a loss function that combines the policy surrogate and a value function error term. This objective can further be augmented by adding an entropy bonus to ensure sufficient exploration, as suggested in past work [Wil92; Mni+16]. Combining these terms, we obtain the following objective, which is (approximately) maximized each iteration:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)], \quad (9)$$

where c_1, c_2 are coefficients, and S denotes an entropy bonus, and L_t^{VF} is a squared-error loss $(V_\theta(s_t) - V_t^{\text{targ}})^2$.

One style of policy gradient implementation, popularized in [Mni+16] and well-suited for use with recurrent neural networks, runs the policy for T timesteps (where T is much less than the episode length), and uses the collected samples for an update. This style requires an advantage estimator that does not look beyond timestep T . The estimator used by [Mni+16] is

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T) \quad (10)$$

where t specifies the time index in $[0, T]$, within a given length- T trajectory segment. Generalizing this choice, we can use a truncated version of generalized advantage estimation, which reduces to Equation (10) when $\lambda = 1$:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1}, \quad (11)$$

$$\text{where } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (12)$$

A proximal policy optimization (PPO) algorithm that uses fixed-length trajectory segments is shown below. Each iteration, each of N (parallel) actors collect T timesteps of data. Then we construct the surrogate loss on these NT timesteps of data, and optimize it with minibatch SGD (or usually for better performance, Adam [KB14]), for K epochs.

Algorithm 1 PPO, Actor-Critic Style

```

for iteration=1, 2, ... do
  for actor=1, 2, ...,  $N$  do
    Run policy  $\pi_{\theta_{\text{old}}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{\text{old}} \leftarrow \theta$ 
end for

```

6 Experiments

6.1 Comparison of Surrogate Objectives

First, we compare several different surrogate objectives under different hyperparameters. Here, we compare the surrogate objective L^{CLIP} to several natural variations and ablated versions.

No clipping or penalty:	$L_t(\theta) = r_t(\theta)\hat{A}_t$
Clipping:	$L_t(\theta) = \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta)), 1 - \epsilon, 1 + \epsilon)\hat{A}_t$
KL penalty (fixed or adaptive)	$L_t(\theta) = r_t(\theta)\hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}, \pi_\theta]$

For the KL penalty, one can either use a fixed penalty coefficient β or an adaptive coefficient as described in Section 4 using target KL value d_{targ} . Note that we also tried clipping in log space, but found the performance to be no better.

Because we are searching over hyperparameters for each algorithm variant, we chose a computationally cheap benchmark to test the algorithms on. Namely, we used 7 simulated robotics tasks² implemented in OpenAI Gym [Bro+16], which use the MuJoCo [TET12] physics engine. We do one million timesteps of training on each one. Besides the hyperparameters used for clipping (ϵ) and the KL penalty (β, d_{targ}), which we search over, the other hyperparameters are provided in in Table 3.

To represent the policy, we used a fully-connected MLP with two hidden layers of 64 units, and tanh nonlinearities, outputting the mean of a Gaussian distribution, with variable standard deviations, following [Sch+15b; Dua+16]. We don’t share parameters between the policy and value function (so coefficient c_1 is irrelevant), and we don’t use an entropy bonus.

Each algorithm was run on all 7 environments, with 3 random seeds on each. We scored each run of the algorithm by computing the average total reward of the last 100 episodes. We shifted and scaled the scores for each environment so that the random policy gave a score of 0 and the best result was set to 1, and averaged over 21 runs to produce a single scalar for each algorithm setting.

The results are shown in Table 1. Note that the score is negative for the setting without clipping or penalties, because for one environment (half cheetah) it leads to a very negative score, which is worse than the initial random policy.

algorithm	avg. normalized score
No clipping or penalty	-0.39
Clipping, $\epsilon = 0.1$	0.76
Clipping, $\epsilon = 0.2$	0.82
Clipping, $\epsilon = 0.3$	0.70
Adaptive KL $d_{\text{targ}} = 0.003$	0.68
Adaptive KL $d_{\text{targ}} = 0.01$	0.74
Adaptive KL $d_{\text{targ}} = 0.03$	0.71
Fixed KL, $\beta = 0.3$	0.62
Fixed KL, $\beta = 1.$	0.71
Fixed KL, $\beta = 3.$	0.72
Fixed KL, $\beta = 10.$	0.69

Table 1: Results from continuous control benchmark. Average normalized scores (over 21 runs of the algorithm, on 7 environments) for each algorithm / hyperparameter setting . β was initialized at 1.

6.2 Comparison to Other Algorithms in the Continuous Domain

Next, we compare PPO (with the “clipped” surrogate objective from Section 3) to several other methods from the literature, which are considered to be effective for continuous problems. We compared against tuned implementations of the following algorithms: trust region policy optimization [Sch+15b], cross-entropy method (CEM) [SL06], vanilla policy gradient with adaptive stepsize³,

²HalfCheetah, Hopper, InvertedDoublePendulum, InvertedPendulum, Reacher, Swimmer, and Walker2d, all “-v1”

³After each batch of data, the Adam stepsize is adjusted based on the KL divergence of the original and updated policy, using a rule similar to the one shown in Section 4. An implementation is available at <https://github.com/berkeleydeeprlcourse/homework/tree/master/hw4>.

A2C [Mni+16], A2C with trust region [Wan+16]. A2C stands for advantage actor critic, and is a synchronous version of A3C, which we found to have the same or better performance than the asynchronous version. For PPO, we used the hyperparameters from the previous section, with $\epsilon = 0.2$. We see that PPO outperforms the previous methods on almost all the continuous control environments.

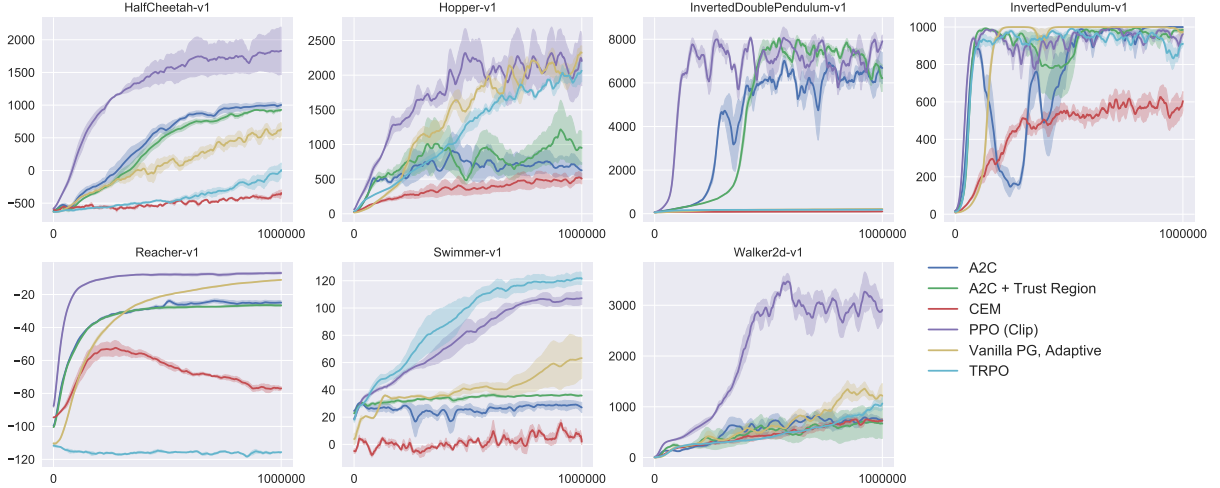


Figure 3: Comparison of several algorithms on several MuJoCo environments, training for one million timesteps.

6.3 Showcase in the Continuous Domain: Humanoid Running and Steering

To showcase the performance of PPO on high-dimensional continuous control problems, we train on a set of problems involving a 3D humanoid, where the robot must run, steer, and get up off the ground, possibly while being pelted by cubes. The three tasks we test on are (1) RoboschoolHumanoid: forward locomotion only, (2) RoboschoolHumanoidFlagrun: position of target is randomly varied every 200 timesteps or whenever the goal is reached, (3) RoboschoolHumanoidFlagrunHarder, where the robot is pelted by cubes and needs to get up off the ground. See Figure 5 for still frames of a learned policy, and Figure 4 for learning curves on the three tasks. Hyperparameters are provided in Table 4. In concurrent work, Heess et al. [Hee+17] used the adaptive KL variant of PPO (Section 4) to learn locomotion policies for 3D robots.

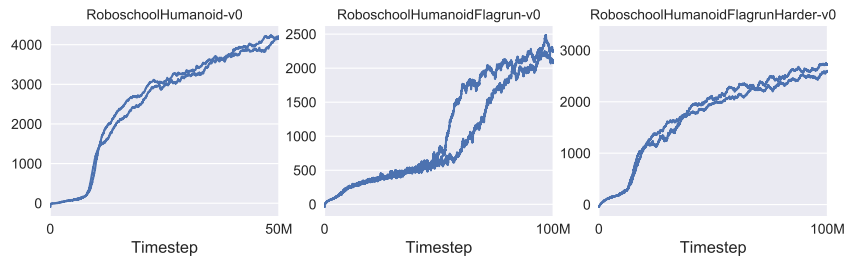


Figure 4: Learning curves from PPO on 3D humanoid control tasks, using Roboschool.

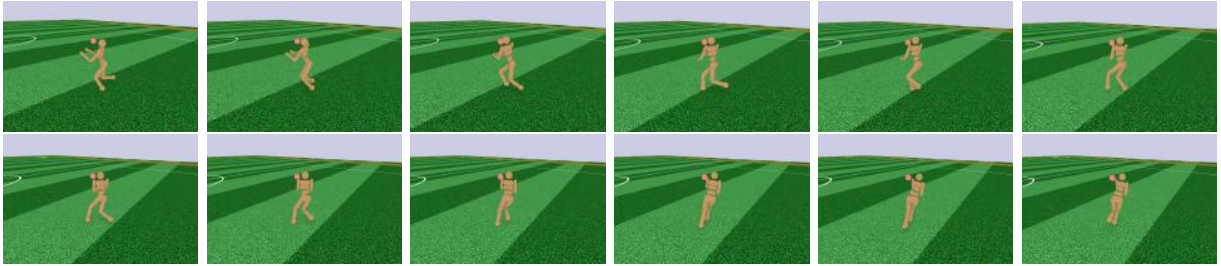


Figure 5: Still frames of the policy learned from RoboschoolHumanoidFlagrun. In the first six frames, the robot runs towards a target. Then the position is randomly changed, and the robot turns and runs toward the new target.

6.4 Comparison to Other Algorithms on the Atari Domain

We also ran PPO on the Arcade Learning Environment [Bel+15] benchmark and compared against well-tuned implementations of A2C [Mni+16] and ACER [Wan+16]. For all three algorithms, we used the same policy network architecture as used in [Mni+16]. The hyperparameters for PPO are provided in Table 5. For the other two algorithms, we used hyperparameters that were tuned to maximize performance on this benchmark.

A table of results and learning curves for all 49 games is provided in Appendix B. We consider the following two scoring metrics: (1) *average reward per episode over entire training period* (which favors fast learning), and (2) *average reward per episode over last 100 episodes of training* (which favors final performance). Table 2 shows the number of games “won” by each algorithm, where we compute the victor by averaging the scoring metric across three trials.

	A2C	ACER	PPO	Tie
(1) avg. episode reward over all of training	1	18	30	0
(2) avg. episode reward over last 100 episodes	1	28	19	1

Table 2: Number of games “won” by each algorithm, where the scoring metric is averaged across three trials.

7 Conclusion

We have introduced proximal policy optimization, a family of policy optimization methods that use multiple epochs of stochastic gradient ascent to perform each policy update. These methods have the stability and reliability of trust-region methods but are much simpler to implement, requiring only few lines of code change to a vanilla policy gradient implementation, applicable in more general settings (for example, when using a joint architecture for the policy and value function), and have better overall performance.

8 Acknowledgements

Thanks to Rocky Duan, Peter Chen, and others at OpenAI for insightful comments.

A Hyperparameters

Hyperparameter	Value
Horizon (T)	2048
Adam stepsize	3×10^{-4}
Num. epochs	10
Minibatch size	64
Discount (γ)	0.99
GAE parameter (λ)	0.95

Table 3: PPO hyperparameters used for the Mujoco 1 million timestep benchmark.

Hyperparameter	Value
Horizon (T)	512
Adam stepsize	*
Num. epochs	15
Minibatch size	4096
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	32 (locomotion), 128 (flagrun)
Log stdev. of action distribution	LinearAnneal($-0.7, -1.6$)

Table 4: PPO hyperparameters used for the Roboschool experiments. Adam stepsize was adjusted based on the target value of the KL divergence.

Hyperparameter	Value
Horizon (T)	128
Adam stepsize	$2.5 \times 10^{-4} \times \alpha$
Num. epochs	3
Minibatch size	32×8
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	8
Clipping parameter ϵ	$0.1 \times \alpha$
VF coeff. c_1 (9)	1
Entropy coeff. c_2 (9)	0.01

Table 5: PPO hyperparameters used in Atari experiments. α is linearly annealed from 1 to 0 over the course of learning.

B Performance on More Atari Games

Here we include a comparison of PPO against A2C on a larger collection of 49 Atari games. Figure 6 shows the learning curves of each of three random seeds, while Table 6 shows the mean performance.

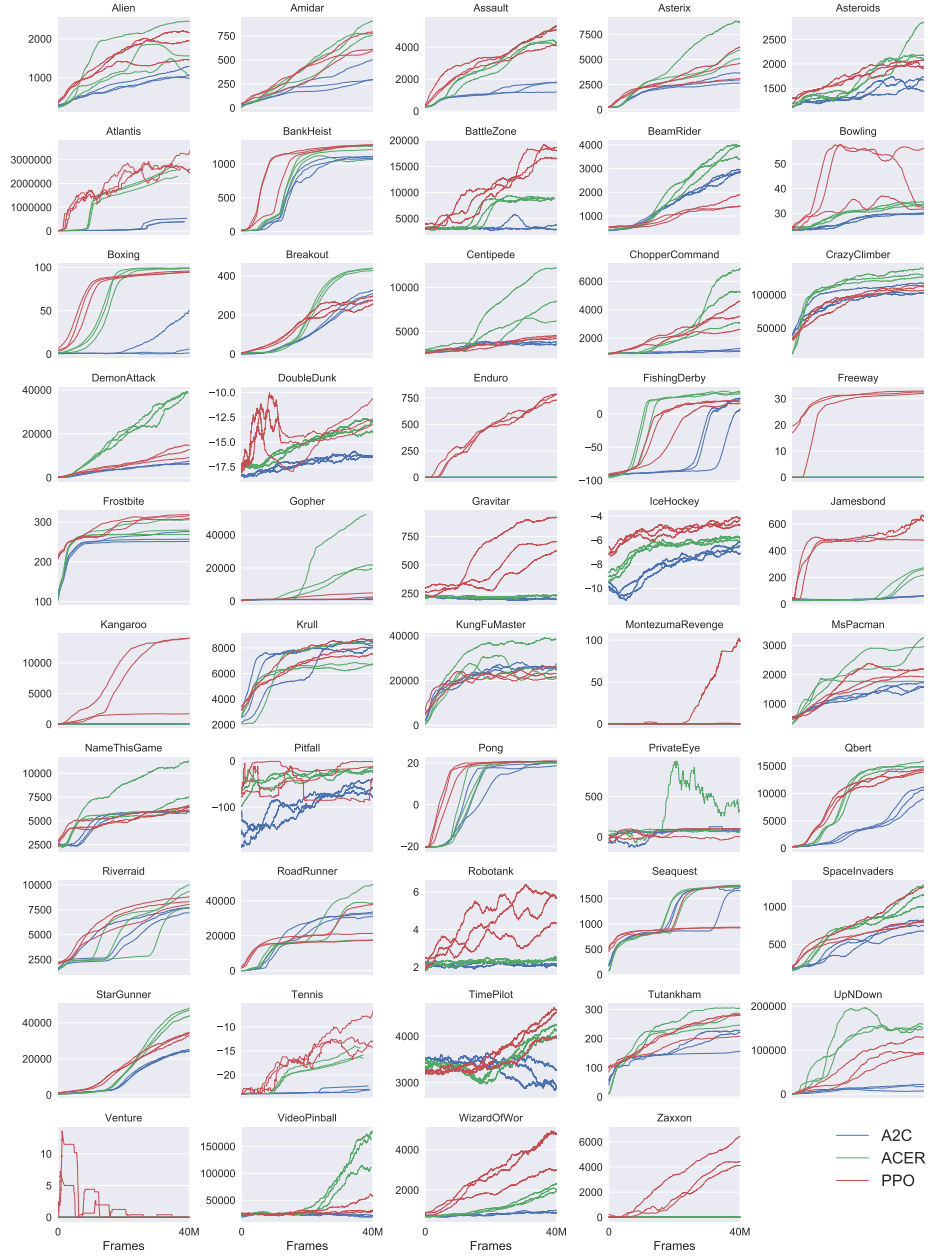


Figure 6: Comparison of PPO and A2C on all 49 ATARI games included in OpenAI Gym at the time of publication.

	A2C	ACER	PPO
Alien	1141.7	1655.4	1850.3
Amidar	380.8	827.6	674.6
Assault	1562.9	4653.8	4971.9
Asterix	3176.3	6801.2	4532.5
Asteroids	1653.3	2389.3	2097.5
Atlantis	729265.3	1841376.0	2311815.0
BankHeist	1095.3	1177.5	1280.6
BattleZone	3080.0	8983.3	17366.7
BeamRider	3031.7	3863.3	1590.0
Bowling	30.1	33.3	40.1
Boxing	17.7	98.9	94.6
Breakout	303.0	456.4	274.8
Centipede	3496.5	8904.8	4386.4
ChopperCommand	1171.7	5287.7	3516.3
CrazyClimber	107770.0	132461.0	110202.0
DemonAttack	6639.1	38808.3	11378.4
DoubleDunk	-16.2	-13.2	-14.9
Enduro	0.0	0.0	758.3
FishingDerby	20.6	34.7	17.8
Freeway	0.0	0.0	32.5
Frostbite	261.8	285.6	314.2
Gopher	1500.9	37802.3	2932.9
Gravitar	194.0	225.3	737.2
IceHockey	-6.4	-5.9	-4.2
Jamesbond	52.3	261.8	560.7
Kangaroo	45.3	50.0	9928.7
Krull	8367.4	7268.4	7942.3
KungFuMaster	24900.3	27599.3	23310.3
MontezumaRevenge	0.0	0.3	42.0
MsPacman	1626.9	2718.5	2096.5
NameThisGame	5961.2	8488.0	6254.9
Pitfall	-55.0	-16.9	-32.9
Pong	19.7	20.7	20.7
PrivateEye	91.3	182.0	69.5
Qbert	10065.7	15316.6	14293.3
Riverraid	7653.5	9125.1	8393.6
RoadRunner	32810.0	35466.0	25076.0
Robotank	2.2	2.5	5.5
Seaquest	1714.3	1739.5	1204.5
SpaceInvaders	744.5	1213.9	942.5
StarGunner	26204.0	49817.7	32689.0
Tennis	-22.2	-17.6	-14.8
TimePilot	2898.0	4175.7	4342.0
Tutankham	206.8	280.8	254.4
UpNDown	17369.8	145051.4	95445.0
Venture	0.0	0.0	0.0
VideoPinball	19735.9	156225.6	37389.0
WizardOfWor	859.0	2308.3	4185.3
Zaxxon	16.3	29.0	5008.7

Table 6: Mean final scores (last 100 episodes) of PPO and A2C on Atari games after 40M game frames (10M timesteps).